

COMPILED COURSE BOOK

CS 201 - Computer Systems

This PDF compiles the current HTML course modules and reference guides into a single print-oriented book. Interactive D2L activities are replaced with notes where necessary.

105 pages compiled from 11 manifest sections

September 8, 2026

Contents

Module 1: C and Integer Representations

1. Module 1 Overview
2. Lesson 1: C++ to C Language
3. Lesson 2: Bits, Bytes, and Data Sizes
4. Lesson 3: Hexadecimal Numbers
5. Lesson 4: Byte Ordering
6. Lesson 5: Bitwise Operations
7. Lesson 6: Bit Flags and Masking
8. Lesson 7: Integer Data Types
9. Lesson 8: Unsigned Integer Encoding
10. Lesson 9: Signed Integer Encoding
11. Lesson 10: Type Casting
12. Lesson 11: Integer Arithmetic
13. Module 1 Summary

Module 2: Floating Point

1. Module 2 Overview
2. Lesson 1: Fractional Binary Numbers
3. Lesson 2: IEEE 754 Floating Point Encoding Scheme
4. Lesson 3: Floating Point Examples
5. Lesson 4: Floating Point Rounding
6. Lesson 5: Floating Point Operations
7. Lesson 6: The Posit Number System
8. Module 2 Summary

Module 3: x86-64 Assembly Fundamentals

1. Module 3 Overview
2. Lesson 1: The Build Process and Your First Assembly Program
3. Lesson 2: Writing Makefiles
4. Lesson 3: Registers, Data Types, and the CPU Environment
5. Lesson 4: Moving Data
6. Lesson 5: The Data Section and Labels
7. Lesson 6: Arithmetic Operations
8. Lesson 7: Logical and Shift Operations
9. Lesson 8: The Stack, Push, and Pop
10. Lesson 9: Putting It All Together
11. Lesson 10: Debugging with GDB
12. Module 3 Summary

Module 4: x86-64 Assembly - Control, Functions, and Data

1. Module 4 Overview
2. Lesson 1: Condition Codes, Comparisons, and Jumps
3. Lesson 2: If Statements
4. Lesson 3: Loops
5. Lesson 4: Switch Statements
6. Lesson 5: Writing Functions
7. Lesson 6: Local Variables and the Stack Frame
8. Lesson 7: Arrays, Structs, and Data Alignment
9. Lesson 8: Buffer Overflows and Security
10. Lesson 9: Floating Point in Assembly
11. Module 4 Summary

Module 5: Processes, Signals, and Forking

1. Module 5 Overview
2. Lesson 1: Flow Control
3. Lesson 2: Exceptions
4. Lesson 3: Classes of Exceptions
5. Lesson 4: Exceptions in Linux/x86-64 Systems
6. Lesson 5: Processes
7. Lesson 6: Process Control
8. Lesson 7: Examples of Using Fork
9. Lesson 8: Signals in Linux Systems
10. Lesson 9: Signal Management
11. Lesson 10: Blocking and Unblocking Signals
12. Lesson 11: Writing Signal Handlers
13. Lesson 12: Signal Handler Examples
14. Module 5 Summary

Module 6: Threads and Synchronization

1. Module 6 Overview
2. Lesson 1: Threads vs. Processes
3. Lesson 2: Creating and Joining Threads
4. Lesson 3: Race Conditions
5. Lesson 4: Mutexes
6. Lesson 5: Condition Variables and Semaphores
7. Lesson 6: Classic Concurrency Problems
8. Module 6 Summary

Module 7: File I/O and Sockets Programming

1. Module 7 Overview
2. Lesson 1: File Descriptors and Low-Level I/O
3. Lesson 2: The FILE API and Directories
4. Lesson 3: TCP/IP and Socket Basics
5. Lesson 4: Building a Server
6. Lesson 5: Clients and Multi-Client Servers
7. Lesson 6: From Echo Server to Web Server
8. Module 7 Summary

Module 8: Memory Hierarchy and Optimization

1. Module 8 Overview
2. Lesson 1: Random Access Memory (RAM)
3. Lesson 2: Locality
4. Lesson 3: Memory Caches
5. Lesson 4: Optimization - Why and When?
6. Lesson 5: gcc Optimization Levels and Limitations
7. Lesson 6: How to Measure Performance
8. Lesson 7: How to Optimize Code
9. Lesson 8: Cache-Aware Programming
10. Module 8 Summary

Module 9: Virtual Memory, Linking, and Dynamic Memory

1. Module 9 Overview
2. Lesson 1: Virtual Memory
3. Lesson 2: Static and Dynamic Linking
4. Lesson 3: Dynamic Memory Allocation
5. Lesson 4: Debugging Memory Errors with Valgrind
6. Lesson 5: Garbage Collection
7. Module 9 Summary

Module 10: Kernel Organization

1. Module 10 Overview
2. Lesson 1: What a Kernel Does
3. Lesson 2: Monolithic Kernels
4. Lesson 3: Microkernel Architectures
5. Lesson 4: Tradeoffs and Modern Approaches
6. Module 10 Summary

Reference: Systems Programming Guides

1. Integer and Floating Point Reference
2. `_start` Assembly Syntax Reference
3. Processes, Signals, and Exceptions Reference
4. Concurrency and Threads Reference
5. File I/O and Sockets Reference
6. Memory Hierarchy and Optimization Reference
7. Virtual Memory, Linking, and Heap Reference
8. Kernel Organization Reference

Module 1 Overview

In Module 1: C and Integer Representations, we will cover the C programming language and how data is represented at the hardware level. We begin by learning the basics of C, a language that maps almost directly to hardware instructions. From there, we explore how numbers are stored inside a computer, including hexadecimal and binary representation, byte ordering, and bitwise operations. The second half of the module focuses on integer data: how signed and unsigned integers are encoded in memory, how integer arithmetic works, and what happens when values overflow their fixed-size containers.

By the end of this module, you will have a solid foundation in both C programming fundamentals and the integer representations that underlie all computation.

Learning objectives

At the end of this learning module, you will be able to:

1. Explain the basic syntax of C, including functions and command line arguments
2. Convert between binary, hexadecimal, and decimal numbering systems
3. Interpret byte ordering (big-endian vs. little-endian) in memory
4. Use bitwise operators, bit flags, and masking techniques
5. Compare and contrast signed and unsigned integer representations
6. Explain Two's Complement encoding for signed integers
7. Describe type casting and conversion between signed and unsigned integers
8. Explain integer arithmetic and identify when overflow occurs

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below
2. Read Lessons 1 to 11 and complete the embedded practice problems in the lessons that include them

3. Take the module quiz

4. Review additional resources on the Module 1 Summary page

Reading

- Bryant & O'Hallaron: Chapter 2
- Matthews: Chapters 0 - 4

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: C++ to C Language

Introduction

History of C and C++

The C language was the original precursor to the C++ language. C was developed in the early 1970s by Dennis Ritchie at Bell Laboratories of AT&T. C++ was later developed by Bjarne Stroustrup, beginning in 1979, and was originally named "C with Classes." It added classes and many other language features on top of a C-like foundation. Both languages have gone through many revisions since then and remain important in systems, embedded, and performance-sensitive software.

C++ and C Are Closely Related

If you are coming from a C++ background you will find much of C syntax familiar. However, modern C++ is **not** a strict superset of C: the languages have diverged, and some valid C programs are not valid C++ programs. For this course, use **gcc** for C code so the compiler parses the source as C and uses the normal C toolchain. Some simple C programs may also compile with **g++**, but that should not be relied on.

What This Summary Is

This is just a brief description of what a C++ coder needs to know in order to write C code. So this is NOT a tutorial on writing C as a beginner. I'm assuming students have taken CS162 and know how to at least program in C++. You can find plenty of tutorials on C on the internet. A widely available and authoritative reference is *The C Programming Language* by Brian W. Kernighan and Dennis M. Ritchie.

Input/Output

One of the largest differences between C and C++ is how IO is handled. For the C language, to get input from the keyboard and send output to the terminal, you will need to use the C standard IO library. This is included by using the line:

```
#include <stdio.h>
```

Note the use of `.h` on the end of the library name. This is used for including all standard libraries.

printf

The first argument to `printf` is a pointer to a **format string**, usually written as a string literal. The remaining arguments supply the values to format and must have the types required by the conversion specifiers. The format need not be a literal, but do not use untrusted input as the format string; print it with `printf("%s", text)`.

Both C and C++ functions can be designed to take a variable number of arguments. This is something they don't teach you in CS161/162, as it isn't done often. If you want to know more about this, here is a resource you can consult. [Tutorialspoint: C - Variable Arguments](#)

Below is an example of using `printf`. We have two variables that need to be printed: a string for a name and an int for an age.

```
printf("Name: %s age: %d\n", name, age);
```

If the name is **Kayla** and the age is **23**, this will print:

```
Name: Kayla age: 23
```

Followed by a new line.

The percent sign starts the beginning of a **format specifier**. Each format specifier will be replaced by the value of a variable in the list of variables following the format string. For our example, the variable `name` is first in the list of variables, and so the first format specifier is replaced by the value of `name`. Likewise, the second format specifier is replaced by the value of the second variable `age`. The specifier's type needs to match the variable's type: a mismatch will still compile, but the behavior at runtime is undefined. GCC will usually warn you about a mismatch (via `-Wformat`), but it compiles the program anyway.

The letters/numbers that follow the % indicate the type of value and optionally other information, such as number of decimal places to print, column width, etc. Each type is indicated by one or two letters. In our example, the letter '**s**' represents a string (C-String as there is no 'string' class in C) and the '**d**' represents a signed integer.

Using `printf` can be more complex sometimes than using the IO stream library. For more information on `printf`, see [Tutorialspoint: C library function - print\(\)](#) [link opens in new

tab]. You can also do your own search for `printf` on Google to see many examples of using `printf`.

scanf

In the C language, the `scanf` function is used to input data from the keyboard. It also takes a variable list of arguments but the first argument is always a format string. The rest of the arguments are variables of any size or type.

Basically, the format string indicates the type(s) of input that is to be entered and in what order. The variable list is the variables to copy into. The variable list is similar to `printf` in that the order of the variables must be in the same order as the format specifiers in the format string.

C passes every argument **by value**. To let `scanf` write into a variable, pass a pointer to it: `&age` supplies the address of `age`. In this call, the array expression `name` converts to a pointer to its first element, so it needs no `&`. The pointer itself is passed by value; arrays are not an exception to this rule.

Below is an example of using `scanf`. We want to pick up from the keyboard two values: a string for a name and an int for an age.

```
#include <stdio.h>

char name[200];
int age;
int main() {
    printf("Enter name followed by a space then an age\n");
    // note the & on the age but not on the name variable
    if (scanf("%199s %d", name, &age) != 2) {
        fprintf(stderr, "Expected a name and an integer age\n");
        return 1;
    }

    printf("name %s age %d\n", name, age);

    return 0;
}
```

Always give `%s` a field width that bounds it to the size of your buffer (here `%199s`, leaving room for the terminating null character in a 200-byte array), and check `scanf`'s return value, which tells you how many items were successfully read.

Do not assume the formats are interchangeable: `scanf("%f", &f)` needs a `float *`, while `scanf("%lf", &d)` needs a `double *`. For `printf`, a `float` argument is promoted to `double`, and `%f` prints either. For more information on `scanf`, here is a link: [Tutorialspoint: C library function - scanf\(\)](#) Again, you can also do your own search for `scanf` on Google to see many examples of using `scanf`.

Compiling and Running Programs

The C compiler on the Linux system is `gcc`. So make sure you are using this compiler instead of `g++`. C language source code files end in `.c` not in `.cpp` though header files still end in `.h`.

The same options as `g++` that are not C++ centric (such as setting the version of C++) will work with `gcc`. Later in the course I will be introducing some new options for `gcc` that you will need for your programming labs.

For example, a basic command line for compiling C code would be:

```
gcc hello.c -o hello
```

The `-o` option does what it does in C++. It changes the name of the program to whatever follows; without this, it has the name `a.out` similar to `g++`.

Once you compile you can just run the program as you would any program.

```
./hello
```

We will go into more details of compiling in `gcc` when we talk about object code in Module 3.

structs

The C language has structs. You define structs the same way you do in C++. For example:

```
struct Employee
{
    char name[200];
    double pay;
};
```

But you need to add the keyword `struct` any time you need to use the name of the struct in a declaration. For example:

```
struct Employee emp;
```

It also is needed when listing parameters:

```
void addEmployee(struct Employee emp)
```

You won't need it when using the variable itself, just whenever you need to declare it or use it in a parameter list.

OR

Structs can be defined like this, and then you don't need to say "struct" everywhere:

```
typedef struct
{
    char name[200];
    double pay;
} Employee;
```

Even better would be:

```
typedef struct
{
    char* name; // dynamically allocate based on string length
    double pay;
} Employee;
```

Passing Parameters by Reference

While the address-of operator **&** does exist in C, it cannot be used to create reference parameters as in C++. To pass something by reference, pass the address of the variable and then use a pointer to change the value inside a function.

For example, the following code works in C++:

```
void changeEmployee(Employee &emp)
{
    ....
    emp.pay = 100.00;
}

...
changeEmployee(emp);
...
```

But in the C language, you will need to pass the address instead and use a pointer inside the function:

```
void changeEmployee(struct Employee* emp)
{
    ....
    emp->pay = 100.00;
}

...
changeEmployee(&emp);
...
```

For primitive data types, you'll need to dereference the pointer to use it in the function:

```
void addEmployee(struct Employee array[], int* count)
{
    int size = *count;
    ....
    (*count)++;
    ....
}
```

Using Constants

A `const` object cannot be modified through that name, but an ordinary `const int` is not an integer constant expression in C. It can be used as a block-scope variable-length array bound where VLAs are supported; it cannot supply a file-scope array bound. For a constant expression, use a macro or an enumeration constant (C23 also adds `constexpr`). For example, instead of:

```
const int MAX_STR=100;
```

You can use:

```
#define MAX_STR 100
```

Use of String Data

C does not have a `string` class. This is the moment you've been preparing for: from here on, you will use null-terminated character arrays (known as "C-strings") for all string data. If

you need string manipulation functions like `strlen` , `strcpy` , or `strcmp` , include the string library:

```
#include <string.h>
```

Dynamic Memory

Another important area of differences is handling dynamic memory. There are no **new** or **delete** operators in C. Memory allocation is done via several functions, the most popular being `malloc` . So to allocate space for an Employee struct on the heap, use:

```
struct Employee *emp = (struct Employee*) malloc(sizeof(struct Employee));
```

This will create one struct Employee on the heap. Note that `malloc` returns a pointer of type `void*` . In C, `void*` implicitly converts to a pointer to any object type, so the cast is optional (though required if compiling as C++). Many C programmers omit the cast because it can mask a missing `#include <stdlib.h>` .

To free up the memory, you will need to use the function **free**:

```
free(emp);
```

The functions `malloc` and `free` are available in the C standard library. You will need to include this library.

```
#include <stdlib.h>
```

Booleans

C99 through C17 provide the built-in type `_Bool` ; including `<stdbool.h>` supplies the convenient names `bool` , `true` , and `false` . In C23 those names are language keywords and need no header. For code that also works in C99 through C17, include:

```
#include <stdbool.h>
```

You can also use an int variable and use the defined constants `TRUE` and `FALSE` for values. Just use the define preprocessor in a header file and then you can use these in your code:

```
// main.h
#define TRUE 1
#define FALSE 0 // you can also use !TRUE here

// main.c
int quit = FALSE;
....
while(!quit)
...
```

Typecasting

The newer C++ type-safe casting operators do not exist in C. So use the C typecasting operator `()` for typecasting.

```
double pay2 = (double) pay1;
```

Using Command Line Arguments

You can pass in input to a C/C++ program by way of command line arguments. This is similar to Bash shell scripts where you can include a space separated list of arguments after the name of the program:

```
./hello Frank Martin 30 5.25
```

Note that if you wanted the name Frank Martin to be one argument, then you need to put it in double quotes:

```
./hello "Frank Martin" 30 5.25
```

So how does your program pick up the arguments? You start by using a different header for your main function that includes an **int** as the first parameter and a pointer to a pointer to a char, or **char****, as the second parameter.

```
int main(int argc, char** argv)
```

They are typically called `argc` and `argv` but you can call them whatever you want to. Just don't change the order or the type of these parameters.

`argv` is the arguments as an array of C-Strings and `argc` is the number of arguments (including the program name). You can access a particular C-String by using indexing. The

first, or 0 index, is the name of the program. The second, or 1 index, is the 1st argument and so on.

```
printf("program name: %s\n", argv[0]);  
printf("1st argument: %s\n", argv[1]);  
.....
```

You can use `argc` in a for loop to go through the list of arguments and print them.

Note that the arguments are C-Strings so if you need them as integer types or floating point types, you can use the functions `atof()` for floating point types and `atoi()` for integer types.

```
int inum = atoi(argv[2]);  
float fnum = atof(argv[3]);
```

You may need to include `stdlib.h` to use these functions.

I should point out that command line arguments are not just a feature of C alone; you can use command line arguments in C++.

Summary Table of Differences

This is a list of differences between C and C++. This is not a comprehensive list; it's only those features you might run into in this class.

C++	C
Has OOP concepts: classes and objects.	Doesn't have any OOP features.
Exception handling is a mechanism built into the language.	Exception handling is not built in but handled through libraries.
Has operator and function overloading	Doesn't have these features.
Parameters can be passed by reference.	All arguments are passed by value. Pass a pointer to let a function modify the pointed-to object; array parameters are adjusted to pointer parameters.
Enumerations (enum types) built into language	Has <code>enum</code> types, but they are essentially integer constants (no type safety).
<code>string</code> type available (via a string class)	Use char arrays to implement strings (C-Strings)
boolean type (<code>bool</code>) built into the language	C99–C17: include <code><stdbool.h></code> for <code>bool</code> , <code>true</code> , and <code>false</code> . C23: these are keywords.
functions with default arguments	Doesn't have this at all.
use of namespaces	Doesn't have this at all.
file extension: <code>.c++</code> or <code>.cpp</code>	file extension: <code>.c</code>
IO: io stream library (<code>iostream</code>) >> and << get, getline ...	IO: standard io library (<code>stdio.h</code>) <code>printf</code> , <code>scanf</code> , <code>fprintf</code> , <code>sprintf</code> ...
dynamic memory: <code>new</code> and <code>delete</code>	dynamic memory: <code>malloc()</code> , <code>free()</code> , <code>calloc()</code> , etc.
no need to use the struct keyword except in defining structs	need to use struct when declaring a variable or in a parameter list.
typecasting: can use <code>()</code> or type-safe casting which gives more control over conversions.	typecasting: can only use <code>()</code>

Lesson 2: Bits, Bytes, and Data Sizes

Information Storage

In this summary we will be covering the binary representation of data within a computer system. Data includes numbers, symbols (letters, digits, etc), graphics, audio, and other data. Throughout this course, we will concentrate on numeric data.

No matter what type of data, all data is represented by binary digits or **bits**. A bit can be one of two values: 0 or 1. Bits are stored in memory in a sequence of cells. Each 8 cells (1 byte) is addressable as a whole by the system. Therefore, the smallest unit of addressable memory is one byte (8 bits).

With the exception of one-byte data values, data is stored across multiple cells. We will talk about how this is done in the lecture on byte ordering.

The **bit pattern** is the series of bits across many cells that represent one data value. Since a user is not likely to know binary, this bit pattern will need to be translated into the actual value (numbers, letters, etc) that the user can understand. This is typically done in hardware.

from bit pattern to number

Address	Contents of Cell	Data Value User Sees
...	...	...
200	10100101	3237
201	00001100	
202	00000000	
203	00000000	
204	11110100	...
....		

Word and Data Sizes

The **word size** describes an architecture's natural unit of integer processing; terminology varies (x86 instruction names still use "word" for 16 bits). On the Linux x86-64 LP64 platform used in this course, general-purpose registers and pointers are 64 bits wide. Pointer width, virtual address width, and physical address width are related but distinct. A 32-bit byte address can name 2^{32} bytes (4 GiB) in one address space, but a 32-bit x86 system with Physical Address Extension (PAE) can have more physical RAM. A full 64-bit byte address could name 2^{64} bytes (16 EiB); actual x86-64 hardware and operating systems implement smaller address ranges.

Do not infer the size of `int` from register or pointer width: on our LP64 platform, `int` is 32 bits while `long` and pointers are 64 bits.

In C/C++ languages, all simple or primitive data types have a fixed size associated with them. Sizes depend on the implementation and its data model, not just the architecture's word size.

Datatypes like `int` can be different byte lengths on different architectures, and the `sizeof()` operator can be used to compensate for that:

```
int a[100];
int n_elements = sizeof(a) / sizeof(int);
```

Below is a table of the most common data types and their sizes for the 64-bit architecture (this is a portion of figure 2.3 in the textbook)

Type	Size in Bytes
char (signedness is implementation-defined)	1
short	2
int	4
long	8 (4 on 64-bit Windows)
char*	8
float	4
double	8

Some things to note: `char *` occupies 8 bytes in the LP64 data model used here. The sizes for `int` and `long` are *not* the same on all 64-bit systems. Linux and macOS use the **LP64** data model where `long` is 8 bytes, but 64-bit Windows uses the **LLP64** data model where `long` stays at 4 bytes (only `long long` and pointers grow to 8 bytes). When portability matters, use the explicit-width types from `<stdint.h>` (`int32_t`, `int64_t`, `uint8_t`, etc.).

There are **signed** and **unsigned** versions of integer numbers. The signed versions can represent both positive, zero, and negative. The unsigned versions can only represent positive and zero. They are typically the same size but have different ranges of values. We'll talk at length about signed and unsigned integer types later in this module (Lessons 7–11). Floating point number types (`float` and `double`) are always signed types. We'll talk about floating point representation in Module 2.

Template JavaScript

Lesson 3: Hexadecimal Numbers

NOTE: This is a summary of chapter 2, section 2.1 in the textbook.

Numeric Representation: Hexadecimal Numbers

Binary numbers are a bit unwieldy to read. Instead of trying to display binary information in binary digits, a more compact numbering system related to binary is used to display binary numbers. This is the **hexadecimal system**.

The hexadecimal system is a numeric system with 16 digits. The digits are represented by the digits 0 to 9 and the letters A to F. The letters are needed to add the extra digits past 9 that a decimal system doesn't have. 'A' represents the equivalent of the decimal number 10, 'B' represents the equivalent of decimal 11, and so on.

Each hexadecimal digit can be directly translated into a set of 4 binary digits. Below is a table of each hex digit and its corresponding 4 binary digits:

Hex Digit	Binary Number
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A (10)	1010
B (11)	1011
C (12)	1100
D (13)	1101
E (14)	1110
F (15)	1111

Since translating between hexadecimal and binary is an exact one-digit to four-digit translation, it's easy to convert between the two numbering systems. For example, if a hex number is:

98A3F

then, the binary equivalent is obtained by substituting each hex digit for its equivalent 4 bit binary number:

9 8 A 3 F

1001 1000 1010 0011 1111

So, the equivalent binary number is 10011000101000111111. Note that the hex number only requires 5 digits, and the binary number requires 20 digits. This makes for a more compact representation in hex and is the reason why it is common to display binary data as hex digits.

Although this is rarely done, you can translate hex numbers into decimal numbers. This is not an exact digits-to-digits translation. To go from hex to decimal, use the formula:

$$\text{decimal} = h(n)*16^n + h(n-1)*16^{(n-1)} + \dots + h(1)*16 + h(0)$$

where **h(n)** is the nth hex digit (replace any letter digit with its decimal equivalent and start n at 0). For example, our number above in decimal is:

$$\text{dec} = 9*16^4 + 8*16^3 + 10*16^2 + 3*16 + 15$$

$$\text{dec} = 589824 + 32768 + 2560 + 48 + 15$$

$$\text{dec} = 625215$$

As I said, this does not often need to be done, and if you need the decimal equivalent, feel free to find a calculator that does the conversion for you (or a web page; there are plenty of them on the internet).

Two hex digits represent one byte. Since much of memory is stored in one-byte increments, you will often see hex numbers written in groups of two to represent one byte per group. For our example (where we need to pad out with a zero the first group to make two digits):

09 8A 3F

In C/C++ language, to indicate that a literal number is a hex number, use **0x** or **0X** in front of the number: **0x098A3F**.

Common misconception: reading `0x10` as ten. It's *sixteen*. Always look for the `0x` prefix — it changes the entire meaning of the digits that follow. Likewise `0xff` is 255, `0x100` is 256, and so on. Forgetting this is the single most common mistake when first reading hex.

Here's an example of a small program that uses hex syntax for arithmetic:

```
#include <stdio.h> // this is how we are able to call printf

int main(){

    int hex1 = 0x1A; // 16 + 10 = 26
    int hex2 = 0xFF; // 15*16 + 15 = 255

    printf("hex1 in decimal is: %d\n", hex1); // the %d directive prints
signed integers in decimal
    printf("hex2 in decimal is: %d\n", hex2);
    printf("The sum in hex is: 0x%x\n", hex1 + hex2); // the hex directive
is %x (or %X if you want UPPERCASE LETTERS)

    return 0;
}
```

Hex Color Codes

A lot of people's first introduction to writing numbers in hex isn't decompiling code or low-level system programming but dealing with color in art programs. For computers, color is generally represented as "24-bit color depth," which means that we're using 24 bits to represent the full RGB color.

We do this by splitting the 24 bits into 8 bits for red, 8 bits for green, and 8 bits for blue.

Now, what numbers can you represent with 8 bits? 0 – 255!

And if you use two digits of hex, what numbers can you represent? Well, that's `0x00`, which is 0 through `0xFF`, which is

15*16 + 15

or 255.

So, in order to represent a 24-bit color, we can just write a hex number with six digits! Pure black is then **0x000000**, pure white is **0xFFFFFF**, and something like teal is **0x39a78e**. If you've never played with hex representations of colors, check out [Color Hex](#) [link opens in new tab].

Template JavaScript

Lesson 4: Byte Ordering

Numeric Representation: Byte Ordering

Byte order is the order in which a multi-byte data type is stored in memory. These are stored as a consecutive set of bytes that are laid out from some low address to a high address, with the lowest address representing the "address" of this data.

There are two ways to lay out the bit pattern of a value across several bytes. One way is to make the most significant byte (the leftmost byte of the bit pattern) be stored starting at the high address. This is called **little endian**, and I won't get into why it's called this (the textbook talks about this weird name). The other way is to store the most significant 8 bits at the low address. This is called **big endian**.

For an example of byte ordering, if we are storing an integer, we need 4 consecutive bytes in memory. Note that for brevity, we will be showing each byte as a two-digit hex number. For example, if we have the hex number 0x12345678 to store, the bytes will be stored in 'big endian' as:

```
address -> 200 201 202 203
```

```
12 34 56 78
```

For little endian, the bytes will be stored:

```
address -> 200 201 202 203
```

```
78 56 34 12
```

Aside: Most computers you will work with are little-endian (x86 and most ARM deployments). Some commercial and embedded systems are still big-endian or bi-endian, so code that handles external binary formats should not assume one fixed byte order.

Common misconception: "endianness affects which order the bits within a byte are laid out." It does not. Within a byte, bit numbering is fixed — the most significant bit is always the leftmost. Endianness is purely about how a multi-byte value (a 4-byte `int` or an 8-byte `long`) is spread across consecutive memory addresses. A single byte has no endianness.

Network protocols specify their multi-byte fields in a defined byte order — commonly called **network byte order**, which is big-endian. So if you stuff a 2-byte port number into a TCP/IP header on a little-endian x86 machine, you must convert it first (using `htons()`; 4-byte values such as IPv4 addresses use `htonl()`) so the receiver reads the bytes correctly. We'll come back to this in Module 7 when we cover sockets.

For more information on the history of byte ordering, you can optionally read [On Holy Wars and a Plea for Peace](#) [link opens in a new tab].

The byte order usually doesn't affect programmers unless you are working across multiple bytes at the bit pattern level. This is pretty rare; you might run into this issue if you are doing very low-level or machine code level programming.

Byte ordering can be an issue when communicating data from one machine to another over a network connection. For more information about this, check the [Wikipedia entry for Endianness](#) [link opens in new tab].

Aside: If you print out a little-endian machine's byte-ordered value in the order of increasing addresses (which is how a char array is stored), each byte will be written in reverse order. If you want to see the actual bit pattern of a number in the right order on a 'little endian' machine, you can change the 'showbytes' function to print out the char bytes from len-1 to 0:

```
for (i = len-1; i >= 0; i--)
```

This will save you some time in deciphering the number in that you don't have to first reverse the hex digits to get the actual bit pattern from the printout.

This `showbytes.c` code file has this function in it.

```

#include <stdio.h>
#include <string.h>

typedef unsigned char * byte_pointer;

void show_bytes(byte_pointer start, size_t len)
{
    size_t i;
    for(i=0; i<len; i++)
        printf(" %.2x", start[i]);
    printf("\n");
}

void show_int(int x)
{
    show_bytes((byte_pointer) &x, sizeof(int));
}

void show_float(float x)
{
    show_bytes((byte_pointer) &x, sizeof(float));
}

void show_pointer(void *x)
{
    show_bytes((byte_pointer) &x, sizeof(void *));
}

void test_show_bytes(int val)
{
    int ival = val;
    float fval = (float) val;
    int *pval = &ival;

    show_int(ival);
    show_float(fval);
    show_pointer(pval);
}

int main()
{
    const char *sval = "12345";
    int i = -23432;

```

```
test_show_bytes(i);  
    show_bytes((byte_pointer) sval, strlen(sval)+1);  
    return 0;  
}
```

Template JavaScript

Lesson 5: Bitwise Operations

NOTE: This is a summary of chapter 2, section 2.1 in the textbook.

Bitwise Operators

Students are familiar with boolean operators `&&` for AND and `||` for OR operations. These work at the numeric level: they treat any nonzero value as true (and zero as false), and the result they produce is always 1 or 0.

There are boolean operators that work on the bit pattern itself. These are called bit-wise boolean operators because they perform boolean operations to each bit one at a time as opposed to the entire number. The following is a table of these bit-wise boolean operators:

Operator	Name	Example
<code>&</code>	bit-wise AND	<code>1010 & 1001 => 1000</code>
<code> </code>	bit-wise OR	<code>1010 1001 => 1011</code>
<code>~</code>	complement or negation	<code>~ 1011 => 0100</code>
<code>^</code>	eXclusive OR	<code>1010 ^ 1001 => 0011</code>

Note that each binary operator compares each bit with the corresponding bit in the same position. For bit-wise AND, if both bits are 1 then the result is 1 otherwise it is 0. For bit-wise OR, if both bits are 0 then the result is 0 otherwise it is 1. For exclusive OR, if both bits are the same the result is 0, otherwise the result is 1. The complement or negation operator flips the bit's value. If the bit is 1 then the result is 0; if the bit is 0 the result is 1.

Note that the bit-wise `&` and `|` use only **one** symbol instead of two. This is a common mistake, so be aware of it.

Shift Operators

Another set of bit-wise operators are bit shift operators. These will move the bit pattern over by some given amount of positions for a given number.

The left shift operator (`num << i`) moves each bit in **num** to the left by **i** amount. For example, if `num=10010011` and `i=2`, the resulting value will be:

```
10010011 => 01001100
```

Note that all left bits that don't fit into the number are just ignored, and the new right bits are filled with 0's.

One caution: left-shifting a signed value into (or past) the sign bit, or shifting by an amount greater than or equal to the width of the type, is undefined behavior in C — stick to unsigned types when shifting.

The right shift operator (`num >> i`) will shift the bit pattern of **num** by **i** amount to the right. For example, if `num` is an unsigned char equal to `10010011` and `i=2`:

```
10010011 => 00100100
```

For all integers, the rightmost bits are ignored. For unsigned integers, the bits on the left are filled in with 0's. This is called a **logical shift**. For signed integers, the C23 standard mandates two's-complement representation, but right shift of a negative signed value remains implementation-defined even in C23 (it was C++20 that defined it as arithmetic). In practice, every mainstream compiler (GCC, Clang, MSVC) performs an **arithmetic shift** — the left side is filled with whatever the original leftmost bit is (1 or 0) — so the behavior is consistent across the systems you will use. So, for our example, if **num** above is a signed char:

```
10010011 => 11100100
```

Another example:

```
01100010 => 00011000
```

Note that the bit that is filled in is whatever the leftmost bit happens to be. We'll discuss why this is so later in this module (Lesson 9) when we talk about how signed integers are represented in memory.

Lesson 6: Bit Flags and Masking

NOTE: This is a summary of chapter 2, section 2.1 in the textbook.

Numeric Representation: Bit Flags and Masking

One area you might need to use bit-wise operators is when working with bit flags. A **bit flag** is a (typically unsigned) integer where each bit represents an attribute that can be on or off. If the `ith` attribute is on, the `ith` bit will be one; if the `ith` attribute is off, the `ith` bit will be zero.

For example, let's say we have some settings for a motor. We can make an 8-bit flag where each represents a setting as listed in the table below (note that position goes from right to left):

Bit Position	Setting
1st position (right-most)	Turn on motor
2nd position	Slow Speed
3rd position	Medium Speed
4th position	Fast Speed
5th position	Reverse Motor
6th thru 8th position	Not Used - reserved for future use.

An example of a bit flag for this motor is one where we want it turned on and moving at a medium speed in reverse. The bit flag would look like:

```
00010101  // motor on, medium speed on, reverse on - all else off
```

Determining the bit in a bit flag

To determine whether a setting is on or off, we need to determine what the bit at the **ith** location is. One way is to set up a **mask**. This is a number where all positions are 0 except the **ith** position, which is set to 1. Let's say we want to determine if the motor is running at medium speed. Our mask would look like this:

```
00000100
```

The mask to find the **ith** bit can be built by setting the mask value to 1 and then left shifting this mask (**i-1**) places over. Note that positions are counted starting at 1 from the right-most bit, so the bit in the **ith** position needs a shift of **i-1**. So, in our example, the medium speed is in the 3rd position, giving a shift of 2:

```
00000001 << 2 -> 00000100
```

Bit-wise ANDing the mask to the number gives a result that is dependent only on the **ith** bit. The other bits will end up being 0 because 0 ANDed with any bit results in 0. So, with our example, we get:

```
00010101 & 00000100 -> 00000100
```

Note that if the medium speed was not turned on:

```
00010001 & 00000100 -> 00000000
```

So, only the medium speed setting determines whether the result is zero or non-zero. This can be tested for in your code.

If, after you have obtained the masked results, you need to pass back a 1 for on and a 0 for off, you can use the right shift operator (**>>**) to move the 1 bit to the right-most bit. Just shift by the position minus 1 of the bit:

```
00000100 >> 2 -> 00000001
```

Turning on or off a bit in a bit flag

For turning a bit on or off, typically you would set up a set of constants to use (more convenient than remembering the decimal number needed for each setting). In our example, we would have them defined at the top of the file:

```
#define MOTOR_ON      1    // mask for 1st bit - 2^0
```

```
#define SPEED_LOW     2    // mask for 2nd bit - 2^1
```

```
#define SPEED_MEDIUM  4    // mask for 3rd bit - 2^2
```

```
#define SPEED_HIGH    8    // mask for 4th bit - 2^3
```

```
#define REVERSE       16   // mask for 5th bit - 2^4
```

Note these create a set of pre-defined masks for each setting. To turn on a bit (or a series of bits), OR the bit flag with the corresponding constant (or set of constants):

```
// turns on the high speed
```

```
bit_flag = bit_flag | SPEED_HIGH
```

```
// turns on the motor and the low speed
```

```
bit_flag = bit_flag | MOTOR_ON | SPEED_LOW
```

To turn off a bit, take the complement of the constant and AND it with the bit flag:

```
// turns off motor and low speed
```

```
bit_flag = bit_flag & ~MOTOR_ON & ~SPEED_LOW
```

Here is a full example:

```
#include <stdio.h>
```

```
#define MOTOR_ON    1    // mask for 1st bit - 2^0
#define SPEED_LOW   2    // mask for 2nd bit - 2^1
#define SPEED_MEDIUM 4    // mask for 3rd bit - 2^2
#define SPEED_HIGH  8    // mask for 4th bit - 2^3
#define REVERSE     16   // mask for 5th bit - 2^4
```

```
void print_menu() {
    printf("\nBit Flag Menu:\n");
    printf("1. Toggle MOTOR_ON\n");
    printf("2. Toggle SPEED_LOW\n");
    printf("3. Toggle SPEED_MEDIUM\n");
    printf("4. Toggle SPEED_HIGH\n");
    printf("5. Toggle REVERSE\n");
    printf("0. Exit\n");
}
```

```
void print_bit_flag(int bit_flag) {
    printf("\nCurrent Bit Flag: %d\n", bit_flag);
    printf("MOTOR_ON: %s\n", (bit_flag & MOTOR_ON) ? "ON" : "OFF");
    printf("SPEED_LOW: %s\n", (bit_flag & SPEED_LOW) ? "ON" : "OFF");
    printf("SPEED_MEDIUM: %s\n", (bit_flag & SPEED_MEDIUM) ? "ON" :
"OFF");
    printf("SPEED_HIGH: %s\n", (bit_flag & SPEED_HIGH) ? "ON" : "OFF");
    printf("REVERSE: %s\n", (bit_flag & REVERSE) ? "ON" : "OFF");
}
```

```
int main() {
    int bit_flag = 0;
    int choice;

    do {
        print_bit_flag(bit_flag);
        print_menu();
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                bit_flag ^= MOTOR_ON;
                break;
            case 2:
                bit_flag ^= SPEED_LOW;
                break;
            case 3:
                bit_flag ^= SPEED_MEDIUM;
                break;
```

```
        case 4:
            bit_flag ^= SPEED_HIGH;
            break;
        case 5:
            bit_flag ^= REVERSE;
            break;
        case 0:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice. Please try again.\n");
    }
} while (choice != 0);

return 0;
}
```

Lesson 7: Integer Data Types

NOTE: This is a summary of chapter 2, sections 2.2 and 2.3 in the textbook.

Integer Representation: Integer Data Types

As mentioned earlier, there are two categories of integer types: **signed** and **unsigned**. The unsigned integer types can only represent positive and zero values. The signed integer types can represent positive, zero and negative values.

Signed values are said to have a **sign** (positive or negative) and a **magnitude** (ie. the size of the value). For example, -5 has a sign of - or negative and a magnitude of 5.

Below is a table of ranges for several unsigned and signed types. This is an abridged version from Figure 2.10 for 64-bit architecture. For unsigned, the minimum value is always 0.

Data Type	Minimum	Maximum
[signed] char	-128	127
unsigned char	0	255
short	-32,768	32,767
unsigned short	0	65,535
int	-2,147,483,648	2,147,483,647
unsigned int	0	4,294,967,295

There are some facts to note. For signed, the max and min magnitudes are about 1/2 of what their unsigned counterparts are. Also note that for signed, the min is one bigger in magnitude than the max. We'll talk about why when we discuss signed encoding later.

Lesson 8: Unsigned Integer Encoding

NOTE: This is a summary of chapter 2, sections 2.2 and 2.3 in the textbook.

Integer Representation: Unsigned Integer Encoding

The encoding of how to represent `unsigned int` in memory is straightforward. The bit pattern is interpreted as a binary number which is then translated to a decimal when displayed to the screen.

For a given size `w` bits, the maximum value of an unsigned numeric type is given by the formula:

$$UMAX = 2^w - 1$$

The minimum value (`UMIN`) for unsigned is always 0.

For example, for `w = 8` (an `unsigned char`):

$$UMAX = 2^8 - 1 = 256 - 1 = 255$$

This maximum value is the all-ones bit pattern:

$$11111111 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

Every bit contributes its power of two, so setting all `w` bits to 1 gives the largest value the type can hold.

Literal numbers (`1` , `0xa23` , etc) are not always treated as signed: C chooses the first integer type that can represent the value (based on the literal's base and size). If you need to force an unsigned literal, use `u` or `U` after the number.

`127556u` or `0x2A234U`

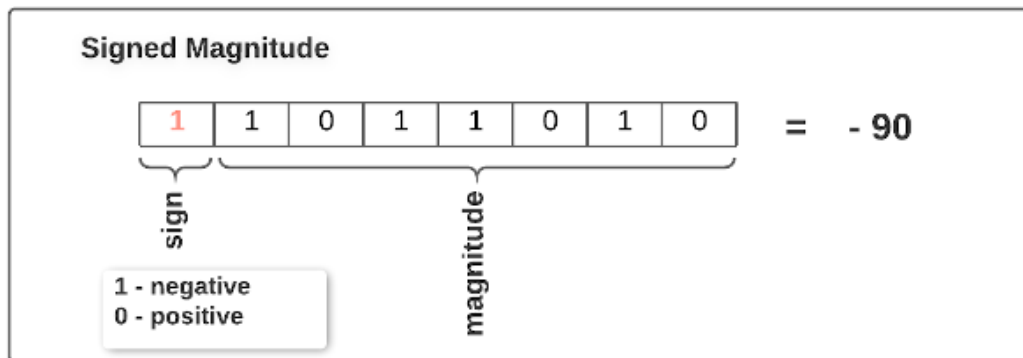
Lesson 9: Signed Integer Encoding

NOTE: This is a summary of chapter 2, sections 2.2 and 2.3 in the textbook.

Signed Integers: Two's Complement

Unsigned numbers are easy to come up with an encoding scheme since all we need is a magnitude to represent a number. For a signed number we need not only a magnitude but we need to indicate a sign as well. This is more of a challenge.

Some very straightforward encodings create added complexity when being designed in hardware. One straightforward way to encode signed numbers is to reserve the left-most bit as a sign bit. If this bit was 0 the number was positive; if it was 1 the number was negative. This is called **sign magnitude** encoding scheme.



signed magnitude method

It looks good at first, however, there are several problems with using this encoding. There are two possible zeros (one with all 0's and one with a 1 followed by all 0's) which makes checking for zero in hardware (a very common task) overly complex. It also makes doing arithmetic more complex.

To see this, first let's talk about how you would add a negative number to another number. The steps are:

- Take all negative numbers and get the complement of each. A complement of a binary number is where each bit is changed to its opposite (0 goes to 1 and 1 goes to 0). For sign

magnitude you would also need to ignore the leftmost bit.

- Add the numbers together.
- If there is a carry out of the magnitude bits (that is, a carry out of the column just to the left of the most significant magnitude bit), add it back into the result. This is called an **end-around carry**.

The above introduces quite a bit of complexity. Getting the complement takes an extra step as well as adding back in the carry at the end.

So two other encoding schemes were invented: **ones' complement** and **two's complement**. In ones' complement, negative numbers were stored as their complement in memory. This gets rid of one extra step in that there is no need to calculate the complement. However, the leftmost carry needed to be added back into the number.

One's Complement	Stores the negative bit pattern as the complement of the positive bit pattern.								
<table border="1"><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	1	1	0	0	1	1	0	= + 102
0	1	1	0	0	1	1	0		
<table border="1"><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td></tr></table>	1	0	0	1	1	0	0	1	= - 102
1	0	0	1	1	0	0	1		

one's complement method

It was an improvement, however, this still had the problem of having two zeros to contend with and one extra step of adding the 1 back in at the end.

Two's complement encoding scheme was invented to compensate for the ones' complement difficulties. It is the same as ones' complement except that negative numbers are stored as their complement plus 1 (again, positives are kept the same). This encoding got rid of the extra zero and extra step at the end. In fact, you can just add numbers, both positive and negative, and it will work just fine as long as there is no overflowing of the range. Note that the left-most carry bit is ignored (it's actually added into the negative number from the start).

A significant motivation for the adoption of two's complement representation for negative numbers was that the same addition hardware that correctly added unsigned integers would work correctly for positive and negative signed integers as well. Most of today's computer systems use two's complement for signed integers.

Two's Complement

Stores the negative numbers as the **complement plus 1** of the positive bit pattern.

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

 = + 102

1	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

 = complement of 102
+ 1

1	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

 = - 102

two's complement method

Note that in order to distinguish positive from negative, the upper range positive numbers that start with a 1 are now treated as negative numbers. This is why signed positive numbers have roughly half the range of unsigned positive numbers. For every bit missing from the left, there is a 1/2 reduction in range of numbers that can be represented by the remaining bits. Also note that the bit pattern that was negative zero now represents the most negative value in the range (**TMIN**). This gives the negative range one extra value over the positive range.

The table below uses 4-bit numbers to show the bit pattern for each given number. Note that the negative magnitudes get smaller as the underlying bit pattern gets larger. With this scheme, the largest bit pattern: 1111 is actually -1. This is always true for other-sized two's complement numbers.

Value	Bit Pattern
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

Next, we will look at how to convert between the bit pattern stored in memory and the actual decimal value. Although the examples are using a 4-bit size, the methods will work with any **w**-bit size type.

Finding the Value from Two's Complement

To find out the actual value of a two's complement bit pattern:

1. Find unsigned representation for all bits but the leftmost bit
2. Minus the leftmost bit (this is the sign bit) times 2^{w-1} from the sum.

For example (we are working with $w=4$):

1011

1. **011** $\rightarrow 2+1=3$
2. **sign bit is 1 so $1 \cdot 2^{(4-1)} = 1 \cdot 8 = 8$**
 $-8 + 3 = -5$.

Another example (we are working with $w=4$):

1101

1. **101** $\rightarrow 4+1=5$
2. **sign bit is 1 so $1 \cdot 2^{(4-1)} = 1 \cdot 8 = 8$**
 $-8 + 5 = -3$.

Find Two's Complement from Value

To find two's complement (in size w) from a given number:

1. Find the bit pattern of the magnitude in w bits. If the number is positive, then this is the two's complement number.
2. For a negative, take the complement of the bit pattern (flip 0's to 1's and 1's to 0's).
3. Do binary addition by adding 1. Ignore any carry from the leftmost bit.

For example:

to find -6 start with bit pattern: 0110
take the complement: 1001
add one to this: $1001 + 0001 = 1010$
so the bit pattern for -6 is 1010

The formulas for maximum and minimum signed integers of size w are:

$$TMAX = 2^{w-1} - 1$$

$$TMIN = -2^{w-1}$$

Note that there is one less positive number than negative number. This is not too much of a problem unless you are negating the TMIN value (there's no corresponding negation value).

What this means is that negating a number might actually cause an overflow to occur.

Lesson 10: Type Casting

NOTE: This is a summary of chapter 2, sections 2.2 and 2.3 in the textbook.

Converting Between Unsigned and Signed

When **typecasting** (changing from one type to another type) between corresponding signed and unsigned types, C defines the result by value: conversion to unsigned uses arithmetic modulo 2^w . On the two's-complement systems used in this course, a same-width conversion therefore preserves the bit pattern and changes how those bits are interpreted. (Conversions between different sizes are covered later in this lesson.) This can lead to several problems.

When going from same-sized (number of bits) signed to unsigned, if the signed value is guaranteed to be positive there is no problem as the bit pattern is the same for positive numbers between signed and unsigned. However, if the signed number turns out to be negative the result will be a positive number not even in the same ballpark. For small negative numbers, the result ends up being a very large positive number.

Typecasting Example: From Same-sized Signed to Unsigned

```
char a;  
...  
unsigned char b = (unsigned char) a;
```



For Positive: No Problems

if a is...

0	1	1	1	0	0	1	1
---	---	---	---	---	---	---	---

 = + 115

then b is...

0	1	1	1	0	0	1	1
---	---	---	---	---	---	---	---

 = + 115



For Negative: Bits are reinterpreted as a positive number quite different in size to the original.

if a is...

1	1	1	1	1	0	0	1
---	---	---	---	---	---	---	---

 = - 7

then b is...

1	1	1	1	1	0	0	1
---	---	---	---	---	---	---	---

 = + 249

typecasting from signed to unsigned

Even with positive values, converting from unsigned to signed can be surprising. Signed and unsigned types of the same width have the same number of bit patterns, but the signed type uses part of its range for negative values. If the unsigned value is larger than `TMAX`, the C standard says the result is implementation-defined (or an implementation-defined signal may be raised). GCC and Clang on the two's-complement systems used in this course preserve the low-order bits, which commonly makes a value with its top bit set appear negative, but portable code should not depend on that result.

Typecasting Example: From Same-sized Unsigned to Signed

```
unsigned char a;  
...  
char b = (char) a;
```



Within Positive Range: No Problems

if a is...

0	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

 = + 58

then b is...

0	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

 = + 58



Larger than Positive Range: For char, the maximum positive number is 127. The original positive number gets interpreted as negative because the leftmost bit is 1.

if a is...

1	0	1	0	1	0	0	1
---	---	---	---	---	---	---	---

 = + 169

then b is...

1	0	1	0	1	0	0	1
---	---	---	---	---	---	---	---

 = - 87

typecasting from unsigned to signed

Typecasting From Smaller to Larger: Expanding Numbers

Changing numbers from smaller to larger sizes requires expanding the number by adding bits to the left of the smaller number to create a larger number. For unsigned numbers, this is done by adding zeros. This is called **zero extension**.

For signed numbers, the left bits are filled in with whatever the left-most bit of the original smaller number was. This is called **sign extension** and is similar to the right shift's arithmetic shift. This is done to preserve the sign and magnitude of negative values.

Typecasting: From Small to Large Size



Unsigned: Zero Extension

0 0 1 1 1 0 1 0 = + 58

0 0 0 0 0 0 1 1 1 0 1 0 = + 58



Signed: Sign Extension

positive

0 0 1 0 1 0 0 1 = + 169

0 0 0 0 0 0 1 0 1 0 0 1 = + 169

negative

1 0 1 0 1 0 0 1 = - 87

1 1 1 1 1 0 1 0 1 0 0 1 = - 87

sign/zero extension: casting a smaller type to a larger type

Typecasting from Larger to Smaller: Truncation

If the original value fits in the destination type, conversion preserves the value. If it does not fit and the destination is **unsigned**, C defines the result modulo 2^w ; on ordinary hardware this keeps the low-order bits and discards the rest. If the destination is **signed** and the value does not fit, however, the result is implementation-defined (or an implementation-defined signal may be raised). Common GCC and Clang targets also keep the low-order bits here, but portable programs should validate the range before a narrowing signed conversion.

Typecasting: From Large to Small Size



Within Range of Smaller Size: no problems

1	1	1	1	1	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---

 = -70

1	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

 = -70



Not Within Range of Smaller Size:
Overflow Problem

1	1	1	1	0	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---

 = -198

0	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

 = +58

truncation: casting a larger type to a smaller type

Implicit Typecasting with Mixed Signed and Unsigned Values

When doing arithmetic or doing comparisons, the compiler does **implicit typecasting** when there is a mixture of signed and unsigned values. The C rules (called the *usual arithmetic conversions*) depend on the relative *rank* of the two types:

- First, the *integer promotions* are applied: operands of types smaller than `int` (such as `char` and `short`) are promoted to `int` before any arithmetic is done.
- If the operands have the same rank, the signed operand is converted to unsigned. For example, `int` mixed with `unsigned int` → both become `unsigned int`.
- If the signed type has a higher rank *and* can represent every value of the unsigned type, the unsigned operand is converted to the signed type. For example, on a typical Linux system `unsigned int` mixed with `long` → both become `long` (because `long` is 64 bits and can hold every `unsigned int` value).

- Otherwise, both operands are converted to the unsigned version of the higher-ranked type.

The rule that matters in practice: `int` compared against `unsigned int` is the dangerous case — the negative `int` gets reinterpreted as a huge positive value. It is recommended not to rely on the compiler to silently choose, especially in comparisons (see textbook for some interesting effects).

Lesson 11: Integer Arithmetic

NOTE: This is a summary of chapter 2, sections 2.2 and 2.3 in the textbook.

Section 2.3: Integer Arithmetic

I have to admit that I was expecting this section to discuss how arithmetic is performed in hardware, but it seems like the authors decided this wasn't as important. The section seems to be more about one consequence of doing arithmetic: **overflow**. Overflow occurs when the full integer result cannot fit within the size limits of the data type.

I guess the reason might be that the underlying mechanism of doing arithmetic tends not to have an effect on software developers until overflow occurs. So knowing about overflow, when it happens and how to detect it, is more important from a programmer's perspective than how arithmetic is performed in hardware.

Overflow In Unsigned Addition

In order to hold the sum of two unsigned numbers of w bits, the resulting size needs to be 1 bit more or $w+1$ bits. For example, if you are using two 8-bit numbers, the sum might need 9 bits to prevent overflow. This represents a range that is twice the range of the two numbers.

For example, the maximum for 8-bit unsigned is 255. Adding two 255 numbers together gives a sum of 510 which will fit in a 9 bit range where the maximum number is 511. Basically, the maximum overflow from addition only requires one more bit to hold the results without overflow.

When overflow occurs, the leftmost bits are ignored. This will cause the sum value to become smaller than either of the two operands (this is proven mathematically in the textbook, so I won't prove it here). This gives us a good way to detect if an overflow has occurred by checking the sum against either of the two operands to see if the sum is smaller. A sum of two positive numbers (remember this is unsigned) is always going to be larger unless overflow occurs.

Overflow in Signed Addition

Addition for two signed integers also needs at least 1 extra bit to fit any sum. Again, this is based on the largest value possible being added to itself (see unsigned). However, overflow can only occur when the two operands have the same sign (either positive or negative). If x and y are both negative, this is called **negative overflow**. If x and y are both positive, this is **positive overflow**.

At the hardware level, if we keep only the low w bits, negative overflow produces a non-negative wrapped bit pattern (the edge case `TMIN + TMIN` wraps to 0), while positive overflow produces a negative one. This explains the processor's overflow flag. It does *not* make a wrap-after-the-fact test valid in portable C, because evaluating an overflowing signed addition is already undefined behavior.

Important caveat about signed integer overflow in C: Although the underlying x86 hardware wraps around predictably as described above, the C standard says that signed integer overflow is *undefined behavior*. Modern compilers (GCC, Clang) at `-O2` and higher routinely exploit this fact, sometimes deleting overflow checks that depend on the wrapped result. The wrap-based detection idioms shown in this lesson work as written for *unsigned* arithmetic, but for signed types you should either use unsigned arithmetic, use compiler builtins like `__builtin_add_overflow`, or check for overflow *before* performing the operation.

WARNING: The above methods are guaranteed to detect overflow. However, using this method:

```
if(x == (sum-y))
    // this is okay
else
    // this is overflow
```

turns out not to work for overflow detection because `x == (sum-y)` is always true in modular arithmetic (adding y and then subtracting y wraps back to the original x), so this test never detects overflow.

Two's Complement Negation

The negation of a signed number is changing the sign of the number while keeping its magnitude. There is no equivalent to negation in unsigned numbers.

For the range of signed numbers, remember that there is one extra negative value than positive value. This has an effect on `TMIN` because there is no positive value with the same magnitude: `TMAX` is one smaller. There is therefore no representable result for `-TMIN`. On ordinary two's-complement hardware the same bit pattern may appear to come back, but in C the expression overflows and has **undefined behavior**. Do not use the following program as an overflow test:

```
int min = INT_MIN;
int nmin = -min;
printf("%d %d\n", min, nmin);
```

The safe takeaway is to check for `TMIN` *before* negating a signed integer.

Overflow in Multiplication

For multiplication, both signed and unsigned need 2^w bits. For example, if we are multiplying two 8-bit numbers, the result may need 16 bits to fit in memory without overflow. This is much more severe than with addition, making multiplication more prone to overflow problems.

For **unsigned** multiplication, the result is reduced modulo 2^w , which is equivalent to keeping the low-order bits. Processor hardware also produces those low bits for a signed multiply, but an out-of-range signed result is undefined behavior in C. Under an explicitly chosen wraparound model, the following textbook identity can detect multiplication overflow:

```
mult = x*y;
if ( x == 0 || mult/x == y)
    // this is okay
else
    // this is overflow
```

works under two's-complement wrap semantics (see problem 2.35 in the textbook, which proves it assuming the multiplication wraps). Two caveats apply in real C code: for signed types, the overflow in `x*y` is undefined behavior (see the warning box above), so the compiler is not obliged to produce the wrapped result the proof relies on; and the case `x = -1, y = INT_MIN` makes `mult/x` itself overflow (`INT_MIN / -1`), which raises SIGFPE and crashes the program on x86-64. The robust practice is to use the compiler builtin `__builtin_mul_overflow`.

Multiply and Divide by Powers of 2

Right and left bit shifting can sometimes express multiplication or division by a power of 2. Optimizing compilers already choose an appropriate shift or multiply instruction, so write the clearest correct expression and let the compiler optimize it. The equivalences below are simplest for unsigned values; signed values require the caveats that follow.

The left shift operator does multiplication:

```
num << i => num*2i
```

The right shift operator does division:

```
num >> i => num/(2i)
```

Right shift on signed numbers is implementation-defined in C: C23 mandates two's-complement representation, but right shift of a negative signed value remains implementation-defined even in C23 (it was C++20 that defined it as an arithmetic shift). In practice, every mainstream compiler (GCC, Clang, MSVC) performs an arithmetic shift (sign-extending). Note that for negative numbers, arithmetic right shift rounds toward negative infinity (floor division), whereas C's `/` operator truncates toward zero. For example, `-7 >> 1` gives `-4`, but `-7 / 2` gives `-3`.

For unsigned values, left shift is defined modulo 2^w . For signed non-negative values, left shift is defined only when the mathematical result is representable; otherwise the behavior is undefined. The right-shift equivalence for negative signed values depends on the implementation and rounds differently from C division on the systems described above.

Module 1 Summary

Wrap up

In this module, we covered the C programming language and integer data representation. Compared to other high-level languages, C maps almost directly into hardware instructions, making code potentially more efficient. C also provides a minimal set of abstractions, which is why it remains the language of choice for systems programming. We then studied how integers are stored in memory, including signed and unsigned representations, type casting between them, and the arithmetic operations that can be performed on integer data.

Key Takeaways

- C is prevalent in critical applications such as operating systems (Windows, Linux, the BSDs, and macOS), web servers, web browsers, and mail servers. Understanding C is essential for understanding how software interacts with hardware.
- Data inside a computer is stored as sequences of bits. Hexadecimal provides a convenient shorthand for binary, and byte ordering (big-endian vs. little-endian) determines how multi-byte values are laid out in memory.
- Bitwise operators allow you to manipulate individual bits within a value, which is fundamental to tasks like masking, setting flags, and low-level hardware control.
- Integer representation comes in two forms: signed (using Two's Complement) and unsigned. The finite word sizes used to represent integer data determine what values can be represented and the behavior of different operations.
- Overflow occurs when the full integer result cannot fit within the size limits of the data type. Recognizing and handling overflow is a common challenge in systems programming.

Additional resources

All links open in a new tab.

[Intro to C Slides set](#) [link]

[Bits and Bytes Slides set](#) [link]

[Bits Bytes and Integers Part 1](#) [link to video lecture from Carnegie Mellon University]

[Bits Bytes and Integers Part 2](#) [link to video lecture from Carnegie Mellon University]

Module 2 Overview

In Module 2: Floating Point, we will explore how computers represent numbers that have fractional parts. Unlike integers, which can be represented exactly in binary, floating point numbers require an approximation scheme that trades off range, precision, and efficiency. We will study the dominant standard (IEEE 754) and also examine an alternative representation called posits.

This module contains 6 lessons:

1. **Lesson 1: Fractional Binary Numbers** — How to represent fractions in binary using a radix point, and converting between binary and decimal fractions.
2. **Lesson 2: IEEE 754 Floating Point Encoding Scheme** — The structure of IEEE 754: sign bit, exponent field, and fraction (mantissa) field for single and double precision.
3. **Lesson 3: Floating Point Examples** — Worked examples encoding and decoding IEEE 754 floating point values, including normalized, denormalized, and special values.
4. **Lesson 4: Floating Point Rounding** — The IEEE 754 rounding modes (round-to-nearest-even, round toward zero, round down, round up; IEEE 754-2008 also added round-to-nearest-ties-away-from-zero) and why round-to-even is the default.
5. **Lesson 5: Floating Point Operations** — How addition, multiplication, and other operations work on floating point numbers, and the precision and overflow problems that can arise.
6. **Lesson 6: Posit Number System** — An alternative to IEEE 754 developed by John Gustafson, featuring variable-length regime bits and tapered precision (this lesson goes beyond the textbook as an enrichment topic).

Learning Objectives

At the end of this learning module, you will be able to:

1. Represent fractional numbers in binary using a radix point
2. Encode and decode IEEE 754 floating point values in both single precision (32-bit) and double precision (64-bit) formats

3. Identify and explain the three categories of IEEE 754 values: normalized, denormalized, and special values (infinity, NaN)
4. Apply the IEEE 754 rounding modes and explain why round-to-even is preferred
5. Recognize precision loss, overflow, and other problems that arise in floating point arithmetic
6. Describe the posit number system and compare it with IEEE 754 representation

Learning Activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Read Lessons 1 through 6 and complete the embedded practice problems in the lessons that include them
2. Read the sections below
3. Participate in Discussion 2
4. Take Quiz 2
5. Complete Assignment 1 (due at the end of this module)
6. Review additional resources on the Module 2 Summary page

Reading

- Bryant & O'Hallaron: Chapter 2, section 2.4

If you have questions, you can post them in the Course Lobby or send your instructor an email.

Lesson 1: Fractional Binary Numbers

NOTE: This is a summary of Chapter 2, section 2.4.1 in the textbook.

Introduction

In this summary, we will look at fractional binary numbers: how values with fractional parts are written in binary and how binary scientific notation works. This is the foundation for the IEEE 754 encoding covered in the next lesson.

In C/C++, there are only two commonly used floating point types: float, which is 32 bit, and double, which is 64 bit. The **float** type is considered to be **single-precision**, and the **double** type is considered to be **double-precision**. Precision is how many digits there are in a floating point number; the more digits, the higher the precision.

Section 2.4.1: Fractional Binary Numbers

Fractional binary numbers have similar notation to decimal numbers. Basically, the fractional part is separated by a period, called the **binary point** (or, more generally, the **radix point**):

1011.101, 0.1101, 0.0001

the digits after the binary point are calculated by using $1/2^n$ in place of 2^n starting at $1/2$ for the first digit after the binary point. So, for the number **1011.101**:

$$1*2^3 + 0*2^2 + 1*2^1 + 1*2^0 + 1*1/2^1 + 0*1/2^2 + 1*1/2^3$$

$$8 + 0 + 2 + 1 = 11$$

$$1/2 + 0 + 1/8 = 5/8 = .625$$

so 1011.101 is 11.625 in decimal

Fractional binary numbers can be represented in scientific notation the same as decimal:

$$1011.101 \Rightarrow 1.011101 \times 2^3$$

Note that technically only the mantissa part (the digits to the left of the 2^3 part) is binary; the base and exponent are typically written in decimal to prevent confusion.

From Fractional Binary to Fully Decimal Number

The above isn't quite in decimal format. To go from fractional binary to full decimal, we need to convert the mantissa to decimal and then multiply by the 2 base exponent part. For example:

$$1.0011 \times 2^{13}$$

$$1.0011 \Rightarrow 1 \times 2^0 + 1 \times 2^{-3} + 1 \times 2^{-4} = 1.1875$$

$$1.1875 \times 2^{13} = 9728$$

$$9.728 \times 10^3$$

Lesson 2: IEEE 754 Floating Point Encoding Scheme

NOTE: This is a summary of chapter 2, sections 2.4.2 to 2.4.5 in the textbook.

IEEE 754 Floating Point Encoding Scheme

Floating Point numbers are stored in a series of bit fields where each field is a part of the bit pattern. The fields represent parts of the floating point number when written in scientific notation. An example of scientific notation would be:

$$-1.1101 \times 2^3$$

There are three parts of this number above that are of interest to floating point encoding:

- minus (-): The **sign** of the number.
- 1.1101: The **significand** (also known as the mantissa). For normalized values, $1 \leq M < 2$ (we will see a second case, $0 \leq M < 1$, for denormalized values later).
- 3: The exponent, which can be positive or negative (or zero).

These three parts are all that need to be stored in memory in order to reproduce the floating point number.

Each of the three parts above is stored in a separate field of the bit pattern. Starting from the left-most bit:

- 1 bit: This is the sign bit field.
- 8 (single) or 11 (double) bits: This is the exponent field.
- 23 (single) or 52 (double) bits: This is the significand field.

Single precision (32 bits):



Double precision (64 bits):



The 3 fields representing a floating point number

The sign bit field is pretty straightforward. If the bit is 1 the number is negative; if the bit is 0 the number is positive. We will discuss the other two parts in detail below.

There are three categories (or cases) that are treated differently. These are normalized cases, which are normal numbers that are not at the smallest exponent; denormalized cases, which represent zero and numbers too small to normalize (exponent field all zeros); and special cases, which are abnormal numbers, such as infinity or 'not a number.' Which case applies depends on the exponent's value. We'll describe how to determine each case in the following sections.

Normalized Cases

This section will discuss normalized cases. A normalized case is where the exponential field is neither all zeros nor all ones. These are the most common cases.

Exponent Field in Normalized Cases

The exponential field is in **biased** format. What this means is the actual number stored is calculated by taking the exponent of the number and adding a bias number. Keep in mind that the exponent of the number can be both positive and negative.

The bias number is chosen so that adding it to any possible exponent within the range for that field always results in a positive number. The bias can also be calculated by this formula:

$$2^{k-1} - 1$$

where k is the number of bits in the exponent field.

If we take the positive range to be roughly equal to the negative range, the single-precision will have a range of -126 to $+127$ (exponents smaller than -126 are denormalized cases; we'll talk about this later). A bias of 127 will keep exponents within the positive range for the bit pattern, so there is no need for a signed bit for the exponent. The exponent bit field ranges from 1 to 254 for single precision.

For double precision with a range of -1022 to $+1023$, the bias number is 1023 , and the exponent field can have a range of 1 to 2046 .

Note that neither an all one pattern (255) nor an all zero pattern (0) is included. These are the other cases discussed below.

As an example, if we have an exponent of -12 for a single-precision float, the bit pattern stored in the exponent field will be equivalent to the decimal number $127 + (-12) = 115$. This pattern is 01110011 .

Another example: if the exponent is 120 , then the bit pattern will be equivalent to the decimal value $127 + 120 = 247$, which is the pattern 11110111 .

Converting from bit field to exponent: $\text{exponent} = \text{bit field} - \text{bias}$

bit field 01110011 ($= 115$) $\rightarrow$ $\text{exponent} = 115 - 127 = -12$

Converting from exponent to bit field: $\text{bit field} = \text{exponent} + \text{bias}$

exponent 120 $\rightarrow$ $\text{bit field} = 120 + 127 = 247 = 11110111$

Converting between the exponent bit field and the exponent (single precision, bias = 127)

An aside: Unlike two's complement, a high-end bit pattern is positive and a low-end bit pattern is negative. With a bias of 127 , biased values 128 – 254 represent unbiased exponents 1 – 127 (so the leftmost bit is 1 for strictly positive exponents), and biased values 1 – 127 represent unbiased exponents -126 ... 0 (so the leftmost bit is 0 for non-positive exponents). For example: a $+1$ exponent is 10000000 , a zero exponent is 01111111 , and a -1 exponent is 01111110 . So roughly speaking, the leftmost bit is 1 for positive exponents and 0 for negative ones — with the caveat that unbiased exponent 0 is neither, and its leftmost bit happens to be 0 .

Significand in Normalized Cases

The significand field is not the straight bit pattern produced by the significand in binary. To allow for 1 bit of extra precision, the 1 in front of the decimal is ignored because, for a normalized case, it is always there. So the significand 1.0111001 for single-precision will be stored as:

```
011100100000000000000000
```

Note that the bit pattern starts from the left-most bit and extends to the right (0's are added to the end).

So when going from bit pattern to significand, you will need to remember to put the one back in front of the number. So if we have 10001110..0, then the significand is 1.1000111.

Since the significand always has a 1 in front, the above scheme does not work with very tiny numbers or zero. The next section talks about another scheme for representing these types of numbers.

Denormalized Cases

This section will discuss denormalized cases. A denormalized case is where the **exponential field is all zeros**. These are numbers that are zero or very close to zero.

Zero

One denormalized case is zero. This can be two bit patterns: all 0's (positive zero) or 1 followed by 0's (negative zero). Yes, there are two zeros in this encoding scheme. If you remember, two's complement was created to get rid of the two zeros, which hardware engineers found to be a problem. However, IEEE 754 was mainly designed with numerical analysis in mind, where having two zeros is actually an asset. By allowing for positive and negative zeros, one can tell in what direction the calculations reached zero. So, what was a liability for hardware engineers is an asset for numerical analysis programmers.

If you want to know more about the decisions that led to IEEE 754, read [An Interview with the Old Man of Floating-Point](#), a condensed version of an interview with one of the original designers of the IEEE 754 standard, William Kahan.

Non-zero

For denormalized cases where the exponent is zero but the significand is non-zero, these represent numbers smaller than 1.0×2^{-126} for single-precision or 1.0×2^{-1022} for double-precision. Technically, the exponent itself is left at -126 (or -1022). In this case, the significand is interpreted as the actual significand without an implied 1. This allows for a gradual decrease to zero, known as "gradual underflow," rather than an abrupt cutoff at 1.0×2^{-126} . For example, if the significand is:

[illegible]

This is interpreted as $.000000000000000000001 \times 2^{-126}$ (note the lack of a leading 1). This equals 2^{-149} , which is about 1.4×10^{-45} in decimal — this is the smallest positive value representable in single-precision (about 5×10^{-324} for double precision, which is 2^{-1074}). Note that at these sizes, the precision is greatly reduced, and these values would be subject to serious round-off errors.

Special Cases

This section will discuss special cases. A **special case** is where the **exponential field is all ones**. There are two special cases: one case when the significand is all zeros and one where it is non-zero.

Infinity

When the significand is all zeros, the value is taken to be **infinity**. This is a valid number in floating point and is printed as **inf** in console output. It can result from a divide by zero or from overflow. This is why you don't get a divide-by-zero error with floating point numbers like you do with integers.

The sign bit can be both 1 or 0. If the sign bit is zero, we have positive infinity. If the sign bit is one, we have negative infinity.

Not a Number (NaN)

When the significand is non-zero (has at least one 1 bit in it somewhere), this is a **NaN** (Not a Number) result. This represents a value that is the result of an impossible calculation, such as taking the square root of a negative number. This is usually printed as **NaN** or **nan** on most systems.

There are two types of NaN's: **quiet NaN** and **signaling NaN**. The quiet NaN does not throw an exception when generated; a signaling NaN might, depending on the system state. A

signaling NaN has a zero as the left-most bit but has at least one 1 in the significand (otherwise, it would be infinity). A quiet NaN has a 1 as the left-most bit. Keep in mind the significand is not a value here, so the hardware will fill in the remaining bits with whatever information it would like to keep track of. This quiet/signaling convention (left-most fraction bit set means quiet) has been standardized since IEEE 754-2008; older architectures varied in how they distinguished the two.

Lesson 3: Floating Point Examples

Floating Point Examples

Here are worked floating point examples in single-precision and double-precision. These show going from the memory's bit pattern to a fractional binary number (or a special case) and vice versa. This lesson is intended as a reference you can come back to when working through problems.

Methodology: Bit Pattern to Value (Decoding)

When given a 32-bit pattern representing an IEEE 754 single-precision float, follow these steps:

- **Step 1:** Identify the **sign bit** (bit 31, the leftmost bit). If it is 1, the number is negative; if 0, positive.
- **Step 2:** Extract the 8-bit **exponent field** (bits 30–23). Convert it to a decimal value. If the field is neither all zeros nor all ones, compute $E = \text{exponent_field} - 127$ (the bias). This is the **normalized** case.
- **Step 3:** Extract the 23-bit **fraction field** (bits 22–0). For a normalized number, the significand is $M = 1.\text{fraction}$ (there is an implicit leading 1). For a denormalized number (exponent field all zeros), the significand is $M = 0.\text{fraction}$ and the exponent is $E = -126$.
- **Step 4:** Compute the value: $\text{value} = (-1)^S \times M \times 2^E$.

For **special cases**: if the exponent field is all ones (255), the value is either $\pm\text{infinity}$ (fraction all zeros) or NaN (fraction nonzero).

Normalized Examples: Bit Pattern to Value

Example 1: A negative normalized number

(Spaces between the fields are for clarity.)

1 01101111 0110110000000000000000

Step 1: Sign bit = 1, so this is a **negative** number.

Step 2: The exponent field is 01101111. Converting: $0 \times 128 + 1 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1 = 64 + 32 + 8 + 4 + 2 + 1 = 111$. (You can also use a calculator.) $E = 111 - 127 = -16$.

Step 3: The fraction bits are 0110110000000000000000, so the significand is $M = 1.011011$.

Step 4: The value is:

$$-1.011011 \times 2^{-16}$$

Keep in mind that 2 and -16 are decimal; the significand part is in binary.

Example 2: A small positive normalized number

0 00000001 0000000000000000000000

Step 1: Sign bit = 0, so this is **positive**.

Step 2: The exponent field is 00000001 = 1. $E = 1 - 127 = -126$. This is the smallest normalized exponent.

Step 3: The fraction bits are all zeros, so $M = 1.0$.

Step 4: The value is:

$$+1.0 \times 2^{-126} \approx 1.175 \times 10^{-38}$$

This is the **smallest positive normalized** single-precision value.

Example 3: A negative number with many fraction bits

1 10000011 1011011010000000000000

Step 1: Sign bit = 1 — negative.

Step 2: Exponent field = 10000011 = 131. $E = 131 - 127 = 4$.

Step 3: Fraction bits = 1011011010000000000000. The significand is $M = 1.101101101$.

Step 4: The fractional binary value is:

$$-1.101101101 \times 2^4$$

Shifting the binary point 4 places right: -11011.01101 . Converting to decimal: $-(16 + 8 + 2 + 1 + 0.25 + 0.125 + 0.03125) = -27.40625$.

Example 4: A moderately large positive number

0 10010100 0100000000000000000000

Step 1: Sign bit = 0 — positive.

Step 2: Exponent field = 10010100 = 148. $E = 148 - 127 = 21$.

Step 3: Fraction bits = 0100000000000000000000. $M = 1.01$.

Step 4: The value is 1.01×2^{21} . In decimal: $1.25 \times 2,097,152 = 2,621,440.0$.

Denormalized Examples

Example 5: A denormalized number

0 00000000 0101110000000000000000

Step 1: Sign bit = 0 — positive.

Step 2: The exponent field is all zeros. This is a **denormalized** case. The exponent is fixed at $E = -126$ (not -127).

Step 3: No implicit leading 1. The significand is $M = 0.0101110000000000000000 = 0.010111$.

Step 4: The value is:

$$+0.010111 \times 2^{-126}$$

This can also be written as 1.0111×2^{-128} .

Example 6: Smallest positive denormalized number

0 00000000 000000000000000000000001

Step 1: Positive. **Step 2:** Denormalized, $E = -126$. **Step 3:** $M = 0.000000000000000000000001 = 2^{-23}$.

Step 4: $\text{Value} = 2^{-23} \times 2^{-126} = 2^{-149} \approx 1.4 \times 10^{-45}$.

This is the **smallest positive value** representable in single precision.

Example 7: Largest denormalized number

0 00000000 111111111111111111111111

Step 1: Positive. **Step 2:** Denormalized, $E = -126$. **Step 3:** $M = 0.111111111111111111111111 = 1 - 2^{-23}$.

Step 4: $\text{Value} = (1 - 2^{-23}) \times 2^{-126} \approx 1.175 \times 10^{-38}$.

Notice this is just **barely below** the smallest normalized value (1.0×2^{-126}), demonstrating the smooth transition between denormalized and normalized numbers. There is no gap.

Special Cases Example

Example 8: A NaN value

0 11111111 100111000000000000000000

Step 1: Sign bit = 0. **Step 2:** Exponent field = 11111111 (all ones) — this is a **special case**.

Since the fraction field is nonzero, this is a **NaN** (Not a Number). Furthermore, the leftmost fraction bit is 1, making it a **quiet NaN** (a signaling NaN would have a 0 as the leftmost fraction bit with at least one other bit set).

Methodology: Value to Bit Pattern (Encoding)

To encode a decimal value as an IEEE 754 single-precision float:

- **Step 1:** Determine the **sign bit**. If the number is negative, the sign bit is 1; otherwise 0. Work with the magnitude from here on.

- **Step 2:** Convert the magnitude to **binary scientific notation**: $1.xxxx \times 2^E$. This means expressing it as a binary number between 1 (inclusive) and 2 (exclusive) times a power of 2.
- **Step 3:** Compute the **exponent field** = $E + 127$ (the bias). Verify it is between 1 and 254 (otherwise the number requires denormalization or is out of range).
- **Step 4:** The **fraction field** is the bits after the leading 1 in the significand, padded with trailing zeros to fill 23 bits.

Encoding Examples: Value to Bit Pattern

Example 9: Encoding $+1.0001 \times 2^{30}$

$$+1.0001 \times 2^{30}$$

Step 1: Positive, so sign bit = 0.

Step 2: Already in binary scientific notation: 1.0001×2^{30} .

Step 3: Exponent field = $30 + 127 = 157$. In binary: 10011101.

Step 4: Drop the leading 1. Fraction field = 00010000000000000000000 (23 bits).

Assembling the fields:

0 10011101 00010000000000000000000

Example 10: Encoding -6.5

Step 1: Negative, so sign bit = 1. Work with magnitude 6.5.

Step 2: Convert 6.5 to binary: $6 = 110$, $0.5 = .1$, so $6.5 = 110.1$. Normalize: 1.101×2^2 .

Step 3: Exponent field = $2 + 127 = 129 = 10000001$ in binary.

Step 4: Fraction field = 10100000000000000000000.

Assembling:

1 10000001 10100000000000000000000

Example 11: Encoding 0.1 (an inexact representation)

Step 1: Positive, sign bit = 0. Work with 0.1.

Step 2: Convert 0.1 to binary by repeatedly multiplying by 2:

```
0.1 × 2 = 0.2 → 0
0.2 × 2 = 0.4 → 0
0.4 × 2 = 0.8 → 0
0.8 × 2 = 1.6 → 1
0.6 × 2 = 1.2 → 1
0.2 × 2 = 0.4 → 0 (pattern repeats: 0011 repeating)
```

So 0.1 in binary is $0.0\overline{0011}$ (the 0011 pattern repeats infinitely). This means **0.1 cannot be represented exactly** in binary floating point. Normalizing: $1.1001100110011... \times 2^{-4}$.

Step 3: Exponent field = $-4 + 127 = 123 = 01111011$.

Step 4: Fraction field = 10011001100110011001101 (23 bits, rounded at the end).

```
0 01111011 10011001100110011001101
```

The stored value is approximately 0.100000001490116..., not exactly 0.1. This is a fundamental limitation of binary floating point and the root cause of many surprising rounding behaviors in programs.

Example 12: Encoding a very small denormalized number

Suppose we want to encode the value 2^{-140} .

Step 1: Positive, sign bit = 0.

Step 2: The value is 1.0×2^{-140} . But $E = -140$ is below the minimum normalized exponent of -126 , so this requires **denormalization**.

Step 3: For a denormalized number, the exponent field is all zeros, and E is fixed at -126 . We must express the value as $M \times 2^{-126}$ where $M = 0.\text{fraction}$. Since $2^{-140} = 2^{-14} \times 2^{-126}$, the significand is 0.00000000000001 (a 1 in the 14th bit position).

Step 4: Fraction field = 00000000000001000000000.

```
0 00000000 00000000000001000000000
```

Note the reduced precision: the fraction only has $23 - 14 = 9$ significant bits remaining for any additional detail.

Double Precision Examples

Double precision works the same way but with an 11-bit exponent (bias = 1023) and 52-bit fraction. There are 64 bits total: 1 sign + 11 exponent + 52 fraction.

Example 13: Decoding a double-precision bit pattern

```
0 10000000011
10110000000000000000000000000000000000000000000000000000
```

Step 1: Sign bit = 0 — positive.

Step 2: Exponent field = 10000000011 = 1027. $E = 1027 - 1023 = 4$.

Step 3: Fraction begins with 1011, the rest are zeros. $M = 1.1011$.

Step 4: Value = $1.1011 \times 2^4 = 11011.0$ in binary = $16 + 8 + 2 + 1 = 27.0$.

Example 14: Encoding -0.75 in double precision

Step 1: Negative, sign bit = 1. Magnitude = 0.75.

Step 2: $0.75 = 0.11$ in binary $= 1.1 \times 2^{-1}$.

Step 3: Exponent field = $-1 + 1023 = 1022 = 0111111110$ (11 bits).

[illegible][illegible]

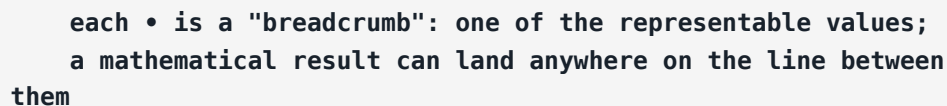
Special Values Reference Table

The following table summarizes the special bit patterns in single precision:

Value	Sign	Exponent	Fraction
-----	-----	-----	-----
+0	0	00000000	000000000000000000000000
-0	1	00000000	000000000000000000000000
+∞	0	11111111	000000000000000000000000
-∞	1	11111111	000000000000000000000000
NaN	x	11111111	(any nonzero fraction)

Here **x** means the sign bit can be either 0 or 1. For NaN, if the leading fraction bit is 1, it is a **quiet NaN**; if 0 (with at least one other fraction bit set), it is a **signaling NaN**. This convention has been standardized since IEEE 754-2008; older architectures varied in how they distinguished the two.

Consider a single precision 32-bit IEEE-754 floating-point number. With 32 bits available, we have 2^{32} possible bit configurations. Think of these 2^{32} bit configurations as "breadcrumbs" that can be placed on the real number line from $-\infty$ to $+\infty$. We can view the IEEE-754 floating-point standard as a system for placing those "breadcrumbs" on the real number line.



(Note that the breadcrumbs are not actually evenly spaced, as this diagram might suggest. The IEEE-754 standard places breadcrumbs that are closer to zero closer together and breadcrumbs that are further from zero further apart.)

In fact, IEEE 754 *requires* that the hardware compute the correctly rounded result. Internally, the floating-point unit uses extra precision bits (called **guard**, **round**, and **sticky** bits) to determine which breadcrumb is closest to the exact mathematical result, even though the exact result itself is not representable. The default rounding mode always picks the nearest breadcrumb. But there are situations where the mathematical result falls *exactly* halfway

between two breadcrumbs, and the hardware needs a tie-breaking rule. This is where different **rounding modes** come in.

IEEE 754-1985 specified four rounding modes; IEEE 754-2008 added a fifth (round-to-nearest, ties away from zero, used in some financial contexts). Most hardware and most code uses the four "classic" modes, which are what we cover here:

- **towards $+\infty$** : always pick the breadcrumb that is closest to $+\infty$
- **towards $-\infty$** : always pick the breadcrumb that is closest to $-\infty$
- **towards 0**: always pick the breadcrumb that is closest to 0
- **towards even (default)**: pick the nearest breadcrumb; when exactly at the midpoint, pick the breadcrumb whose rightmost bit is 0

The **towards even** rounding mode is the default. Some explanation of it is necessary here. As you go along the number line in either direction, breadcrumbs whose rightmost bit is 0 always alternate with breadcrumbs whose rightmost bit is 1.



...0 ...1 ...0 ...1 ...0

(the rightmost bit of each breadcrumb's bit pattern)

Adjacent breadcrumbs alternate between "even" (rightmost bit 0) and "odd" (rightmost bit 1)

Therefore, any mathematical result will *always* fall between a breadcrumb whose rightmost bit is 0 and one whose rightmost bit is 1. The idea behind this rounding mode is that sometimes a mathematical result will be rounded up towards $+\infty$ and sometimes down towards $-\infty$. The hope is that upwards and downwards roundoff errors will cancel each other out, producing a more accurate result. Clearly, this is only a hope, and that hope may not always come true.

Concrete Rounding Examples in Decimal

Before looking at binary, let us build intuition by rounding decimal numbers to one decimal place. Suppose the exact result of some computation is 2.35 and we must round to one decimal place. The two nearest representable values are 2.3 and 2.4, and 2.35 falls *exactly* at the midpoint.

- **Toward $+\infty$** : 2.4 (round up toward positive infinity)
- **Toward $-\infty$** : 2.3 (round down toward negative infinity)
- **Toward 0**: 2.3 (round toward zero, which is downward for a positive number)

- **Toward even:** 2.4 (the digit in the last kept place would be 4, which is even)

Now consider 2.45 rounded to one decimal place. The two nearest values are 2.4 and 2.5, and 2.45 is exactly at the midpoint.

- **Toward $+\infty$:** 2.5
- **Toward $-\infty$:** 2.4
- **Toward 0:** 2.4
- **Toward even:** 2.4 (the digit in the last kept place is 4, which is even — so we round down)

Notice that the three directed modes (toward $+\infty$, toward $-\infty$, and toward 0) always round in their fixed direction, no matter how close the value is to either side. Round-to-even, by contrast, always picks the *nearest* representable value; its "choose the even one" rule only comes into play when the value is *exactly* at the midpoint. In other words, round-to-even differs from the simple "round to nearest" you learned in school only at exact midpoints.

Binary Rounding Examples

Now let us look at rounding in binary, which is what the hardware actually does. Suppose our floating point format has **3 fraction bits** (so the significand looks like 1.xxx). We need to round values that have more than 3 fraction bits down to exactly 3.

Below the midpoint: round down

Consider the value 1.00101. This falls between the representable values 1.001 and 1.010. The dropped bits are **01**, which is below the midpoint (the midpoint would be 10). All rounding modes round toward the closer value:

1.00101 → 1.001 (rounded down; the dropped bits 01 are below the midpoint)

Above the midpoint: round up

Consider the value 1.00111. Again between 1.001 and 1.010. The dropped bits are **11**, which is above the midpoint. All modes round toward the closer value:

1.00111 → 1.010 (rounded up; the dropped bits 11 are above the midpoint)

Exactly at the midpoint: round-to-even decides

Consider the value 1.00110. The dropped bits are **10**, which is *exactly* the midpoint between 1.001 and 1.010. Now round-to-even looks at the last kept bit:

```
1.001|10 → last kept bit is 1 (odd), so round up to make it 0 (even)
Result: 1.010
```

Now consider 1.01010. This is exactly at the midpoint between 1.010 and 1.011. The dropped bits are **10**:

```
1.010|10 → last kept bit is 0 (even), so round down to keep it even
Result: 1.010
```

In both midpoint cases, round-to-even chose the result whose last fraction bit is 0. This is the key rule: **at an exact midpoint, choose the value whose least significant kept bit is 0.**

Guard, Round, and Sticky Bits

When the floating point hardware computes an arithmetic result (such as an addition or multiplication), the exact result typically has more bits than fit in the fraction field. The hardware cannot keep all of these extra bits, but it keeps three extra bits beyond the fraction field to determine the correct rounding direction:

- **Guard bit (G):** the first bit beyond the fraction field
- **Round bit (R):** the second bit beyond the fraction field
- **Sticky bit (S):** the logical OR of *all* remaining bits beyond the round bit

The sticky bit is crucial: it tells us whether there is *any* nonzero information beyond the first two extra bits, without needing to store all those bits individually.

These three bits determine the rounding direction under round-to-even:

```
GRS = 0xx → below midpoint → round down (truncate)
GRS = 100 → exactly at midpoint → round to even
GRS = 101, 110, or 111 → above midpoint → round up
```

Example: Adding with guard, round, and sticky bits

Suppose we have a 3-fraction-bit system and want to compute $1.001 + 0.0101$. First, align the exponents by shifting the smaller number right:

$$\begin{array}{r} 1.001 \text{ (this is } 1.001 \times 2^0) \\ + 0.0101 \text{ (this is } 1.01 \times 2^{-2}, \text{ shifted right by 2)} \\ \hline 1.0111 \end{array}$$

The exact result is 1.0111. With 3 fraction bits, we keep 1.011 and the extra bits are: G=1, R=0, S=0. This is GRS = 100 — exactly at the midpoint. The last kept bit is 1 (odd), so round-to-even rounds up:

$$1.011 + 0.001 = 1.100$$

The correctly rounded result is 1.100.

Why Round-to-Even Is the Default

Consider what happens if we always round *up* at the midpoint (as many people learn in grade school). Over many operations, every midpoint case adds a small positive error. These errors **accumulate**, introducing a systematic upward bias. After thousands of operations, the result can be significantly too high.

Similarly, always rounding toward zero introduces a systematic bias that makes positive results too small and negative results too large (in magnitude, too small).

Round-to-even is **statistically unbiased**: at midpoint cases, it rounds up half the time (when the last kept bit is odd) and rounds down half the time (when the last kept bit is already even). Over many operations, the positive and negative rounding errors tend to cancel out, producing more accurate accumulated results. This is why IEEE 754 chose it as the default.

The other three modes — toward $+\infty$, toward $-\infty$, and toward 0 — still exist because they are useful for **interval arithmetic**, where a programmer computes both an upper bound (rounding toward $+\infty$) and a lower bound (rounding toward $-\infty$) to determine the range of possible results. This technique is used in scientific computing to rigorously bound the error in a computation. In C, the rounding mode can be changed at runtime using the `fesetround()` function from `<fenv.h>`.

Practical Impact: Accumulated Rounding Error

Here is a simple C program that demonstrates how rounding errors accumulate:

```
float sum = 0.0f;
for (int i = 0; i < 10000000; i++) {
    sum += 0.1f;
}
printf("Expected: 1000000.0, Got: %f\n", sum);
```

With ordinary IEEE 754 binary32 evaluation and the default round-to-nearest-even mode, the output is **1087937.000000**, not 1000000.0. (The exact C result can differ on implementations that keep excess intermediate precision or under compiler options that permit reassociation.) Notice how enormous the accumulated error is: about 88,000, or roughly 9%. The reason is twofold: first, 0.1 cannot be represented exactly in binary (as we saw in Lesson 3), so every addition introduces a small error; second, once `sum` grows large, 0.1f is far below the spacing between adjacent breadcrumbs at that magnitude, so each addition is rounded very coarsely and the errors pile up into the huge discrepancy you see.

This is not a bug in the hardware — it is the inevitable consequence of finite precision arithmetic. Understanding rounding is essential for writing software that produces reliable numerical results.

Computers must necessarily represent numbers as bits. But *bits are not numbers*, and getting bits to act like numbers cannot be perfectly accomplished. Computer scientists have gotten *close*, but perfection is impossible. It's necessary for any competent programmer to understand the ways in which bits *do not* act like numbers and to make sure that this does not cause incorrect results in the software they write.

Lesson 5: Floating Point Operations

Floating Point Operations

Floating point operations suffer from two problems: loss of precision and overflow. Loss of precision is usually caused by rounding, and overflow is caused by the exponent being out of the range for the floating point type used.

Loss of precision when rounding on floating point numbers creates a peculiar behavior compared to integers. It almost seems like the results are breaking some mathematical laws. But keep in mind that mathematical laws assume infinite precision, whereas a computer has only finite precision.

How Floating Point Addition Works Internally

To understand why precision loss happens, it helps to know what the hardware actually does when it adds two floats. The process has four steps:

- **Step 1: Align the exponents.** The number with the smaller exponent has its significand shifted right until both numbers share the same exponent. Bits shifted off the right end are lost (though the guard, round, and sticky bits from Lesson 4 capture some of this).
- **Step 2: Add the significands.** With both numbers now aligned to the same exponent, the hardware adds (or subtracts) the significands as fixed-point numbers.
- **Step 3: Normalize the result.** If the result is not in the form $1.xxxx \times 2^E$, the hardware shifts the significand and adjusts the exponent.
- **Step 4: Round.** The result is rounded to fit the fraction field, using the rounding mode (typically round-to-even).

The alignment step (Step 1) is where precision gets lost: when you add a large and small number, the small number's significant bits get shifted off the right end. This is the root cause of non-associativity.

Non-Associativity of Addition

For exact real-number arithmetic, addition and multiplication are commutative and associative. IEEE 754 floating-point addition *is* commutative (`a + b` always equals `b + a`), but it is *not* associative because intermediate rounding changes results. For example, the two expressions below should give the same answer mathematically, but due to how rounding works, they don't (NOTE: `1e10f` is the C format for 1×10^{10} as a single-precision `float` ; in double precision, which has enough significand bits to hold $1e10 + 3.14$, the first expression would come out ≈ 3.14):

```
(3.14f + 1e10f) - 1e10f = 0.0
3.14f + (1e10f - 1e10f) = 3.14
```

For the first expression, 3.14 is much smaller than 1×10^{10} so the bits that represent 3.14 are effectively rounded out of existence during the alignment step. Then, the resulting 1×10^{10} is subtracted from itself to give 0. In the second expression, we are doing the subtraction first to get 0 and adding this to 3.14 to get 3.14.

More Precision Loss Examples

Small values vanishing entirely

In single precision, the significand has 23 fraction bits, giving about 7 decimal digits of precision. If two numbers differ by more than about a factor of 2^{24} ($\approx 1.7 \times 10^7$), the smaller number's bits are completely shifted away during alignment:

```
float a = 1.0f;
float b = 1.0e-8f;
printf("%d\n", (a + b) == a); // prints 1 (true!)
```

The value `1.0e-8` is so much smaller than 1.0 that adding it to 1.0 produces exactly 1.0 in single precision. The small value is below the **precision threshold** — it has no effect whatsoever.

Summation order matters

When summing a large array of small numbers, the order of addition matters. If you accumulate into a large running sum, each small number may lose precision when aligned to the sum's exponent. A better strategy is to sort the numbers by magnitude and sum from

smallest to largest, or to use pairwise summation or Kahan summation (a technique that tracks the lost low-order bits in a separate compensation variable).

```
// Naive summation: small values may be lost as sum grows large
float sum = 0.0f;
for (int i = 0; i < n; i++)
    sum += small_values[i];

// Better: sort ascending so small values are added to other
small values first
sort(small_values, n); // ascending order
float sum = 0.0f;
for (int i = 0; i < n; i++)
    sum += small_values[i];
```

Overflow

Multiplication can behave well except in cases where overflow occurs during the calculations. Overflow for floating point is when a result is beyond the range of an exponent for that type. For single precision, this is a number larger than about 2^{128} (or about 3.4×10^{38}). For double precision, this is about 2^{1024} (or about 1.8×10^{308}).

When an intermediate operation overflows, that operation produces infinity (and raises the overflow flag), and subsequent operations continue using that infinity. To see this in single precision, where the largest representable value is about 3.4×10^{38} , the expression `(1e20f*1e20f)*1e-20f` ends up `+inf` (because `1e20f * 1e20f = 1e40`, which already overflows `float`), while the expression `1e20f*(1e20f*1e-20f)` ends up being `1e20f`. (Note: in `double` precision, both expressions evaluate to about `1e20` because `double` has plenty of headroom above `1e40` — the suffix `f` is what forces the overflow in this example.)

Underflow

Underflow is the opposite problem: a result is too small to represent as a normalized number. When this happens, the result enters the denormalized range, where it gradually loses precision (fewer and fewer significant bits). Eventually, if the result becomes smaller than the smallest denormalized value, it rounds to zero.

```

float x = 1.5e-38f;
printf("x          = %e\n", x);           // normalized (smallest
normal is about 1.18e-38)
printf("x/2       = %e\n", x / 2);       // denormalized, losing
precision
printf("x/256     = %e\n", x / 256);     // denormalized, fewer
significant bits
printf("x/2^50    = %e\n", x * powf(2, -50)); // underflows to
0.0

```

The gradual transition through denormalized numbers (rather than an abrupt jump to zero) is one of the important design features of IEEE 754. It ensures that `a - b == 0` if and only if `a == b`, even for very small values.

Multiplication and Precision

Multiplication can also lose precision, though it is usually less dramatic than addition of differently-scaled numbers. When you multiply two significands that each have 23 fraction bits, the exact product has up to 46 fraction bits. This must be rounded back to 23 bits. However, since both operands contribute proportionally to the result, you do not lose entire operands the way you can with addition — the error from rounding a multiplication is at most half a unit in the last place (0.5 ulp).

Floating Point Comparison Pitfalls

Due to rounding, two computations that *should* give the same result mathematically often produce slightly different floating point values. This makes direct equality comparison unreliable:

```

// BAD: may fail even when a and b are "equal" up to rounding
if (a == b) { ... }

// BETTER: use an epsilon-based comparison
if (fabs(a - b) < 1e-6) { ... }

```

The appropriate value of epsilon depends on the magnitude of the values being compared and the precision requirements of the application. For single-precision floats, an epsilon of around $1e-6$ to $1e-7$ is common for values near 1.0. For values that may be much larger or smaller, a **relative** epsilon is more appropriate:


```
// Relative comparison
if (fabs(a - b) < epsilon * fabs(a)) { ... }
```

Note that this form silently fails when `a` is zero or very close to it (the right-hand side collapses to nearly zero); a more robust version scales by `epsilon * fmax(fabs(a), fabs(b))` and also accepts an absolute floor (a small fixed tolerance) for comparisons near zero.

Never use `==` to compare floating point results unless you have a specific reason to believe the values are *exactly* equal (for example, when comparing against 0.0 for a value you explicitly set to 0.0).

IEEE 754 Correctness Guarantee

Despite all these precision issues, IEEE 754 provides a very strong guarantee: every basic arithmetic operation (+, −, ×, ÷, and square root) must produce the **correctly rounded result**. This means the hardware must compute the result *as if* the operation were performed with infinite precision and then rounded to the target format using the current rounding mode.

For a specified IEEE 754 operation, format, rounding mode, and operands, correct rounding determines the numerical result. Whole C programs can still differ when compilers contract operations (for example into fused multiply-add), reassociate expressions under permissive options, retain excess precision, choose different library algorithms, or propagate NaN payloads differently. Rounding error is structured rather than random, which makes numerical error analysis possible.

Rounding and overflow are problems that need to be addressed when doing numerical analysis in programs. There is actually a whole branch of computational science on mitigating these problems in code that the textbook doesn't go into.

Floating Point Typecasting

Typecasting between floating point types and between floating point and integers has problems stemming from both overflow and rounding. This is discussed below:

- `int` to `float`: There's no overflow possible, but since the significand has fewer bits than the integer's magnitude may need (24 significand bits — 23 stored plus 1 implicit — vs up to 31 magnitude bits in a 32-bit `int`), the integer number may be rounded to another integer number. I should point out that some integers can't be represented exactly in floating point representation. For example:

```
int n = 123456789;
float f = (float)n;
printf("%f\n", f);    // prints 123456792.000000
```

The value 123456789 requires 27 significant bits, but a float only has 24 bits of significand (23 fraction + 1 implicit). So the value is rounded to the nearest representable float, which is 123456792.

- int or float to double: Because of the greater range and precision (32 bits for an `int` versus a 53-bit significand for double — 52 fraction bits plus 1 implicit leading 1), every `int` value can be represented exactly in a double (but see below for why you may not want to use a double to represent integer-only values). A float likewise can fit into a double. In fact, many programmers use only doubles for floating point if there's no good reason to do otherwise (such as saving memory space or a library needs floats).
- double to float: With IEEE binary64 and binary32, the destination offers 24 significand bits rather than 53. A value already representable in binary32 (such as 1.5) converts exactly; other in-range values may round. Magnitudes below the smallest normal binary32 value ($\sim 1.18 \times 10^{-38}$) may round to a subnormal, and sufficiently tiny values may round to zero. IEEE-conforming conversions can overflow to infinity in round-to-nearest mode; directed modes can instead produce the largest finite value. Core C does not itself guarantee infinity for an out-of-range conversion, so do not rely on that without the implementation's IEEE support. Choose the precision your application needs.
- float or double to int: The value will be rounded toward zero. This is called **truncation** since it just cuts off the decimal part without any rounding being done. This may not be very desirable. For example:

```
printf("%d\n", (int)3.9f);    // prints 3 (truncated toward
zero)
printf("%d\n", (int)(-3.9f)); // prints -3 (truncated
toward zero)
```

Furthermore, when going from a double to an int, the value may overflow. Casting a value outside the range of `int` (such as `(int)1e20f`) is **undefined behavior** in C — the result is unpredictable and the program may crash or produce garbage.

Although it might be okay to represent integer-only values as doubles, there is a downside to this. The performance comparison depends on the processor, operation, vectorization, and conversions; floating-point arithmetic is not universally much slower than integer arithmetic. To increase range, you can use unsigned versions of int for twice the range, or you can use a 64-bit signed type (`long` on LP64 systems, `long long` on LLP64), which represents all

positive integers up to about 9.2×10^{18} . The unsigned 64-bit type extends this to about 1.8×10^{19} ($2^{64} \approx 1.8 \times 10^{19}$). Depending on the version of C and compiler you are using, there might be even larger integer types, but these might not be portable to other compilers.

The biggest benefit is that all integers are represented between the maximum and minimum in an integer type. Floating point types have 'gaps' where some integers can't be represented. Just because the range on a double is roughly 10^{308} doesn't mean every integer in that range is representable. A double can represent every integer exactly only up to 2^{53} in magnitude (about 9×10^{15}); beyond that, representable values skip integers. So do use an integer type for integers whenever possible; it is better suited for integers than floating point (unless greater range is imperative and having gaps in representation is acceptable).

Lesson 6: The Posit Number System

Introduction: Why Posits?

Throughout this module we have studied IEEE 754, the standard that has dominated floating point arithmetic since 1985. While IEEE 754 is enormously successful, it has well-known limitations:

- **Many special-value encodings:** IEEE 754 binary32 has over 16 million NaN encodings. They share NaN comparison semantics, but signaling/quiet distinctions and payload bits can carry diagnostic information. Posit proponents prefer to spend nearly all encodings on finite real values.
- **Constant relative precision within a binade:** Normal IEEE binary formats provide a fixed significand width. Posits instead taper precision, allocating more fraction bits near 1 and fewer at extreme magnitudes. Which distribution is better depends on the workload.
- **Several special cases:** Signed zeros, infinities, NaNs, and subnormal numbers provide useful semantics but add cases that hardware, libraries, and programmers must handle.

In 2017, computer scientist **John L. Gustafson** proposed an alternative called the **posit number system**. Posits trade IEEE 754's fixed fields and rich special-value semantics for tapered precision, a wide dynamic range, one zero, and one NaR (Not a Real) encoding. Accuracy comparisons depend on the format parameters and workload.

Posit Format: Posit<n, es>

A posit is characterized by two parameters:

- **n** — the total number of bits
- **es** — the maximum number of exponent bits

We write this as **Posit<n, es>**. For example, Posit<8, 2> is an 8-bit posit with up to 2 exponent bits. (Note that the 2022 Posit Standard fixes $es = 2$ for all bit widths; this lesson teaches the general Posit<n, es> scheme from Gustafson's original 2017 formulation, which is why some examples use other es values.)

A posit number is divided into four fields, read left to right:

	sign		regime		exponent		fraction	
	1		variable		0 to es		remaining bits	

The key innovation is the **variable-length regime field**. Because the regime can expand or shrink, the exponent and fraction fields adjust in size accordingly. This gives posits their characteristic **tapered precision**: values near 1.0 get a short regime, leaving more bits for the exponent and fraction (high precision), while very large or very small values get a long regime, consuming bits that would otherwise go to the fraction (lower precision, but greater range).

The Sign Bit

The leading bit is the sign bit, just like IEEE 754:

- **0** = positive (or zero)
- **1** = negative

However, unlike IEEE 754 where the sign bit is simply a flag, in posits a negative value is stored as the **two's complement** of its positive encoding. So to decode a negative posit, you first take the two's complement of the entire bit string to get the positive version, then decode that, and finally negate the result.

The Regime Field

The regime is a variable-length field that immediately follows the sign bit. It consists of a **run of identical bits** terminated by the **opposite bit** (or by the end of the number if no opposite bit fits).

There are two cases:

- **A run of 1s terminated by a 0** (or end of number): If the run contains **k** ones, the regime value is **k - 1**.
- **A run of 0s terminated by a 1** (or end of number): If the run contains **k** zeros, the regime value is **-k**.

Examples of regime fields and their values:

Regime bits	Run length	Regime value

0001	3 zeros	-3
001	2 zeros	-2
01	1 zero	-1
10	1 one	0
110	2 ones	1
1110	3 ones	2
11110	4 ones	3

Note that the terminating bit (the opposite bit) is part of the regime field but is **not** counted in the run length. If the run extends to the end of the number with no terminating bit, we simply count all the identical bits.

The Useed Value

The regime field contributes a factor of **useed** raised to the regime value. The useed is defined as:

$$\text{useed} = 2^{(2^{\text{es}})}$$

For different values of es:

$$\begin{aligned} \text{es} = 0: \quad \text{useed} &= 2^{(2^0)} = 2^1 = 2 \\ \text{es} = 1: \quad \text{useed} &= 2^{(2^1)} = 2^2 = 4 \\ \text{es} = 2: \quad \text{useed} &= 2^{(2^2)} = 2^4 = 16 \\ \text{es} = 3: \quad \text{useed} &= 2^{(2^3)} = 2^8 = 256 \end{aligned}$$

The useed acts as a "super-exponent" that allows posits to cover a huge dynamic range. With es = 2, each step of the regime multiplies or divides the value by 16.

The Exponent Field

After the regime (and its terminating bit), the next **es** bits form the exponent field. If fewer than es bits remain in the number, the exponent is padded with trailing zeros on the right. The exponent is interpreted as an unsigned integer **e**, and it contributes a factor of 2^e to the value.

The Fraction Field

Any remaining bits after the exponent form the fraction field. Just like IEEE 754 normalized values, there is an **implicit leading 1**. So if the fraction bits are **fff...f**, the fraction value is

1.fff...f in binary. If there are no fraction bits remaining, the fraction value is simply **1.0**.

Putting It All Together

The value of a positive posit is:

$$\text{value} = \text{used}^{\text{regime}} * 2^{\text{exponent}} * (1 + \text{fraction})$$

Or equivalently, since $\text{used} = 2^{(2^{\text{es}})}$:

$$\text{value} = 2^{(\text{regime} * 2^{\text{es}} + \text{exponent})} * (1 + \text{fraction})$$

For a negative posit, take the two's complement first, decode as above, then negate.

Special Values

Posits have exactly **two** special bit patterns:

- **Zero:** all bits are 0. For example, in Posit<8, 2>: **0000 0000** = 0.
- **NaR (Not a Real):** only the sign bit is 1, all other bits are 0. For example, in Posit<8, 2>: **1000 0000** = NaR.

There is no positive or negative infinity, no signed zero, and no family of NaN payloads. NaR is the single non-real result used for operations the posit rules do not represent as a real value, and it propagates through subsequent arithmetic. This simplifies the set of special encodings but conveys less diagnostic information than IEEE 754 NaNs.

Worked Example 1: Decoding a Posit<8, 2> Value

Let us decode the Posit<8, 2> bit pattern **0 110 01 10** step by step.

Step 1: Identify the sign bit

```
Bit pattern:  0  110  01  10
               ^
Sign bit = 0  -->  positive
```

Step 2: Extract the regime

Remaining bits: 1 1 0 0 1 1 0
 ^^^^

We read from the left: 1, 1, 0
This is a run of two 1s terminated by a 0.

Run length $k = 2$

Regime value = $k - 1 = 2 - 1 = 1$

The regime field consumed 3 bits: "110"

Step 3: Extract the exponent

Remaining bits after regime: 0 1 1 0
 $es = 2$, so we take the next 2 bits: "01"

Exponent (unsigned) = 01 in binary = 1

Step 4: Extract the fraction

Remaining bits after exponent: 1 0
Fraction bits = "10"

Fraction value = 1.10 in binary = $1 + 0.5 = 1.5$
(Remember the implicit leading 1)

Step 5: Compute the value

Parameters:

$es = 2$

$used = 2^{(2^2)} = 2^4 = 16$

regime value = 1

exponent = 1

fraction = 1.5

value = $used^{regime}$	*	$2^{exponent}$	*	fraction
= 16^1	*	2^1	*	1.5
= 16	*	2	*	1.5
= 48				

The Posit<8, 2> bit pattern 0 110 01 10 represents the value 48.

Worked Example 2: Encoding a Decimal Value as Posit<8, 2>

Let us encode the decimal value -3.5 as a Posit<8, 2>.

Step 1: Handle the sign

The value is negative, so:
sign bit = 1
We will encode the magnitude 3.5 first, then take the two's complement.

Step 2: Determine the regime

We need: $used^{regime} * 2^{exponent} * fraction = 3.5$
With $es = 2$, $used = 16$.

Try regime = 0:
 $used^0 = 16^0 = 1$
So we need: $2^{exponent} * fraction = 3.5$
Since exponent can be 0..3 (2 bits), and $fraction \geq 1.0$:
With exponent = 1: $2^1 * fraction = 3.5 \rightarrow fraction = 1.75$
 $fraction = 1.75 = 1.11$ in binary. This works!

Regime value = 0 $\rightarrow$ regime bits = "10" (one 1, terminated by 0)

Step 3: Encode the exponent

Exponent = 1 = "01" in binary (2 bits since $es = 2$)

Step 4: Encode the fraction

Fraction = 1.75

Remove the implicit leading 1: $0.75 = 0.11$ in binary

Fraction bits = "11"

Remaining bits for fraction: $8 - 1(\text{sign}) - 2(\text{regime}) - 2(\text{exponent}) = 3$ bits

We have 2 fraction bits, padded to fill: "110"

(The third bit is 0 since there are no further fractional bits.)

Step 5: Assemble the positive encoding

sign	regime	exponent	fraction
0	10	01	110

Total: 1 sign + 2 regime + 2 exponent + 3 fraction = 8 bits.

Positive encoding: 0 10 01 110

As 8 bits: 01001110

Step 6: Apply two's complement for the negative value

Positive encoding: 01001110

Bitwise complement: 10110001

Add 1: + 00000001

Two's complement: 10110010

So Posit<8, 2> encoding of -3.5 is: 10110010

Step 7: Verify

```
Start with: 10110010
Sign bit = 1 --> negative. Take two's complement:
Complement: 01001101
Add 1:      01001110
Now decode 01001110:
Sign = 0 (positive)
Regime: "10" --> one 1, terminated by 0 --> regime = 0
Exponent: "01" --> e = 1
Fraction: "110" --> 1.110 in binary = 1 + 0.5 + 0.25 = 1.75
Value =  $16^0 * 2^1 * 1.75 = 1 * 2 * 1.75 = 3.5$ 
Negate: -3.5. Correct!
```

Comparison with IEEE 754

Here is a side-by-side comparison of the key differences between IEEE 754 and posits:

Property	IEEE 754	Posit<n, es>
Field structure	Fixed-size sign, exponent, fraction	Fixed sign, variable-length regime, then exponent, then fraction
Special values	+0, -0, +Inf, -Inf, NaN (millions of bit patterns)	Only two: 0 (all zeros) and NaR (sign bit only)
Precision distribution	Roughly uniform across all exponent ranges	Tapered: highest precision near 1.0, less at extremes
Encoding allocation	Many NaN encodings, including payload and quiet/signaling information	Every encoding other than 0 and NaR represents a nonzero finite real value
Signed zero	Yes (+0 and -0 are distinct)	No — there is exactly one zero
Denormalized numbers	Yes (gradual underflow)	No separate subnormal encoding; spacing becomes coarser in relative terms near the smallest magnitudes
Dynamic range (for same bit width)	Large	Can be larger, depending on es
Hardware support	Widespread native support across general-purpose CPUs and GPUs (feature details and IEEE conformance vary)	Limited — experimental hardware and software implementations

Precision Near 1.0

When a posit value is close to 1.0, the regime is short ("10" for regime value 0), leaving more bits for exponent and fraction. A same-width posit can therefore provide finer spacing than a same-width IEEE format in that region. Whether this improves an application depends on where its values occur and on accumulation, rounding, exceptional-value, and implementation behavior.

Different Use of the Encoding Space

In IEEE 754 binary32, exponent field 11111111 with a nonzero fraction encodes NaN. There are $2^{23} - 1$ fraction patterns for each sign, totaling over 16 million NaN encodings; payloads and the quiet/signaling distinction can convey information even though NaNs are unordered in ordinary comparisons. In a posit format, every encoding other than 0 and NaR represents a nonzero finite real value. This is a design tradeoff, not free capacity: posits give up IEEE 754's infinities, signed zero, and varied NaN semantics.

Limitations of Posits

Despite their advantages, posits are not yet a replacement for IEEE 754 in practice:

- **Hardware support:** IEEE-style binary floating point has widespread native support in CPUs, GPUs, and DSPs. Posit hardware remains limited, and software implementations are generally much slower than native floating-point instructions.
- **Variable-length fields:** The variable-length regime makes hardware design more complex. Extracting the regime requires a leading-zero or leading-one count, and the shifting logic to separate the exponent and fraction fields adds latency.
- **Ecosystem and tooling:** Decades of libraries, compilers, debugging tools, and numerical analysis have been built around IEEE 754. The posit ecosystem is still in its early stages.
- **Tapered precision is a trade-off:** While posits give more precision near 1.0, they give **less** precision for very large or very small values. Applications that need uniform precision across a wide range may not benefit.
- **No infinity:** IEEE 754 can represent overflow as infinity. Posit rounding instead clamps a finite result beyond the representable range to the signed maximum finite value; a nonzero result smaller in magnitude than minPos rounds to signed minPos. NaR is used for results that are not uniquely defined real numbers, such as division by zero, not for ordinary finite overflow.

Summary

The posit number system is a thoughtful alternative design for representing real numbers. Variable-length regime bits produce tapered precision, while one zero and one NaR replace IEEE 754's larger set of special encodings. Those choices can improve spacing for some value ranges but give up useful IEEE semantics and mature hardware/software support. Understanding posits helps expose the design tradeoffs in numeric representation.

Module 2 Summary

Wrap Up

In this module we studied floating point representation — how computers store and manipulate numbers that have fractional parts. We began with fractional binary numbers and built up to the full IEEE 754 encoding scheme, examining how sign, exponent, and fraction fields work together to represent an enormous range of values. We explored rounding modes and the arithmetic pitfalls that arise from finite precision. Finally, we looked at the posit number system as a modern alternative to IEEE 754.

Key Takeaways

- **Fractional binary numbers** use a radix point, with bit positions to the right representing negative powers of 2 ($1/2$, $1/4$, $1/8$, etc.).
- **IEEE 754** is the dominant standard for floating point. A value is encoded using three fields: a sign bit, a biased exponent, and a fractional mantissa with an implicit leading 1 (for normalized values).
- **Special values** in IEEE 754 include positive/negative infinity (exponent all 1s, fraction all 0s) and NaN (exponent all 1s, fraction nonzero). Denormalized numbers (exponent all 0s) allow representation of values very close to zero.
- **Rounding** is unavoidable because most real numbers cannot be represented exactly. The default mode, round-to-even, minimizes statistical bias over many operations.
- **Precision loss** is a fundamental limitation: floating point addition is not associative, and small numbers added to large numbers may be lost entirely. Overflow and underflow are also concerns.
- **Posits** (Posit<n, es>) offer an alternative encoding with variable-length regime bits, tapered precision (more precision near 1.0, less at extremes), one zero, and one NaR value. They allocate nearly all other encodings to finite real values while giving up IEEE 754's infinities, signed zero, and NaN payload semantics.

Additional Resources

[Floating Point](#) (Video Lecture from Carnegie Mellon University)

[PositHub](#) — Resources and community for the posit number system

[Beating Floating Point at its Own Game: Posit Arithmetic](#) — John Gustafson's paper introducing posits

Module 3 Overview

In Module 3: x86-64 Assembly Fundamentals, you will learn to **write** x86-64 assembly programs by hand. This is something that most students have not encountered in their previous programming experience, and it is one of the most important topics in computer systems.

We start by examining the build process and writing your first assembly program using the GNU tools `as` and `ld`, then write Makefiles to automate assembling and linking. From there, we cover registers, data movement and addressing modes, the data section and labels, arithmetic operations, logical and shift operations, stack operations, and the Linux syscall interface for bare `read`, `write`, and `exit`. The module closes with debugging assembly programs in GDB, stepping through them instruction by instruction. Each topic follows a "write it by hand first, then see how gcc does it" approach: you write a working assembly program, then compare it with `gcc -S` output to understand what the compiler generates.

Learning objectives

At the end of this learning module, you will be able to:

1. Describe the four-stage build process and the role of the compiler, assembler, and linker
2. Use `as` and `ld` to assemble and link hand-written assembly programs
3. Write Makefiles to automate the assembly and linking of single-file and multi-file projects
4. Use `gcc -S` and `objdump -d` to examine compiler-generated assembly
5. Identify x86-64 registers, their sub-register names, and their typical roles
6. Write programs that use the `.data`, `.rodata`, and `.bss` sections with RIP-relative addressing
7. Use the `mov` and `lea` instructions with all addressing modes
8. Write programs using arithmetic instructions (`add`, `sub`, `imul`, `idiv`) and logical/shift instructions
9. Use `push`, `pop`, and manual stack manipulation for temporary storage

10. Use the Linux x86-64 syscall convention to perform bare `read` , `write` , and `exit` operations
11. Write a complete assembly program that reads input, computes a result, and prints output
12. Use GDB to set breakpoints, step through instructions, and inspect registers and memory

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below
2. Read Lessons 1 to 10, work through the hand-written assembly examples, and complete the embedded practice problems in the lessons that include them
3. Take the module quiz
4. Review additional resources on the Module 3 Summary page

Reading

- Bryant & O'Hallaron: Chapter 3, sections 3.1 - 3.5
- Dive into Systems: Chapters 6 & 7
- [Course _start Assembly Syntax Reference](#)

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: The Build Process and Your First Assembly Program

From Source Code to Executable

The term "building" software is often used interchangeably with compiling, as in "compile and run your code." But compiling is actually just one step in a four-stage pipeline. Understanding each stage matters because in this module you will work with these stages individually.

- **Stage 1: Preprocessing** — A preprocessor handles directives (lines starting with `#`). For `#include`, the contents of the header file are pasted into the source. For `#define`, the symbol is substituted throughout the file.
- **Stage 2: Compiling** — The compiler translates preprocessed C into **assembly code**, a human-readable representation of machine instructions. This stage does the heavy lifting: parsing, type-checking, and optimizing. If you stop here with `gcc -S`, you get a `.s` text file for each `.c` file.
- **Stage 3: Assembling** — The assembler (`as`) converts the `.s` file into an **object file** (`.o`). This is nearly machine code but with some unresolved references. Stop here with `gcc -c`.
- **Stage 4: Linking** — The linker (`ld`) combines all `.o` files plus library code into a final executable containing pure machine code.

Two Paths to an Executable

With `gcc`, all four stages happen behind the scenes in a single command:

```
$ gcc -o myprogram myprogram.c
```

But there is another path — the one we will use for the rest of this module. You write assembly by hand in a `.s` file, then use the assembler and linker directly:

```
$ as -o myprogram.o myprogram.s
$ ld -o myprogram myprogram.o
```

The `as` command assembles your `.s` file into an object file. The `ld` command links it into a runnable executable. No compiler involved — you are the compiler.

Your First Assembly Program

Here is the smallest possible x86-64 Linux program. It does nothing but exit with a status code:

```
.text
.global _start
_start:
    mov $60, %rax
    mov $42, %rdi
    syscall
```

Let's break this down line by line:

- `.text` — This directive tells the assembler that what follows is executable code.
- `.global _start` — This makes the label `_start` visible to the linker. The linker needs to know where your program begins, and by convention that label is `_start`.
- `_start:` — A label. It marks a location in the code that can be referred to by name.
- `mov $60, %rax` — Load the value 60 into register `%rax`. On Linux, `syscall` number 60 is `exit`.
- `mov $42, %rdi` — Load 42 into `%rdi`. This is the exit status code — the value the program returns to the shell.
- `syscall` — Transfer control to the Linux kernel to execute the system call.

Build and run it

Save the above as `smallest.s`, then:

```
$ as -o smallest.o smallest.s
$ ld -o smallest smallest.o
$ ./smallest
$ echo $?
42
```

The `echo $?` command prints the exit status of the last program. You should see `42`. Congratulations — you just wrote, assembled, linked, and ran a program with no compiler at all.

Exit codes as output: Until we learn how to print text, we will use the program's exit code (checked with `echo $?`) as a convenient way to verify our programs produce the right answer. Exit codes are limited to 0–255 (values are taken mod 256), but that is enough for now.

gcc Comparison: `return 42;`

Now let's see how `gcc` handles the same idea. Write this C program:

```
int main() {  
    return 42;  
}
```

Compile it with optimization to keep the output readable:

```
$ gcc -Og -S return42.c  
$ cat return42.s
```

You will see something like:

```
main:  
    movl    $42, %eax  
    ret
```

(Plus some assembler directives we can ignore for now.) Notice the differences:

- `gcc` uses `main:` as the entry point. The C runtime provides the actual `_start` and calls `main` for you.
- `gcc` returns the value in `%eax` (the 32-bit portion of `%rax`) and uses `ret` to return to the caller. The C runtime then calls `exit` with that value.
- Our hand-written program used `_start` and invoked the `exit` syscall directly. There is no caller to return to.

Throughout this module, we will follow this pattern: **write it by hand first**, then see how `gcc` does the same thing. This way you understand both the raw machine and the compiler's

conventions.

Lesson 2: Writing Makefiles

Why Makefiles?

In Lesson 1 we built programs by typing `as` and `ld` commands manually at the terminal. For a single source file, that is perfectly manageable. But as programs grow to multiple source files — as we will see in Lesson 9, where a program is split across `main.s`, `readInt.s`, and `writeInt.s` — re-typing the assembler and linker commands for every file after every change becomes tedious and error-prone. You might forget to reassemble a file you edited, or mistype a flag, and then spend twenty minutes debugging a problem that does not actually exist in your code.

Make is a build automation tool that solves this problem. You describe the build process once in a file called a `Makefile`, and from then on a single command — `make` — does all the work for you.

What Make Does

Make reads a file called `Makefile` (or `makefile`) in the current directory. This file describes which files depend on which, and the shell commands needed to build them. When you type `make`, it checks the modification timestamps of all the files involved and **only rebuilds what has changed**. If you edit one `.s` file out of five, Make will reassemble just that one file and re-link the final executable — skipping the four files that have not changed. For large projects, this saves a tremendous amount of time.

Basic Syntax

A Makefile consists of **rules**. Each rule has three parts:

```
target: prerequisites
      recipe
```

- The **target** is the file to be built (or a name for an action).

- The **prerequisites** (also called dependencies) are the files that the target depends on. If any prerequisite is newer than the target, Make will run the recipe.
- The **recipe** is one or more shell commands that build the target from the prerequisites.

Tab requirement: The recipe lines **must** be indented with a literal TAB character, not spaces. This is the single most common Makefile syntax error. If Make complains about "missing separator," check that your editor is inserting a real tab. In `vim`, a tab is the default. In `nano`, press `Ctrl-I` or the Tab key. In VS Code, check the status bar for "Spaces" vs. "Tabs" and switch if needed.

A First Makefile

Let's write a Makefile for the `smallest` exit program from Lesson 1. Recall that building it by hand requires two commands:

```
$ as -o smallest.o smallest.s
$ ld -o smallest smallest.o
```

Here is a Makefile that automates this:

```
smallest: smallest.o
    ld -o smallest smallest.o

smallest.o: smallest.s
    as -o smallest.o smallest.s

clean:
    rm -f smallest smallest.o

.PHONY: clean
```

How it works

When you type `make` with no arguments, Make builds the **first target** in the file — in this case, `smallest`. It sees that `smallest` depends on `smallest.o`, so it checks the rule for `smallest.o`. That rule depends on `smallest.s`. If `smallest.s` is newer than `smallest.o` (or if `smallest.o` does not exist), Make runs `as -o smallest.o smallest.s`. Then, if `smallest.o` is newer than `smallest`, Make runs `ld -o smallest smallest.o`.

The `clean` rule has no prerequisites — it simply removes the build products. You run it with `make clean`. The `.PHONY: clean` line tells Make that `clean` is not an actual file; without this, if a file named `clean` happened to exist in the directory, Make would see that it is "up to date" and skip the rule.

Variables

If you decide to switch assemblers or add flags, you would have to change every recipe line. **Makefile variables** (also called macros) let you define a value once and use it everywhere:

```
AS = as
LD = ld

smallest: smallest.o
    $(LD) -o smallest smallest.o

smallest.o: smallest.s
    $(AS) -o smallest.o smallest.s
```

You define a variable with `NAME = value` and reference it with `$(NAME)`. Now if you want to add a flag — say, `-g` for debug symbols — you change one line (`AS = as -g` or, better, add an `ASFLAGS` variable) instead of editing every recipe.

Automatic Variables

Make provides several **automatic variables** that refer to parts of the current rule. The three most useful are:

- `$@` — the **target** of the current rule.
- `$<` — the **first prerequisite**.
- `$$` — **all prerequisites**, separated by spaces.

Using these, our rules become more concise and less repetitive:

```
AS = as
LD = ld

smallest: smallest.o
    $(LD) -o $@ $^

smallest.o: smallest.s
    $(AS) -o $@ $<
```

In the first rule, `$@` expands to `smallest` (the target) and `$^` expands to `smallest.o` (all prerequisites). In the second rule, `$@` expands to `smallest.o` and `$<` expands to `smallest.s` (the first — and only — prerequisite). The advantage of automatic variables becomes clear when you have many similar rules: you write the pattern once, and the filenames fill themselves in.

Pattern Rules

When a project has many `.s` files, writing a separate rule for each `.o` file is tedious. A **pattern rule** uses the `%` wildcard to match any stem:

```
%.o: %.s
    $(AS) -o $@ $<
```

This tells Make: "for any file ending in `.o`, if there is a corresponding file ending in `.s`, build the `.o` by running the assembler." The `%` matches the same string on both sides — so `main.o` depends on `main.s`, `readInt.o` depends on `readInt.s`, and so on. You write one rule, and it handles all of them.

A Multi-File Makefile

Here is a complete Makefile for a project with three source files — `main.s`, `readInt.s`, and `writeInt.s` — foreshadowing the multi-file programs we will build in Lesson 9:

```

AS = as
LD = ld
ASFLAGS = -g

OBJS = main.o readInt.o writeInt.o

program: $(OBJS)
    $(LD) -o $@ $^

%.o: %.s
    $(AS) $(ASFLAGS) -o $@ $<

clean:
    rm -f program $(OBJS)

.PHONY: clean

```

A few things to note:

- The `ASFLAGS = -g` variable adds the `-g` flag for debug symbols. This will be essential in Lesson 10 when we use GDB to step through our programs.
- The `OBJS` variable lists all the object files. It is used in both the link rule and the `clean` rule, so adding a new source file means changing just one line.
- The pattern rule `%.o: %.s` handles all three `.s` → `.o` compilations.
- Typing `make` builds the `program` target. Typing `make clean` removes all generated files.

Makefiles for C Programs

Makefiles are not specific to assembly — they are used for C, C++, and many other languages. Here is the equivalent Makefile for a C program:

```
CC = gcc
CFLAGS = -Wall -g

myprogram: myprogram.o
    $(CC) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm -f myprogram *.o

.PHONY: clean
```

The structure is identical. The only differences are the tool names (`gcc` instead of `as / ld`), the flag variable (`CFLAGS` instead of `ASFLAGS`), the file extension (`.c` instead of `.s`), and the `-c` flag that tells `gcc` to compile without linking. The concepts — targets, prerequisites, recipes, variables, pattern rules — are exactly the same.

Common Conventions

Most Makefiles follow a few widely adopted conventions:

- `all` as the default target. If a Makefile builds more than one program, the first rule is typically named `all` and lists all the programs as prerequisites:

```
all: program1 program2

program1: program1.o
    $(LD) -o $@ $^

program2: program2.o utils.o
    $(LD) -o $@ $^
```

Since `all` is the first target, typing `make` builds everything.

- `clean` for removing generated files. Every Makefile should have a `clean` target that deletes all object files and executables, returning the directory to a source-only state.
- `.PHONY` for non-file targets. Any target that does not correspond to a real file — such as `clean` or `all` — should be declared `.PHONY` so that Make does not confuse it with a file of the same name.

Putting It All Together

Here is the workflow you will use throughout this module and the rest of the course:

1. Write your assembly source files (`.s` files).
2. Write a Makefile (or copy and adapt one from a previous project).
3. Run `make` to build.
4. Test your program.
5. Edit a source file to fix a bug or add a feature.
6. Run `make` again — it rebuilds only what changed.
7. When finished, run `make clean` to remove generated files.

This cycle — edit, make, test — is the standard development loop in systems programming. It works the same way whether you are writing assembly, C, or C++.

Lesson 3: Registers, Data Types, and the CPU Environment

What a Processor Has to Work With

A processor is a remarkably simple machine. It has a tiny amount of fast storage called **registers** — just 16 general-purpose registers on x86-64, totaling 128 bytes. Everything the CPU does involves moving data between these registers and main memory, and performing arithmetic or logical operations on register contents.

The key components of the processor state are:

- **Register file** — 16 general-purpose 64-bit registers for integer data and addresses.
- **Program counter** (`%rip`) — holds the address of the next instruction to execute.
- **Condition code registers** — single-bit flags set by arithmetic operations, used for branching (covered in Module 4).
- **Floating point / vector registers** — a separate bank of 16 XMM/YMM registers for floating point math (covered later).
- **Main memory** — a large byte-addressable array that holds code, data, and the stack.

The 16 General-Purpose Registers

Each of the 16 registers is 64 bits (8 bytes) wide. You can also access smaller portions of each register using different names. This is a legacy of the x86 architecture growing from 16-bit to 32-bit to 64-bit over the decades.

64-bit	32-bit	16-bit	8-bit	Typical use
<code>%rax</code>	<code>%eax</code>	<code>%ax</code>	<code>%al</code>	Return value
<code>%rbx</code>	<code>%ebx</code>	<code>%bx</code>	<code>%bl</code>	Callee-saved
<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code>	<code>%cl</code>	4th argument
<code>%rdx</code>	<code>%edx</code>	<code>%dx</code>	<code>%dl</code>	3rd argument
<code>%rsi</code>	<code>%esi</code>	<code>%si</code>	<code>%sil</code>	2nd argument
<code>%rdi</code>	<code>%edi</code>	<code>%di</code>	<code>%dil</code>	1st argument
<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code>	<code>%bpl</code>	Base pointer
<code>%rsp</code>	<code>%esp</code>	<code>%sp</code>	<code>%spl</code>	Stack pointer
<code>%r8</code>	<code>%r8d</code>	<code>%r8w</code>	<code>%r8b</code>	5th argument
<code>%r9</code>	<code>%r9d</code>	<code>%r9w</code>	<code>%r9b</code>	6th argument
<code>%r10</code>	<code>%r10d</code>	<code>%r10w</code>	<code>%r10b</code>	Caller-saved
<code>%r11</code>	<code>%r11d</code>	<code>%r11w</code>	<code>%r11b</code>	Caller-saved
<code>%r12</code>	<code>%r12d</code>	<code>%r12w</code>	<code>%r12b</code>	Callee-saved
<code>%r13</code>	<code>%r13d</code>	<code>%r13w</code>	<code>%r13b</code>	Callee-saved
<code>%r14</code>	<code>%r14d</code>	<code>%r14w</code>	<code>%r14b</code>	Callee-saved
<code>%r15</code>	<code>%r15d</code>	<code>%r15w</code>	<code>%r15b</code>	Callee-saved

The "Typical use" column reflects the System V AMD64 calling convention used on Linux. We will cover calling conventions in detail in Module 4. For now, just note that `%rax` is where return values go, `%rdi` is the first argument, and `%rsp` is the stack pointer.

AT&T Syntax

There are two common syntaxes for x86 assembly: Intel and AT&T. We use **AT&T syntax**, which is what `gcc` and the GNU assembler (`as`) produce by default. The key conventions are:

- Register names are prefixed with `%` : `%rax` , `%rdi` , etc.
- Immediate (constant) values are prefixed with `$` : `$42` , `$0xFF` .
- In two-operand instructions, the **source comes first** and the **destination comes second**:
`mov $5, %rax` means "move 5 into %rax."
- Instructions have a **size suffix** indicating the data width:

Common misconception — the operand order trap. If you've seen x86 assembly anywhere outside this course (Intel manuals, Stack Overflow answers, Windows debuggers, most online tutorials), you've probably seen *Intel syntax*, where the operand order is the **opposite**: destination first, source second. The same instruction is written:

AT&T (this course):	<code>mov \$5, %rax</code>	<code># "move 5 into rax"</code>
Intel (most other docs):	<code>mov rax, 5</code>	<code># "rax := 5"</code>

Reading `mov %rax, %rbx` as "rax becomes rbx" instead of "rbx becomes rax" is the single most common mistake when transitioning between syntaxes. Whenever you copy an example off the internet, look for the `%` register prefixes and the `$` immediate prefixes — if you see them, it's AT&T (source-first). If you don't, it's Intel (destination-first) and the operands need to be swapped before they'll assemble with `as` .

Suffix	Name	Size	C type
<code>b</code>	Byte	1 byte (8 bits)	<code>char</code>
<code>w</code>	Word	2 bytes (16 bits)	<code>short</code>
<code>l</code>	Long / Double word	4 bytes (32 bits)	<code>int</code>
<code>q</code>	Quad word	8 bytes (64 bits)	<code>long</code> , pointers

For example, `movq` moves 8 bytes, `movl` moves 4 bytes, and `movb` moves 1 byte. When the size is obvious from context (e.g., you are writing to `%eax` , which is 32 bits), the suffix can sometimes be omitted, but it is good practice to include it.

Hand-Write: Registers in Action

Let's write a program that uses several registers and does some arithmetic. We will add two numbers and exit with the result:

```
.text
.global _start
_start:
    mov $200, %rax
    mov $100, %rbx
    add %rbx, %rax      # %rax = 200 + 100 = 300
    mov %rax, %rdi      # exit code = 300
    mov $60, %rax       # syscall number for exit
    syscall
```

Build and run:

```
$ as -o registers.o registers.s
$ ld -o registers registers.o
$ ./registers
$ echo $?
44
```

Wait — we computed 300 but got 44? Remember that exit codes are a single unsigned byte (0–255). The value $300 \bmod 256 = 44$. This is a good reminder that data sizes matter in assembly. We will take the first step in Lesson 4 when we learn to print text, and address it fully in Lesson 9 when we build `writeInt` to print integer values.

gcc Comparison: Disassembly with `objdump`

Let's see the other direction — starting from C and examining the assembly that `gcc` produces. Write this C file:

```
long add_two(long a, long b) {
    return a + b;
}
```

Compile to assembly:

```
$ gcc -Og -S add_two.c
$ cat add_two.s
```

You will see gcc's output, which includes the function label and instructions using the register names from the calling convention (`%rdi` for the first argument, `%rsi` for the second).

You can also compile to an object file and use `objdump` to see both machine code and assembly side by side:

```
$ gcc -Og -c add_two.c
$ objdump -d add_two.o
```

The output shows each instruction's address (offset), the raw bytes of machine code in hexadecimal, and the assembly instruction. Machine instructions on x86-64 range from 1 to 15 bytes in length — this is a variable-length instruction set.

Lesson 4: Moving Data

The Most Common Instruction

The `mov` instruction is the workhorse of assembly. It copies data from a source to a destination. On x86-64, many arithmetic instructions allow one memory operand, but arithmetic cannot use two memory operands at once. So most programs still spend a large fraction of their instructions moving data between memory and registers.

Three Kinds of Operands

Every instruction operand falls into one of three categories:

- **Immediate** — A constant value, written with a `$` prefix: `$42` , `$0xFF` , `$-1` .
- **Register** — The contents of a register, written with a `%` prefix: `%rax` , `%edi` .
- **Memory** — A value stored at a memory address. A register name in parentheses means "the value at the address stored in this register": `(%rax)` means "the value in memory at the address contained in `%rax` ."

The `mov` Family

The `mov` instruction copies data without modifying it. Add a size suffix to specify how many bytes to move:

- `movb $0x56, %al` — Move 1 byte (immediate to register)
- `movw %ax, %bx` — Move 2 bytes (register to register)
- `movl $0x56, %ebx` — Move 4 bytes (immediate to register)
- `movq %rax, %rcx` — Move 8 bytes (register to register)
- `movw (%rcx), %bx` — Move 2 bytes from the memory address in `%rcx` into `%bx`
- `movq %rax, (%rdx)` — Move 8 bytes from `%rax` to the memory address in `%rdx`

Critical restriction: You **cannot** move directly from one memory location to another. If you need to copy memory-to-memory, you must use two instructions with a register as an intermediary:

```
movq (%rax), %rcx  then  movq %rcx, (%rdx)
```

Zero- and Sign-Extending Moves

When copying a small value into a larger register, you must say what happens to the extra high bits: fill them with zeros (**zero-extend**) or with copies of the sign bit (**sign-extend**). You will meet these instructions immediately in `gcc -S` output:

Instruction	Meaning
<code>movzbl S, D</code>	Zero-extend a byte to 32 bits (high bits become 0)
<code>movzbq S, D</code>	Zero-extend a byte to 64 bits
<code>movsbl S, D</code>	Sign-extend a byte to 32 bits (high bits copy the sign bit)
<code>movslq S, D</code>	Sign-extend 32 bits to 64 bits
<code>cltq</code>	Sign-extend <code>%eax</code> into <code>%rax</code> (no operands)

```
movb  $-1, %al      # %al = 0xFF (-1 as a byte)
movzbl %al, %eax     # %eax = 0x000000FF = 255 (movsbl here would
give -1)
```

One quirk to remember: writing a 32-bit register automatically zeroes the upper 32 bits of the containing 64-bit register, so `movzbl` also clears bits 32–63 — which is why `gcc` uses `movzbl` where you might expect `movzbq`.

Effective Address Computation

Memory operands can be more complex than just `(%reg)`. The full form of a memory address in AT&T syntax is:

displacement(base, index, scale)

This computes the address: $\text{displacement} + \text{base} + \text{index} \times \text{scale}$

Any part can be omitted. The scale must be 1, 2, 4, or 8. Some examples:

Syntax	Address computed	Typical use
<code>(%rax)</code>	value in <code>%rax</code>	Simple pointer dereference
<code>8(%rax)</code>	<code>%rax + 8</code>	Struct field access
<code>(%rax,%rcx)</code>	<code>%rax + %rcx</code>	Base + offset
<code>(%rax,%rcx,8)</code>	<code>%rax + %rcx × 8</code>	Array of 8-byte elements
<code>200(%rbp,%r10,4)</code>	<code>200 + %rbp + %r10 × 4</code>	Complex array indexing

For example, if `%rbp` holds 10 and `%r10` holds 2, then `200(%rbp, %r10, 4)` computes: $200 + 10 + 2 \times 4 = 218$.

The `lea` Instruction

`lea` stands for "Load Effective Address." Unlike `mov`, which loads the *value* at a computed address, `lea` loads the *address itself* into a register. Think of it as the assembly equivalent of C's address-of operator `&`.

`leaq (%rax,%rcx,8), %rdx` `# %rdx = %rax + %rcx * 8 (the address, not the value)`

This makes `lea` extremely useful for two purposes: computing addresses of variables in memory, and doing quick arithmetic without a dedicated `add` / `mul` instruction.

Hand-Write: Hello, World

Let's use `mov` and `lea` to write a "Hello, world!" program. This uses the `write` system call (syscall number 1) which takes three arguments: file descriptor, buffer address, and byte count.

```

        .section .rodata
msg:     .asciz "Hello, world!\n"
msglen = . - msg - 1

        .text
        .global _start
_start:
    mov $1, %rax           # syscall number: write
    mov $1, %rdi           # file descriptor: stdout
    lea msg(%rip), %rsi    # address of the string
    mov $msglen, %rdx      # number of bytes to write
    syscall

    mov $60, %rax          # syscall number: exit
    xor %rdi, %rdi         # exit code 0
    syscall

```

Build and run:

```

$ as -o hello.o hello.s
$ ld -o hello hello.o
$ ./hello
Hello, world!

```

Key points:

- `.section .rodata` places the string in read-only data (it will not be modified at runtime).
- `.asciz` creates a null-terminated ASCII string.
- `msglen = . - msg - 1` computes the printable string length at assembly time (subtracting the null terminator added by `.asciz`). The `.` symbol means "current address."
- `lea msg(%rip), %rsi` uses RIP-relative addressing to load the string's address. RIP-relative addressing computes an address relative to the current instruction pointer, which is required for position-independent code on x86-64.
- `xor %rdi, %rdi` is a common idiom for setting a register to zero (any value XORed with itself is zero). It is shorter and faster than `mov $0, %rdi`.

The syscall convention: Syscall arguments go in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9` — note `%r10` in the fourth slot, where ordinary function calls use `%rcx` (we'll see the full contrast in Module 4, Lesson 5). The syscall number goes in `%rax`. Crucially, the `syscall` instruction **clobbers** `%rax` (replaced by the return value), `%rcx`, and `%r11`. Keeping a live value in one of those three registers across a `syscall` — and expecting it to still be there afterward — is the most common bug in hand-written `_start` programs.

From write to read

The `write` syscall sends bytes from your program's memory to the outside world. The `read` syscall does the reverse: it asks the kernel to place incoming bytes into a buffer in your memory. On Linux x86-64, `read` is syscall number 0.

- `%rax = 0` — syscall number for `read`
- `%rdi = 0` — file descriptor for standard input
- `%rsi` — address of the destination buffer
- `%rdx` — maximum number of bytes to read
- Return value in `%rax` — number of bytes actually read

This return value matters because `read` does not add a null terminator for us. It simply reports how many bytes it placed into memory. If we want to write those same bytes back out, we pass that returned count to `write`.

Hand-Write: Raw Echo with read and write

The following program reads up to 32 bytes from standard input and immediately writes back exactly the bytes it received. This is the simplest bare-syscall input/output program. For now, we assume the `read` succeeds; in Module 4 we will have the control flow machinery needed to check for EOF and errors.

```

        .section .bss
buffer: .skip 32

        .text
        .global _start
_start:
    mov $0, %rax          # syscall number: read
    mov $0, %rdi          # file descriptor: stdin
    lea buffer(%rip), %rsi # address of input buffer
    mov $32, %rdx         # max bytes to read
    syscall

    mov %rax, %rdx        # byte count for write
    mov $1, %rax          # syscall number: write
    mov $1, %rdi          # file descriptor: stdout
    lea buffer(%rip), %rsi # address of data to print
    syscall

    mov $60, %rax
    xor %rdi, %rdi
    syscall

```

Build and run:

```

$ as -o raw_echo.o raw_echo.s
$ ld -o raw_echo raw_echo.o
$ ./raw_echo
hello
hello

```

Key points:

- `.bss` reserves space for uninitialized writable data. Our buffer starts out zeroed, but `read` overwrites it with the bytes the user typed.
- After the `read` syscall, `%rax` no longer holds the syscall number. It now holds the *result*: the number of bytes read.
- `mov %rax, %rdx` reuses that returned byte count as the third argument to `write`, so we print exactly what was read.
- When reading from a terminal, input is usually line-buffered: the kernel delivers the bytes after you press Enter, including the newline character.

gcc Comparison: `printf("Hello")`

If you write the equivalent in C and compile with `gcc -Og -S` :

```
#include <stdio.h>
int main() {
    printf("Hello, world!\n");
    return 0;
}
```

The generated assembly calls the C library function `puts` (gcc optimizes `printf` with a simple string to `puts`). Where our hand-written version used a single `write` syscall plus an `exit` syscall (8 instructions total), the gcc version calls into the C standard library, which internally does much more setup. The library approach is more portable, but now you can see what it actually comes down to at the hardware level: loading register arguments and making a system call.

Lesson 5: The Data Section and Labels

Where Data Lives

So far, all of our values have been immediates baked directly into instructions (`mov $42, %rax`) or computed at runtime in registers. But real programs need to store data that persists across instructions — global variables, strings, arrays. This data lives in dedicated **sections** of the executable.

The Three Data Sections

An assembly program is divided into sections. The assembler and linker use these to lay out the final executable:

Directive	Section	Contents
<code>.text</code> or <code>.section .text</code>	Text	Executable instructions (read-only at runtime)
<code>.data</code> or <code>.section .data</code>	Data	Initialized read-write global variables
<code>.section .rodata</code>	Read-only data	Constants, string literals (read-only at runtime)
<code>.bss</code> or <code>.section .bss</code>	BSS	Uninitialized global variables (zero-filled at load time)

The `.bss` section is special: it takes up no space in the executable file on disk. The operating system simply allocates the requested amount of zero-filled memory when the program starts.

Data Directives

Inside the `.data` or `.rodata` sections, you use directives to declare values:

Directive	Size	Example
<code>.byte</code>	1 byte	<code>.byte 0xFF</code>
<code>.word</code>	2 bytes	<code>.word 1000</code>
<code>.long</code>	4 bytes	<code>.long 100000</code>
<code>.quad</code>	8 bytes	<code>.quad 5000000000</code>
<code>.ascii</code>	string (no null)	<code>.ascii "Hello"</code>
<code>.asciz</code>	string + null byte	<code>.asciz "Hello"</code>
<code>.space N</code> or <code>.skip N</code>	N zero bytes	<code>.space 100</code> (in <code>.bss</code>)

You can also declare multiple values on one line: `.quad 1, 2, 3, 4, 5` allocates five consecutive 8-byte values — this is how arrays are created in assembly.

Labels

A **label** is a name followed by a colon that marks a location in the program. Labels in the `.data` section name your variables; labels in `.text` name your code locations (jump targets, function entry points):

```
.section .data
x:    .long 10
y:    .long 25

.text
.global _start
_start:
    # code that uses x and y
```

The label `x:` refers to the memory address where the value 10 is stored. The label `_start:` refers to the address of the first instruction.

RIP-Relative Addressing

To access data stored in the `.data` or `.rodata` section, you use **RIP-relative addressing**:

```
mov x(%rip), %eax    # load the 4-byte value at label x into %eax
```

The `(%rip)` part tells the assembler to encode the address as an offset from the current instruction pointer. This is the standard way to access global data on x86-64 Linux — it produces position-independent code that works regardless of where the program is loaded in memory.

To *store* a value back:

```
mov %eax, x(%rip)    # store %eax into the 4-byte location at label x
```

To get the *address* of a label (rather than the value stored there), use `lea` :

```
lea x(%rip), %rax    # %rax now holds the memory address of x
```

Hand-Write: Adding Two Global Variables

Let's write a program that declares two integers in `.data` , adds them, and exits with the result:

```
.section .data
a:      .long 17
b:      .long 25

.text
.global _start
_start:
    mov a(%rip), %eax    # load a into %eax
    add b(%rip), %eax    # %eax = a + b = 42
    mov %eax, %edi       # exit code = result
    mov $60, %rax        # syscall: exit
    syscall
```

Build and run:

```
$ as -o data1.o data1.s
$ ld -o data1 data1.o
$ ./data1
$ echo $?
42
```

Notice that we used `.long` (4 bytes) and the corresponding 32-bit register names (`%eax` , `%edi`) with the `movl` / `addl` implicit sizing. If we had used `.quad` (8 bytes), we would use `%rax` and `%rdi` with `movq` / `addq` .

Hand-Write: A Mutable Variable

Here is a program that reads a value, modifies it, and stores it back:

```
        .section .data
counter: .quad 0

        .text
        .global _start
_start:
    mov counter(%rip), %rax    # load counter (0)
    add $10, %rax              # counter = 10
    add $32, %rax              # counter = 42
    mov %rax, counter(%rip)    # store back (optional here, but
shows the pattern)
    mov %rax, %rdi
    mov $60, %rax
    syscall
```

This pattern — load from memory, operate in registers, store back to memory — is the fundamental rhythm of assembly programming.

gcc Comparison: Global Variables

Consider this C program:

```
int x = 17;
int y = 25;

int main() {
    return x + y;
}
```

Compile with `gcc -Og -S` and you will see that `gcc` puts `x` and `y` in a data section and accesses them with the same `x(%rip)` pattern. The main difference is that `gcc` adds type metadata (e.g., `.size`, `.type`) and uses `.globl` to make the symbols visible. The actual instructions for loading and adding the values look very similar to what we wrote by hand.

Lesson 6: Arithmetic Operations

Computing with Values

Up to this point, we have mostly moved data around. Now we will discuss the instructions that actually compute new values. Arithmetic instructions take a size suffix (`b` , `w` , `l` , `q`); the suffix can be omitted when a register operand implies the size, but it is required when no operand does — for example, when adding an immediate directly to memory. They store their result into the destination operand, overwriting what was there.

Arithmetic Instruction Reference

The `add` and `sub` instructions work on both signed and unsigned values. `imul` and `idiv` are for signed integers; `mul` and `div` are for unsigned.

Instruction	Effect	Description
<code>add S, D</code>	$D + S \rightarrow D$	Adds S to D. Result stored in D.
<code>sub S, D</code>	$D - S \rightarrow D$	Subtracts S from D. Note: this is D minus S, not S minus D.
<code>imul S, D</code>	$D \times S \rightarrow D$	Signed multiply. Two-operand form: result goes in D.
<code>idiv S</code>	$\text{\%rdx:\%rax} / S \rightarrow \text{\%rax}$ $\text{\%rdx:\%rax} \bmod S \rightarrow \text{\%rdx}$	Signed divide. Takes one operand; divides the 128-bit value <code>\%rdx:\%rax</code> by S. Quotient in <code>\%rax</code> , remainder in <code>\%rdx</code> . You must sign-extend <code>\%rax</code> into <code>\%rdx</code> with <code>cqto</code> first.
<code>inc D</code>	$D + 1 \rightarrow D$	Increment by one. Equivalent to <code>x++</code> in C.
<code>dec D</code>	$D - 1 \rightarrow D$	Decrement by one. Equivalent to <code>x--</code> in C.
<code>neg D</code>	$-D \rightarrow D$	Arithmetic negation. Same as <code>x = -x</code> in C.

Division in Detail: `idiv` and `cqto`

Division is more complex than the other operations because it uses **implicit operands**. The `idiv` instruction always divides the 128-bit value formed by `\%rdx:\%rax` (high bits in `\%rdx`, low bits in `\%rax`) by the single operand. Before using `idiv`, you must sign-extend `\%rax` into `\%rdx` using `cqto` (Convert Quadword to Octword — "octword" is the 128-bit value `\%rdx:\%rax`):


```
# Compute 17 / 5
mov $17, %rax      # dividend
cqto               # sign-extend %rax into %rdx:%rax
mov $5, %rcx
idiv %rcx           # %rax = 3 (quotient), %rdx = 2 (remainder)
```

Common mistake: Forgetting `cqto` before `idiv`. If `%rdx` contains leftover data from a previous computation, you will get a wrong answer or a floating-point exception. Always use `cqto` (or `cld` for 32-bit division) immediately before `idiv`.

Hand-Write: $(a + b) * c$

Let's compute an expression using data from the `.data` section:

```
.section .data
a:    .long 3
b:    .long 7
c:    .long 7

.text
.global _start
_start:
    mov a(%rip), %eax    # %eax = 3
    add b(%rip), %eax    # %eax = 3 + 7 = 10
    imull c(%rip), %eax  # %eax = 10 * 7 = 70
    mov %eax, %edi       # exit code = 70
    mov $60, %rax
    syscall
```

```
$ as -o arithmetic.o arithmetic.s
$ ld -o arithmetic arithmetic.o
$ ./arithmetic
$ echo $?
70
```

Hand-Write: Division with Remainder

This program divides 100 by 7 and exits with the remainder:

```

        .text
        .global _start
_start:
        mov $100, %rax      # dividend
        cqto                # sign-extend into %rdx:%rax
        mov $7, %rcx        # divisor
        idiv %rcx           # %rax = 14 (quotient), %rdx = 2
        (remainder)
        mov %rdx, %rdi      # exit with remainder
        mov $60, %rax
        syscall

```

```

$ echo $?
2

```

gcc Comparison: The `leaq` Trick

We introduced `leaq` in Lesson 4 as a way to compute addresses. But `gcc` frequently uses `leaq` for pure arithmetic, because the address-computation hardware is fast and can do addition and multiplication by small constants in a single instruction.

For example, if `gcc` needs to compute $x * 5$ (where x is in `%rdx`), it might generate:

```
leaq (%rdx, %rdx, 4), %rdx    # %rdx = %rdx + %rdx * 4 = %rdx * 5
```

Recall the address formula: `base + index × scale`. Here the base is `%rdx`, the index is also `%rdx`, and the scale is 4, giving $x + x*4 = x*5$. This is faster than using `imul`.

Similarly, $x * 3$:

```
leaq (%rdx, %rdx, 2), %rdx    # %rdx = %rdx + %rdx * 2 = %rdx * 3
```

And $x * 2 + 10$:

```
leaq 10(%rdx, %rdx), %rdx     # %rdx = 10 + %rdx + %rdx = 2*x + 10
```

Note that for `leaq` the size suffix applies to the destination. In 64-bit mode the address-computation registers in the source default to 64 bits as well, so in practice the registers inside the source operand are written as `%rax`, `%rbx`, etc. (Strictly speaking, an address-size override prefix can switch them to 32 bits, but you won't encounter that in this course.)

Lesson 7: Logical and Shift Operations

Bit-Level Operations

In Module 1 you learned about bitwise operations in C: `&`, `|`, `^`, `~`, `<<`, `>>`.

Assembly provides the same operations as instructions. These are essential for tasks like masking bits, setting flags, and fast multiplication/division by powers of two.

Logical Instruction Reference

Instruction	Effect	Description
<code>not D</code>	$\sim D \rightarrow D$	Bitwise NOT (flip every bit). One operand.
<code>and S, D</code>	$D \& S \rightarrow D$	Bitwise AND. Useful for masking bits.
<code>or S, D</code>	$D S \rightarrow D$	Bitwise OR. Useful for setting bits.
<code>xor S, D</code>	$D \wedge S \rightarrow D$	Bitwise XOR. <code>xor %rax, %rax</code> is the standard idiom for zeroing a register.

Shift Instruction Reference

Instruction	Effect	Description
<code>shl k, D / sal k, D</code>	$D \ll k \rightarrow D$	Shift left by <code>k</code> bits. Fills with zeros on the right. <code>shl</code> and <code>sal</code> are identical.
<code>shr k, D</code>	$D \gg k \rightarrow D$ (logical)	Logical shift right. Fills with zeros on the left. For unsigned values.
<code>sar k, D</code>	$D \gg k \rightarrow D$ (arithmetic)	Arithmetic shift right. Fills with copies of the sign bit. For signed values.

The shift amount `k` can be an immediate value or the register `%cl` (the low byte of `%rcx`). It cannot be any other register — this is a hardware restriction.

Hand-Write: Bit Manipulation

This program demonstrates extracting the low byte with `and` and multiplying by 8 with `shl`:

```
.text
.global _start
_start:
    mov $0x1234, %rax
    and $0xFF, %rax      # extract low byte: %rax = 0x34 = 52
    shl $3, %rax         # multiply by 8: %rax = 52 * 8 =
416                      # 416 mod 256 = 160
    mov %rax, %rdi
    mov $60, %rax
    syscall
```

```
$ echo $?
160
```

The `and $0xFF` operation masks off everything except the lowest 8 bits — this is the assembly equivalent of `x & 0xFF` in C. Shifting left by 3 is equivalent to multiplying by $2^3 = 8$.

If the masking pattern feels familiar: it's the same trick from Module 1 Lesson 6 (the motor-flag example). Testing the "medium speed" bit there was `flags & 0x04` in C; in assembly it's `and $0x04, %rax` followed by checking whether the result is zero. Setting a flag was `flags |= 0x04` in C; in assembly it's `or $0x04, %rax`. The bitwise operators you learned for C are the same operators here, with the same bit patterns — only the syntax has changed.

Hand-Write: Swap Without a Temporary

The classic XOR swap trick works in assembly too:

```
.text
.global _start
_start:
    mov $10, %rax      # a = 10
    mov $25, %rbx      # b = 25
    xor %rbx, %rax     # a = a ^ b
    xor %rax, %rbx     # b = a ^ b (now b = original a =
10)
    xor %rbx, %rax     # a = a ^ b (now a = original b =
25)
    mov %rax, %rdi     # exit with a = 25
    mov $60, %rax
    syscall
```

```
$ echo $?
25
```

In practice you would just use `mov` with a spare register, but this demonstrates how XOR works at the instruction level.

gcc Comparison

When you write bitwise operations in C, `gcc` translates them directly to the corresponding assembly instructions:

C expression	gcc output (typical)
<code>x & 0xFF</code>	<code>andl \$255, %eax</code>
<code>x 0x80</code>	<code>orl \$128, %eax</code>
<code>x ^ y</code>	<code>xorl %edx, %eax</code>
<code>x << 3</code>	<code>sall \$3, %eax</code>
<code>x >> 1</code> (signed)	<code>sarl \$1, %eax</code>
<code>x >> 1</code> (unsigned)	<code>shrl \$1, %eax</code>

Notice that `gcc` chooses `sar` vs `shr` based on whether the C variable is signed or unsigned. This is exactly what you learned in Module 1 about logical vs. arithmetic right shift.

Lesson 8: The Stack, Push, and Pop

What is the Program Stack?

The **program stack** (or "call stack") is a region of memory that the program uses for temporary storage: local variables, saved register values, return addresses, and function arguments that don't fit in registers. It is the backbone of function calls.

The stack is a **LIFO** (Last In, First Out) data structure, like a stack of plates. You **push** a value onto the top and **pop** a value off the top. The "top" of the stack is always the address stored in the **stack pointer** register, `%rsp`.

The Stack Grows Downward

On x86-64 (and most modern architectures), the stack grows **toward lower memory addresses**. This is counterintuitive at first:

- Pushing a value **decreases** `%rsp` (moves it to a lower address).
- Popping a value **increases** `%rsp` (moves it to a higher address).

If you imagine memory as a vertical diagram with low addresses at the bottom and high addresses at the top, the stack grows *downward* from the top of the address space.

push and pop

The `push` instruction does two things in one:

1. Subtract 8 from `%rsp` (make room for a 64-bit value)
2. Write the operand to the memory address now pointed to by `%rsp`

In other words, `pushq %rax` is equivalent to:

```
subq $8, %rsp
movq %rax, (%rsp)
```

The `pop` instruction reverses this:

1. Read the value at `(%rsp)` into the destination operand
2. Add 8 to `%rsp`

So `popq %rax` is equivalent to:

```
movq (%rsp), %rax
addq $8, %rsp
```

Note: In normal 64-bit code, `push` and `pop` are typically used with 64-bit (8-byte) values. There are also 16-bit forms (such as `pushw` / `popw`), but there is no 32-bit `push` / `pop` form in x86-64 mode. The System V ABI requires the stack to be aligned to 16-byte boundaries before a `call` instruction.

Manual Stack Manipulation

Sometimes you need more stack space than `push` / `pop` provides — for instance, to allocate room for several local variables at once. You can do this by directly adjusting `%rsp`:

```
# Allocate 32 bytes of local variable space
sub $32, %rsp

# Use the space via offsets from %rsp
movq $10, (%rsp)      # first local (bytes 0-7)
movq $20, 8(%rsp)     # second local (bytes 8-15)
movq $30, 16(%rsp)    # third local (bytes 16-23)
movq $40, 24(%rsp)    # fourth local (bytes 24-31)

# Deallocate when done
add $32, %rsp
```

This is exactly what compilers do to create space for local variables. We will see this pattern in detail in Module 4 when we cover function stack frames.

Hand-Write: Using the Stack for Temporary Storage

Here is a complete program that uses `push` and `pop` to save and restore register values. It computes two results, saves the first on the stack while computing the second, then retrieves it:

```

        .text
        .global _start
_start:
    # Compute first result: 5 + 7 = 12
    mov $5, %rax
    add $7, %rax          # %rax = 12
    push %rax             # save 12 on the stack

    # Compute second result: 10 + 20 = 30
    mov $10, %rax
    add $20, %rax         # %rax = 30

    # Now %rax = 30 and the stack holds 12
    pop %rbx              # %rbx = 12 (restored from stack)
    add %rbx, %rax        # %rax = 30 + 12 = 42

    mov %rax, %rdi
    mov $60, %rax
    syscall

```

```

$ echo $?
42

```

Without the stack, we would have needed a third register to hold the intermediate result. With only 16 registers, the stack provides essentially unlimited temporary storage — at the cost of being slower than register access.

Hand-Write: Stack-Based Local Variables

This program allocates space for two local variables on the stack, stores values, reads them back, and computes a result:

```

        .text
        .global _start
_start:
        sub $16, %rsp           # allocate 16 bytes for 2 locals

        movq $17, (%rsp)        # local1 = 17
        movq $25, 8(%rsp)       # local2 = 25

        mov (%rsp), %rax        # load local1
        add 8(%rsp), %rax        # %rax = local1 + local2 = 42

        add $16, %rsp           # deallocate locals

        mov %rax, %rdi
        mov $60, %rax
        syscall

```

```

$ echo $?
42

```

gcc Comparison: Function Prologue and Epilogue

When `gcc` compiles a function at `-O0` (no optimization), it generates a standard pattern called the **prologue** and **epilogue**. You can see this yourself by compiling a simple function:

```

$ cat simple.c
int add(int a, int b) { return a + b; }

$ gcc -O0 -S -o simple.s simple.c
$ cat simple.s      # look for the add: label

```

The generated assembly will follow this pattern:

```

# Prologue (at function entry)
pushq   %rbp           # save old base pointer
movq    %rsp, %rbp     # set base pointer to current stack top
subq    $16, %rsp      # allocate space for local variables

# ... function body ...

# Epilogue (at function exit)
leave   # equivalent to: movq %rbp, %rsp; popq
%rbp
ret     # return to caller

```

The `leave` instruction is a shorthand that restores `%rsp` and `%rbp` in one step. We will cover the full calling convention — including why `%rbp` is saved and what `ret` does — in Module 4.

With optimization enabled (`-Og` or higher), `gcc` often skips the prologue/epilogue for simple "leaf" functions that don't call other functions, since it can keep everything in registers.

Lesson 9: Putting It All Together

A Complete Program

In this capstone lesson we combine everything from Module 3 — the build process, registers, data movement, the data section, arithmetic, logical operations, and the stack — into a complete, working assembly program. We also introduce `readInt` and `writeInt`, two helper routines that let our programs read and print integer values.

I/O Helper Routines: `readInt` and `writeInt`

Up to now, our only output mechanism has been the exit code (limited to 0–255). For real programs, we need to read input from the keyboard and print results to the screen. Writing I/O from scratch requires parsing ASCII digits — doable but tedious. Instead, we will use two pre-written helper routines:

- `readInt` — Reads decimal digits from standard input and returns the integer in `%rax`. For the simple course helper, enter one nonempty line of nonnegative decimal digits followed by Enter; stay within the signed 64-bit range.
- `writeInt` — Takes an integer in `%rdi`, converts it to decimal ASCII, and writes it to standard output. The helper in the Fall 2026 source materials does not append a newline; print one separately if needed.

These are provided as separate `.s` files (`readInt.s` and `writeInt.s`) on the course D2L page; download both before working through the example below. To use them, you assemble each file separately and link them all together:

```
$ as -o readInt.o readInt.s
$ as -o writeInt.o writeInt.s
$ as -o myprogram.o myprogram.s
$ ld -o myprogram myprogram.o readInt.o writeInt.o
```

What the Helpers Are Hiding

The kernel only provides byte-oriented primitives. A bare `read` syscall copies bytes from file descriptor 0 into a buffer and returns a byte count in `%rax`. A bare `write` syscall copies bytes from a buffer to file descriptor 1. That means if the user types `42` and presses Enter, the kernel gives us the ASCII bytes `'4'`, `'2'`, and `'\n'` — not the integer value 42.

`readInt` and `writeInt` are small layers on top of those raw syscalls:

- `readInt` performs a raw `read`, then converts decimal ASCII digits into a binary integer in `%rax`.
- `writeInt` converts a binary integer into ASCII digits in a buffer, then performs a raw `write`.

So when you call these helpers, you are not avoiding syscalls — you are reusing them through a cleaner interface.

In your source file, you declare them as external symbols:

```
.extern readInt  
.extern writeInt
```

Then you can call them like functions: put arguments in the right registers and use `call`.

The simple `readInt` helper assumes a complete newline-terminated input arrives in one read. It does not validate signs, nondigits, overflow, EOF, or read errors. Use the interactive inputs shown here; it is not a general-purpose parser for redirected input or arbitrary user data.

Multi-File Assembly

To make a label visible to other files, use `.global`:

```
.global myfunction      # other files can call myfunction
```

To use a label defined in another file, use `.extern`:

```
.extern readInt         # readInt is defined elsewhere
```

The linker resolves these references when combining the `.o` files into a final executable.

The `call` and `ret` Instructions (Preview)

To call a function, use the `call` instruction. It pushes the return address onto the stack and jumps to the target label. The `ret` instruction pops the return address and jumps back. We will cover functions in full detail in Module 4, but we need `call` now to use our I/O helpers.

```
call readInt      # call the readInt function; result in %rax
call writeInt     # call writeInt; value to print should be
in %rdi
```

Stack alignment: The [System V AMD64 ABI](#) requires `%rsp` to be 16-byte aligned at the point of a `call` instruction (see §3.2.2). Since `call` pushes an 8-byte return address, `%rsp` will be 8 bytes off from 16-byte alignment at the entry of the called function. The callee's prologue (e.g., `push %rbp`) often restores 16-byte alignment for local stack usage. In a `_start` program, the kernel sets the initial `%rsp` to a 16-byte aligned value (per the ABI §3.4, "Initial Stack and Register State"), so you can call functions directly at the start of the program. But if you `push` one 8-byte value and then want to make another call, you must add another 8 bytes of padding (or reserve 16 bytes total) before that call.

Echo Program: Read and Print One Integer

Let's start simple — read a number and print it back:

```
.extern readInt
.extern writeInt

.text
.global _start
_start:
    call readInt      # read integer into %rax
    mov %rax, %rdi    # move result to %rdi (argument for
writeInt)
    call writeInt     # print it

    mov $60, %rax
    xor %rdi, %rdi
    syscall
```

```
$ ./echoInt  
42  
42
```

Complete Program: Read Two Integers, Compute, and Print

Now let's build a program that reads two integers, adds them, multiplies the sum by 3, and prints the result. This uses everything from the module:


```

        .extern readInt
        .extern writeInt

        .section .data
result: .quad 0

        .section .rodata
prompt1: .asciz "Enter first number: "
p1len = . - prompt1 - 1
prompt2: .asciz "Enter second number: "
p2len = . - prompt2 - 1
label:   .asciz "Result: "
lablen = . - label - 1

        .text
        .global _start
_start:
    # Print prompt 1
    mov $1, %rax          # write syscall
    mov $1, %rdi          # stdout
    lea prompt1(%rip), %rsi
    mov $p1len, %rdx
    syscall

    # Read first number
    call readInt
    push %rax              # save first number on stack

    # Keep %rsp 16-byte aligned before the next call
    sub $8, %rsp

    # Print prompt 2
    mov $1, %rax
    mov $1, %rdi
    lea prompt2(%rip), %rsi
    mov $p2len, %rdx
    syscall

    # Read second number
    call readInt
    add $8, %rsp           # discard alignment padding
    mov %rax, %rbx         # %rbx = second number

    # Compute: (first + second) * 3
    pop %rax               # restore first number from stack
    add %rbx, %rax         # %rax = first + second
    imul $3, %rax, %rax    # %rax = sum * 3

```

```

    # Store result
    mov %rax, result(%rip) # (optional here, but shows the
pattern)

    # Print label
    push %rax                # save result (the write syscall
will overwrite %rax)
    mov $1, %rax
    mov $1, %rdi
    lea label(%rip), %rsi
    mov $lablen, %rdx
    syscall

    # Print result
    pop %rdi                 # restore result into %rdi (argument
for writeInt)
    call writeInt

    # Exit
    mov $60, %rax
    xor %rdi, %rdi
    syscall

```

```

$ ./compute
Enter first number: 10
Enter second number: 4
Result: 42

```

What this program uses from the module

- **Sections:** `.data` for mutable storage, `.rodata` for string constants, `.text` for code
- **Data directives:** `.quad`, `.asciz`, computed lengths with `. - label - 1` when using `.asciz`
- **Registers:** `%rax`, `%rbx`, `%rdi`, `%rsi`, `%rdx`, `%rsp`
- **Data movement:** `mov`, `lea`, RIP-relative addressing
- **Arithmetic:** `add`, `imul` (three-operand form)
- **Stack:** `push` / `pop` to save values across function calls
- **System calls:** direct `write` and `exit` syscalls in the main program; `readInt` / `writeInt` themselves are built on top of raw `read` / `write`
- **Multi-file linking:** `.extern`, `call`, `ret`

gcc Comparison: The C Equivalent

The equivalent C program would be:

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    int result = (a + b) * 3;
    printf("Result: %d\n", result);
    return 0;
}
```

If you compile this with `gcc -Og -S`, you will see calls to `printf` and `scanf`, which internally make the same `write` and `read` system calls our program uses directly for prompts and indirectly through `readInt` / `writeInt` — but with thousands of lines of library code in between for formatting, buffering, and error handling. Our hand-written version does the same job in about 30 instructions.

What's Next

In Module 4, we will add **control flow** (comparisons, branches, loops, switch statements) and **functions** (calling conventions, stack frames, recursion). By the end of that module, you will be able to write assembly programs as complex as anything you would write in C.

Lesson 10: Debugging with GDB

Why You Need a Debugger

So far, our main debugging tool has been `echo $?` — running the program and checking the exit code. This works for tiny programs, but it is painfully limited. If a 50-line assembly program produces the wrong answer, all `echo $?` tells you is *what* the answer was, not *where* things went wrong.

GDB (the GNU Debugger) lets you pause your program at any point, inspect every register and memory location, and step through instructions one at a time. It is the single most important tool for understanding what your assembly code is actually doing.

Building with Debug Symbols

For GDB to show you source-level information (source lines and line numbers), you need to include **debug symbols** when you assemble. Add the `-g` flag:

```
$ as -g -o program.o program.s
$ ld -o program program.o
```

If you are using `gcc` instead:

```
$ gcc -g -nostdlib -o program program.s
```

Without `-g`, GDB can still use label names retained in the ordinary symbol table, such as `_start`, and you can disassemble and step instructions. What is missing is source-line information. Stripping the symbol table can remove those ordinary names as well.

Starting GDB

Launch GDB with your executable as the argument:

```
$ gdb ./program
```

You will see a welcome message and then the `(gdb)` prompt. Your program is **not running yet** — GDB has loaded it but is waiting for your instructions.

```
GNU gdb (Ubuntu 12.1-0ubuntu1) 12.1
...
Reading symbols from ./program...
(gdb)
```

Essential Commands

Here are the commands you will use constantly. You can abbreviate most of them to their first letter or two.

Setting Breakpoints

A **breakpoint** tells GDB to pause execution when it reaches a specific location. You can break on a label name or an address:

```
(gdb) break _start
Breakpoint 1 at 0x401000: file program.s, line 5.
(gdb) break myfunction
Breakpoint 2 at 0x401030: file program.s, line 20.
```

You can also break at a specific line number:

```
(gdb) break program.s:12
```

Running the Program

The `run` command starts execution. The program runs until it hits a breakpoint, crashes, or finishes:

```
(gdb) run
Starting program: /home/user/program

Breakpoint 1, _start () at program.s:5
5          mov $1, %rax
```

Now the program is paused at the first instruction of `_start`.

Stepping Through Instructions

Once paused, you can execute one instruction at a time:

- `stepi` (or `si`) — Execute the next single machine instruction. If the instruction is a `call`, step **into** the called function.
- `nexti` (or `ni`) — Execute the next single machine instruction. If the instruction is a `call`, execute the entire called function and stop after it returns (step **over**).

```
(gdb) stepi
6          call readInt
(gdb) nexti
7          mov %rax, %rdi
```

Use `stepi` when you want to see what happens inside a function call. Use `nexti` when you trust the function and just want to get past it.

Continuing and Quitting

- `continue` (or `c`) — Resume running until the next breakpoint or program exit.
- `quit` (or `q`) — Exit GDB. If the program is still running, GDB will ask for confirmation.

Inspecting State

The real power of GDB is in inspecting what your program is doing.

Registers

View all general-purpose registers:

```
(gdb) info registers
rax          0xa          10
rbx          0x0          0
rcx          0x0          0
rdx          0x0          0
rsi          0x0          0
rdi          0x0          0
rbp          0x0          0x0
rsp          0x7fffffff398 0x7fffffff398
...
rip          0x401005      0x401005 <_start+5>
```

View a single register:

```
(gdb) print $rax
$1 = 10
(gdb) print/x $rax
$2 = 0xa
```

The `/x` format prints in hexadecimal. Other useful formats: `/d` (signed decimal), `/u` (unsigned decimal), `/t` (binary).

Examining Memory

The `x` (examine) command reads memory at a given address. The syntax is `x/NFU address`, where:

- **N** = number of units to display
- **F** = format (`x` hex, `d` decimal, `c` character, `s` string, `i` instruction)
- **U** = unit size (`b` byte, `h` halfword/2 bytes, `w` word/4 bytes, `g` giant/8 bytes)

Examples:

```

# Show 8 bytes in hex starting at the address in %rsp
(gdb) x/8xb $rsp
0x7fffffff398: 0xa0    0xe3    0xff    0xff    0xff    0x7f
0x00    0x00

# Show 4 quad-words (8 bytes each) at the top of the stack
(gdb) x/4xg $rsp
0x7fffffff398: 0x00007fffffff3a0    0x0000000000000001
0x7fffffff3a8: 0x00007fffffff5b2    0x0000000000000000

# Show a string at a label
(gdb) x/s &prompt1
0x402000:      "Enter first number: "

# Disassemble instructions at the current location
(gdb) x/5i $rip
=> 0x401005 <_start+5>:  call    0x401080 <readInt>
    0x40100a <_start+10>: push    %rax
    0x40100b <_start+11>: sub     $0x8,%rsp
    0x40100f <_start+15>: mov     $0x1,%rax
    0x401016 <_start+22>: mov     $0x1,%rdi

```

Walkthrough: Debugging the Compute Program

Let's debug the "compute" program from Lesson 9 step by step. Recall that it reads two integers, adds them, multiplies the sum by 3, and prints the result.

```

$ as -g -o compute.o compute.s
$ as -o readInt.o readInt.s
$ as -o writeInt.o writeInt.s
$ ld -o compute compute.o readInt.o writeInt.o
$ gdb ./compute

```

Step 1: Set breakpoints and run

Rather than stepping through the prompt-printing and I/O code one instruction at a time, we set breakpoints at the interesting points — right after each `readInt` call and at the arithmetic section:


```
(gdb) break _start
Breakpoint 1 at 0x401000: file compute.s, line 19.
(gdb) break compute.s:27
Breakpoint 2 at 0x401023: file compute.s, line 27.
(gdb) break compute.s:45
Breakpoint 3 at 0x401052: file compute.s, line 45.
(gdb) run
Starting program: /home/user/compute

Breakpoint 1, _start () at compute.s:19
19             mov $1, %rax
```

Step 2: Run to the first read and check the result

We use `continue` to skip ahead to breakpoint 2, which is just after the first `readInt` call. GDB will print the prompt and wait for our input along the way:

```
(gdb) continue
Continuing.
Enter first number: 10

Breakpoint 2, _start () at compute.s:27
27             push %rax
```

Now check what `readInt` returned:

```
(gdb) print $rax
$1 = 10
```

Good — `%rax` holds 10 as expected.

Step 3: Run to the arithmetic and step through it

```
(gdb) continue
Continuing.
Enter second number: 4

Breakpoint 3, _start () at compute.s:45
45             pop %rax
```

Now we are at the computation section. Let's step through it one instruction at a time with `nexti`:

```
(gdb) nexti
46          add %rbx, %rax
(gdb) print $rax
$2 = 10
(gdb) print $rbx
$3 = 4
```

The first number was restored from the stack into `%rax` , and the second number is in `%rbx` . After the `add` :

```
(gdb) nexti
47          imul $3, %rax, %rax
(gdb) print $rax
$4 = 14
```

After the `imul` :

```
(gdb) nexti
(gdb) print $rax
$5 = 42
```

The computation is correct: $(10 + 4) \times 3 = 42$.

Step 4: Let it finish

```
(gdb) continue
Continuing.
Result: 42
[Inferior 1 (process 12345) exited normally]
(gdb) quit
```

This workflow — set breakpoints at the interesting locations, `continue` between them, and `nexti` through the tricky parts — is how you will typically use GDB in practice.

Quick Reference

Command	Short	Description
<code>break <label></code>	<code>b</code>	Set a breakpoint at a label or address
<code>run</code>	<code>r</code>	Start (or restart) the program
<code>stepi</code>	<code>si</code>	Execute one instruction (step into calls)
<code>nexti</code>	<code>ni</code>	Execute one instruction (step over calls)
<code>continue</code>	<code>c</code>	Resume execution until next breakpoint
<code>info registers</code>	<code>i r</code>	Show all general-purpose registers
<code>print \$reg</code>	<code>p</code>	Print the value of a register or expression
<code>print/x \$reg</code>	<code>p/x</code>	Print in hexadecimal
<code>x/NFU addr</code>	Examine memory (N units, Format, Unit size)	
<code>disassemble</code>	<code>disas</code>	Show disassembly of the current function
<code>info breakpoints</code>	<code>i b</code>	List all breakpoints
<code>delete N</code>	<code>d N</code>	Delete breakpoint number N
<code>layout asm</code>	Switch to TUI mode showing registers +	
<code>layout regs</code>		

Command	Short	Description
	assembly	
quit	q	Exit GDB

Tips

- **Pressing Enter** with no command repeats the last command. This is especially useful with `stepi` — just keep pressing Enter to step one instruction at a time.
- **layout asm** opens a split-screen view that shows the assembly code with an arrow pointing to the current instruction. `layout regs` adds a register display at the top. Press `Ctrl-x a` to exit TUI mode.
- **Watchpoints:** `watch $rax` will pause the program whenever `%rax` changes. Useful for tracking down where a value gets corrupted. (Note: register watchpoints are slow — GDB must single-step internally since hardware watchpoints only work on memory addresses. For small programs this is fine.)
- **Conditional breakpoints:** `break _start if $rdi == 5` pauses only when the condition is true.

Connection to picoCTF: Many of the discussion exercises use picoCTF challenges that require reading assembly, setting breakpoints, and inspecting memory in GDB. The commands in this lesson — especially `break`, `run`, `stepi`, `info registers`, and `x` — are exactly what you will use in those exercises.

Module 3 Summary

Wrap up

In this module, you wrote x86-64 assembly programs by hand using the `as` and `ld` toolchain, and compared your programs with the output of `gcc -S`. You progressed from a 3-line exit program to complete programs that use bare `read / write` syscalls, compute results using arithmetic and stack operations, and print output.

Key Takeaways

- The build process has four stages: preprocessing, compiling, assembling, and linking. You can write assembly directly in `.s` files and use `as` and `ld` to bypass the compiler entirely.
- **Makefiles** automate the build process. A Makefile specifies targets, prerequisites, and recipes so that `make` only rebuilds files whose dependencies have changed. Pattern rules (`%.o: %.s`) and automatic variables (`$@` , `$<` , `$^`) keep Makefiles concise even for multi-file projects.
- x86-64 processors have 16 general-purpose registers, each accessible at 64-bit, 32-bit, 16-bit, or 8-bit widths. AT&T syntax uses `%` for registers, `$` for immediates, and size suffixes (`b` / `w` / `l` / `q`).
- The `mov` instruction copies data between registers and memory but cannot move memory-to-memory. The `lea` instruction computes addresses and is also used for fast arithmetic.
- Global data lives in `.data` (read-write), `.rodata` (read-only), and `.bss` (zero-initialized) sections. Data is accessed using RIP-relative addressing: `label(%rip)` .
- Arithmetic instructions (`add` , `sub` , `imul` , `idiv`) and logical/shift instructions (`and` , `or` , `xor` , `shl` , `shr`) operate on registers and memory. Division uses implicit `%rdx:%rax` operands and requires `cqto` for sign extension.
- Linux syscalls use `%rax` for the syscall number and return value. For bare byte I/O, `read` (0) reads from `stdin` into a buffer, and `write` (1) writes bytes from a buffer to `stdout` .
- The program stack grows downward. `push` / `pop` and manual `sub` / `add` on `%rsp` provide temporary storage. Multi-file programs use `.global` / `.extern` and link with `ld` .

Additional resources

All links open in a new tab.

[Course _start Assembly Syntax Reference](#) [internal reference page]

[Object Code Slides set](#) [link]

Object Code - Machine Level Programming Basics [[link to video lecture from Carnegie Mellon University](#)]

Module 4 Overview

In Module 4: x86-64 Assembly — Control, Functions, and Data, you will continue writing assembly programs by hand while learning how the processor handles control flow, function calls, and structured data. This module builds directly on the fundamentals from Module 3.

We begin with condition codes, comparisons, and jump instructions, then use them to implement `if` statements, loops, and `switch` statements. Next, we write functions in assembly: learning the calling convention, the stack frame layout, recursive function calls, and how to wrap bare `read / write` syscalls inside reusable functions. We also look at how arrays and structs are laid out in memory, examine buffer overflow vulnerabilities and their defenses, and conclude with floating point operations. Each topic follows the "write it by hand first, then see how gcc does it" approach.

Learning objectives

At the end of this learning module, you will be able to:

1. Explain how condition codes are set by `cmp` and `test`, and use them with `set` and jump instructions
2. Write assembly programs that implement `if` statements, loops, and `switch` statements
3. Write assembly functions using the System V AMD64 calling convention (`call`, `ret`, argument registers)
4. Distinguish the System V function calling convention from the Linux syscall convention, and write simple syscall wrapper functions
5. Identify the components of a stack frame and explain the function prologue/epilogue
6. Explain callee-saved vs. caller-saved registers and use them correctly in hand-written functions
7. Write recursive assembly functions
8. Describe how arrays and structures are laid out in memory, including alignment and padding

9. Explain buffer overflow attacks and the defenses against them (ASLR, stack canaries, NX bit)
10. Describe how floating point values are handled using XMM registers and SSE2 instructions

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below
2. Read Lessons 1 to 9, work through the hand-written assembly examples, and complete the embedded practice problems in the lessons that include them
3. Take the module quiz
4. Review additional resources on the Module 4 Summary page

Reading

- Bryant & O'Hallaron: Chapter 3, sections 3.6 - 3.11
- Dive into Systems: Chapters 6 & 7

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: Condition Codes, Comparisons, and Jumps

How the CPU Makes Decisions

High-level languages have `if`, `while`, and `for`. The CPU has none of these. Instead, it has **condition codes** (single-bit flags set by arithmetic operations) and **jump instructions** that branch based on those flags. All control flow in assembly — every `if`, every loop, every `switch` — is built from these two primitives.

The Condition Code Flags

The CPU maintains six arithmetic flags that are updated automatically by most arithmetic and logical instructions. The first four rows are the ones you'll use for control flow; `PF` and `AF` are included for completeness. We'll meet `PF` again briefly when we cover floating-point comparisons in Lesson 9.

Flag	Name	Set when...
CF	Carry Flag	The operation produced a carry out of (or borrow into) the most significant bit. Indicates unsigned overflow.
ZF	Zero Flag	The result of the operation is zero.
SF	Sign Flag	The result is negative (most significant bit is 1).
OF	Overflow Flag	The operation caused a two's complement (signed) overflow.
PF	Parity Flag	The low byte of the result has an even number of 1 bits. Used by floating-point comparison instructions to flag NaN results.
AF	Auxiliary Carry Flag	Carry out of bit 3 (used by BCD instructions; rarely seen in modern code).

These flags are set as a *side effect* of instructions like `add` , `sub` , `and` , `xor` , etc. The `lea` instruction does **not** set condition codes (it is an address computation, not an arithmetic operation).

Setting Flags Without Storing a Result: `cmp` and `test`

Often you want to compare two values without changing either one. Two special instructions do this:

Instruction	Computes	Description
<code>cmp S, D</code>	$D - S$ (result discarded)	Subtracts S from D, sets flags, but does not store the result. Used to compare two values.
<code>test S, D</code>	$D \& S$ (result discarded)	Bitwise AND of S and D, sets flags, but does not store the result. Used to test individual bits.

Operand order matters for `cmp` : `cmp %rbx, %rax` computes `%rax - %rbx` .

The flags reflect the relationship of the *second* operand to the *first*. So after `cmp %rbx, %rax` , a "jump if greater" instruction checks whether `%rax > %rbx` . This is confusing — think of it as "compare %rax against %rbx."

The `set` Instructions

The `set` family reads the condition flags and stores a 0 or 1 in a single-byte register:

Instruction	Condition	Description
<code>sete D</code> / <code>setz D</code>	$ZF = 1$	Equal / zero
<code>setne D</code> / <code>setnz D</code>	$ZF = 0$	Not equal / not zero
<code>sets D</code>	$SF = 1$	Negative
<code>setns D</code>	$SF = 0$	Non-negative
<code>setg D</code>	$\sim(SF \wedge OF) \ \& \ \sim ZF$	Greater (signed)
<code>setge D</code>	$\sim(SF \wedge OF)$	Greater or equal (signed)
<code>setl D</code>	$SF \wedge OF$	Less (signed)
<code>setle D</code>	$(SF \wedge OF) \mid ZF$	Less or equal (signed)
<code>seta D</code>	$\sim CF \ \& \ \sim ZF$	Above (unsigned greater)
<code>setae D</code>	$\sim CF$	Above or equal (unsigned)
<code>setb D</code>	CF	Below (unsigned less)
<code>setbe D</code>	$CF \mid ZF$	Below or equal (unsigned)

The destination is one byte: either an 8-bit register (e.g., `%al` , `%cl` , `%r8b`) or a byte in memory.

Jump Instructions

Jump instructions change the program counter (`%rip`) to a target label, either unconditionally or based on condition flags:

Instruction	Condition	Description
<code>jmp label</code>	(always)	Unconditional jump
<code>je label / jz label</code>	$ZF = 1$	Jump if equal / zero
<code>jne label / jnz label</code>	$ZF = 0$	Jump if not equal / not zero
<code>js label</code>	$SF = 1$	Jump if negative
<code>jns label</code>	$SF = 0$	Jump if non-negative
<code>jg label</code>	$\sim(SF \wedge OF) \ \& \ \sim ZF$	Jump if greater (signed)
<code>jge label</code>	$\sim(SF \wedge OF)$	Jump if greater or equal (signed)
<code>jl label</code>	$SF \wedge OF$	Jump if less (signed)
<code>jle label</code>	$(SF \wedge OF) \mid ZF$	Jump if less or equal (signed)
<code>ja label</code>	$\sim CF \ \& \ \sim ZF$	Jump if above (unsigned)
<code>jae label</code>	$\sim CF$	Jump if above or equal (unsigned)
<code>jb label</code>	CF	Jump if below (unsigned)
<code>jbe label</code>	$CF \mid ZF$	Jump if below or equal (unsigned)

Jump targets can be **direct** (a label name, like `jmp loop_start`) or **indirect** (an address stored in a register, like `jmp *%rax`). Indirect jumps are used in switch statements.

Hand-Write: Compare and Branch

This program compares two values and branches based on the result:

```

        .text
        .global _start
_start:
        mov $1, %rax
        mov $2, %rbx
        cmp %rbx, %rax      # compute %rax - %rbx (sets flags)
        jl less             # jump if less (signed): 1 < 2, so
we jump
        mov $0, %rdi        # this line is skipped
        jmp finish
less:
        mov $10, %rdi       # we land here
finish:
        mov $60, %rax
        syscall

```

```

$ echo $?
10

```

After `cmp %rbx, %rax`, the flags reflect `%rax - %rbx = 1 - 2 = -1`. The result is negative with no signed overflow ($SF = 1$, $OF = 0$), so $SF \wedge OF = 1$ and `jl` (jump if less, signed) takes the branch.

gcc Comparison: `int max(int a, int b)`

Consider this C function:

```

int max(int a, int b) {
    if (a > b)
        return a;
    return b;
}

```

Compiled with `gcc -Og -S`, you get something like:

```

max:
        cmpl    %esi, %edi    # compare a (edi) vs b (esi)
        movl    %esi, %eax    # assume b (default return)
        cmovgl  %edi, %eax    # if a > b, overwrite with a
        ret

```

Here `gcc` uses `cmovg` (conditional move if greater) instead of a jump. Conditional moves avoid branch misprediction penalties and are preferred by the compiler for simple two-way choices. We will see conditional moves again in Lesson 2.

Lesson 2: If Statements

Branching in Assembly

In C, you write `if (condition) { ... } else { ... }`. In assembly, this becomes a compare-and-jump pattern. The general strategy is:

1. Evaluate the condition using `cmp` or `test`
2. Use a conditional jump to skip over one branch
3. Use an unconditional `jmp` to skip the other branch

Hand-Write: Positive, Zero, or Negative

This program checks whether a value is positive, zero, or negative, and exits with a different code for each case:

```

        .text
        .global _start
_start:
        mov $-5, %rax           # value to test (try changing this)

        cmp $0, %rax
        jg positive             # jump if %rax > 0
        je is_zero              # jump if %rax == 0

negative:
        mov $1, %rdi            # exit code 1 = negative
        jmp done

is_zero:
        mov $2, %rdi            # exit code 2 = zero
        jmp done

positive:
        mov $3, %rdi            # exit code 3 = positive

done:
        mov $60, %rax
        syscall

```

```

$ echo $?
1

```

With `%rax = -5`, neither `jg` (jump if greater than 0) nor `je` (jump if equal to 0) fires, so execution falls through to the `negative` label. Try changing `$-5` to `$0` or `$7` and verify the exit code changes.

The pattern

Notice the structure. The C equivalent would be:

```

if (x > 0)      goto positive;
if (x == 0)     goto is_zero;
/* fall through to negative */

```

Assembly "if statements" are really just conditional gotos. The `jmp done` after each case prevents falling through into the next case.

Inverted Conditions

A common pattern that `gcc` uses is to **invert** the condition. Instead of "jump to the then-branch if true," the compiler writes "jump *past* the then-branch if false." This avoids an extra unconditional jump:

```
# C: if (a == 0) { result = -1; }
    cmp $0, %eax
    jne skip_if          # jump PAST the body if NOT equal
    mov $-1, %eax        # this is the if-body
skip_if:
    ...
```

When reading compiler output, watch for these inverted conditions — a `jne` right after a comparison that checks for equality means "skip if the equality test fails."

gcc Comparison: Division by Zero Check

Consider this C function:

```
int divide(int a, int b) {
    int ans = 0;
    if (b == 0) {
        ans = -1;
    } else {
        ans = a / b;
    }
    return ans;
}
```

Compiled with `gcc -Og -S` :

```
divide:
    movl    $-1, %eax        # assume ans = -1 (b == 0 case)
    testl   %esi, %esi       # test b against itself (sets ZF if
b == 0)
    je      .L1              # if b == 0, jump to return
    movl    %edi, %eax        # %eax = a
    cltd                                         # sign-extend %eax into %edx:%eax
    idivl   %esi              # %eax = a / b
.L1:
    ret
```

Key observations:

- `gcc` uses `testl %esi, %esi` to check if `b` is zero. This is equivalent to `cmpl $0, %esi` but is one byte shorter.
- The condition is **not inverted** here: `je` jumps when `b == 0`, skipping the division. The compiler initialized `%eax` to `-1` first, so if the jump is taken, `-1` is already in the return register.
- Notice `cld` before `idivl` — this is the 32-bit version of `cqto` that sign-extends `%eax` into `%edx:%eax`.

Conditional Move: `cmov`

For simple two-way choices, modern CPUs offer **conditional move** instructions. A `cmov` copies a value only if the condition flags match:

```
cmp %esi, %edi
cmovgl %edi, %eax    # if edi > esi (signed), move edi into eax
```

Conditional moves can be more efficient than branching for simple two-way choices. Instead of jumping over code, the CPU computes both values and selects one, avoiding any penalty from guessing the wrong branch direction (a topic we will cover in detail in Module 8). `gcc` often uses `cmov` when both branches are simple, side-effect-free assignments; the exact output varies by `gcc` version and flags.

The condition suffixes for `cmov` are the same as for `j` instructions: `cmove`, `cmovne`, `cmovg`, `cmovl`, etc.

Lesson 3: Loops

Loops Are Just Conditional Jumps

Assembly has no built-in loop construct. Every loop is implemented with a backward jump: compare, and if the condition holds, jump back to an earlier label. There are three standard patterns, corresponding to do-while, while, and for loops in C.

Hand-Write: Sum 1 to N (Do-While Pattern)

The simplest loop pattern is **do-while**: execute the body, then check the condition and jump back if it holds. Let's sum the integers from 1 to 10:

```
.text
.global _start
_start:
    mov $0, %rax        # sum = 0
    mov $1, %rcx        # i = 1

loop:
    add %rcx, %rax       # sum += i
    inc %rcx            # i++
    cmp $11, %rcx       # compare i to 11
    jl loop             # if i < 11, repeat

    mov %rax, %rdi       # exit with sum
    mov $60, %rax
    syscall
```

```
$ echo $?
55
```

The sum $1+2+\dots+10 = 55$. The loop body always executes at least once, then the condition is checked at the bottom. This corresponds to:

```
do {
    sum += i;
    i++;
} while (i < 11);
```

While Loop: Jump-to-Middle Pattern

A **while** loop checks the condition *before* each iteration. One way to implement this is the **jump-to-middle** pattern: start with an unconditional jump to the condition check, then loop back from there:

```
.text
.global _start
_start:
    mov $0, %rax        # sum = 0
    mov $1, %rcx        # i = 1
    jmp test            # jump to condition first

loop:
    add %rcx, %rax       # sum += i
    inc %rcx            # i++

test:
    cmp $11, %rcx
    jl loop              # if i < 11, go to loop body

    mov %rax, %rdi
    mov $60, %rax
    syscall
```

This produces the same result (55), but if `%rcx` were already 11 or higher at the start, the body would never execute — just like a C `while` loop.

This corresponds to:

```
while (i < 11) {
    sum += i;
    i++;
}
```

For Loops

A C `for` loop is really just a `while` loop with the initialization and update made explicit:

```
for (init; test; update) { body; }  
// is equivalent to:  
init;  
while (test) { body; update; }
```

So the assembly pattern is the same as the while loop. Our sum example is already essentially a for loop: `for (int i = 1; i < 11; i++) { sum += i; }`

Hand-Write: Countdown with Output

Here is a loop that counts down from 5 to 1 and exits with the final value:

```
.text  
.global _start  
_start:  
    mov $5, %rcx          # start at 5  
  
loop:  
    dec %rcx              # i--  
    cmp $0, %rcx  
    jg loop               # if i > 0, keep going  
  
    # %rcx is now 0  
    mov %rcx, %rdi  
    mov $60, %rax  
    syscall
```

```
$ echo $?  
0
```

`dec` sets the condition flags, so we could also write `jnz loop` (jump if not zero) instead of the `cmp / jg` pair. Using `dec + jnz` is a common loop optimization.

gcc Comparison: Three Loop Forms

For positive `n` small enough to avoid signed overflow, these three C functions compute the same sum. For `n < 1`, the do-while form returns 1 while the other forms return 0 because

they skip the body. Here is representative output from `gcc -Og -S` :

Do-While

```
long dowhile(long n) {
    long sum = 0;
    long i = 1;
    do {
        sum += i;
        i++;
    } while (i <= n);
    return sum;
}
```

gcc output (simplified):

```
dowhile:
    movl    $0, %eax        # sum = 0
    movl    $1, %edx        # i = 1
.L2:
    addq    %rdx, %rax       # sum += i
    addq    $1, %rdx        # i++
    cmpq    %rdi, %rdx      # compare i to n
    jle     .L2              # if i <= n, loop
    ret
```

While

```
long awhile(long n) {
    long sum = 0;
    long i = 1;
    while (i <= n) {
        sum += i;
        i++;
    }
    return sum;
}
```

gcc output (jump-to-middle):


```

awhile:
    movl    $0, %eax        # sum = 0
    movl    $1, %edx        # i = 1
    jmp     .L5             # jump to test

.L6:
    addq    %rdx, %rax       # sum += i
    addq    $1, %rdx        # i++

.L5:
    cmpq    %rdi, %rdx      # compare i to n
    jle     .L6             # if i <= n, loop
    ret

```

For

```

long forloop(long n) {
    long sum = 0;
    for (long i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}

```

gcc produces nearly identical code to the while version — it converts the `for` to a `while` internally.

Notice how all three versions use the same registers (`%rax` for sum, `%rdx` for i, `%rdi` for n). The only difference is whether the test comes before or after the body.

Lesson 4: Switch Statements

Jump Tables

A `switch` statement with many cases could be implemented as a chain of `if-else` checks, but that would be slow for large switches. Instead, the compiler generates a **jump table**: an array of code addresses, one per case value. The switch variable is used as an index into this table, and an indirect jump goes directly to the correct case code in $O(1)$ time.

Hand-Write: A Simple Dispatch Table

Let's write a program that takes a value (0, 1, or 2), looks it up in a jump table, and exits with a different code for each case:

```

        .section .data
value:   .long 1                # try changing to 0 or 2

        .section .rodata
        .align 8
jump_table:
        .quad case0
        .quad case1
        .quad case2

        .text
        .global _start
_start:
        movl value(%rip), %eax    # load the switch variable
        cmpl $2, %eax            # range check
        ja default                # if above 2, go to default

        # Look up the target address in the jump table
        lea jump_table(%rip), %rdx
        movslq %eax, %rax         # widen the 32-bit int to a
64-bit index
        jmp *(%rdx,%rax,8)        # indirect jump:
jump_table[value]

case0:
        mov $10, %rdi
        jmp done

case1:
        mov $20, %rdi
        jmp done

case2:
        mov $30, %rdi
        jmp done

default:
        mov $99, %rdi

done:
        mov $60, %rax
        syscall

```

```

$ echo $?
20

```

With `value = 1`, the program looks up `jump_table[1]`, which holds the address of `case1`, and jumps there directly.

How the jump table works

- The jump table is stored in `.rodata` (it is constant data).
- Each entry is a `.quad` (8-byte address) pointing to a case label.
- The **range check** (`cmpl $2, %eax / ja default`) catches values outside 0–2 and sends them to the default case. Using `ja` (unsigned above) handles negative values too, since negative values become large positive values when treated as unsigned.
- The **indirect jump** `jmp *(%rdx,%rax,8)` reads the address from `jump_table + value*8` and jumps to it.
- The `movslq` widens the 32-bit value to the 64 bits needed for address computation. gcc emits a *sign*-extending move here because the C switch variable is a signed `int`; since the range check has already sent every out-of-range value to `default`, only small non-negative values reach this instruction, and sign- versus zero-extension makes no difference.

gcc Comparison: Full Switch Example

Consider a more complex switch:

```
long doswitch(long x) {
    long result;
    switch (x) {
        case 0: result = 10; break;
        case 1: result = 20; break;
        case 2:
        case 3: result = 30; break;    // shared case
        case 5: result = 50; break;    // gap at case 4
        default: result = -1;
    }
    return result;
}
```

gcc generates a jump table with entries for cases 0 through 5. For the missing case 4, the table entry points to the default code. For the shared cases 2 and 3, both table entries point to the same code block. The compiler:

1. Subtracts the lowest case value (if not 0) to normalize the index
2. Performs a range check using `cmp` and `ja`
3. Uses the result as an index into the jump table stored in `.rodata`
4. Executes an indirect jump via `jmp *table(,%rax,8)`

Here is representative `gcc -O1` output for `doswitch` (compiled without PIE for readability; exact labels and layout vary by gcc version):

```
doswitch:
    cmpq    $5, %rdi           # range check: table covers
cases 0-5
    ja      .L8                # above 5 (or negative) ->
default
    jmp     *.L4(,%rdi,8)      # indirect jump through the
table
.L3:
    movl    $10, %eax          # case 0
    ret
.L5:
    movl    $20, %eax          # case 1
    ret
.L6:
    movl    $30, %eax          # cases 2 and 3 (shared code)
    ret
.L7:
    movl    $50, %eax          # case 5
    ret
.L8:
    movq    $-1, %rax          # default
    ret

    .section .rodata
    .align 8
.L4:
    .quad   .L3                # case 0
    .quad   .L5                # case 1
    .quad   .L6                # case 2
    .quad   .L6                # case 3 (same target as case
2)
    .quad   .L8                # case 4 -> default (fills
the gap)
    .quad   .L7                # case 5
```

Notice how the shared cases 2 and 3 point at the same label, and the missing case 4 is filled with the default label so the table stays dense.

When gcc Uses If-Else Instead

The compiler does **not** always use a jump table. If the case values are sparse (e.g., `case 1`, `case 100`, `case 10000`), a jump table would waste enormous amounts of memory. In

that situation, `gcc` falls back to a chain of compare-and-jump instructions, essentially an if-else ladder. For a small number of cases (typically 3 or fewer), if-else is also used since the jump table overhead is not worthwhile.

Lesson 5: Writing Functions

Functions in Assembly

In C, functions are a fundamental abstraction. In assembly, there is no built-in concept of a "function" — just labels you can jump to. What makes a label into a function is **convention**: the **calling convention** defines how arguments are passed, where the return value goes, and which registers must be preserved. On Linux x86-64, this is the **System V AMD64 ABI**.

`call` and `ret`

Two instructions provide the mechanism for function calls:

- `call label` — Pushes the address of the *next* instruction (the return address) onto the stack, then jumps to `label`. Conceptually: `push [return address]; jmp label` (note: `push %rip` is not an actual valid instruction — the hardware does this internally)
- `ret` — Pops the return address from the stack and jumps to it. Conceptually: `pop [into %rip]` (again, `pop %rip` is not a valid instruction — the hardware does this internally)

The Calling Convention

The System V AMD64 ABI defines three key rules:

1. Argument Passing

The first six integer/pointer arguments are passed in registers, in this order:

Argument #	Register
1st	<code>%rdi</code>
2nd	<code>%rsi</code>
3rd	<code>%rdx</code>
4th	<code>%rcx</code>
5th	<code>%r8</code>
6th	<code>%r9</code>

Arguments beyond the sixth are pushed onto the stack (right to left). The return value is placed in `%rax`.

2. Callee-Saved Registers

The called function must **preserve** these registers — if it uses them, it must save and restore their original values:

`%rbx` , `%rbp` , `%r12` , `%r13` , `%r14` , `%r15`

3. Caller-Saved Registers

All other registers may be **clobbered** by the called function. If the caller needs their values after the call, it must save them itself (typically by pushing them onto the stack before the call):

`%rax` , `%rcx` , `%rdx` , `%rsi` , `%rdi` , `%r8` , `%r9` , `%r10` , `%r11`

Hand-Write: A Simple Function

Let's write a function `add_nums` that takes two arguments and returns their sum:


```

        .text
        .global _start

# Function: add_nums(a, b) -> a + b
add_nums:
        push %rbp                # save old base pointer
        mov %rsp, %rbp          # set up new base pointer
        mov %rdi, %rax          # %rax = first arg (a)
        add %rsi, %rax          # %rax = a + b (return value)
        pop %rbp                # restore base pointer
        ret

_start:
        mov $17, %rdi           # first argument
        mov $25, %rsi           # second argument
        call add_nums           # result in %rax

        mov %rax, %rdi          # exit with result
        mov $60, %rax
        syscall

```

```

$ echo $?
42

```

The `push %rbp` / `mov %rsp, %rbp` at the start and `pop %rbp` / `ret` at the end form the **prologue** and **epilogue**. For a function this simple, the prologue is not strictly necessary, but it is the standard pattern.

Function Calls Versus Syscalls

A normal function call and a Linux syscall are related but not identical interfaces. The `call` instruction follows the System V AMD64 function calling convention. The `syscall` instruction follows the Linux kernel's syscall convention.

Interface	Argument registers	Return register	Important clobbers
Function call	<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> , <code>%r9</code>	<code>%rax</code>	caller-saved registers may be overwritten
Syscall	<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%r10</code> , <code>%r8</code> , <code>%r9</code>	<code>%rax</code>	<code>%rcx</code> , <code>%r11</code>

For `read` and `write` , only the first three arguments are used, so the two conventions line up nicely. That makes them perfect first examples of assembly wrapper functions. If you later wrap a `syscall` with four or more arguments, you must remember to move the fourth argument from `%rcx` into `%r10` before executing `syscall` .

Hand-Write: Reusable `read` / `write` Wrappers

We can package bare syscalls inside ordinary assembly functions. The caller uses `call` with the normal function convention; the wrapper translates that request into a kernel `syscall` . In this example, we also use a branch to avoid writing anything when `read` returns EOF or an error.

```

        .section .rodata
prompt: .asciz "Type a line: "
plen = . - prompt - 1

        .section .bss
buffer: .skip 64

        .text
        .global _start

# write_buf(fd, buf, count) -> bytes written or negative error code
write_buf:
    mov $1, %rax
    syscall
    ret

# read_buf(fd, buf, count) -> bytes read or negative error code
read_buf:
    mov $0, %rax
    syscall
    ret

_start:
    mov $1, %rdi
    lea prompt(%rip), %rsi
    mov $plen, %rdx
    call write_buf

    mov $0, %rdi
    lea buffer(%rip), %rsi
    mov $64, %rdx
    call read_buf

    cmp $0, %rax
    jle done                # 0 = EOF, negative = error

    mov %rax, %rdx
    mov $1, %rdi
    lea buffer(%rip), %rsi
    call write_buf

done:
    mov $60, %rax
    xor %rdi, %rdi
    syscall

```

```
$ ./echo_line
Type a line: hello, syscalls
hello, syscalls
```

Key points in this program:

- `write_buf` and `read_buf` are **leaf functions**: they do not call any other functions, so they need no stack frame here.
- The caller passes `(fd, buf, count)` in `%rdi`, `%rsi`, and `%rdx`. The wrappers only need to place the syscall number in `%rax`.
- `read_buf` returns the number of bytes read in `%rax`, just like the raw syscall. The caller then moves that value to `%rdx` before calling `write_buf`.
- `syscall` clobbers `%rcx` and `%r11`. That is fine here because both are caller-saved registers anyway.

Hand-Write: Recursive Fibonacci

Recursion is the real test of understanding functions, because each call creates a new stack frame. Here is recursive Fibonacci:

```

        .text
        .global _start

# fib(n): returns the nth Fibonacci number
# fib(0) = 0, fib(1) = 1, fib(n) = fib(n-1) + fib(n-2)
fib:
    push %rbp
    mov %rsp, %rbp
    push %rbx                # save callee-saved register
    sub $8, %rsp             # 16-byte align %rsp and reserve a
save slot

    cmp $1, %rdi             # base case: n <= 1
    jle fib_base

    # Recursive case
    mov %rdi, %rbx           # save n in callee-saved %rbx
    dec %rdi                 # n - 1
    call fib                 # fib(n-1), result in %rax
    mov %rax, (%rsp)         # save fib(n-1) in our reserved slot

    lea -2(%rbx), %rdi       # n - 2
    call fib                 # fib(n-2), result in %rax

    add (%rsp), %rax         # %rax = fib(n-1) + fib(n-2)
    jmp fib_done

fib_base:
    mov %rdi, %rax           # return n (0 or 1)

fib_done:
    add $8, %rsp             # release reserved slot
    pop %rbx                 # restore callee-saved register
    pop %rbp
    ret

_start:
    mov $10, %rdi           # compute fib(10)
    call fib
    mov %rax, %rdi          # exit with result
    mov $60, %rax
    syscall

```

```

$ echo $?
55

```

Key points in this program:

- `%rbx` is callee-saved, so `fib` pushes and pops it. We use `%rbx` to hold `n` across the recursive calls because we know `fib` won't clobber it.
- We save `fib(n-1)` on the stack (into the reserved slot at `(%rsp)`) before calling `fib(n-2)` because `%rax` is caller-saved and will be overwritten by the second call.
- The `sub $8, %rsp` after pushing `%rbx` serves two purposes: it 16-byte aligns the stack before the recursive `call`s (the System V AMD64 ABI requires `%rsp` to be 16-byte aligned at every `call` instruction), and it reserves an 8-byte slot we use to save `fib(n-1)`. Two pushes (`%rbp` and `%rbx`) plus the entry misalignment of 8 bytes leaves `%rsp` at `8 mod 16` — without that `sub $8`, the first recursive `call` would violate the alignment rule.
- Each recursive call creates a new stack frame. For `fib(10)`, there will be many nested frames on the stack simultaneously.

To make that last point concrete, here is what the stack looks like at the moment the call chain has reached `fib(1)` while computing `fib(3)`:

```

higher addresses
+-----+
|  _start's stack stuff  |
+-----+
|  return addr to _start | <- pushed by call fib(3)
+-----+
|  saved %rbp (= 0)      | <- fib(3)'s frame starts here
|  saved %rbx (junk)    |
|  8 bytes alignment +  |
|  slot for fib(n-1)    |
+-----+
|  return addr in fib(3) | <- pushed by call fib(2) inside
fib(3)
+-----+
|  saved %rbp (-> fib3)  | <- fib(2)'s frame
|  saved %rbx (= 3)     |
|  8 bytes alignment +  |
|  slot for fib(n-1)    |
+-----+
|  return addr in fib(2) | <- pushed by call fib(1) inside
fib(2)
+-----+
|  saved %rbp (-> fib2)  | <- fib(1)'s frame, currently
running
|  saved %rbx (= 2)     |
|  8 bytes alignment +  |
|  slot for fib(n-1)    |
+-----+ <- %rsp right now
lower addresses

```

Each frame chains back to the previous one through saved `%rbp`, forming what's sometimes called a "frame pointer chain." A debugger walks this chain to print a backtrace.

gcc Comparison: Function Compilation

The C equivalent:

```

int add_numbers(int a, int b) {
    int sum = a + b;
    return sum;
}

```

Compiled with `gcc -Og -S :`

```
add_numbers:
    leal    (%rdi,%rsi), %eax
    ret
```

gcc uses `leal` to compute `a + b` directly into `%eax` (the 32-bit return register). No prologue or epilogue — this is a **leaf function** (it doesn't call other functions) and doesn't need stack space, so gcc optimizes away the frame setup entirely.

Lesson 6: Local Variables and the Stack Frame

When Locals Need the Stack

In Lesson 5, our simple functions kept everything in registers. But a function with many local variables, or one that needs to take the address of a local (using `&` in C), must store those locals on the stack. The region of the stack belonging to a single function call is its **stack frame**.

Anatomy of a Stack Frame

When a function is called, its stack frame is laid out (from higher to lower addresses):

1. **Return address** — Pushed by the `call` instruction. Located at `(%rsp)` upon function entry.
2. **Saved `%rbp`** — Pushed by the prologue. After `mov %rsp, %rbp`, `%rbp` points here.
3. **Saved callee-saved registers** — Pushed if the function needs to use `%rbx`, `%r12` – `%r15`.
4. **Local variables** — Allocated by `sub $N, %rsp`. Accessed via negative offsets from `%rbp`.

```

higher addresses
+-----+
| caller's frame ... |
| (arguments past 6, |
| if any, live here) |
+-----+
| return address     | <- (%rbp + 8); pushed by `call`
+-----+
| saved %rbp         | <- (%rbp); set by `mov %rsp, %rbp`
+-----+
| saved %rbx         | <- if used, pushed in prologue
+-----+
| saved %r12         | <- if used
+-----+
| local var #1       | <- e.g., -8(%rbp)
+-----+
| local var #2       | <- e.g., -16(%rbp)
+-----+
| ...               |
+-----+ <- (%rsp); top of stack right now
lower addresses

```

Stack grows downward (toward lower addresses), so "pushing" decreases `%rsp`. This function's frame is the region between the return address and `%rsp`. Memory below `%rsp` is not safe to use in general (the ABI guarantees only a 128-byte "red zone", and only for leaf functions); memory above the return address belongs to the caller.

The Prologue and Epilogue

The standard function prologue sets up the frame:

```

push %rbp          # save caller's base pointer
mov %rsp, %rbp     # establish our base pointer
sub $N, %rsp       # allocate N bytes for locals

```

Local variables are then accessed as `-8(%rbp)`, `-16(%rbp)`, `-24(%rbp)`, etc. Using `%rbp` as a fixed reference point means the offsets don't change even if the function pushes more values onto the stack.

The standard epilogue tears it down:

leave
ret

equivalent to: mov %rbp, %rsp; pop %rbp
pop return address and jump

The `leave` instruction is a convenient shorthand that restores both `%rsp` and `%rbp` in one instruction.

Hand-Write: A Function with Local Variables

Here is a function that stores its arguments and an intermediate result as local variables on the stack:

```

        .text
        .global _start

# compute(a, b): returns (a + b) + (b * 2)
compute:
    push %rbp
    mov %rsp, %rbp
    sub $24, %rsp          # space for 3 locals (8 bytes each)

    mov %rdi, -8(%rbp)     # local1 = a
    mov %rsi, -16(%rbp)    # local2 = b

    # Compute a + b
    mov -8(%rbp), %rax
    add -16(%rbp), %rax
    mov %rax, -24(%rbp)    # local3 = a + b

    # Compute b * 2
    mov -16(%rbp), %rax
    shl $1, %rax           # %rax = b * 2

    # Final result = local3 + (b * 2)
    add -24(%rbp), %rax

    leave
    ret

_start:
    mov $10, %rdi          # a = 10
    mov $8, %rsi           # b = 8
    call compute           # (10 + 8) + (8 * 2) = 18 + 16 = 34
    mov %rax, %rdi
    mov $60, %rax
    syscall

```

```

$ echo $?
34

```

In practice, this function could keep everything in registers. We use stack locals here to illustrate the pattern. The compiler generates code like this when there are too many live values for the available registers.

gcc Comparison: -O0 vs -Og

Consider:

```
long compute(long a, long b) {
    long sum = a + b;
    long doubled = b * 2;
    return sum + doubled;
}
```

At -O0 (no optimization), gcc stores every variable on the stack, producing code almost identical to our hand-written version: prologue, `sub $N, %rsp`, moves to/from `-8(%rbp)`, `-16(%rbp)`, etc.

At -Og (basic optimization), gcc keeps everything in registers and often eliminates the prologue entirely:

```
compute:
    leaq    (%rdi,%rsi), %rax    # sum = a + b
    leaq    (%rax,%rsi,2), %rax  # sum + b*2 = a + 3b
    ret
```

gcc optimized `sum + doubled` into `(a + b) + 2b = a + 3b` and computed it with two `leaq` instructions. This is why reading -O0 output is better for learning — it maps directly to the C source — while -Og or higher shows what the optimizer does.

Leaf Functions and Inlining

A **leaf function** is one that does not call any other functions. The compiler can optimize leaf functions aggressively:

- No prologue/epilogue if all locals fit in registers
- No need to save callee-saved registers if the function doesn't use them
- May be **inlined**: the function body is copied directly into the caller, eliminating the `call / ret` overhead entirely

This is why small helper functions in C (like `max(a, b)`) have essentially zero overhead at higher optimization levels.

16-Byte Stack Alignment

The System V ABI requires that `%rsp` be aligned to a **16-byte boundary** at the point of a `call` instruction. Since `call` pushes an 8-byte return address, `%rsp` will be 8 bytes off from 16-byte alignment at function entry. The prologue's `push %rbp` restores 16-byte alignment. If you allocate stack space with `sub`, the amount should keep `%rsp` 16-byte aligned. Our `compute` example above got away with `sub $24, %rsp` (leaving `%rsp` only 8-byte aligned) because `compute` makes no calls — it is a leaf function, so the misalignment is harmless; a non-leaf function would round the allocation up to a multiple of 16 (e.g., 32 bytes).

If you get a segfault when calling library functions, misaligned stack is a common cause.

Variable-Size Stack Frames

If gcc can often skip the frame pointer entirely, why keep `%rbp` at all? One reason: sometimes the compiler cannot know the stack frame size at compile time — for instance, when using `alloca()` or variable-length arrays. In this case, gcc uses `%rbp` as a fixed reference point and lets `%rsp` change dynamically. Local variables at fixed offsets from `%rbp` remain accessible even as the stack grows.

Lesson 7: Arrays, Structs, and Data Alignment

Arrays in Assembly

An array in C is a contiguous block of same-sized elements. In assembly, an array is simply a sequence of values at a labeled address. To access the i th element, you compute:

```
base_address + i × element_size .
```

Hand-Write: Sum an Array

This program declares an array of five 8-byte integers and computes their sum using a loop with scaled index addressing:

```

        .section .data
arr:     .quad 10, 20, 30, 15, 25
len:     .quad 5

        .text
        .global _start
_start:
        lea arr(%rip), %r8      # %r8 = base address of array
        mov $0, %r9             # %r9 = sum = 0
        mov $0, %rcx            # %rcx = index i = 0

loop:
        cmp len(%rip), %rcx     # memory-operand compare: reads the
count from .data (computes i - len)
        jge done                # if i >= len, exit loop
        add (%r8,%rcx,8), %r9    # sum += arr[i]
        inc %rcx                # i++
        jmp loop

done:
        mov %r9, %rdi            # exit with sum
        mov $60, %rax
        syscall

```

```

$ echo $?
100

```

The key instruction is `add (%r8,%rcx,8), %r9`. This uses the scaled index addressing mode: `base + index × 8`. Since each `.quad` is 8 bytes, `%rcx` serves as both the loop counter and the array index.

Structures in Memory

A C struct is laid out as contiguous fields, each at a fixed offset from the start of the struct. The compiler calculates these offsets at compile time. In assembly, struct access is just memory access with a constant displacement:


```
# struct { long x; long y; int z; }
# Offsets: x at 0, y at 8, z at 16
# If %rdi points to the struct:
movq (%rdi), %rax      # load x (offset 0)
movq 8(%rdi), %rbx     # load y (offset 8)
movl 16(%rdi), %ecx    # load z (offset 16)
```

Unions

A C `union` stores all fields at the *same* memory offset (offset 0). The total size of the union equals the size of its largest field. Only one field can hold a valid value at a time:

```
# union { long l; int i; char c; }
# Size: 8 bytes (size of long)
# All fields start at offset 0
movq (%rdi), %rax      # read as long
movl (%rdi), %eax      # read as int (lower 4 bytes)
movb (%rdi), %al       # read as char (lowest byte)
```

Data Alignment

Modern processors access memory most efficiently when data is **aligned** — stored at an address that is a multiple of the data's size. The rules are:

- A 1-byte value (`char`) can be at any address
- A 2-byte value (`short`) should be at an even address
- A 4-byte value (`int`) should be at a multiple of 4
- An 8-byte value (`long` , pointer) should be at a multiple of 8

To satisfy alignment, the compiler inserts **padding bytes** between struct fields. This can make a struct larger than the sum of its field sizes.

Alignment Walkthrough

Consider this struct:

```
struct example {
    char a;      // 1 byte
    int  b;      // 4 bytes
    char c;      // 1 byte
    long d;      // 8 bytes
};
```

Without padding, this would be $1 + 4 + 1 + 8 = 14$ bytes. With alignment:

Offset	Field	Size	Notes
0	a	1	
1–3	(padding)	3	Align b to multiple of 4
4	b	4	
8	c	1	
9–15	(padding)	7	Align d to multiple of 8
16	d	8	

Total size: 24 bytes (not 14!). Additionally, the struct's overall size is padded to a multiple of 8 (its largest field's alignment), so arrays of this struct work correctly.

Reordering Fields to Save Space

If we reorder the fields from largest to smallest:

```
struct example_compact {
    long d;      // 8 bytes at offset 0
    int  b;      // 4 bytes at offset 8
    char a;      // 1 byte at offset 12
    char c;      // 1 byte at offset 13
    // 2 bytes padding to reach multiple of 8
};
```

Total size: 16 bytes — 8 bytes smaller! This is a practical optimization when working with large numbers of structs.

Why this matters beyond memory size: when we get to caches in Module 8, you'll see that the CPU loads memory in fixed-size *cache lines* (typically 64 bytes). A 16-byte struct lets you pack four of them per cache line; a 24-byte struct only fits two-and-a-bit. For code that walks through arrays of structs, the smaller layout can be more than just smaller in memory — it can be measurably faster, because the cache and the memory bus are doing less work per logical record.

Lesson 8: Buffer Overflows and Security

Out-of-Bounds Memory Access

C does not perform bounds checking on array accesses. If you write past the end of a buffer, you silently overwrite whatever happens to be next in memory — other local variables, the saved base pointer, or the **return address**. This is a **buffer overflow**.

Since local variables and the return address live on the stack, overflowing a stack-allocated buffer can corrupt the return address. When the function executes `ret`, it jumps to the corrupted address — which could be attacker-controlled code.

Hand-Write: Overflow Demonstration

This program allocates a small buffer on the stack and deliberately writes past it, demonstrating corruption of nearby stack metadata (here, the saved base pointer):

```

        .text
        .global _start

_start:
    push %rbp
    mov %rsp, %rbp
    sub $8, %rsp          # allocate 8-byte buffer at -8(%rbp)

    # Fill buffer (normal)
    movq $0x41414141, -8(%rbp)    # "AAAA" + 4 zero bytes
    (movq stores 8 bytes)

    # Overflow! Write past the buffer into saved %rbp
    movq $0x42424242, 0(%rbp)    # overwrites saved %rbp!

    # This corrupts frame-chain state. To hijack control flow
    directly,
    # an overflow typically targets the saved return address at
    8(%rbp).
    # We exit safely instead of returning through the corrupted
    frame.
    mov $0, %rdi
    mov $60, %rax
    syscall

```

In a real attack scenario, the attacker would overflow the buffer until they overwrite the return address with the address of their own injected code (or a useful existing code sequence).

Why `gets()` Is Dangerous

The classic vulnerable function is `gets()` from the C standard library:

```

char buf[64];
gets(buf);    // reads unlimited input into 64-byte buffer!

```

`gets()` reads until a newline with no maximum length. If the user types more than 64 characters, it overflows the buffer. This is why `fgets()` (which takes a maximum length) should always be used instead. The `gets()` function has been removed from the C standard as of C11.

Thwarting Buffer Overflows

Expecting programmers to avoid all buffer overflows has not worked. Modern systems use multiple defense layers:

1. Stack Randomization (ASLR)

Address Space Layout Randomization allocates the stack at a random address each time a program runs. An attacker cannot predict where their injected code will be in memory.

One countermeasure: a **NOP sled** — the attacker prefixes injected code with thousands of `nop` instructions. If the return address lands anywhere in the sled, execution slides into the payload. ASLR's randomization range is usually large enough to defeat this.

2. Stack Canaries

The compiler inserts a random **canary value** between the buffer and the return address. Before the function returns, it checks whether the canary has been modified. If it has, a buffer overflow occurred and the program aborts with a "stack smashing detected" error.

```
# gcc inserts something like:
function_entry:
    mov %fs:0x28, %rax    # load canary from thread-local
storage
    mov %rax, -8(%rbp)    # place canary on stack
    ...
function_exit:
    mov -8(%rbp), %rax
    xor %fs:0x28, %rax    # compare with original
    jne __stack_chk_fail  # if different, abort!
    leave
    ret
```

This is enabled by default in the `gcc` builds shipped by most Linux distributions (as `-fstack-protector-strong`); upstream `gcc` does not enable it by default. To disable it (for educational experiments only!), use `-fno-stack-protector`.

3. Non-Executable Stack (NX / DEP)

Modern hardware marks the stack as **non-executable**. Even if an attacker overwrites the return address to point into the stack, the CPU refuses to execute instructions from that memory region. This is enforced at the hardware level (the "NX bit" in page tables).

Attackers have responded with **return-oriented programming** (ROP), which chains together short instruction sequences ("gadgets") already present in executable code. Defending against ROP is an active area of research.

gcc Comparison: With and Without Protection

Compile the same vulnerable program two ways:

```
# With protection (default)
$ gcc -o vuln vuln.c

# Without protection
$ gcc -fno-stack-protector -o vuln_unsafe vuln.c
```

Disassemble both with `objdump -d` and compare. The protected version has extra instructions to load, store, and check the canary value. The unprotected version has none — and is vulnerable to a straightforward buffer overflow attack.

Lesson 9: Floating Point in Assembly

NOTE: This lesson provides a summary of floating point operations in x86-64 assembly. Floating point assembly will not be heavily tested on quizzes or exams — you should understand the general concepts and register layout, but detailed instruction memorization is not required.

Separate Hardware for Floating Point

Floating point numbers use a completely separate set of registers and instructions from the integer operations we have been studying. This reflects the hardware: the CPU has dedicated floating point/vector processing units designed for high-throughput parallel arithmetic.

The architecture has evolved through several generations: **MMX**, then **SSE** ("Streaming SIMD Extensions"), and most recently **AVX** ("Advanced Vector Extensions"). Modern `gcc` on x86-64 uses SSE2 instructions by default.

Floating Point Registers

There are several banks of FP/SIMD registers, added in successive architecture generations:

Name	Count	Size	Notes
<code>%mm0</code> – <code>%mm7</code>	8	64 bits	Legacy MMX registers (rarely used directly; aliased to the x87 FPU stack, not to XMM).
<code>%xmm0</code> – <code>%xmm15</code>	16	128 bits	SSE registers. Used for scalar float/double operations.
<code>%ymm0</code> – <code>%ymm15</code>	16	256 bits	AVX registers. The XMM registers are the lower half of these.

Each XMM register can hold a single `float` (32 bits in the low portion), a single `double` (64 bits in the low portion), or multiple values for SIMD parallel operations (e.g., 4 floats or 2 doubles). We will focus on scalar (single-value) operations.

Calling Convention for Floats

Floating point arguments are passed in `%xmm0` through `%xmm7` (up to 8 FP arguments). These are counted separately from integer arguments. For example:

```
void f(float a, int b, float c)
// a -> %xmm0, b -> %edi, c -> %xmm1
```

The return value goes in `%xmm0`. All XMM registers are **caller-saved**.

Moving Floating Point Data

The move instructions for floating point are:

Instruction	Description
<code>movss S, D</code>	Move single-precision (float, 4 bytes)
<code>movsd S, D</code>	Move double-precision (double, 8 bytes)

The same memory-to-memory restriction applies: you cannot `movsd` from one memory location to another.

Type Conversion Instructions

Instruction	Direction	Description
<code>cvtts2si S, D</code>	float → int	Convert single-precision to integer (truncate). Source: XMM or memory. Dest: integer register.
<code>cvttsd2si S, D</code>	double → int	Convert double-precision to integer (truncate).
<code>cvtsi2ss S, D</code>	int → float	Convert integer to single-precision. Source: integer register or memory. Dest: XMM register.
<code>cvtsi2sd S, D</code>	int → double	Convert integer to double-precision.

Arithmetic Operations

Single	Double	Effect	Description
<code>addss S, D</code>	<code>addsd S, D</code>	$S + D \rightarrow D$	Addition
<code>subss S, D</code>	<code>subsd S, D</code>	$D - S \rightarrow D$	Subtraction
<code>mulss S, D</code>	<code>mulsd S, D</code>	$S \times D \rightarrow D$	Multiplication
<code>divss S, D</code>	<code>divsd S, D</code>	$D / S \rightarrow D$	Division
<code>maxss S, D</code>	<code>maxsd S, D</code>	$\max(S,D) \rightarrow D$	Maximum
<code>minss S, D</code>	<code>minsd S, D</code>	$\min(S,D) \rightarrow D$	Minimum
<code>sqrtps S, D</code>	<code>sqrtsd S, D</code>	$\sqrt{S} \rightarrow D$	Square root

Hand-Write: Floating Point Demo

This program loads two doubles from the data section, multiplies them, converts the result to an integer, and exits with it:

```
.section .data
val_a: .double 3.5
val_b: .double 12.0

.text
.global _start
_start:
    movsd val_a(%rip), %xmm0    # load 3.5
    movsd val_b(%rip), %xmm1    # load 12.0
    mulsd %xmm1, %xmm0          # %xmm0 = 3.5 * 12.0 = 42.0
    cvtsd2si %xmm0, %rdi         # convert to integer: 42
    mov $60, %rax
    syscall

$ echo $?
42
```

Floating Point Constants

Unlike integer instructions, most floating point instructions cannot use immediate values. Constants must be stored in memory (typically in `.rodata`) and loaded from there. The assembler directive `.float` stores a single-precision value; `.double` stores double-precision.

Comparisons

Floating point comparisons use `ucomiss` (single) and `ucomisd` (double). These report the comparison in **ZF, PF, and CF** and clear **OF, SF, and AF** to zero. That means you must use the *unsigned-style* conditional jumps (`je`, `jne`, `ja`, `jae`, `jb`, `jbe`) after a floating-point comparison; the signed-style jumps (`jg`, `jle`, etc.) won't give the right answer because SF and OF carry no useful information.

```
ucomisd %xmm1, %xmm0    # compare xmm0 vs xmm1
ja label                # jump if xmm0 > xmm1
                        # (use ja/jb, NOT jg/jl)
```

The **parity flag** (PF) is set when either operand is NaN, which can be tested with `jp` (jump if parity). NaN comparisons are *unordered*: a NaN compared against anything sets ZF=1, PF=1, CF=1, so a plain `je` will treat NaN as "equal" unless you also check PF first.

Bitwise Operations on Floats

There are a few bitwise operations available for floating point: `xorps` / `xorpd` (XOR) and `andps` / `andpd` (AND). The most common use is zeroing a register:

```
xorpd %xmm0, %xmm0      # set %xmm0 to 0.0
```

gcc Comparison

Here is a C function and what `gcc -Og -S` produces:

```
double afunc(double ad, int count, double bd) {  
    double sd = ad * bd;  
    double res = sd * count;  
    return res;  
}
```

```
afunct:  
    mulsd    %xmm1, %xmm0          # sd = ad * bd  
    cvtsi2sd %edi, %xmm1          # convert count to double  
    mulsd    %xmm1, %xmm0          # res = sd * count  
    ret
```

Notice that `ad` is in `%xmm0`, `bd` is in `%xmm1` (floating point arguments), and `count` is in `%edi` (integer argument). The compiler converts `count` to double before multiplying.

Module 4 Summary

Wrap up

In this module, you wrote assembly programs with control flow, functions, and structured data — completing the full arc from a 3-line exit program in Module 3 to recursive functions, syscall wrappers, and complex data structures. You also examined buffer overflow vulnerabilities and floating point operations.

Key Takeaways

- Condition codes (CF, ZF, SF, OF) are set by arithmetic and comparison instructions. The `cmp` and `test` instructions set flags without storing a result. Jump instructions and conditional moves read these flags to implement branching.
- All control flow — `if` statements, loops, and `switch` statements — is built from compare-and-jump patterns. Switch statements with many cases use jump tables for O(1) dispatch.
- Functions in assembly are defined by convention, not hardware. The System V AMD64 calling convention specifies argument registers (`%rdi` , `%rsi` , ...), the return register (`%rax`), and which registers are callee-saved vs. caller-saved.
- The Linux `syscall` convention is close to, but not identical to, the normal function calling convention. In particular, the fourth `syscall` argument uses `%r10` instead of `%rcx` , and `syscall` clobbers `%rcx` and `%r11` .
- A function's stack frame holds the return address, saved registers, and local variables. The prologue (`push %rbp` / `mov %rsp, %rbp` / `sub`) and epilogue (`leave` / `ret`) manage this frame.
- Arrays use scaled-index addressing (`base + index × size`). Structs use fixed offsets with alignment padding. Reordering struct fields from largest to smallest minimizes wasted space.
- Buffer overflows exploit C's lack of bounds checking. Defenses include ASLR (randomizing addresses), stack canaries (detecting corruption), and the NX bit (preventing code execution from the stack).
- Floating point uses a separate set of 16 XMM registers and SSE2 instructions (`movsd` , `addsd` , `mulsd` , etc.). FP arguments pass in `%xmm0` – `%xmm7` , separately from integer arguments.

Additional resources

All links open in a new tab.

[Conditionals and Statements Slides set](#) [link]

[Conditionals and Statements](#) [link to video lecture from Carnegie Mellon University]

[Object Code - Procedures Slides set](#) [link]

Object Code - Procedures [\[link to video lecture from Carnegie Mellon University\]](#)

Module 5 Overview

In Module 5: Processes, Signals, and Forking, we move from the instruction level up to the operating system level. We will study how the operating system manages running programs through processes and how it handles unexpected events through exceptional control flow.

We start by discussing control flow and exceptional control flow. The sole job of the CPU is to process one instruction after another, but sometimes events -- both hardware and software -- cause an abrupt change in that flow. We will talk about exceptions, what causes them, and how they are handled. From there, we learn about processes: what they are, how the operating system creates and manages them, and how to use system calls such as `fork`, `exec`, and `wait` to create and coordinate child processes. The module concludes with signal handling: how signals work as software-level exceptions, how to write signal handlers, and how to block signals safely from within your C code.

This module consolidates what were previously two separate modules (Processes and Signal Handling) into one comprehensive unit.

How this module is organized

The twelve lessons form three arcs:

- **Part A: Exceptions (Lessons 1–4)** — control flow, exceptional control flow, and the four classes of exceptions.
- **Part B: Processes (Lessons 5–7)** — what a process is, and how to create and manage child processes with `fork`, `exec`, and `wait` / `waitpid`.
- **Part C: Signals (Lessons 8–12)** — how signals work, writing and installing signal handlers, and blocking signals safely.

Learning objectives

At the end of this learning module, you will be able to:

1. Explain control flow and exceptional control flow

2. Define the four classes of exceptions (interrupts, traps, faults, and aborts)
3. Describe what a process is and how processes are managed by the operating system
4. Use `fork` to create child processes and predict the behavior of programs that use it
5. Use `exec` to replace a process image and `wait / waitpid` to synchronize parent and child processes
6. Explain how signals and signal handlers work
7. Write signal handlers and install them using `signal` or `sigaction`
8. Block and unblock signals from your C code using the appropriate library functions

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below
2. Read Lessons 1 to 12 and complete the embedded practice problems in the lessons that include them
3. Take the module quiz
4. Review additional resources on the Module 5 Summary page

Reading

- Bryant & O'Hallaron: Chapter 8

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: Flow Control

Part A: Exceptions (Lessons 1–4)

NOTE: This is a summary of chapter 8, sections 8.1 to 8.4 in the textbook.

Flow Control

Up to now, every control transfer in your assembly programs has been requested by the program itself: a `jmp`, a `je`, a `call`, a `ret`. The CPU just walked from one instruction to the next, occasionally taking a branch when the program asked it to. We call this sequence of transfers the program's **flow of control** or **control flow**.

This module is about everything that happens when control transfers *without the program asking*. Think about pressing Ctrl-C in a terminal: the program was happily running an infinite loop, and a fraction of a second later it has died. Nothing in the program's source code says "exit on Ctrl-C." Yet the keypress somehow reached into the running process and stopped it. How?

The answer is something the textbook calls **exceptional control flow** (ECF) — abrupt changes to a process's flow that the process itself did not initiate. ECF is the mechanism behind a long list of things you already use without thinking about them:

- A hardware timer fires every few milliseconds so the OS can rotate through running processes.
- You request data from a disk; the disk eventually finishes and signals back.
- A child process dies and the parent gets notified.
- You hit Ctrl-C and the program receives SIGINT.
- You divide by zero, dereference a null pointer, or call a system call.

All of these are detours: the program was doing one thing, an event happened, and control jumps elsewhere. Over the next dozen lessons we'll work through the four kinds of events that cause these detours (interrupts, traps, faults, aborts), the operating-system abstractions built

on top of them (processes, fork/exec/wait), and a major user-visible application of all that machinery (signals).

Lesson 2: Exceptions

Part A: Exceptions (Lessons 1–4)

Exceptions

The umbrella name for all the events that cause exceptional control flow is **exceptions**. An exception is an abrupt change in control flow that comes from a change in the processor's state — either something the running instruction did (such as dereferencing a null pointer and triggering a segfault), or something completely external (such as a network card noticing a packet has arrived).

Let's walk through what mechanically happens. Imagine your program executes `movq (%rax), %rbx` and `%rax` happens to be 0. The CPU tries to read from address 0, the MMU detects that address 0 is unmapped, and the CPU raises exception number 14 (the page fault). What happens next is a four-step dance:

1. **Look up the handler.** Long before any of this happened, the OS set up an **exception table** in memory: an array indexed by exception number, where each entry is the address of an **exception handler**. Entry 14 holds the address of the kernel's page-fault handler.
2. **Save the user state.** The hardware pushes some processor state (at least the faulting instruction's address and the flags) onto the kernel stack so it can resume the user program later.
3. **Transfer to the handler.** The CPU switches to kernel mode and jumps to the handler's address. From the user program's perspective this is a procedure call it didn't make.
4. **Decide what to do.** The handler runs. When it finishes, one of three things happens:
 - **Resume the same instruction.** The handler fixed the underlying problem (e.g., paged in a missing page). The faulting instruction re-executes, this time successfully. (This is what makes virtual memory work.)
 - **Resume the next instruction.** The exception was triggered intentionally by the program (e.g., a system call). The instruction after the trap-causing one continues.
 - **Abort the program.** The exception is fatal (e.g., dereferencing a NULL pointer in a process with no handler). The OS terminates the process and returns control to the rest of the system.

You'll see this same four-step pattern over and over in the rest of the module. What changes is which exception number gets raised, who runs the handler, and what the handler decides to do.

Lesson 3: Classes of Exceptions

Part A: Exceptions (Lessons 1–4)

Classes of Exceptions

The single most useful axis to organize exceptions on is **synchronous vs. asynchronous**: was the event caused by the instruction the CPU was just executing (sync), or did it come from outside (async)? Synchronous means tied to the instruction that causes the event, not necessarily repeatable across runs: a load might page-fault on one run but find its page resident on another. Asynchronous events arrive independently of the instruction stream.

The textbook splits exceptions into four classes. Three of them are synchronous (caused by the current instruction); only interrupts are asynchronous.

Class	Sync / Async	Caused by	What the handler does
Interrupt	Async	An I/O device signaling the CPU	Service the device, then resume the next instruction
Trap	Sync	An intentional instruction (e.g., <code>syscall</code>)	Do the requested work, return to the next instruction
Fault	Sync	An error during execution	Try to fix it; if successful, retry the same instruction; otherwise abort
Abort	Sync	An unrecoverable hardware error	Terminate the process; control never returns

Now the details of each class:

- **Interrupts:** these occur *asynchronously* — the CPU was executing one of your instructions and the network card finished receiving a packet, or the disk controller finished a read, or the timer's quantum expired. The hardware sets a "pin" on the CPU; the CPU notices once it finishes the current instruction and jumps to an **interrupt handler**. The exception number on the bus tells it which handler. Because interrupts can happen at any time, kernel code that protects shared data has to disable interrupts (or use locks) for any region that the handler might touch.
- **Traps:** these are *synchronous and intentional*. The program executed an instruction whose entire purpose is to trigger a trap, almost always to ask the OS for service. The canonical example is the `syscall` instruction (Module 3 Lesson 4), which the kernel handles by reading the syscall number out of `%rax` and dispatching to the right kernel routine. Debuggers also use traps via the `int3` breakpoint instruction. After the handler runs, execution continues at the *next* instruction.
- **Faults:** these are *synchronous and unintentional*. The program tried to do something it can't do at the moment, and the CPU traps to a fault handler that tries to fix the situation. If it can, the handler returns and the *same* instruction re-executes — this time successfully. The page fault is the most common example: your program touches a virtual page that's been swapped out, the kernel reads it back from disk, and the access succeeds the second time around. If the fault is unrecoverable (NULL dereference, divide by zero), the kernel turns it into a fatal signal and the process dies.
- **Aborts:** these are *synchronous* too, but the hardware has determined the situation is unrecoverable. There is no resuming. The classic example is the **machine check exception**, raised when the CPU detects something like uncorrectable ECC errors in DRAM. The system typically panics. Faults and aborts shade into each other on real CPUs — for example, integer divide-by-zero is technically a fault, but Linux turns it into a fatal SIGFPE which makes it look like an abort from a user's perspective.

Lesson 4: Exceptions in Linux/x86-64 Systems

Part A: Exceptions (Lessons 1–4)

Exceptions in Linux/x86-64 Systems

In modern Linux systems, there are 256 possible exceptions. Numbers 0-31 are hardware exceptions defined by the architecture (Intel x86-64) and numbers 32-255 are interrupts and traps defined by the operating system (Linux).

The 256 exceptions are too numerous to list. Here is a short list of some common exceptions:

- **Divide Error (0)** - This is classified as a fault. The kernel delivers SIGFPE, which terminates the process by default. This only works with integer arithmetic as floating point just sets the resulting value to infinity (inf) when a divide by zero is performed. Ironically, this is called a 'floating point exception' even if by default it only happens with integer math.
- **General Protection Fault (13)** - This fault is triggered by segment-level protection violations such as privilege violations or certain misaligned accesses. Although the kernel delivers SIGSEGV for this exception, it is distinct from the more common Page Fault (exception 14) that produces the familiar "segmentation fault" when a program accesses invalid memory.
- **Page Fault (14)** - The processor raises this exception when a virtual memory access cannot be completed using the current page-table entries. There are two main flavors: *recoverable* page faults — the page is valid but currently swapped out, lazily allocated, or copy-on-write — in which case the kernel handles the fault and resumes the user program transparently; and *invalid* page faults — the program tried to access memory it doesn't have permission for, or that isn't mapped at all (e.g., dereferencing `NULL` or walking off the end of a buffer). The invalid case is what application programmers see as the familiar "segmentation fault" via SIGSEGV. A 'page' is how virtual memory is divided into fixed-size chunks (typically 4 KiB on x86-64).
- **Machine check (18)** - Machine check is typically what is generated when hardware fails. This is always an abort (and may cause the operating system to abort).

Linux System Calls

There are hundreds of system calls in Linux. These are basically functions you can call within your C/C++ program to ask for services of the operating system. Typical ones are file I/O, managing processes (starting child processes or exiting the current process) and so on. Figure 8.10 in the textbook shows a table of some of these calls. We will discuss using some of these in future sections.

Lesson 5: Processes

Part B: Processes (Lessons 5–7)

Processes

To understand better how exceptions are handled, it helps to understand the workings of processes. According to the textbook, "A process is the instance of a program in execution". A program is a series of instructions that are run as a process (or as several processes as a program can be executed several times concurrently).

A process runs in a **context** which consists of the sum total of state (or information) needed to successfully run the process to completion. This state includes:

- the program's code and data stored in memory
- its program stack
- the contents of its general-purpose registers
- its program counter
- environment variables
- the set of open file descriptors

When a program name is invoked at the command line (shell), the shell program creates a new process and the context needed to run the process to completion. An application program can also create a new process. This is called **forking** and we will talk more about this in the following sections.

How the operating system handles processes is complex. The textbook only discusses two things the operating system provides processes:

- logical control flow.
- address space.

For each process, these are independent from all other processes. This gives the illusion from a process's perspective that they have exclusive use of the CPU and the memory space.

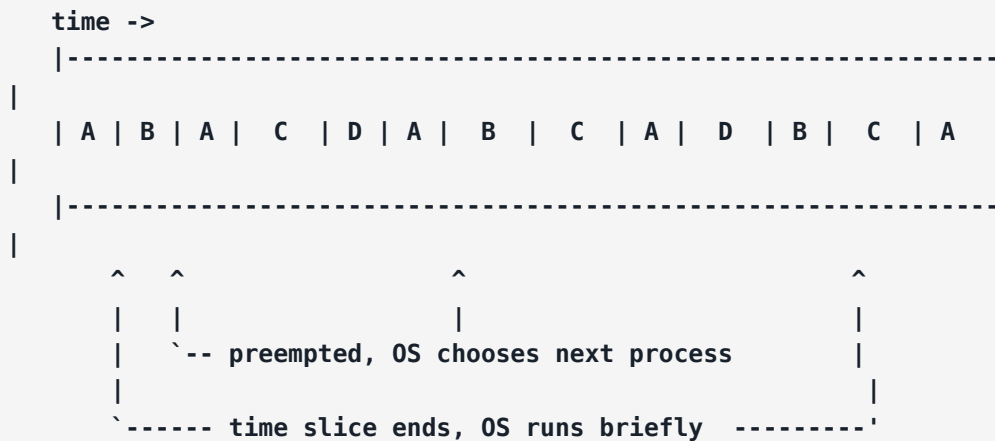
Logical Control Flow

This is defined in the book as the series of PC (program counter) values that the CPU runs for just this one process. Basically it's the sequence of instructions a process is running from beginning to end. This is also abbreviated to logical flow.

A process is typically running **concurrently** with other processes in the system. This means that its logical flow overlaps in time with other processes. This makes it seem like these processes are running at the same time. However, if they share a CPU, only one process can run at the same time.

How does the operating system and hardware run several processes on the same CPU? This is done by **interleaving** the running of the processes. Each process takes turns using the CPU and memory for very short imperceptible time intervals.

CPU usage over time (single core, four processes A, B, C, D):



At any single instant, only ONE process is executing on this CPU. But across a wall-clock second, every process gets some CPU time, giving the illusion that they all run at once.

Above is an illustration of what interleaving is. There are 4 processes running on one CPU. They take turns running a portion of their instruction set before being **preempted** which means being suspended for a time. Each 'time slice' has one process running for a variable amount of time. How long and what process to run next is called **scheduling** and is decided by the operating system, which is run in between time slices.

The textbook puts more emphasis on a one-CPU system, but most modern CPUs have multiple CPUs called **cores**. The textbook does this for keeping the discussion simple and not having to get into the complex intricacies of scheduling on a multi-core CPU. So the picture of how programs are run is more complex than what's being presented here.

The preempting is done via a hardware interrupt that goes off every few milliseconds. This causes the currently running process to stop running (be preempted), the context is saved and

the operating system's kernel is run. This is to allow the operating system to do its own tasks in-between each process (this is assuming it doesn't run in its own core). When the operating system is done it will choose another process to run and preempt itself.

Concurrent Flows

When one process's execution time overlaps another process's execution time they are said to be **concurrent**. Concurrency comes in two types: **interleaving** and **parallelism**. Interleaving is described as above. In today's multi-core systems, it's possible to have two processes concurrently run side-by-side (no interleaving) in two different cores. This is called parallel flows, parallel execution or running in parallel.

Private Address Space

Each process is run within a private address space that starts with 0 and goes (theoretically) to the limit set by the machine's address width — on x86-64, user programs see 48 bits of address space (about 256 TiB), regardless of how much RAM is actually installed. In reality, the process is not likely to use all of this space and since portions of memory might be reserved for the operating system, it won't be able to use the full memory that the computer has to offer.

The system manages to create this illusion by mapping the program's addresses to actual physical addresses. Each program running at the same time is given a range of the actual memory addresses (usually this is 'growable' or 'shrinkable' depending on what each program needs). These are mapped starting at 0 and going to whatever the process needs. If the program needs more than the allocated memory it's given, part of the memory may be moved to disk to make room for the creation of more memory. So the same address (say, address 0x1000) in two different processes maps to two *different* physical memory locations — each process gets its own private mapping. This scheme is called **virtual memory**.

Figure 8.13 gives a much fuller view of this private address space. Note that the **kernel** (the core of the operating system) is mapped into the high portion of every process's address space. The kernel uses this region for its own data — I/O buffers for files the process has opened, syscall data, etc. The process itself is not allowed to read or write this kernel region; the MMU catches attempts to do so and the OS turns them into a segmentation fault.

User and Kernel Modes

Ordinary processes need to be restricted in what they can do and access while running; however, the operating system needs full access to everything. So processes can run in two modes: **user mode** for restricted access and **kernel mode** for full access. This is determined

by a 'mode bit' inside a 'control register' that lets the CPU know whether the process running has access to sensitive parts of memory or can do only restricted tasks.

A process running in kernel mode can execute any instruction in the instruction set and access any memory location in the system. However, a processor in user mode is not allowed to execute privileged instructions that do things such as halt the processor, change the mode bit, or initiate an I/O operation. Nor is it allowed to directly reference code or data in the kernel area of the address space. Any such attempt results in a fatal protection fault. User programs must instead access kernel code and data indirectly via the system call interface.

Initially a process runs in user mode. It can be switched to kernel mode if an exception happens and an exception handler needs to be run. The mode bit is switched and then the exception handler is run in kernel mode. Then it is switched back to user mode to continue to run the rest of the process.

Context Switches

Every time a process needs to be switched out and in, its entire state (or 'context') needs to be switched in and out. The context of each process is kept by the kernel in its part of the memory space. An exception in hardware is used to do context switching.

Switching between processes is called **scheduling** and this is decided by the kernel's scheduler code.

It's not just interleaving processes on the same core that causes preemptive switching. If a process makes a system call, the process is switched out either because the operating system needs to run or because the process will be waiting on some long running action such as reading from the disk. Interrupts and other exceptions will cause processes to be switched out.

Looking ahead to Module 6: a major theme of this lesson is that processes are deliberately isolated — each one has its own private address space and its own copy of every global variable. That isolation is a feature when you want safety, but it also makes it harder for two pieces of work to *cooperate* on shared data (you can't just write to a global). Module 6 introduces **threads**, which look a lot like processes (each has its own logical control flow and stack) but deliberately *share* the same address space, file descriptor table, and signal handlers. That shared memory is what makes threads cheap to communicate through — and what makes them dangerous in the ways the next module will explore.

Lesson 6: Process Control

Part B: Processes (Lessons 5–7)

Process Control

This section goes over the use of some popular library functions for creating and managing processes. A process can create a separate process while it is running. The process doing the creating is called the **parent** process. The process being created is called the **child** process.

Obtaining Process ID

All processes have a unique ID called a **process ID** or just `pid`. A process can obtain its own pid by using `getpid()` function. It can get its parent's pid by using `getppid()`.

```
// caller
pid_t pid = getpid();
// parent of caller
pid_t ppid = getppid();
```

The type `pid_t` is currently an `int` type. This could change in the future.

Creating and Terminating Processes

Before we discuss creating and terminating processes, here are some states (the status of a process) that a process can be in:

- **running** - currently executing or waiting to be executed.
- **stopped** - suspended temporarily (signals: `SIGSTOP`, `SIGTSTP`, `SIGTTIN`, or `SIGTTOU`). While stopped the process won't be scheduled until the status changes back to running (signal: `SIGCONT`).
- **terminated** - the process is permanently suspended and won't ever be scheduled to run.

This is the result of:

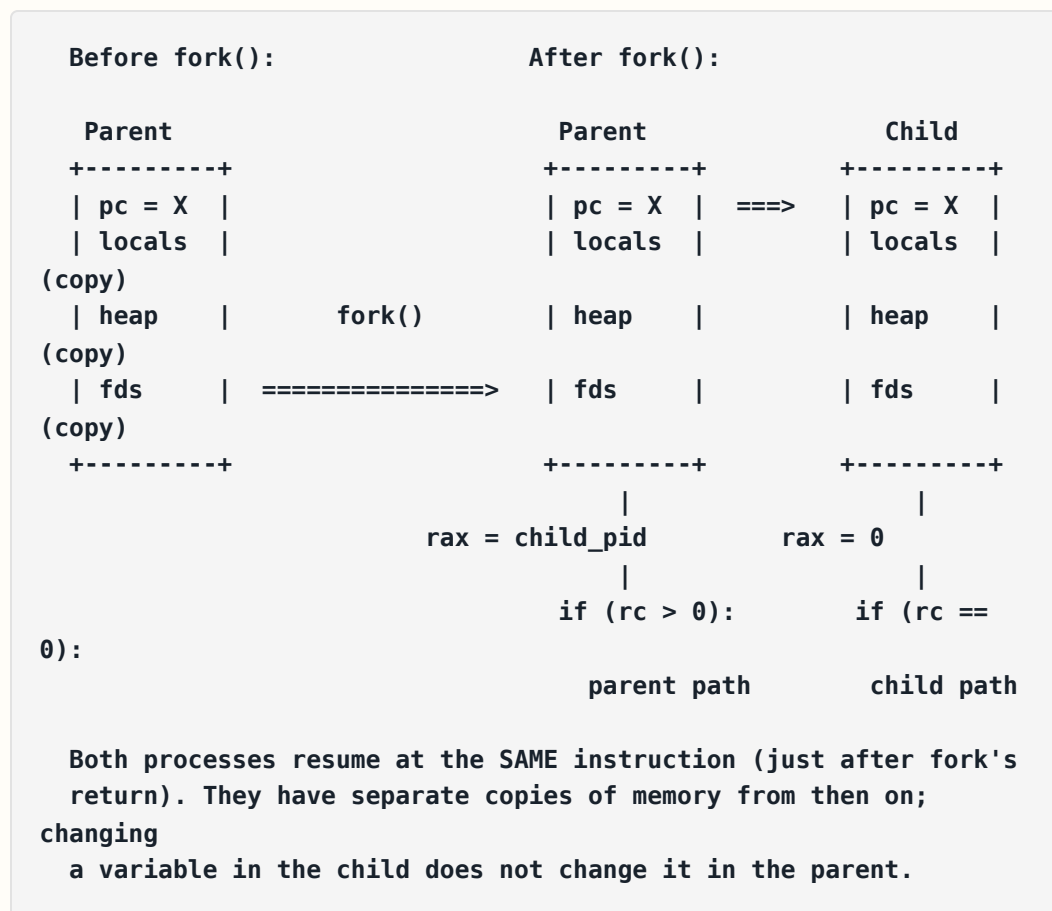
1. receiving a signal whose default action is to terminate the process (and that is not caught).

2. returning from its main function.
3. The `exit()` function has been called.

The main return value or the exit status number from an `exit()` call can be picked up by the parent program. This allows the parent program to find out how the child process was terminated.

Creating a Process Within a Process

The `fork()` function will create another process (a child process) within the running of another process (a parent process). The fork will create another separate process with a separate context but one where the initial values are copied from the parent's context. The thing that trips students up is the return value: `fork()` returns *once in each of two processes*. In the parent, it returns the child's `pid`; in the brand-new child, it returns 0. PIDs are always non-zero positive numbers, so the code after `fork()` can tell who it is by checking the return value:



Because context is duplicated, variables are in separate memory spaces but files are shared (since file descriptor numbers are copied between each process).

Reaping Child Processes

When the child process terminates, it is not removed from the system. The parent process needs to **reap** the child process first in order to completely terminate the child process. Until the child process is reaped, it is called a **zombie process**.

If the parent process terminates before the child process does, the child process becomes a child of the `init` process (first process in the chain of processes that start running when the operating system boots up). The `init` process will then reap the child process.

If the parent process is a long running program (such as a daemon that runs in the background permanently) then the child may become a permanent zombie. Zombies hold a slot in the kernel's process table and a small amount of bookkeeping memory; the practical worry is not raw memory pressure but eventual **PID exhaustion** — once the process table fills up, no more processes can be created on the system, including new instances of system services. It is important for long running programs to reap any child processes that have terminated.

To do this, use either the `wait()` function or the `waitpid()` function.

```
pid_t waitpid(pid_t pid, int* statusp, int options)
```

The three parameters are as follows:

- **pid** - This is either the pid of the child to reap (this will only reap one child) OR -1 in which any one child in a **wait set** is reaped (the wait set is all child processes of the calling process, unless options have changed the behavior; `waitpid` returns when a member of the wait set terminates). The pid of the reaped child is returned.
- **statusp** - This is a pointer to an integer. This is filled in by the function itself to give back information about how the process exited. There are a set of macros (these are pre-processor functions) that take the passed back int and either give further info or determine whether some state is true or false:
 - **WIFEXITED(status)** - True if the child terminated normally (via `exit()` or a return from main), regardless of what exit status was used. Use `WEXITSTATUS` to get the actual exit code.
 - **WEXITSTATUS(status)** - returns the exit status (the number put into an exit call or the return value in main). WARNING: Only the low 8 bits of the exit status are available (values 0-255). If the value is larger than 255, it is truncated to the low 8 bits (e.g., `exit(256)` results in a status of 0).
 - **WIFSIGNALED(status)** - True if terminated by a signal.

- **WTERMSIG(status)** - If above is true, this will return the number of the signal that did the termination.
- **WIFSTOPPED(status)** - True if the child is stopped (suspended).
- **WSTOPSIG(status)** - Gets the signal number if above is true.
- **WIFCONTINUED(status)** - True if child was restarted.
- **options** - This allows for modifying the behavior of the call by passing in a pre-defined constant. You can OR (|) together several option constants to get multiple settings.

The settings are:

- **WNOHANG** - return immediately: a positive PID if a child has a reportable state change, 0 if matching children exist but none is ready, or -1 on error (including `ECHILD` when none remain). Without this flag, `waitpid()` can block.
- **WUNTRACED** - `waitpid` already suspends the caller until a child changes state; this flag *extends* what counts as a reportable change. Without `WUNTRACED`, only terminated children cause `waitpid` to return. With `WUNTRACED`, stopped children (e.g., a child that received `SIGSTOP` or `SIGTSTP`) also cause it to return.
- **WCONTINUED** - similar extension: also report children that were stopped and have since been resumed via `SIGCONT`.

If an error has happened then `waitpid()` will send back a -1 and sets the `errno` (error code) to `ECHILD`. If the `waitpid()` was interrupted itself the error code will be `EINTR`.

The `wait()` function is as below:

```
pid_t wait(int *statusp);
```

It is a simpler version of `waitpid()` in that it's equivalent to:

```
waitpid(-1,&status,0);
```

In other words it will reap any child with no options (takes the default of waiting for a child to reap).

Putting Processes to Sleep

A function that can be used to suspend (or stop) the currently running process is `sleep()`:

```
unsigned int sleep(unsigned int secs);
```

The function takes a time in seconds and returns how many actual seconds were left. This is because the process can be woken early if interrupted by a caught signal (any signal for which a handler has been installed).

Another function is the `pause()` function, which stops the process until it receives a signal:

```
int pause(void);
```

Loading and Running Different Processes

The `fork` command creates a process that runs the same code. But what if you want to run different code from another program? This is where the `execve()` function comes in. It is the actual system call; the various `exec*` library functions are convenience wrappers around it.

The `execve()` function can be used to load a program and run it as a part of the process currently running. Note the current process becomes the loaded process so it no longer runs its own code. So it is said to NOT return to the same code if successful. It will return with a -1 if it is NOT successful.

```
int execve(const char *pathname,  
           char *const argv[], char *const envp[]);
```

The `pathname` is a path to the program. The `argv` and `envp` parameters are required: an array of argument strings to pass to the program and an array of environment variables (you may pass minimal arrays, but you cannot omit them). Note that `getenv()` returns the value of one named environment variable; to access the whole environment, use the global `environ` variable. (Note the qualifier order: `char *const argv[]` is an array of *constant pointers to char*, not `const char *argv[]` — this matters if you compile against the real `<unistd.h>` header.)

Note that if you call `execve()` within the program, the current program is replaced by the new program. If you want to keep the current program running, do a `fork()` first, and inside the child process, call the `execve()` function. This will keep the parent running as the current program but have the child become the new program.

You can look at the interesting example of a shell program in the textbook to see how `execve()` is used in code. I won't have you use `execve()` in labs or ask questions about it on the quiz/exam. So just basically know what it does.

Lesson 7: Examples of Using Fork

Part B: Processes (Lessons 5–7)

Examples of Using Fork to Create Child Processes

Below are some simple examples of using `fork()` and `waitpid()` to create and manage child processes. These will be helpful when doing the Module 5 lab. Note that these examples are under the Example Code submodule for Module 5.

Example 1: `fork1.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>          // for fork()
#include <sys/types.h>       // for pid_t
#include <sys/wait.h>        // for waitpid()

int main()
{
    pid_t cpid1;

    // up to this point only the parent is running
    cpid1 = fork();

    // after this point BOTH parent and child
    // are running the SAME code below
    // However, the cpid1 number is DIFFERENT
    // in the parent and the child.

    if(cpid1 < 0) // error condition
    {
        // only parent exists if this happens
        printf("ERROR in forking\n");
    }
    else if(cpid1 == 0) // this is the child
    {
        // This code will be run by the child
        // ONLY as the parent's cpid1 will not be 0.
        pid_t cpid2 = getpid(); // child's pid
        pid_t ppid2 = getppid(); // parent's pid
        printf("CHILD: child's pid is %d\n", cpid2);
        printf("CHILD: parent's pid is %d\n", ppid2);
    }
    // cpid1 > 0 so this is the parent, and cpid1 is the child's pid
    else
    {
        // This code will be run by the parent
        // ONLY as it will have non-zero returned
        pid_t ppid1 = getpid(); // parent's pid
        printf("PARENT: child's pid is %d\n", cpid1);
        printf("PARENT: parent's pid is %d\n", ppid1);
    }
    return 0;
}

```

Example Output (your output will have different pid numbers and might be in a different order):

```
PARENT: child's pid is 7491
PARENT: parent's pid is 7490
CHILD: child's pid is 7491
CHILD: parent's pid is 7490
```

One caveat about this sample output: the parent here never waits for the child, so the parent can exit before the child runs its `printf()` calls. If that happens, the child is reparented (as described in Lesson 6) and its "parent's pid" line will show the pid of `init` (or a subreaper process) rather than 7490. Adding a `wait()` call to the parent would guarantee the output shown.

Note that before any `fork` call, only the parent is running. After `fork` is called, there are two processes running the same code. The return value for `fork` is one of three things:

- `pid < 0` - This is an error where the `fork` couldn't be done. Assume only the parent is running at this point.
- `pid == 0` - This is what is passed back in the child process. You can use an `if` statement to process any code that the child needs to run.
- `pid > 0` - This is what is passed back in the parent process. In fact, the `pid` is the child's pid. Again, you can use an `if` statement to run any code that only the parent needs to run.

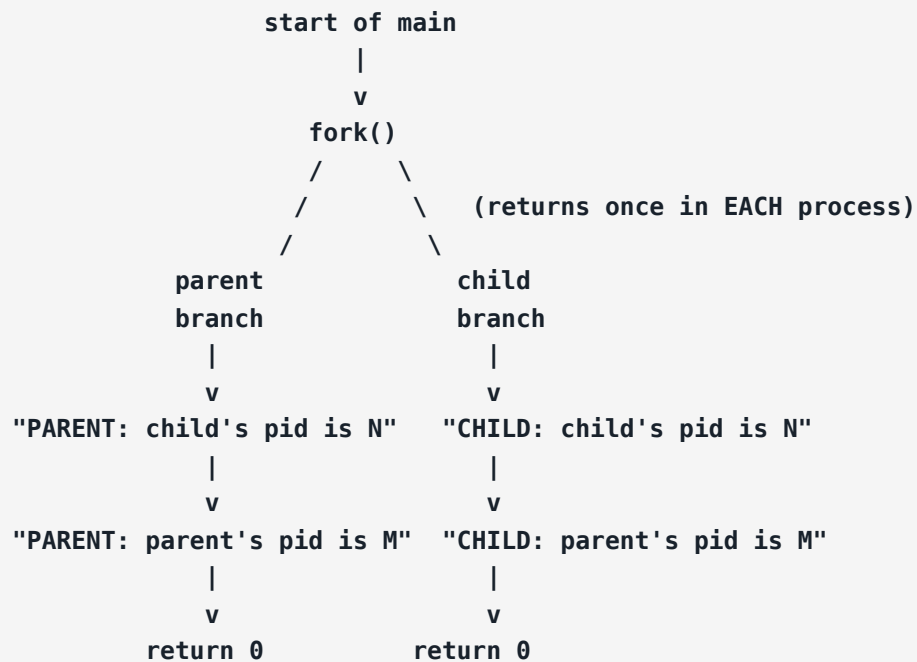
Note that the child's pid is available to the parent, but the child itself will need to call `getpid()` to get its own pid.

Control Order of Parent and Child

Since either process can be preempted at any time, this may change the order that the child and parent print to the terminal. There are some restrictions though. The order of output must match the order in which each `printf()` statement appears in each process. For example, the `printf()` statements for PARENT must be in the order shown in the code (they are only being run by the parent). However, you could have any CHILD `printf()` statements printed before, after or in-between any of the PARENT statements.

Figure 8.16 in Section 8.4.2 shows a technique called a **process graph** that you can use to determine the order that each process will do a task (print to terminal, etc). This technique will be handy for doing the Module 5 lab where you will need to determine the possible order of output to the terminal.

A process graph reads top-to-bottom along each branch (statements within a single process must execute in the order written), but the two branches can interleave freely. Here is the process graph for the program above:



Reading the graph:

- Each branch's statements happen in order from top to bottom.
- Statements on different branches can occur in any order relative to each other.

The printouts BETWEEN two parallel branches can be printed in any order. The printouts INSIDE one branch have to be in the order they are in. So the following output is possible:

```
PARENT: child's pid is 7491
CHILD: child's pid is 7491
PARENT: parent's pid is 7490
CHILD: parent's pid is 7490
```

because the order inside each branch is kept. However, the following output is NOT possible.

```
PARENT: parent's pid is 7490
PARENT: child's pid is 7491
CHILD: child's pid is 7491
CHILD: parent's pid is 7490
```

This is because it violates the order in the parent's branch.

Example 2: fork2.c


```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>          // for fork()
#include <sys/types.h>       // for pid_t
#include <sys/wait.h>        // for waitpid()

int global = 10;

int main()
{
    pid_t pid, cpid1;
    int number = 1;

    // up to this point only the parent is running
    // global is 10 and number is 1.
    cpid1 = fork();
    // after this point BOTH parent and child
    // are running. The child inherits the
    // variables declared before the fork
    // with their current values. However,
    // the 'number' variable is DIFFERENT
    // in each process.

    if(cpid1 < 0)
    {
        // only parent exists if this happens
        printf("ERROR in forking\n");
        return 0;
    }
    // these lines are run by BOTH processes in their own flow
    pid = getpid(); // gets either the child or parent pid
    printf("BEFORE: pid %d number is %d global is %d\n", pid, number,
global);
    if(cpid1 == 0) // this is the child
    {
        number = -100;
        global = -100;
    }
    else // pid > 0 so this is the parent
    {
        number = 200;
        global = 200;
    }
    sleep(1); // delay the AFTER prints so the BEFORE prints likely
              // come first. (sleep-based ordering is NOT a guarantee)

    // NOTE: this will print TWICE because two different
    // processes are going to print it.

```

```
    printf("AFTER: pid: %d number is %d global is %d\n", pid, number,
global);

    return 0;
}
```

Example Output (your output may be in a different order)

```
BEFORE: pid 7851 number is 1 global is 10
BEFORE: pid 7852 number is 1 global is 10
AFTER: pid: 7851 number is 200 global is 200
AFTER: pid: 7852 number is -100 global is -100
```

This is illustrating what happens with variables. Any variable declared either as a global variable or as a local one is not shared between the two processes. Note that they do share the same values just after the fork. But when they change the value in their respective flows, the change doesn't affect the other process's values.

Example 3: fork3.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>          // for fork()
#include <sys/types.h>       // for pid_t
#include <sys/wait.h>        // for waitpid()

int main()
{
    pid_t cpid1, pid;

    cpid1 = fork();

    if(cpid1 < 0)
    {
        // only parent exists if this happens
        printf("ERROR in forking\n");
        return 0;
    }

    if(cpid1 == 0) // this is the child
    {
        printf("CHILD's pid is %d\n", getpid());
        return 100;
    }

    // cpid1 > 0 so this is the parent (child may still be running)
    printf("PARENT's pid is %d\n", getpid());
    int status;
    pid_t wpid = waitpid(-1, &status, 0); // taking defaults
    if(WIFEXITED(status))
    {
        int chstat = WEXITSTATUS(status);
        printf("PARENT: child %d exited with a status of %d\n", wpid,
chstat);
    }
    return 0; // ONLY parent returns here
}

```

Example Output

```

PARENT's pid is 8864
CHILD's pid is 8865
PARENT: child 8865 exited with a status of 100

```

This is an example of using waitpid() to 'reap' a child process. Also, this picks up the child's return value in the parent, allowing for some information to be passed back to the parent.

I might point out that this method of getting information back to the parent is very limited. Only the low 8 bits of the exit status are available to the parent (values 0-255). If the return value is larger than 255, it is truncated to the low 8 bits (e.g., returning 256 results in a status of 0). There are better ways to pass information between processes, such as using shared memory via the `mmap()` system call, but they are beyond the scope of this course. Also, if processes need to share memory, it's much better to make them **threads** instead of separate processes. Threads are separate logical flows within the same process that share memory and context but have a different flow of instructions. Again, this is not something that is discussed in this course (although in my opinion they ought to).

Note that if the child terminates before the parent can get to the `waitpid` line, it will still be 'reaped' properly. This is because the child will be put into a 'zombie' state until the parent calls `waitpid`. Also, if the parent gets to the `waitpid` line before any child does terminate, then the program will hang until a child does terminate. Then it will 'reap' that child and go on. Which is why you need to use a while loop to 'reap' multiple children.

Example 4: fork4.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>          // for fork()
#include <sys/types.h>       // for pid_t
#include <sys/wait.h>        // for waitpid()

int main()
{
    int pid = 0;

    for(int i=1; i<=4; i++)
    {
        pid_t cpid = fork();
        if(cpid == 0) // this is a child
        {
            pid = getpid();
            printf("the %d child (pid %d) has been launched\n",i,pid);
            return i;
        }
    }

    // only parent gets this far
    int status;
    while ((pid = waitpid(-1, &status, 0)) > 0)
    {
        if (WIFEXITED(status))
            printf("child %d terminated normally with exit status=%d\n",pid,
WEXITSTATUS(status));
        else
            printf("child %d terminated abnormally\n", pid);
    }
    return 0;
}

```

Example Output:

```

the 2 child (pid 9536) has been launched
the 1 child (pid 9535) has been launched
the 4 child (pid 9538) has been launched
the 3 child (pid 9537) has been launched
child 9535 terminated normally with exit status=1
child 9536 terminated normally with exit status=2
child 9537 terminated normally with exit status=3
child 9538 terminated normally with exit status=4

```

This is an example of launching multiple child processes and reaping them in the end. Note that I need to either return, exit or break out of the loop when in the child processes. If this isn't done, the child processes will generate their own child processes and so on. Try removing the if statement inside the for loop and see what happens.

To see why the early `return` in the child matters, here is the process tree with and without it. With the early return, the parent simply has four children (five processes total). Without it, every process keeps running the loop and forking, so the process count doubles each iteration, ending at sixteen processes.

WITH the early return (correct):

Parent runs the loop 4 times. Each iteration spawns one child; the child immediately returns, so it does NOT execute any further `fork()` calls.

```
Parent
 /  |  \  \
C1 C2 C3 C4
```

Total processes: 4 children + 1 parent = 5.

Each child exits with a unique status (1, 2, 3, 4).

WITHOUT the early return (the bug):

After `fork()` returns, BOTH the parent and the child continue the for-loop. Every process keeps spawning new processes at every iteration. The number of processes doubles each iteration.

```
i=1:          P
             / \
i=2         P   C
           /\   /\
i=3        P C C C
          /\ /\ /\ /\
i=4    ...16 leaves...
```

Total processes after the loop: $2^4 = 16$, with 15 children (and no exit status is 1..4 anymore -- without the early return, every process falls through to the reaping loop and returns 0, so all 15 children exit with status 0).

Try it. Counting unique PIDs in the output is illuminating.

Also, note that in order to pick up all children, I need a while loop. The way it works is if there are any child processes in a zombie state, one of them will be reaped and their

information will be printed. Then waitpid is called again to pick up the next child process. If there are no child processes running or in a zombie state, waitpid will pass back a -1 and the while loop will end (since -1 is not greater than 0).

Lesson 8: Signals in Linux Systems

Part C: Signals (Lessons 8–12)

NOTE: This is a summary of chapter 8, section 8.5 in the textbook.

Signals in Linux Systems

Signals are higher-level or software-level exceptions that processes, including the operating system, can send to other processes (or to itself). These can be 'caught' by the process through a **signal handler**: a function that is to be called when the signal is caught by the process. If the process doesn't have a signal handler, the operating system has a default action that it takes.

The list of POSIX signals on the [Signals \(IPC\) wiki](#) shows all of the 31 standard signal types that can be sent in a Linux system. You don't need to memorize these signal types. Below is a list of some important signals:

- **SIGINT (2): default action: terminates**
this is sent to the foreground process running in a terminal when the user hits <ctrl> c. Also, can be sent to any user process by using the command "kill -2".
- **SIGFPE (8): default action: terminates with core dump**
Called a 'floating point exception', it is sent when dividing by 0 is attempted. Ironically, by default, floating point divide by zero isn't affected by this signal, only integers are.
- **SIGKILL (9): default action: terminates**
can send this by using the command **kill -9** followed by the PID of the process. This is one of two signals that can't be overridden by a signal handler.
- **SIGSEGV (11): default action: terminates with core dump**
the infamous 'segmentation fault' when accessing invalid memory.
- **SIGALRM (14): default action: terminates**
can be used to set up a one-shot timer. The `alarm()` function sends this signal once after n seconds. To get repeated alarms, `alarm()` must be called again (e.g., from within the signal handler).

- **SIGCHLD (17): *default action: ignores***
sent by the kernel to a parent process when one of its children terminates, stops, or resumes (the child does not call `kill()` itself; the kernel generates and delivers SIGCHLD on the parent's behalf).
- **SIGCONT (18): *default action: continues a stopped process***
Sent to resume a stopped process. A caught SIGCONT also resumes the process before its handler runs.
- **SIGSTOP (19): *default action: stops until SIGCONT***
Sent to a process to stop or suspend a process. This is NOT sent from the terminal when hitting `<ctrl> z`. SIGSTOP can be sent by any process with permission (e.g., `kill -STOP <pid>`) or by debuggers and job-control mechanisms. Like SIGKILL, its behavior **cannot** be caught, blocked, or ignored — these are the two signals that can't be overridden by a signal handler.
- **SIGTSTP (20): *default action: stops until SIGCONT***
Sent from the terminal when hitting `<ctrl> z`. Unlike SIGSTOP, **SIGTSTP can be caught, blocked, or ignored** by a signal handler — that's how programs like text editors and shells decide what cleanup to do (saving cursor position, restoring terminal modes, etc.) before suspending.

SIGCHLD and Reaping Children: The Classic Pattern

SIGCHLD ties directly back to the zombie problem from Lesson 6. Because the kernel sends SIGCHLD to the parent whenever a child terminates, a parent doesn't have to sit blocked in `waitpid()` waiting for children to finish. Instead it can install a handler for SIGCHLD (using the functions we'll see in the next lesson) and do the reaping there:

```
void sigchld_handler(int sig)
{
    int saved_errno = errno;
    int status;
    (void)sig;
    // reap EVERY zombie that is ready, without blocking
    for (;;) {
        pid_t pid = waitpid(-1, &status, WNOHANG);
        if (pid > 0) continue;
        if (pid == -1 && errno == EINTR) continue;
        break; // 0: none ready; -1: no children or another error
    }
    errno = saved_errno;
}
```

Include `<errno.h>` for `errno` and `<sys/wait.h>` for `waitpid()`. Saving and restoring `errno` prevents an asynchronous handler from changing an error value the interrupted code was about to inspect. The `WNOHANG` option (from Lesson 6) makes `waitpid()` return immediately with 0 when children remain but none has a reportable state change, or -1 with `ECHILD` when no matching children remain, so the handler never blocks. The reaping loop matters because ordinary **standard** signals are not queued: multiple occurrences can collapse into one pending signal. If three children terminate in quick succession, the parent may therefore observe only one pending `SIGCHLD`. Reaping in a loop until `waitpid()` has nothing left picks them all up. POSIX real-time signals have different queuing rules.

We'll discuss how a process can deal with some of these in a later lesson.

Lesson 9: Signal Management

Part C: Signals (Lessons 8–12)

Signal Management

A signal is sent from the operating system to one or more processes due to some system event. The event can be a run-time error, such as divide by zero, or the user typing `<ctrl> c` at a terminal. Many of the signals available are low-level events that are only handled by the operating system. Processes can send signals to other processes including to themselves.

All signals are processed by the operating system (even ones sent to itself). The operating system then records the signal as pending in the process's context. For standard signals, you can think of this as a bit mask where each bit position corresponds to a signal number. Pending signals are not received by the process until the process is in a state where the kernel can deliver them.

The operating system will check first another bit flag to see if this signal has been **blocked**. A process can block most signals (except SIGKILL and SIGSTOP). When a signal is blocked, it cannot be received by the process and stays a pending signal until unblocked. This blocks even the default behavior. Note that blocking does not get rid of the signal; once the signal is unblocked, it is delivered to the process. If a process wants to prevent even the default behavior of the signal, it can use the signal library to **ignore** the signal. We'll discuss the signal library in detail in later sections.

For **standard** signals, this bit-mask model means that multiple occurrences of the same signal can collapse while one is already pending; applications cannot count occurrences from deliveries. POSIX real-time signals use queued delivery under their specified rules. Signals of different types are tracked independently.

Group ID

Signals can target either a specific process or an entire process group, depending on how they are sent. For example, terminal-generated signals like `<ctrl> c` go to the foreground process group, while `kill(pid, sig)` targets one process. Each process belongs to one process

group and has a process group ID (pgrp or pgid). Any child processes that a process forks are automatically put into the process's own group unless changed with process-group APIs.

To get the pgid:

```
#include <unistd.h>
pid_t getpgrp(void);
```

To set the pgid for itself or another process:

```
#include <unistd.h>
int setpgid(pid_t pid, pid_t pgid);
```

The parameters are the pid of the process and the group id of the group to put the process in. There are some special values that do something different:

- **if pid is 0:** Set the current process to the given group pgid.
- **if pgid is 0:** Set the group ID to the given pid.
- **if both are 0:** Create a new group with the current process's current pid and put the current pid into this group.

Sending Signals

There are several ways to send a signal:

- **Sending Signals with the kill command:** The **kill** command can be used to send not just terminate signals but any type of signal. The signal can be sent to either a specific process or it can be sent to all processes in a group.

sending SIGKILL (signal 9) to a particular process (pid 15213, pgid 15111):

```
kill -9 15213
```

sending SIGKILL to all processes in a group, use a minus sign in front of the group ID:

```
kill -9 -15111
```

- **Sending signals with the keyboard:** Both **<ctrl> c** and **<ctrl> z** send a signal to every process in the foreground's process group. So it affects not only the starting process but all of the process's children. If you want to distinguish what gets killed or stopped, you will need to provide a signal handler.

- **Sending signals with the `kill()` function:** This function is in the C language signal library. It will send any signal (not just SIGKILL) to another process or itself:

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
```

`sig` is the signal to send. If `pid` is positive, the signal is sent only to that process. if `pid` is zero, the signal is sent to all processes in the current process's group including the process itself. if the `pid` is negative, the signal is sent to all processes in the process group whose ID is the absolute value of `pid` (including the current process if it belongs to that group).

- **Sending Signals with the `alarm()` function:** The alarm function in the signal library sends a SIGALRM to the process itself after n seconds.

```
#include <unistd.h>

unsigned int alarm(unsigned int secs);
```

The return value is how many seconds were left on the last alarm sent or 0 if there was no previous alarm. Setting secs to 0 will cancel any pending alarms and not set any new ones. Note that this only sets one alarm not an interval of alarms sent every n seconds. There are other functions for doing this. Check out `timer_create` function for how to do repeated time intervals.

Receiving Signals: Signal Handlers

Whenever the kernel is about to resume user-mode execution for a process or thread (for example, after a system call or when scheduling it onto the CPU), it checks for pending unblocked signals. If a standard signal is pending and not blocked, the kernel delivers it and clears that pending bit. Standard signals are not queued, and if multiple different standard signals are pending, the delivery order is unspecified.

The process can do three things with this signal:

1. If the disposition for this signal has been set to the special value `SIG_IGN`, the kernel silently discards the signal.
2. If there is a signal handler, it will be called.
3. If there is no signal handler and it is not being ignored, it will do the default action for this signal: this can be terminate, terminate with a core dump, stop the process, continue a

stopped process, or ignore the signal.

Signal Handlers

A signal handler modifies the default action for a particular signal. This is a function that is 'installed' using the **signal()** function from the signal library:

```
#include <signal.h>
typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

It takes a signal number and a pointer to a function. A pointer to a function allows a function to be passed into another function to be called by that other function. To pass in a pointer to a function, just use the function name without any quotes around it. The function needs to have the header:

```
void {name}(int sig)
```

where {name} is replaced with the function name. the passed-in **sig** value is the signal number, as you can set the same handler to handle more than one signal (you need to install each one individually).

Instead of a pointer to a function, the **handler** argument can be one of two constants:

- **SIG_IGN** : This sets the signal to be ignored completely. Works with all signals except SIGKILL and SIGSTOP.
- **SIG_DFL** : This sets the signal to have the default behavior.

The **signal()** function returns a pointer to the previous signal handler (which may be a user-defined function, **SIG_DFL**, or **SIG_IGN**). It returns **SIG_ERR** on error.

The Better Alternative: sigaction()

Use `sigaction()` for any code you actually want to ship. The `signal()` function is shown above because it's short and convenient for in-class examples, but it has a portability gotcha: on classic System V Unix, `signal()` resets the handler to `SIG_DFL` after each delivery, so a second occurrence of the same signal kills the process unless you reinstall the handler. Linux glibc and BSD do the more useful thing (the handler stays installed), but if you want predictable behavior across platforms, or if you need to control exactly which signals are blocked while the handler runs, or if you want interrupted system calls to restart automatically, you want `sigaction()`.

```
#include <signal.h>

int sigaction(int signum,
               const struct sigaction *act,
               struct sigaction *oldact);

struct sigaction {
    void      (*sa_handler)(int); /* simple-style handler, like signal()
    */
    void      (*sa_sigaction)(int, siginfo_t *, void *); /* extended */
    sigset_t   sa_mask;           /* signals to block during the handler
    */
    int        sa_flags;           /* SA_RESTART, SA_NOCLDSTOP, ... */
    /* ... */
};
```

One detail about the struct: in glibc, `sa_handler` and `sa_sigaction` occupy the same storage (a union), so you set one or the other, never both — adding `SA_SIGINFO` to `sa_flags` is what tells the kernel to use the extended `sa_sigaction` form.

A typical use looks like this:

```
struct sigaction sa;
sa.sa_handler = my_handler;
sigemptyset(&sa.sa_mask); /* don't block extra signals */
sa.sa_flags = SA_RESTART; /* restart interrupted syscalls */

if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction");
    exit(1);
}
```

The two flags worth knowing:

- `SA_RESTART` — if a signal is delivered while the process is blocked in a "slow" syscall like `read()` or `accept()`, restart that syscall transparently after the handler returns instead of failing it with `EINTR`. It does not restart every blocking operation: for example, `sleep()` and `sigsuspend()` still return after a handler. Check the interrupted function's contract and handle `EINTR` or partial progress where appropriate, even when using `SA_RESTART`.
- `SA_NOCLDSTOP` — for `SIGCHLD` only: don't deliver `SIGCHLD` when a child is just stopped (only when it terminates).

Throughout the rest of this module we'll keep using `signal()` in the small examples for brevity, but `sigaction()` is the right answer in real code.

Lesson 10: Blocking and Unblocking Signals

Part C: Signals (Lessons 8–12)

Blocking and Unblocking Signals

You can block signals from your C code by using a series of functions for blocking signals.

The function `sigprocmask()` takes an action and a bit flag (or a `mask`). The mask has a 1 for every signal needed to be blocked or a 0 if the signal is not to be blocked.

```
#include <signal.h>
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

The `set` argument is a mask with the signals' bits set to 1 that are to be affected. `oldset` is the current (before changing) blocking bit flag.

The `how` is a constant that indicates what to do with the `set` mask:

- **SIG_BLOCK** - Just add any 1 bits to the current bit flag. This turns on blocking.
- **SIG_UNBLOCK** - unset (set to 0) any 1 bits in the current bit flag. This turns off blocking.
- **SIG_SETMASK** - this replaces the block bit flag with the passed-in `set` mask.

`sigprocmask()` returns 0 on OK and -1 on error.

`sigprocmask()` can also be used to get the current signal mask without changing anything by passing `NULL` as the second parameter (when `set` is `NULL`, the `how` argument is ignored, so any value works) and a pointer to a `sigset_t` for the third parameter. The third parameter will hold the current signal mask when the function returns.

Setting up `set` Mask

There are several other functions that can be used to create the `set` mask from scratch. These are used with any of the three `sigprocmask` actions (`SIG_BLOCK`,

SIG_UNBLOCK, or SIG_SETMASK) to build the signal set.

The following (sigemptyset) creates an empty signal mask to be filled in by the other functions below:

```
int sigemptyset(sigset_t *set);
```

The following (sigfillset) will create a signal mask that has all signals set to one. This is useful if you want to block every signal. But remember that SIGKILL and SIGSTOP cannot be blocked.

```
int sigfillset(sigset_t *set);
```

The following (sigaddset) will add (set to 1) a signal to the passed-in signal mask:

```
int sigaddset(sigset_t *set, int signum);
```

The following (sigdelset) will delete (set to 0) a signal from the passed-in signal mask.

```
int sigdelset(sigset_t *set, int signum);
```

All of these functions return a 0 if OK, -1 on error.

There is one more that can be useful for finding out if a signal is blocked or not:

```
int sigismember(const sigset_t *set, int signum);
```

Sends back a 1 if a member (is set to 1), a 0 if not a member (set to 0), or a -1 on error.

An Example: Blocking the SIGINT and SIGTSTP Signals

Here is a snippet of code that blocks the SIGINT and SIGTSTP signals. Afterwards this program can't be killed by using `<ctrl> c` OR suspended by using `<ctrl> z`. It can still be killed by using `kill -9` though.

```
sigset_t mask, prev_mask; // type to use for bit mask

sigemptyset(&mask); // make a blank bit mask
sigaddset(&mask, SIGINT); // add SIGINT to bit mask
sigaddset(&mask, SIGTSTP); // add SIGTSTP to bit mask

/* Block SIGINT and SIGTSTP and save previous blocked set */
sigprocmask(SIG_BLOCK, &mask, &prev_mask);
....

/* Restore previous blocked set, unblocking SIGINT
and SIGTSTP */
sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

Lesson 11: Writing Signal Handlers

Part C: Signals (Lessons 8–12)

Writing Signal Handlers

Signal handlers are run differently than ordinary functions. There is no call located in the rest of the code to this function. Please don't call them yourself in your code. They are called from outside by the operating system while the rest of the process is running (usually this is done just when the process gets back from being preempted). A call can happen at any point in the running of your code. It can happen in the middle of processing a C code line (C code lines are usually equivalent to several machine code lines).

This causes some problems when sharing resources between signal handler and the main code in your program. For example, let's say an int type global variable called `flag` is being read or written to by both the handler and the main part of the program. The handler decides to write to `flag` at the same time the main program is reading `flag`. Since there may be several machine-level instructions in writing to memory, the main program may not pick up the right value.

To avoid this problem, whenever a global variable is written to and read by both the handler and the main program, use the 'volatile' keyword and the 'sig_atomic_t' type.

```
volatile sig_atomic_t flag;
```

Note that global variables that are constant and don't change don't need to be declared this way.

Also, remember that your handler is blocking the main code from running to completion. So it's suggested to keep signal handlers short and simple so they run quickly. If the main code needs to know that a signal was handled, use a global flag.

Also, there are some functions in the C standard libraries that may cause problems when called from the signal handler. Figure 8.33 in section 8.5.5 has a list of `async-signal-safe` functions that you can use. Note that `printf()` is not on the list. For output from the signal handler, you can use the `write()` function. Example 1 and Example 2 in the next

lesson show how to use it to print a message. For Example 4 in the next lesson, I used a flag to get the main to print a signal from a child process.

Lesson 12: Signal Handler Examples

Part C: Signals (Lessons 8–12)

Signal Handler Examples

Here are some signal handler examples.

Example 1: Simple SIGINT Handler

This installs a simple handler to pick up any SIGINT signals. These are caused by hitting **<ctrl> c** at the terminal. Compile and run this program, then hit **<ctrl> c** to see what happens.

```

#include <unistd.h> // for sleep and write functions
#include <stdio.h> // for printf
#include <signal.h> // needed for the signal handling

#define MAX 10
#define STDOUT 1

void sigint_handler(int signum)
{
    char mess[100] = "\nOuch!\n";

    write(STDOUT, mess, 7);
}

int main(){
    int signum = SIGINT;

    //install signal handler for SIGINT
    signal(signum, sigint_handler);

    // do some busy work
    int i =0;
    while(i < MAX)
    {
        sleep(1);
        i++;
    }
    printf("I've had enough! Goodbye!\n");
    return 0;
}

```

Note that I'm using the `write()` function within the signal handler to be on the safe side. The write function takes a file handle (1 is for standard out which means print to the terminal), an array of char, and the number of bytes to send, which is 7.

Example 2: Blocking SIGTSTP and SIGINT

This program shows how to block signals from being delivered. While signals are blocked, any signal handlers for those signals will not execute. Once the signals are unblocked, any pending signals that arrived during the blocked period will then be delivered.

```

#include <unistd.h>
#include <stdio.h>
#include <signal.h>

#define STDOUT 1

void sigint_handler(int signum) {
    char msg[] = "\nCaught SIGINT!\n";
    write(STDOUT, msg, sizeof(msg) - 1);
}

void busyWork() {
    for (int i = 5; i > 0; i--) {
        printf(" Working... %d seconds left\n", i);
        sleep(1);
    }
}

int main() {
    // Install handler for SIGINT
    signal(SIGINT, sigint_handler);

    printf("Phase 1: Signals are NOT blocked. Try Ctrl+C.\n");
    busyWork();

    // Block SIGINT and SIGTSTP
    sigset_t mask, prev_mask;
    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);
    sigaddset(&mask, SIGTSTP);
    sigprocmask(SIG_BLOCK, &mask, &prev_mask);

    printf("\nPhase 2: SIGINT and SIGTSTP are BLOCKED. Try Ctrl+C
now.\n");
    busyWork();

    // Restore the previous signal mask (unblock)
    printf("\nPhase 3: Unblocking signals now...\n");
    sigprocmask(SIG_SETMASK, &prev_mask, NULL);
    // Any SIGINT that arrived during Phase 2 will be delivered here

    printf("Done.\n");
    return 0;
}

```

Notice that during Phase 2, hitting **<ctrl> c** appears to do nothing. But once we restore the previous mask with `SIG_SETMASK`, the pending SIGINT is delivered and the handler runs.

Remember: blocked signals are not lost, they are held as pending until unblocked.

One warning: this program installs no handler for SIGTSTP, so if you press <ctrl> z during Phase 2, the pending SIGTSTP will take its default action (stop the process) the moment it is unblocked in Phase 3 — the program will appear to hang before printing "Done." (it is just stopped; type `fg` to resume it).

Example 3: Using a global flag

This is a complete example of using a global flag to communicate between a signal handler and the main program. The handler only sets the flag (which is safe), and the main loop checks it. Note the use of `volatile sig_atomic_t` to ensure the compiler does not optimize away reads of the flag.

```

#include <unistd.h>
#include <stdio.h>
#include <signal.h>

volatile sig_atomic_t flag = 0;

void signal_handler(int signum) {
    if (signum == SIGINT)
        flag = 1;
    else if (signum == SIGTSTP)
        flag = 2;
}

int main() {
    // Install the same handler for both signals
    signal(SIGINT, signal_handler);
    signal(SIGTSTP, signal_handler);

    printf("Running. Press Ctrl+C to quit, Ctrl+Z to poke.\n");

    while (1) {
        sleep(1);
        if (flag == 1) {
            printf("Got a SIGINT! We'll quit now!\n");
            break;
        } else if (flag == 2) {
            printf("Got a SIGTSTP! Not quitting.\n");
            flag = 0; // reset flag so we can detect the next one
        }
    }

    printf("Goodbye.\n");
    return 0;
}

```

Key points: the handler itself does nothing dangerous -- it just sets an integer. The main loop polls the flag each iteration. This pattern avoids calling `printf()` or other non-async-signal-safe functions inside the handler.

One limitation to be aware of: because there is only one application flag, closely spaced signals can be silently lost at this layer. If a SIGINT and a SIGTSTP arrive within the same polling interval, the second overwrites the first; and the `flag = 0` reset can erase a signal delivered just after the check. Using one `sig_atomic_t` flag per signal avoids cross-signal overwrites, though standard signals of the *same* type can still coalesce while pending as described in Lesson 9.

Example 4: Child Processes and Signals

This example demonstrates sending signals between a parent and child process. When `fork()` is called, the child inherits the parent's signal handlers. However, each process has its own copy of global variables, so the flag in the parent and the flag in the child are **independent** -- you cannot use a global variable to communicate between processes. Instead, we use `kill()` to send signals between them.

```

#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/wait.h>

#define STDOUT 1

volatile sig_atomic_t got_signal = 0;

void handler(int signum) {
    // Use write() since it is async-signal-safe
    if (signum == SIGUSR1) {
        char msg[] = " [Handler] Received SIGUSR1\n";
        write(STDOUT, msg, sizeof(msg) - 1);
        got_signal = 1;
    }
}

int main() {
    // Install handler for SIGUSR1 with sigaction (preferred over
    signal()).
    // Note: SA_RESTART restarts slow system calls like read(), but it
    does
    // NOT apply to sleep() -- sleep() always returns early when a handler
    // runs. The while(!got_signal) loops below are what make this program
    // correct: an early return from sleep() just re-checks the flag.
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
        perror("sigaction");
        exit(1);
    }
    // Child will inherit this handler across fork().

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // --- Child process ---
        printf("Child (PID %d): sleeping, waiting for signal from
parent...\n",

```

```

        getpid());

    // Wait for a signal from the parent
    while (!got_signal) {
        sleep(1);
    }

    printf("Child: got the signal! Sending SIGUSR1 back to parent (PID
%d).\n",
        getppid());
    kill(getppid(), SIGUSR1);

    printf("Child: done.\n");
    exit(0);

} else {
    // --- Parent process ---
    printf("Parent (PID %d): child is PID %d\n", getpid(), pid);
    sleep(2); // give the child time to set up

    printf("Parent: sending SIGUSR1 to child...\n");
    kill(pid, SIGUSR1);

    // Now wait for the child to signal us back
    printf("Parent: waiting for child to signal back...\n");
    while (!got_signal) {
        sleep(1);
    }

    printf("Parent: got the signal back from child!\n");

    // Wait for child to finish
    waitpid(pid, NULL, 0);
    printf("Parent: child has exited. Done.\n");
}

return 0;
}

```

Trace through the code for each process:

1. The parent installs a SIGUSR1 handler, then `fork()` s. Both processes now have the handler.
2. The child sleeps, polling its `got_signal` flag.
3. The parent sleeps briefly, then sends SIGUSR1 to the child via `kill(pid, SIGUSR1)` .

4. The child's handler fires, setting the child's copy of `got_signal` to 1.
5. The child wakes up, sees the flag, and sends SIGUSR1 back to the parent via `kill(getppid(), SIGUSR1)`.
6. The parent's handler fires, setting the parent's copy of `got_signal` to 1.
7. The parent wakes up, reaps the child with `waitpid()`, and both exit.

Key takeaway: after `fork()`, each process has its own address space. The `got_signal` variable is separate in each process. To communicate between processes, you must use mechanisms like signals, pipes, or shared memory.

A Note on Waiting for Signals: `sigsuspend()`

The `while (!got_signal) sleep(1);` loops in these examples are a simple workaround: they are correct, but they poll, so the program can waste up to a full second noticing a signal that has already arrived. You might be tempted to just unblock the signal and call `pause()` instead — but that has a race: if the signal is delivered in the gap *between* the unblocking `sigprocmask()` call and the `pause()` call, the handler runs first and `pause()` then sleeps forever waiting for a signal that already came. The production tool is `sigsuspend()`, which atomically swaps in a mask and waits, so there is no gap:

```
sigset_t mask, prev;
sigemptyset(&mask);
sigaddset(&mask, SIGUSR1);
sigprocmask(SIG_BLOCK, &mask, &prev); // block SIGUSR1 first
while (!got_signal)
    sigsuspend(&prev);                // atomically unblock + wait
sigprocmask(SIG_SETMASK, &prev, NULL); // restore the old mask
```

See CS:APP section 8.5.7 for the full discussion of this pattern.

Module 5 Summary

Wrap up

In this module, we moved from the instruction level to the operating system level. We discussed exceptional control flow and the four classes of exceptions. We learned about processes -- how the operating system creates, schedules, and terminates them -- and practiced using `fork`, `exec`, and `wait` to create and manage child processes. Finally, we studied signals: what they are, how to write signal handlers, and how to safely block and unblock them.

Key Takeaways

- The sole job of the CPU is to process one instruction after another. Going from one instruction to the next is called a control transfer, and a sequence of control transfers is called the control flow. Exceptions are abrupt changes in the control flow triggered by some change in the processor's state.
- In modern Linux systems, there are 256 possible exceptions. Numbers 0-31 are hardware exceptions defined by the architecture (Intel x86-64), and numbers 32-255 are interrupts and traps defined by the operating system.
- A process is an instance of a program in execution. Each process has its own private address space and the illusion that it has exclusive use of the processor. The `fork` system call creates a new child process that is a copy of the parent, and the child and parent continue executing concurrently from the point of the fork.
- The `fork` call returns twice: once in the parent (returning the child's PID) and once in the child (returning 0). Programs that use `fork` can produce different outputs on different runs because the order in which the parent and child execute is not guaranteed.
- Signals are software-level exceptions that processes can send to other processes. A signal can be caught by installing a signal handler -- a function that the operating system calls when the signal is delivered to the process. If no handler is installed, the operating system takes a default action.
- Signal handlers run asynchronously, outside the normal flow of the program. Because of this, signal handlers must be written carefully: they should use only async-signal-safe functions. A related technique worth knowing about (though not demonstrated in this module's examples) is protecting shared data structures by temporarily blocking signals around a critical section with `sigprocmask` — Lesson 10 covers the blocking mechanics.

Additional resources

All links open in a new tab.

[Exceptions and Processes Slides set](#) [link]

[Exceptions and Processes](#) [link to video lecture from Carnegie Mellon University]

[Exceptional Control Flow - Signal Slides set](#) [link]

Exceptional Control Flow - Signals [\[link to video lecture from Carnegie Mellon University\]](#)

Module 6 Overview

In this Module 6: Threads and Synchronization, you will learn about:

- Threads: what they are, how they differ from processes, and why they are a fundamental building block of concurrent programs.
- The POSIX threads (pthreads) library: how to create, manage, and join threads in C programs.
- Race conditions: what happens when multiple threads access shared data without coordination, and why the results are unpredictable.
- Mutexes: the primary synchronization primitive for protecting critical sections of code.
- Condition variables and semaphores: more advanced synchronization tools for coordinating thread activity.
- Classic concurrency problems: the Dining Philosophers problem and the Reader-Writer problem, which illustrate the challenges of concurrent programming.

Background

In earlier modules, you learned how to create new processes using **fork()** and how processes communicate using signals. Processes are heavyweight: each fork creates a complete copy of the address space. Threads offer a lighter-weight alternative. Multiple threads within the same process share the same address space (code, heap, global variables) but each has its own stack and set of registers. This makes threads cheaper to create and allows them to communicate through shared memory. However, shared memory introduces new problems: when multiple threads read and write the same data simultaneously, the results can be unpredictable and incorrect. This module teaches you how to write correct concurrent programs by using synchronization primitives.

Learning Objectives

At the end of this learning module, you will be able to:

1. Explain the difference between threads and processes, including what is shared and what is private to each thread.
2. Create and join POSIX threads using **pthread_create()** and **pthread_join()**, passing arguments and receiving return values.
3. Identify race conditions in concurrent code and explain why they produce incorrect results.
4. Use mutexes (**pthread_mutex_t**) to protect critical sections and prevent race conditions.
5. Use condition variables (**pthread_cond_t**) and semaphores (**sem_t**) to coordinate threads that must wait for specific conditions.
6. Describe classic concurrency problems (Dining Philosophers, Reader-Writer) and implement solutions that avoid deadlock and starvation.
7. Identify and avoid common pitfalls in concurrent programming, including deadlock, starvation, and data races.

Learning Activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Read Lessons 1 through 6 and study the code examples carefully. Type them in and run them yourself.
2. Compile and experiment with the provided C programs. Observe how race conditions produce different results on each run.
3. Complete Discussion 6
4. Take Quiz 6
5. Review additional resources in the Module 6 Summary

Important: The code examples in this module require the pthreads library. When compiling, prefer the **-pthread** flag. For example: **gcc -o myprogram myprogram.c -pthread**

If you have questions, you can post them in the Course Lobby or send your instructor an email.

Lesson 1: Threads vs. Processes

Introduction

You already know how to create a new process using **fork()**. When a process calls `fork()`, the operating system creates an entirely new process with its own copy of the address space. The parent and child processes do not share memory: changes made in one process are invisible to the other (unless you set up explicit inter-process communication like pipes or shared memory segments).

A **thread** is a different kind of concurrent execution unit. Threads exist *within* a process. A single process can contain multiple threads, and all threads within that process share the same address space. This means they share the same code, the same global variables, and the same heap memory. However, each thread has its own **stack** and its own set of **CPU registers** (including its own program counter).

What Is a Thread?

A thread is sometimes called a **lightweight process**. It is the smallest unit of execution that the operating system can schedule. Every process has at least one thread (the **main thread**), which is the thread that begins executing at **main()**.

When you create additional threads, they begin executing a function you specify. All threads in the process run concurrently. On a multi-core CPU, threads can run truly in parallel (at the same instant on different cores). On a single-core CPU, the operating system rapidly switches between threads, giving the illusion of parallelism.

Threads vs Processes: Key Differences

Property	Process (fork)	Thread (pthread_create)
Address space	Each process has its own copy	All threads share the same address space
Code segment	Shared (read-only)	Shared
Global variables	Each process has its own copy	Shared among all threads
Heap memory	Each process has its own copy	Shared among all threads
Stack	Each process has its own	Each thread has its own
Registers	Each process has its own	Each thread has its own
File descriptors	Copied at fork time	Shared among all threads
Creation cost	Expensive (copy address space)	Cheap (just allocate a stack)
Communication	Requires IPC (pipes, signals, shared memory)	Direct access to shared variables
Isolation	Strong: crash in one process does not affect another	Weak: a bug in one thread can corrupt data for all threads

The Thread Memory Model

The diagram below shows how memory is laid out when a process has multiple threads. Notice that the code, global/static data, and heap are shared, while each thread has its own stack.

+-----+		
	PROCESS	
+-----+		
	Code (Text) Segment [SHARED]	
	(program instructions)	
+-----+		
	Global / Static Data [SHARED]	
	(global variables, static variables)	
+-----+		
	Heap [SHARED]	
	(malloc'd memory)	
	grows upward -->	
+-----+		
	... free space ...	
+-----+		
	Thread 3 Stack [PRIVATE]	
	(local variables, return addresses)	
+-----+		
	Thread 2 Stack [PRIVATE]	
	(local variables, return addresses)	
+-----+		
	Thread 1 Stack (Main) [PRIVATE]	
	(local variables, return addresses)	
+-----+		

Compare this to what happens with fork():

+-----+			+-----+		
PARENT PROCESS			CHILD PROCESS		
+-----+			+-----+		
	Code	[OWN COPY]		Code	[OWN COPY]
	Globals	[OWN COPY]		Globals	[OWN COPY]
	Heap	[OWN COPY]		Heap	[OWN COPY]
	Stack	[OWN COPY]		Stack	[OWN COPY]
+-----+			+-----+		
Completely separate address spaces			Completely separate address spaces		

Why Use Threads?

There are several important reasons to use threads instead of (or in addition to) processes:

1. Performance and Efficiency

Creating a thread is much faster than creating a new process. With `fork()`, the operating system must duplicate the entire address space (even with copy-on-write optimization, there is overhead for setting up page tables). Creating a thread only requires allocating a new stack and some bookkeeping. Context switching between threads within the same process is also faster than switching between processes, because the memory mappings do not need to change.

2. Shared Memory Communication

Because threads share the same address space, they can communicate simply by reading and writing shared variables. With separate processes, you need to set up pipes, shared memory segments, message queues, or other IPC mechanisms. Thread communication is more direct, though as we will see, it requires careful synchronization.

3. Parallelism

On a multi-core system, threads allow your program to do real work in parallel. For example, if you need to process a large array, you can divide it among several threads, each working on a portion simultaneously. This can give you a significant speedup.

4. Responsiveness

In a program with a user interface, one thread can handle user input while another thread performs a long computation in the background. Without threads, the program would freeze during the computation.

When to Use Threads vs fork()

Use **threads** when:

- The concurrent tasks need to share data frequently.
- You need low overhead for creating and managing concurrent tasks.
- The tasks are parts of the same logical program (e.g., a web server handling multiple requests).

Use **fork()** when:

- You need strong isolation between concurrent tasks (a bug in one should not crash the other).
- You want to run a completely different program (fork + exec).
- Security is a concern and you want to limit what each task can access.

What Threads Share and What They Do Not

Let us be precise about what is shared and what is private:

Shared by all threads in a process:

- The code (text) segment
- Global and static variables
- The heap (anything allocated with malloc)
- Open file descriptors
- Signal handlers and signal dispositions
- The current working directory
- User and group IDs

Private to each thread:

- Thread ID (pthread_t)
- Stack (local variables, function call chain)
- CPU registers (including the program counter and stack pointer)
- The errno variable (each thread has its own copy)
- Signal mask (which signals are blocked)
- Thread-specific data (set with pthread_setspecific)

A Simple Illustration

Here is a small program to demonstrate that threads share global variables but have separate stacks (local variables). We will learn the details of `pthread_create` and `pthread_join` in the next lesson; for now, just focus on the output.

```
#include <stdio.h>
#include <pthread.h>

int shared_global = 0; /* shared by all threads */

void *thread_func(void *arg) {
    int local_var = 42; /* private to this thread */
    shared_global = 100; /* modifies the shared global */

    printf("Thread: shared_global = %d\n", shared_global);
    printf("Thread: local_var = %d\n", local_var);
    printf("Thread: address of local_var = %p\n",
           (void *)&local_var);
    return NULL;
}

int main() {
    int local_var = 7; /* private to main thread */
    pthread_t tid;

    printf("Main before: shared_global = %d\n", shared_global);

    pthread_create(&tid, NULL, thread_func, NULL);
    pthread_join(tid, NULL);

    printf("Main after: shared_global = %d\n", shared_global);
    printf("Main: local_var = %d\n", local_var);
    printf("Main: address of local_var = %p\n",
           (void *)&local_var);

    return 0;
}
```

Compile and run:

```
gcc -pthread -o threads_demo threads_demo.c
./threads_demo
```

Sample output:

```
Main before: shared_global = 0
Thread: shared_global = 100
Thread: local_var = 42
Thread: address of local_var = 0x7f2a1c000ef4
Main after: shared_global = 100
Main: local_var = 7
Main: address of local_var = 0x7ffd5a3b1c2c
```

Notice the following:

- The thread changed **shared_global** to 100, and main can see that change (it prints 100 after the thread finishes). This confirms that global variables are shared.
- The thread's **local_var** is 42, but main's **local_var** is still 7. Local variables live on each thread's private stack and are independent.
- The addresses of the two **local_var** variables are very different, confirming they are on separate stacks.

Warning: The fact that threads share memory is both their greatest strength and their greatest danger. If two threads modify the same variable at the same time without synchronization, the results are unpredictable. We will explore this problem in detail starting in Lesson 3.

Key Takeaways

- A thread is a lightweight unit of execution within a process.
- All threads in a process share the same address space (code, globals, heap).
- Each thread has its own stack and registers.
- Threads are cheaper to create and faster to context-switch than processes.
- Shared memory makes communication easy but introduces the risk of race conditions.
- Use threads for concurrent tasks that need to share data; use `fork()` when you need isolation.

Lesson 2: Creating and Joining Threads

The POSIX Threads Library (pthreads)

On Linux and other Unix-like systems, the standard library for thread programming is **POSIX Threads**, commonly called **pthread**s. To use it, include the header `<pthread.h>` and compile/link with `-pthread` (preferred on Linux). The older `-lpthread` form also works for many of the examples in this course.

```
#include <pthread.h>
```

pthread functions and types generally use the **pthread_** prefix. Most pthread routines that report an error return 0 on success or an error number directly; they do *not* report that error through `errno`. A few routines, such as `pthread_self()` and `pthread_equal()`, return other kinds of values, so always check the specific function's contract.

The pthread_t Type

Every thread is identified by a value of type **pthread_t**. This is an opaque type; you should not make assumptions about its internal structure. You use it to refer to a specific thread when calling functions like `pthread_join()`.

```
pthread_t tid; /* declare a thread ID variable */
```

Creating a Thread: pthread_create()

The function **pthread_create()** creates a new thread. Its signature is:

```
int pthread_create(pthread_t *thread,
                  const pthread_attr_t *attr,
                  void *(*start_routine)(void *),
                  void *arg);
```

Let us break down each parameter:

Parameter	Description
thread	Pointer to a pthread_t variable. On success, the new thread's ID is stored here.
attr	Pointer to thread attributes (stack size, scheduling, etc). Pass NULL for default attributes. You will almost always pass NULL .
start_routine	A pointer to the function the new thread will execute. This function must take a single void * parameter and return void * .
arg	The argument to pass to start_routine. Use NULL if you do not need to pass anything.

The third parameter deserves special attention. The type **void *(*start_routine)(void *)** means "a pointer to a function that takes a void pointer and returns a void pointer." Your thread function must have exactly this signature.

A Minimal Example

```
#include <stdio.h>
#include <pthread.h>

void *say_hello(void *arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t tid;

    /* Create a new thread that runs say_hello */
    pthread_create(&tid, NULL, say_hello, NULL);

    /* Wait for the thread to finish */
    pthread_join(tid, NULL);

    printf("Thread has finished. Back in main.\n");
    return 0;
}
```

Compile and run:

```
gcc -pthread -o hello_thread hello_thread.c
./hello_thread
```

Output:

```
Hello from the new thread!
Thread has finished. Back in main.
```

Waiting for a Thread: pthread_join()

The function **pthread_join()** blocks the calling thread until the specified thread terminates. It is analogous to **waitpid()** for processes.

```
int pthread_join(pthread_t thread, void **retval);
```

Parameter	Description
thread	The thread ID of the thread to wait for.
retval	If not NULL, the return value of the thread's start function is stored here. Pass NULL if you do not need the return value.

Important: If you do not call `pthread_join()` (and the thread is not detached), the thread's resources are not fully released when it terminates. This is similar to a zombie process. Always join your threads or detach them.

To detach a thread, use **`pthread_detach()`**:

```
int pthread_detach(pthread_t thread);
```

A detached thread's resources are reclaimed automatically when it exits, but it can no longer be joined, so you cannot wait for it or retrieve its return value.

Watch out: Returning from `main()` terminates the *entire process*, including any threads that are still running. If your worker threads must finish their work, either join them all before `main` returns or have `main` call **`pthread_exit()`** instead of returning, which ends only the main thread and lets the others keep running.

Passing Arguments to Threads

The fourth parameter to `pthread_create` is a **`void *`**. This is the generic pointer type in C: it can point to anything. To pass data to your thread function, you pass the address of the data, and inside the function you cast the void pointer back to the appropriate type.

Passing a Single Integer

A common pattern is to pass an integer to a thread. You must be careful: you cannot simply cast an int to a void pointer and back (the result of this conversion is implementation-defined, though it works on most platforms). The safe approach is to pass a pointer to the integer:

```
#include <stdio.h>
#include <pthread.h>

void *print_number(void *arg) {
    int *num_ptr = (int *)arg;    /* cast void* back to int* */
    printf("Thread received: %d\n", *num_ptr);
    return NULL;
}

int main() {
    pthread_t tid;
    int value = 42;

    pthread_create(&tid, NULL, print_number, &value);
    pthread_join(tid, NULL);

    return 0;
}
```

Output:

```
Thread received: 42
```

The Danger of Passing Stack Variables to Multiple Threads

When creating multiple threads in a loop, a very common mistake is to pass the address of the loop variable. Consider this **BROKEN** example:

```

/* WARNING: This code has a bug! */
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5

void *print_id(void *arg) {
    int *id_ptr = (int *)arg;
    printf("I am thread %d\n", *id_ptr);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        /* BUG: passing address of i, which changes! */
        pthread_create(&threads[i], NULL, print_id, &i);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}

```

This is a **race condition**. The loop variable `i` changes as the loop progresses, and all threads receive a pointer to the same variable. By the time a thread runs and dereferences the pointer, `i` may have already moved to the next value. Worse, since C99 the loop-scoped `i` ceases to exist once the loop finishes, so a thread that runs late may dereference a dangling pointer to a variable that no longer exists—undefined behavior, not merely a stale value. You might see output like:

```

I am thread 2
I am thread 3
I am thread 5
I am thread 5
I am thread 5

```

The correct approach is to give each thread its own copy of the argument. One way is to use an array:


```

/* CORRECT version */
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5

void *print_id(void *arg) {
    int *id_ptr = (int *)arg;
    printf("I am thread %d\n", *id_ptr);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        thread_args[i] = i; /* each thread gets its own copy */
        pthread_create(&threads[i], NULL, print_id,
                      &thread_args[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    return 0;
}

```

Now each thread has its own integer that will not change. The output (in some order) will be:

```

I am thread 0
I am thread 1
I am thread 2
I am thread 3
I am thread 4

```

(The order may vary because threads run concurrently, but each thread will correctly print its own ID.)

Passing a Struct

If you need to pass multiple values to a thread, define a struct and pass a pointer to it:

```

#include <stdio.h>
#include <pthread.h>

typedef struct {
    int id;
    char name[32];
    double value;
} thread_data_t;

void *worker(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;
    printf("Thread %d: name=%s, value=%.2f\n",
        data->id, data->name, data->value);
    return NULL;
}

int main() {
    pthread_t tid;
    thread_data_t data = {1, "alpha", 3.14};

    pthread_create(&tid, NULL, worker, &data);
    pthread_join(tid, NULL);

    return 0;
}

```

Output:

```
Thread 1: name=alpha, value=3.14
```

Returning Values from Threads

A thread function returns **void ***. You can return a value by returning a pointer. The calling thread retrieves this value through **pthread_join()**.

Warning: Do not return a pointer to a local variable! The thread's stack is deallocated when it finishes. Use malloc to allocate the return value on the heap, or use a global/static variable.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *compute_sum(void *arg) {
    int *range = (int *)arg; /* array of two ints: start, end */
    int start = range[0];
    int end = range[1];

    /* Allocate result on the heap so it survives after return */
    long *result = malloc(sizeof(long));
    *result = 0;

    for (int i = start; i <= end; i++) {
        *result += i;
    }

    printf("Thread computed sum from %d to %d = %ld\n",
        start, end, *result);
    return (void *)result;
}

int main() {
    pthread_t tid;
    int range[] = {1, 1000000};
    void *retval;

    pthread_create(&tid, NULL, compute_sum, range);
    pthread_join(tid, &retval);

    long *sum = (long *)retval;
    printf("Main received result: %ld\n", *sum);

    free(sum); /* free the memory allocated by the thread */
    return 0;
}

```

Compile and run:

```

gcc -pthread -o sum_thread sum_thread.c
./sum_thread

```

Output:

```
Thread computed sum from 1 to 1000000 = 500000500000  
Main received result: 500000500000
```

Creating Multiple Threads

Here is a complete example that creates several threads, each performing work, and then joins them all:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define NUM_THREADS 4

void *do_work(void *arg) {
    int id = *(int *)arg;
    long sum = 0;

    /* Each thread sums a different range */
    int start = id * 250000 + 1;
    int end = (id + 1) * 250000;

    for (int i = start; i <= end; i++) {
        sum += i;
    }

    printf("Thread %d: sum of %d to %d = %ld\n",
        id, start, end, sum);

    long *result = malloc(sizeof(long));
    *result = sum;
    return (void *)result;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];
    long total = 0;

    /* Create all threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        int ret = pthread_create(&threads[i], NULL, do_work,
                                &thread_ids[i]);

        if (ret != 0) {
            fprintf(stderr, "pthread_create failed: %s\n",
                strerror(ret));
            exit(1);
        }
    }

    /* Join all threads and collect results */
    for (int i = 0; i < NUM_THREADS; i++) {
        void *retval;
        pthread_join(threads[i], &retval);
    }
}

```

```

        total += *(long *)retval;
        free(retval);
    }

    printf("Total sum: %ld\n", total);
    printf("Expected:  %ld\n", (long)1000000 * 1000001 / 2);

    return 0;
}

```

This program divides the task of summing numbers 1 through 1,000,000 among 4 threads. Each thread computes a partial sum, and main adds them together.

Compile and run:

```

gcc -pthread -o parallel_sum parallel_sum.c
./parallel_sum

```

Sample output:

```

Thread 0: sum of 1 to 250000 = 31250125000
Thread 1: sum of 250001 to 500000 = 93750125000
Thread 2: sum of 500001 to 750000 = 156250125000
Thread 3: sum of 750001 to 1000000 = 218750125000
Total sum: 500000500000
Expected:  500000500000

```

Error Handling

Error-reporting pthread routines such as `pthread_create()` and `pthread_join()` return 0 on success and an error number directly on failure. They do *not* return -1 and report that error through `errno`. Check the contract and return value:

```

int ret = pthread_create(&tid, NULL, my_func, NULL);
if (ret != 0) {
    fprintf(stderr, "pthread_create failed: %s\n",
            strerror(ret));
    exit(1);
}

```

Include `<string.h>` to use `strerror()`.

Compiling with pthreads

You must link the pthreads library when compiling. Either of these works:

```
gcc -pthread -o myprogram myprogram.c
gcc -o myprogram myprogram.c -lpthread
```

The **-pthread** flag (without the "l") is actually preferred on Linux because it sets both the compiler and linker flags correctly. Both will work for the examples in this course.

Key Takeaways

- **pthread_create()** creates a new thread that begins executing a specified function.
- **pthread_join()** waits for a thread to finish and retrieves its return value.
- Thread functions must have the signature **void *func(void *arg)**.
- Pass arguments via the void* parameter; cast to the correct type inside the thread function.
- Be careful when passing addresses of loop variables; give each thread its own copy.
- Return values from threads using malloc'd memory; do not return pointers to local variables.
- Use `-pthread` for both compilation and linking. `-lpthread` only requests a library at link time and is not a general substitute for the compiler option.

Lesson 3: Race Conditions

Introduction

In Lesson 1, we learned that threads share the same address space, including global variables and heap memory. In Lesson 2, we learned how to create threads and pass data to them. Now we encounter the fundamental problem of concurrent programming: **race conditions**.

A **race condition** occurs when two or more threads access shared data at the same time, and at least one of them modifies it. The final result depends on the unpredictable order in which the threads execute, which is determined by the OS scheduler and can vary from run to run.

A Motivating Example: The Shared Counter

Consider a simple task: two threads each increment a shared counter 1,000,000 times. You would expect the final value to be 2,000,000. Let us see what actually happens.


```

/* race_condition.c - demonstrates a race condition */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define ITERATIONS 1000000

int counter = 0; /* shared global variable */

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        counter++; /* this is NOT atomic! */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Expected: %d\n", 2 * ITERATIONS);
    printf("Actual:    %d\n", counter);

    return 0;
}

```

Compile and run this several times. Note that we deliberately use `-O0` and `-pthread`: at higher optimization levels, gcc may hoist `counter` into a register inside the loop, which (ironically) makes the bug invisible because the compiler is allowed to assume there's no data race. We *want* to see the bug here, so we turn optimization off:

```

gcc -O0 -pthread -o race race_condition.c
for i in $(seq 1 10); do ./race; done

```

You will get different results each time! Here is what typical output might look like:

Expected: 2000000

Actual: 1247832

Expected: 2000000

Actual: 1389241

Expected: 2000000

Actual: 1156903

The actual value is almost always less than 2,000,000 and varies from run to run. This is a race condition in action.

If you do *not* see different values across runs — for example, if every run prints exactly 2000000 — the most likely culprits are: (1) you forgot `-O0` and optimization changed how the data race manifests, (2) you ran on a single-core VM where the threads happen to never get preempted mid-update, or (3) ITERATIONS is too small. Try cranking ITERATIONS up to 10 million and running 20 times in the loop. If you still see no variation, the bug exists but isn't being triggered — that's a defining feature of race conditions, and exactly why "it works on my machine" is a deeply unsatisfying answer in concurrent code.

Why Does This Happen?

The statement `counter++` looks like a single operation in C, but it is actually **three** separate operations at the machine level:

1. **Read** the current value of counter from memory into a register.
2. **Modify** the value by adding 1 in the register.
3. **Write** the new value from the register back to memory.

This is called a **read-modify-write** sequence. The problem is that a thread can be interrupted (by the OS scheduler) between any of these steps. Consider this scenario with two threads:

Suppose `counter = 5`

Thread A:

1. Read `counter` (5)

2. Add 1 (6)

3. Write 6 to `counter`

Thread B:

1. Read `counter` (5)

2. Add 1 (6)

3. Write 6 to `counter`

Result: `counter = 6` (should be 7!)

Both threads read the value 5, both add 1 to get 6, and both write 6 back. One increment is completely lost! This is called a **lost update**.

An **atomic** operation is indivisible with respect to other threads observing the same object. This does not mean the thread cannot be interrupted. `counter++` on an ordinary shared C integer is not an atomic read-modify-write, regardless of whether the compiler emits one instruction or several. The load/add/store interleaving above illustrates a common failure on our target; it is not a limit on what C's undefined data-race behavior permits.

The Concept of a Critical Section

A **critical section** is a section of code that accesses shared data and must not be executed by more than one thread at a time. In our counter example, the line `counter++` is a critical section.

More formally, a critical section is any code where:

- A shared resource (variable, file, data structure) is being accessed, AND
- At least one thread is writing to (modifying) that resource.

To prevent race conditions, we must ensure **mutual exclusion**: only one thread can be in the critical section at a time. We will learn how to achieve this using mutexes in Lesson 4.

Non-Deterministic Behavior

One of the most frustrating aspects of race conditions is that they cause **non-deterministic behavior**. The program does not always produce the same output for the same input. Whether a race condition manifests depends on:

- How the OS scheduler happens to interleave the threads

- The number of CPU cores on your machine
- System load from other processes
- Timing variations at the microsecond level

This means:

- A buggy program might appear to work correctly during testing and fail in production.
- Adding a print statement can change the timing enough to make the bug disappear (the **Heisenbug** phenomenon).
- Running under a debugger changes timing and may hide the bug.
- The bug might appear on one machine but not another.

Key insight: Race conditions are among the hardest bugs to find and fix. A program with a race condition is incorrect even if it sometimes produces the right answer. "It works on my machine" is not evidence of correctness.

A More Detailed Example

Let us expand our example to make the race condition more visible. This version prints each thread's contribution and shows the final discrepancy:

```

/* race_detailed.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define ITERATIONS 1000000
#define NUM_THREADS 4

int counter = 0;

void *increment(void *arg) {
    int id = *(int *)arg;
    int local_count = 0;

    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
        local_count++;
    }

    printf("Thread %d: incremented %d times\n",
           id, local_count);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    printf("Starting %d threads, each incrementing %d times\n",
           NUM_THREADS, ITERATIONS);
    printf("Expected final counter: %d\n\n",
           NUM_THREADS * ITERATIONS);

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, increment, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nActual final counter: %d\n", counter);
    printf("Lost updates: %d\n",
           NUM_THREADS * ITERATIONS - counter);
}

```

```
    return 0;
}
```

Sample output:

```
Starting 4 threads, each incrementing 1000000 times
Expected final counter: 4000000
```

```
Thread 0: incremented 1000000 times
Thread 2: incremented 1000000 times
Thread 1: incremented 1000000 times
Thread 3: incremented 1000000 times
```

```
Actual final counter: 2641873
Lost updates: 1358127
```

Notice that each thread correctly performed 1,000,000 increments (as counted by its local variable), but more than a million updates were lost due to the race condition. The exact numbers will differ each time you run the program.

It Is Not Just Counters

Race conditions can occur with any shared data, not just integer counters. Here are other common scenarios:

- **Linked list operations:** Two threads inserting into a linked list simultaneously can corrupt the list structure, causing lost nodes or infinite loops.
- **Bank account transfers:** Reading a balance, checking if sufficient, and deducting must be done atomically.
- **Array updates:** Two threads writing to the same array index.
- **File operations:** Two threads writing to the same file descriptor.
- **Checking then acting:** Thread A checks if a resource is available, but Thread B grabs it before A can act.

A Linked List Race Condition

Consider two threads both trying to insert at the head of a linked list:

```

/* Shared linked list head */
node_t *head = NULL;

/* Thread A and Thread B both do this: */
void insert(int value) {
    node_t *new = malloc(sizeof(node_t));
    new->data = value;
    new->next = head;    /* Step 1: read head */
    head = new;          /* Step 2: update head */
}

```

If both threads execute Step 1 before either executes Step 2, both new nodes will point to the old head. Then whichever thread writes to head last will win, and the other thread's node will be lost (a memory leak and data loss).

```

Initial state: head --> [C] --> [D] --> NULL

Thread A: new_a->next = head    (new_a --> [C])
Thread B: new_b->next = head    (new_b --> [C])
Thread A: head = new_a          (head --> [A] --> [C] --> [D])
Thread B: head = new_b          (head --> [B] --> [C] --> [D])

Result: Node A is lost! head --> [B] --> [C] --> [D]

```

Identifying Race Conditions

To identify potential race conditions in your code, ask these questions:

1. Is there any data that is accessible by more than one thread?
2. Is at least one thread writing to (modifying) that data?
3. Can the accesses happen simultaneously (without synchronization)?

If the answer to all three is yes, you have a potential race condition.

Note: If all threads only *read* shared data (and no one writes), there is no race condition.

Multiple readers are safe. The problem arises when at least one writer is present.

What Does the C Standard Say?

The C11 standard (and later) formally defines a **data race** as undefined behavior. This means the compiler is allowed to assume your program is free of data races. If your program has one, the compiler might optimize your code in ways that make the bug even worse. For example,

the compiler might reorder instructions, cache values in registers indefinitely, or eliminate "redundant" reads. This is another reason why race conditions must be fixed with proper synchronization, not with tricks like adding volatile keywords.

Can We Fix This Without Synchronization?

You might be tempted to try fixes like:

- **Making the variable volatile:** This tells the compiler not to cache the variable in a register, but it does NOT make the read-modify-write atomic. It does not prevent race conditions.
- **Using a temporary variable:** This does not help because the fundamental problem is the non-atomic read-modify-write.
- **Adding sleep() calls:** This might change timing but does not guarantee correctness.
- **Sprinkling printf() everywhere:** Same story — it changes the timing without fixing the bug. (See the misconception below for what printf actually guarantees.)

The only correct solution is to use proper **synchronization primitives**. We will learn about the most important one, the **mutex**, in the next lesson.

Common misconception — " `printf` is automatically synchronized." This is partially true and very misleading.

- What POSIX *does* guarantee: a single call to `printf` on a given `FILE*` appears atomic relative to other calls on the same stream — you won't see one thread's bytes shuffled into the middle of another thread's `printf`.
- What POSIX does *not* guarantee: anything about your program's logical state. If two threads both do `printf("counter = %d\n", counter)` concurrently, the lines won't be physically intermixed, but the values printed are still subject to the race — you may see the same value twice, miss values, or print stale values cached in a register.

The shared-counter program above, with a `printf` in the middle of the loop, would still lose updates. `printf` protects its own output buffer, not your data.

Key Takeaways

- A **race condition** occurs when multiple threads access shared data concurrently and at least one writes to it.

- On an ordinary shared integer, **counter++** performs a read-modify-write that is not atomic; its instruction sequence depends on the compiler.
- Race conditions cause **non-deterministic behavior**: different results on every run.
- A **critical section** is code that accesses shared data and must be executed by only one thread at a time.
- Race conditions can affect any shared data structure, not just counters.
- The C standard defines data races as **undefined behavior**.
- Volatile, sleep, and other tricks do not fix race conditions. Proper synchronization is required.

Lesson 4: Mutexes

Introduction

In Lesson 3, we saw that race conditions occur when multiple threads access shared data without coordination. The solution is to enforce **mutual exclusion**: only one thread can execute the critical section at a time. The primary tool for achieving mutual exclusion in pthreads is the **mutex** (short for "mutual exclusion").

A mutex is like a lock on a door. Before entering the critical section, a thread must **lock** the mutex. If another thread already holds the lock, the requesting thread **blocks** (waits) until the lock is released. When a thread is done with the critical section, it **unlocks** the mutex, allowing another waiting thread to proceed.

The Mutex API

The pthreads library provides the following functions and types for working with mutexes:

Function / Type	Purpose
pthread_mutex_t	The mutex data type
PTHREAD_MUTEX_INITIALIZER	Static initializer for a mutex (used for global/static mutexes)
pthread_mutex_init(&mutex, attr)	Initialize a mutex at runtime (used for local/dynamic mutexes). Pass NULL for attr to get default behavior.
pthread_mutex_lock(&mutex)	Acquire the lock. Blocks if already locked by another thread.
pthread_mutex_unlock(&mutex)	Release the lock. Another waiting thread can now acquire it.
pthread_mutex_destroy(&mutex)	Free resources associated with the mutex. Call when done with it.

Initializing a Mutex

There are two ways to initialize a mutex:

Static Initialization (for global or static mutexes)

```
/* Simple and preferred for global mutexes */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

This is the easiest approach when the mutex is declared at global scope.

Dynamic Initialization (for local or heap-allocated mutexes)

```
pthread_mutex_t lock;
pthread_mutex_init(&lock, NULL); /* NULL = default attributes */
```

Use this when the mutex is a local variable or when you need to set special attributes. When using dynamic initialization, you must call **pthread_mutex_destroy()** when you are done with the mutex.

Locking and Unlocking

The basic pattern for protecting a critical section is:

```
pthread_mutex_lock(&lock);    /* acquire the lock */

/* ... critical section ... */
/* only one thread at a time can be here */

pthread_mutex_unlock(&lock);  /* release the lock */
```

When a thread calls **pthread_mutex_lock()**:

- If the mutex is **unlocked**, the thread acquires it immediately and continues.
- If the mutex is **locked** by another thread, the calling thread blocks (sleeps) until the mutex becomes available.

When a thread calls **pthread_mutex_unlock()**:

- The mutex is released. If any threads are waiting on this mutex, one of them is woken up and acquires it.

Rule: A mutex must be unlocked by the same thread that locked it. It is an error for one thread to unlock a mutex that was locked by a different thread.

Fixing the Race Condition

Let us fix the counter program from Lesson 3 by adding a mutex:

```

/* fixed_counter.c - race condition fixed with a mutex */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define ITERATIONS 1000000

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);    /* enter critical section */
        counter++;
        pthread_mutex_unlock(&lock); /* leave critical section */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Expected: %d\n", 2 * ITERATIONS);
    printf("Actual:   %d\n", counter);

    return 0;
}

```

Compile and run:

```

gcc -pthread -o fixed fixed_counter.c
./fixed
./fixed
./fixed

```

Output (every time):

Expected: 2000000
Actual: 2000000

The answer is now correct every time. The mutex ensures that only one thread can execute the **counter++** line at a time, preventing the read-modify-write interleaving we saw in Lesson 3.

A Complete Example with Dynamic Initialization

Here is a more complete example using dynamic initialization and proper cleanup:

```

/* mutex_dynamic.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 500000

typedef struct {
    int counter;
    pthread_mutex_t lock;
} shared_data_t;

void *increment(void *arg) {
    shared_data_t *data = (shared_data_t *)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&data->lock);
        data->counter++;
        pthread_mutex_unlock(&data->lock);
    }

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    shared_data_t shared;

    /* Initialize shared data */
    shared.counter = 0;
    pthread_mutex_init(&shared.lock, NULL);

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, &shared);
    }

    /* Join threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected: %d\n", NUM_THREADS * ITERATIONS);
    printf("Actual:    %d\n", shared.counter);

    /* Clean up */
    pthread_mutex_destroy(&shared.lock);
}

```

```
    return 0;
}
```

Output:

```
Expected: 2000000
Actual:   2000000
```

Protecting Multiple Operations

Sometimes the critical section involves more than one line of code. You must hold the lock for the entire sequence of operations that must be atomic. For example, transferring money between two accounts:

```
/* Both accounts share the same mutex */
pthread_mutex_lock(&bank_lock);
if (accounts[from] >= amount) {
    accounts[from] -= amount;
    accounts[to] += amount;
}
pthread_mutex_unlock(&bank_lock);
```

You would not want to lock and unlock around each individual line, because another thread could see an inconsistent state where money has left one account but not yet arrived in the other.

Performance Implications

Mutexes add overhead. Every lock and unlock operation requires atomic CPU instructions and potential context switches. If you lock and unlock very frequently (like locking for every single counter increment in a tight loop), the overhead can be significant.

Here are some strategies to manage performance:

Minimize Time in the Critical Section

Do as little work as possible while holding the lock. Move any computation that does not need shared data outside the critical section:


```

/* Slower: unnecessary work inside lock */
pthread_mutex_lock(&lock);
result = expensive_calculation(input);
shared_array[index] = result;
pthread_mutex_unlock(&lock);

/* Faster: only the shared access is inside the lock */
result = expensive_calculation(input);
pthread_mutex_lock(&lock);
shared_array[index] = result;
pthread_mutex_unlock(&lock);

```

Accumulate Locally, Then Update Shared Data Once

```

/* Slow: lock/unlock 1,000,000 times */
void *increment_slow(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

/* Faster: lock/unlock only once */
void *increment_fast(void *arg) {
    int local_sum = 0;
    for (int i = 0; i < ITERATIONS; i++) {
        local_sum++;
    }
    pthread_mutex_lock(&lock);
    counter += local_sum;
    pthread_mutex_unlock(&lock);
    return NULL;
}

```

The second version is much faster because each thread accumulates its count in a private local variable and only acquires the lock once at the end.

Deadlock

A **deadlock** occurs when two or more threads are each waiting for a lock held by the other, so none of them can make progress. This is a permanent state: the program hangs forever.

Example of Deadlock

```
pthread_mutex_t lock_a = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_b = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg) {
    pthread_mutex_lock(&lock_a);    /* holds A */
    sleep(1); /* simulate some work */
    pthread_mutex_lock(&lock_b);    /* waits for B */

    /* ... critical section ... */

    pthread_mutex_unlock(&lock_b);
    pthread_mutex_unlock(&lock_a);
    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lock_b);    /* holds B */
    sleep(1); /* simulate some work */
    pthread_mutex_lock(&lock_a);    /* waits for A */

    /* ... critical section ... */

    pthread_mutex_unlock(&lock_a);
    pthread_mutex_unlock(&lock_b);
    return NULL;
}
```

Here, Thread 1 holds lock_a and waits for lock_b. Thread 2 holds lock_b and waits for lock_a. Neither can proceed. The program is deadlocked.

```
Thread 1:  holds lock_a  ---waits for--->  lock_b
                                              |
                                              held by
                                              |
Thread 2:  holds lock_b  ---waits for--->  lock_a
                                              |
                                              held by
                                              |
                                              Thread 1 (circular!)
```

Four Conditions for Deadlock

All four of these conditions must be true simultaneously for deadlock to occur:

1. **Mutual exclusion:** At least one resource is held in a non-shareable way.
2. **Hold and wait:** A thread holds one resource while waiting for another.
3. **No preemption:** Resources cannot be forcibly taken from a thread.
4. **Circular wait:** There is a circular chain of threads, each waiting for a resource held by the next.

Preventing Deadlock

The most practical way to prevent deadlock is to break the **circular wait** condition by always acquiring locks in a consistent, global order:

```
/* SAFE: both threads lock in the same order (A first, then B) */

void *thread1(void *arg) {
    pthread_mutex_lock(&lock_a);    /* lock A first */
    pthread_mutex_lock(&lock_b);    /* then lock B */
    /* ... critical section ... */
    pthread_mutex_unlock(&lock_b);
    pthread_mutex_unlock(&lock_a);
    return NULL;
}

void *thread2(void *arg) {
    pthread_mutex_lock(&lock_a);    /* lock A first (same order!)
*/
    pthread_mutex_lock(&lock_b);    /* then lock B */
    /* ... critical section ... */
    pthread_mutex_unlock(&lock_b);
    pthread_mutex_unlock(&lock_a);
    return NULL;
}
```

If all threads always acquire locks in the same order, the circular wait cannot form, and deadlock is impossible.

Common Mutex Mistakes

Here are some common mistakes to avoid:

- **Forgetting to unlock:** If a thread returns or exits without unlocking a mutex, other threads will wait forever.
- **Locking twice without unlocking:** Calling `pthread_mutex_lock` on a mutex you already hold causes undefined behavior (for the default mutex type, this typically causes the

thread to deadlock, blocking forever waiting for itself to release the lock). With

`PTHREAD_MUTEX_ERRORCHECK` type, an error code is returned instead.

- **Using different mutexes:** Two threads must lock the *same* mutex to achieve mutual exclusion. If thread A locks `mutex_1` and thread B locks `mutex_2`, they are not coordinating at all.
- **Unlocking a mutex you do not hold:** Only the thread that locked a mutex should unlock it.

```
/* BUG: forgot to unlock in the error case! */
pthread_mutex_lock(&lock);
if (error_condition) {
    return NULL; /* OOPS! mutex is still locked forever */
}
/* ... do work ... */
pthread_mutex_unlock(&lock);

/* FIXED: unlock before returning */
pthread_mutex_lock(&lock);
if (error_condition) {
    pthread_mutex_unlock(&lock); /* unlock first! */
    return NULL;
}
/* ... do work ... */
pthread_mutex_unlock(&lock);
```

trylock: Non-Blocking Lock Attempt

Sometimes you want to attempt to acquire a lock without blocking. The function `pthread_mutex_trylock()` does this:

```
int ret = pthread_mutex_trylock(&lock);
if (ret == 0) {
    /* lock acquired successfully */
    /* ... critical section ... */
    pthread_mutex_unlock(&lock);
} else {
    /* lock is held by someone else */
    /* do something else instead of waiting */
}
```

This is useful in situations where a thread has other work it could do instead of blocking.

In real code, check specifically for **EBUSY** (the return value meaning "the lock is currently held"); other nonzero returns such as **EINVAL** indicate a real error, not a held lock, and should be handled as failures.

Key Takeaways

- A **mutex** enforces mutual exclusion: only one thread can hold it at a time.
- Protect every critical section with **pthread_mutex_lock()** and **pthread_mutex_unlock()**.
- Initialize with **PTHREAD_MUTEX_INITIALIZER** for global mutexes or **pthread_mutex_init()** for dynamic ones.
- Always unlock the mutex when leaving the critical section, including error paths.
- **Deadlock** occurs when threads form a circular chain of lock dependencies. Prevent it by always acquiring locks in a consistent order.
- Minimize time spent holding a lock for better performance.
- When done, call **pthread_mutex_destroy()** on dynamically initialized mutexes.

Lesson 5: Condition Variables and Semaphores

Introduction: Why Mutexes Alone Are Not Enough

Mutexes solve the problem of ensuring only one thread accesses shared data at a time. But there is another common pattern that mutexes alone cannot handle efficiently: one thread needs to **wait for a specific condition** that another thread will make true.

Consider the **producer-consumer** problem: a producer thread generates data and places it in a buffer, while a consumer thread removes data from the buffer and processes it. The consumer must wait when the buffer is empty, and the producer must wait when the buffer is full.

With only a mutex, the consumer would have to do something like this:

```
/* Busy waiting - WASTEFUL! */
while (1) {
    pthread_mutex_lock(&lock);
    if (count > 0) {
        /* consume an item */
        pthread_mutex_unlock(&lock);
        break;
    }
    pthread_mutex_unlock(&lock);
    /* spin and try again... wastes CPU! */
}
```

This is called **busy waiting** (or spinning). The consumer constantly locks and unlocks the mutex, checking the condition over and over, wasting CPU cycles. What we need is a way for the consumer to **sleep** until the producer signals that data is available. This is exactly what **condition variables** provide.

Condition Variables

A **condition variable** allows a thread to block until a particular condition is true, and allows another thread to signal when that condition has changed. Condition variables are always used

together with a mutex.

The Condition Variable API

Function / Type	Purpose
pthread_cond_t	The condition variable data type
PTHREAD_COND_INITIALIZER	Static initializer for a condition variable
pthread_cond_init(&cond, attr)	Initialize dynamically. Pass NULL for default attributes.
pthread_cond_wait(&cond, &mutex)	Atomically release the mutex and block until signaled. When it returns, the mutex is re-acquired.
pthread_cond_signal(&cond)	Wake up one thread waiting on this condition variable.
pthread_cond_broadcast(&cond)	Wake up all threads waiting on this condition variable.
pthread_cond_destroy(&cond)	Free resources associated with the condition variable.

How pthread_cond_wait() Works

The function **pthread_cond_wait(&cond, &mutex)** performs this sequence:

1. Releases the mutex (so other threads can make progress).
2. Puts the calling thread to sleep, waiting on the condition variable.
3. After a signal, broadcast, or spurious wakeup, it re-acquires the mutex before returning; this may require waiting for another thread to release the mutex.

Steps 1 and 2 are atomic with respect to another thread acquiring the mutex and then signaling the condition. Step 3 happens later; the entire wait is not one indivisible operation. This distinction is critical. If they were not, another thread could signal the condition between the unlock and the sleep, and the signal would be lost.

The Correct Pattern

You must always use condition variables with a **while loop**, not an if statement:

```
pthread_mutex_lock(&mutex);
while (condition_is_not_true) {
    pthread_cond_wait(&cond, &mutex);
}
/* condition is now true, and we hold the mutex */
/* ... do work ... */
pthread_mutex_unlock(&mutex);
```

Why a **while** loop instead of **if**? Two reasons:

- **Spurious wakeups:** The POSIX standard allows `pthread_cond_wait` to return even if no one signaled. This is rare but possible. The while loop handles this by rechecking the condition.
- **Stolen wakeups:** If multiple threads are waiting and one is woken up, another thread might acquire the mutex first and consume the resource before the woken thread gets to run. The while loop handles this by rechecking.

Critical rule: Always check the condition in a **while** loop, never with just an **if** statement. This is one of the most common mistakes in concurrent programming.

Producer-Consumer with Condition Variables

Here is a complete implementation of the bounded-buffer producer-consumer problem using condition variables:


```

/* producer_consumer.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 20

int buffer[BUFFER_SIZE];
int count = 0;          /* number of items in buffer */
int in_index = 0;       /* next position to insert */
int out_index = 0;      /* next position to remove */

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = i + 1; /* produce an item */

        pthread_mutex_lock(&mutex);

        /* Wait while buffer is full */
        while (count == BUFFER_SIZE) {
            printf("Producer: buffer full, waiting...\n");
            pthread_cond_wait(&not_full, &mutex);
        }

        /* Insert item into buffer */
        buffer[in_index] = item;
        in_index = (in_index + 1) % BUFFER_SIZE;
        count++;
        printf("Producer: produced item %d (count=%d)\n",
            item, count);

        /* Signal consumer that buffer is not empty */
        pthread_cond_signal(&not_empty);

        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        pthread_mutex_lock(&mutex);

```

```

    /* Wait while buffer is empty */
    while (count == 0) {
        printf("Consumer: buffer empty, waiting...\n");
        pthread_cond_wait(&not_empty, &mutex);
    }

    /* Remove item from buffer */
    int item = buffer[out_index];
    out_index = (out_index + 1) % BUFFER_SIZE;
    count--;
    printf("Consumer: consumed item %d (count=%d)\n",
        item, count);

    /* Signal producer that buffer is not full */
    pthread_cond_signal(&not_full);

    pthread_mutex_unlock(&mutex);
}
return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    printf("All items produced and consumed.\n");
    return 0;
}

```

Compile and run:

```

gcc -pthread -o prodcons producer_consumer.c
./prodcons

```

Sample output (abbreviated):

```
Producer: produced item 1 (count=1)
Producer: produced item 2 (count=2)
Producer: produced item 3 (count=3)
Consumer: consumed item 1 (count=2)
Consumer: consumed item 2 (count=1)
Producer: produced item 4 (count=2)
Producer: produced item 5 (count=3)
Producer: produced item 6 (count=4)
Producer: produced item 7 (count=5)
Producer: buffer full, waiting...
Consumer: consumed item 3 (count=4)
...
All items produced and consumed.
```

Notice how the producer blocks when the buffer is full and the consumer blocks when it is empty. The condition variables handle the coordination efficiently without busy waiting.

Understanding the Flow

Let us trace through the key interactions:

1. The producer locks the mutex, checks if the buffer is full. If not, it inserts an item and signals **not_empty** to wake the consumer if it was sleeping.
2. The consumer locks the mutex, checks if the buffer is empty. If so, it calls **pthread_cond_wait(¬_empty, &mutex)**, which atomically releases the mutex and puts the consumer to sleep.
3. When the producer signals **not_empty**, the consumer wakes up. **pthread_cond_wait** re-acquires the mutex before returning. The consumer then re-checks the condition (while loop), finds the buffer is not empty, and consumes the item.

Signal vs Broadcast

pthread_cond_signal wakes up **one** waiting thread. **pthread_cond_broadcast** wakes up **all** waiting threads.

Use **signal** when:

- Only one waiting thread can make progress at a time (e.g., there is exactly one item available, so only one consumer should wake up).

Use **broadcast** when:

- The condition change might allow multiple waiters to proceed.
- Different threads are waiting for different conditions on the same condition variable.

- You are unsure which waiting threads can proceed.

When in doubt, use **broadcast**. It is less efficient (wakes up threads that may just go back to sleep) but always correct. Using signal incorrectly can cause threads to sleep forever.

Semaphores

A **semaphore** is another synchronization primitive, older than condition variables. A semaphore maintains an integer counter that can be incremented and decremented atomically. It is useful for controlling access to a resource with a limited number of instances.

To use semaphores on Linux, include `<semaphore.h>`.

The Semaphore API

Function / Type	Purpose
sem_t	The semaphore data type
sem_init(&sem, pshared, value)	Initialize an unnamed semaphore. pshared=0 for threads in the same process. value is the initial count.
sem_wait(&sem)	Decrement the counter. If the counter is 0, the thread blocks until it becomes positive. Also called "P" or "down".
sem_post(&sem)	Increment the counter. If threads are waiting, one is woken up. Also called "V" or "up".
sem_destroy(&sem)	Free resources associated with the semaphore.

How Semaphores Work

Think of a semaphore as a counter with special rules:

- **sem_wait:** "I want to use a resource." If the counter is greater than 0, decrement it and proceed. If the counter is 0, block until someone posts.
- **sem_post:** "I am done with a resource." Increment the counter. If anyone is waiting, wake one up.

A semaphore initialized to 1 behaves like a mutex (a "binary semaphore"). A semaphore initialized to N allows up to N threads to access a resource simultaneously.

Using a Semaphore as a Mutex

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem;
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        sem_wait(&sem);    /* like pthread_mutex_lock */
        counter++;
        sem_post(&sem);    /* like pthread_mutex_unlock */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    sem_init(&sem, 0, 1); /* initial value 1 = binary semaphore */

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter: %d\n", counter);

    sem_destroy(&sem);
    return 0;
}
```

Producer-Consumer with Semaphores

Semaphores can elegantly solve the producer-consumer problem. We use three semaphores:

- **empty:** counts empty slots (initialized to BUFFER_SIZE)
- **full:** counts full slots (initialized to 0)
- **mutex:** binary semaphore for mutual exclusion (initialized to 1)

```

/* prodcons_sem.c - Producer-Consumer with semaphores */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 20

int buffer[BUFFER_SIZE];
int in_index = 0;
int out_index = 0;

sem_t empty_slots; /* counts empty positions */
sem_t full_slots; /* counts filled positions */
sem_t mutex; /* binary semaphore for mutual exclusion */

void *producer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = i + 1;

        sem_wait(&empty_slots); /* wait for an empty slot */
        sem_wait(&mutex); /* enter critical section */

        buffer[in_index] = item;
        in_index = (in_index + 1) % BUFFER_SIZE;
        printf("Producer: produced %d\n", item);

        sem_post(&mutex); /* leave critical section */
        sem_post(&full_slots); /* signal a full slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full_slots); /* wait for a full slot */
        sem_wait(&mutex); /* enter critical section */

        int item = buffer[out_index];
        out_index = (out_index + 1) % BUFFER_SIZE;
        printf("Consumer: consumed %d\n", item);

        sem_post(&mutex); /* leave critical section */
        sem_post(&empty_slots); /* signal an empty slot */
    }
    return NULL;
}

```

```

int main() {
    pthread_t prod, cons;

    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    sem_destroy(&mutex);

    printf("Done.\n");
    return 0;
}

```

Compile and run:

```

gcc -pthread -o prodcons_sem prodcons_sem.c
./prodcons_sem

```

Named vs Unnamed Semaphores

The semaphores we have used so far are **unnamed semaphores**, initialized with **sem_init()**. They exist in memory and are shared between threads in the same process (when pshared=0).

Named semaphores are created with **sem_open()** and have a name in the filesystem (typically under /dev/shm/ on Linux). They can be shared between *different processes*.

```

#include <fcntl.h>
#include <semaphore.h>

/* Create or open a named semaphore */
sem_t *sem = sem_open("/my_semaphore", O_CREAT, 0644, 1);

/* Use it just like an unnamed semaphore */
sem_wait(sem);
/* ... critical section ... */
sem_post(sem);

/* Close it when done */
sem_close(sem);

/* Remove it from the system (once, when no longer needed) */
sem_unlink("/my_semaphore");

```

Named semaphores are useful when you need synchronization between separate processes (not just threads within one process).

Condition Variables vs Semaphores

Feature	Condition Variable	Semaphore
Must be used with a mutex	Yes (always)	No (self-contained)
Maintains a count	No	Yes (integer counter)
Signal without waiter	Signal is lost	Count is incremented (remembered)
Spurious wakeups	Possible (use while loop)	Not applicable
Broadcast	Yes (wake all waiters)	No (must post N times)
Cross-process	Possible but complex	Easy with named semaphores

In general, condition variables with mutexes are more flexible and are the preferred approach in most pthreads programs. Semaphores are simpler for certain patterns like resource counting and are familiar from operating systems theory.

Key Takeaways

- Mutexes alone cannot efficiently handle "wait for a condition" patterns.
- **Condition variables** let a thread sleep until another thread signals that a condition has changed.
- Always use condition variables with a **mutex** and a **while loop** around the wait.
- **pthread_cond_signal** wakes one waiter; **pthread_cond_broadcast** wakes all waiters.
- **Semaphores** maintain a counter and provide **sem_wait** (decrement/block) and **sem_post** (increment/wake).
- A semaphore initialized to 1 works like a mutex.
- **Named semaphores** can be shared between processes; **unnamed semaphores** are for threads within a process.

Lesson 6: Classic Concurrency Problems

Introduction

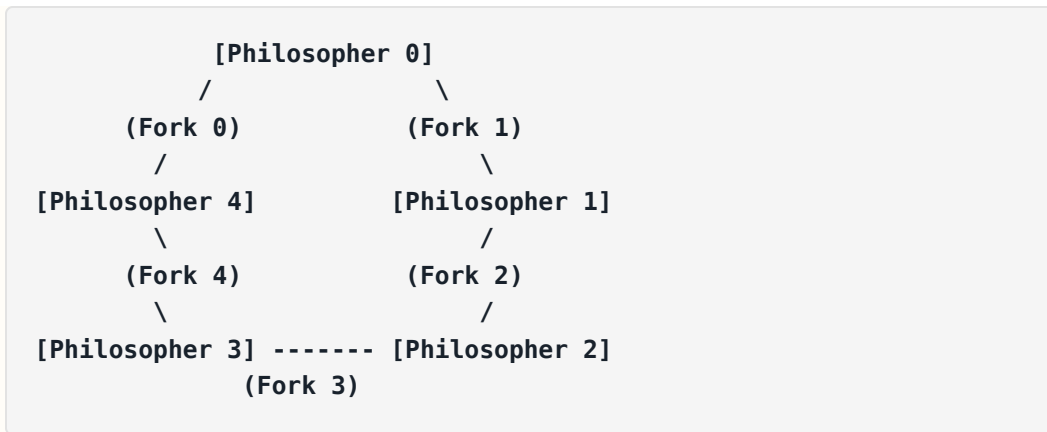
Throughout the history of computer science, certain concurrency problems have become famous because they cleanly illustrate the challenges of writing correct concurrent programs. In this lesson, we will study two of the most well-known: the **Dining Philosophers** problem and the **Reader-Writer** problem. These problems appear in nearly every operating systems textbook and are frequently asked about in technical interviews. (You have already solved a third classic, the **producer-consumer** problem, in Lesson 5.)

More importantly, these problems represent real patterns that appear in actual systems. The Dining Philosophers problem models any situation where multiple processes compete for a limited set of shared resources. The Reader-Writer problem models any situation where some threads only read shared data while others need to modify it.

The Dining Philosophers Problem

Problem Description

Five philosophers sit around a circular table. Between each pair of adjacent philosophers, there is a single fork (so there are 5 forks total). Each philosopher alternates between **thinking** and **eating**. To eat, a philosopher must pick up both the fork on their left and the fork on their right. After eating, they put both forks down and resume thinking.



In the diagram, the five philosophers and five forks alternate around the circle: fork i sits to the left of philosopher i , and philosopher i 's right fork is fork $(i+1) \bmod 5$.

The challenge: design a protocol so that:

- No philosopher starves (everyone eventually gets to eat).
- No deadlock occurs (the system does not freeze with everyone waiting).
- Maximum concurrency is achieved (as many philosophers as possible eat simultaneously).

Each fork is a shared resource, and each philosopher is a thread.

Naïve Solution (Deadlock!)

The most natural approach is for each philosopher to pick up their left fork, then their right fork, eat, and put both down. Let us implement this and see what goes wrong:

```

/* dining_deadlock.c - naive solution that DEADLOCKS */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5
#define NUM_MEALS 3

pthread_mutex_t forks[NUM_PHILOSOPHERS];

void think(int id) {
    printf("Philosopher %d is thinking.\n", id);
    usleep(rand() % 500000); /* think for a random time */
}

void eat(int id) {
    printf("Philosopher %d is EATING.\n", id);
    usleep(rand() % 500000); /* eat for a random time */
}

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    for (int meal = 0; meal < NUM_MEALS; meal++) {
        think(id);

        /* Pick up left fork */
        printf("Philosopher %d picks up left fork %d.\n",
            id, left);
        pthread_mutex_lock(&forks[left]);

        /* Pick up right fork */
        printf("Philosopher %d picks up right fork %d.\n",
            id, right);
        pthread_mutex_lock(&forks[right]);

        eat(id);

        /* Put down forks */
        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
    }
    return NULL;
}

```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    srand(42);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    printf("All done.\n");
    return 0;
}

```

One caveat about these examples: the worker threads call **rand()** to get random delays, and POSIX does not require **rand()** to be thread-safe. That is fine here, where the random numbers only add timing variety, but code that needs reliable per-thread random numbers should use **rand_r()** with a per-thread seed.

If you compile and run this, it may work sometimes, but it can also **deadlock**. The deadlock scenario is:

1. Philosopher 0 picks up fork 0 (left).
2. Philosopher 1 picks up fork 1 (left).
3. Philosopher 2 picks up fork 2 (left).
4. Philosopher 3 picks up fork 3 (left).
5. Philosopher 4 picks up fork 4 (left).
6. Now every philosopher tries to pick up their right fork, but every fork is already held.
Everyone is waiting for someone else. Deadlock!

```
Phil 0: holds fork 0, waits for fork 1
Phil 1: holds fork 1, waits for fork 2
Phil 2: holds fork 2, waits for fork 3
Phil 3: holds fork 3, waits for fork 4
Phil 4: holds fork 4, waits for fork 0    <-- circular!
```

This is the classic circular wait condition for deadlock.

Solution 1: Break the Symmetry (Resource Ordering)

The simplest and most elegant solution is to break the circular wait by having **one philosopher pick up forks in a different order**. Specifically, we have all philosophers pick up the lower-numbered fork first, then the higher-numbered fork. Since Philosopher 4's left fork is fork 4 and right fork is fork 0, this philosopher will pick up fork 0 first (the lower number) instead of fork 4.

This breaks the cycle because no circular wait can form when all threads acquire locks in a consistent global order.

```

/* dining_ordered.c - correct solution using resource ordering */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5
#define NUM_MEALS 5

pthread_mutex_t forks[NUM_PHILOSOPHERS];

void think(int id) {
    printf("Philosopher %d is thinking.\n", id);
    usleep(rand() % 500000);
}

void eat(int id) {
    printf("Philosopher %d is EATING.\n", id);
    usleep(rand() % 500000);
}

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    /* Determine which fork has the lower number */
    int first, second;
    if (left < right) {
        first = left;
        second = right;
    } else {
        first = right;
        second = left;
    }

    for (int meal = 0; meal < NUM_MEALS; meal++) {
        think(id);

        /* Always pick up the lower-numbered fork first */
        printf("Philosopher %d picks up fork %d.\n", id, first);
        pthread_mutex_lock(&forks[first]);

        printf("Philosopher %d picks up fork %d.\n", id, second);
        pthread_mutex_lock(&forks[second]);

        eat(id);
    }
}

```

```

        /* Put down forks (order does not matter for unlocking) */
        pthread_mutex_unlock(&forks[second]);
        pthread_mutex_unlock(&forks[first]);

        printf("Philosopher %d puts down forks.\n", id);
    }

    printf("Philosopher %d is done.\n", id);
    return NULL;
}

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    srand(42);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    printf("All philosophers have finished dining.\n");
    return 0;
}

```

Compile and run:

```

gcc -pthread -o dining dining_ordered.c
./dining

```

This solution will never deadlock. The key insight is that Philosopher 4, who sits between fork 4 and fork 0, picks up fork 0 first (the lower number). This means they cannot complete the circular chain.

Solution 2: Limit the Number of Diners

Another approach is to use a semaphore to ensure that at most 4 philosophers (out of 5) can attempt to pick up forks at the same time. If at most 4 try, at least one will always be able to get both forks and eat, preventing deadlock.

```

/* dining_semaphore.c - limit concurrent diners */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5
#define NUM_MEALS 5

pthread_mutex_t forks[NUM_PHILOSOPHERS];
sem_t seats; /* limits concurrent diners to N-1 */

void think(int id) {
    printf("Philosopher %d is thinking.\n", id);
    usleep(rand() % 500000);
}

void eat(int id) {
    printf("Philosopher %d is EATING.\n", id);
    usleep(rand() % 500000);
}

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    for (int meal = 0; meal < NUM_MEALS; meal++) {
        think(id);

        sem_wait(&seats); /* sit down (at most 4 at a time) */

        pthread_mutex_lock(&forks[left]);
        pthread_mutex_lock(&forks[right]);

        eat(id);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);

        sem_post(&seats); /* stand up */
    }

    printf("Philosopher %d is done.\n", id);
    return NULL;
}

```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    srand(42);

    /* Allow at most NUM_PHILOSOPHERS - 1 concurrent diners */
    sem_init(&seats, 0, NUM_PHILOSOPHERS - 1);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    sem_destroy(&seats);

    printf("All philosophers have finished dining.\n");
    return 0;
}

```

This works because with only 4 philosophers attempting at a time around 5 forks, at least one will always be able to pick up both forks. The pigeonhole principle guarantees that deadlock cannot occur.

The Reader-Writer Problem

Problem Description

Consider a shared data structure (such as a database or a file) that is accessed by two types of threads:

- **Readers** only read the data. Multiple readers can access the data simultaneously without any problem, because reading does not change the data.

- **Writers** modify the data. A writer needs exclusive access: no other reader or writer should access the data while a writer is writing.

The rules are:

1. Multiple readers can read at the same time (shared access).
2. Only one writer can write at a time (exclusive access).
3. No reader can read while a writer is writing.
4. No writer can write while any reader is reading.

Using a single mutex for all access would work for correctness, but it would be too restrictive: it would prevent multiple readers from reading simultaneously, even though that is perfectly safe.

Reader-Writer Lock Concept

The idea is to maintain a count of active readers. The first reader locks out writers, and the last reader unlocks for writers. Writers need exclusive access.

+-----+			
New	Readers	Writer	
request	active	active	
+-----+			
Reader	Allowed	Must	
		wait	
+-----+			
Writer	Must	Must	
	wait	wait	
+-----+			

Implementation Using a Semaphore, Mutex, and Counter

The reader count is protected by a mutex. The resource gate is a binary semaphore: the first reader waits on it and the last reader posts it, and these can be different threads. A POSIX mutex cannot be used for that gate because only its owning thread may unlock it.

```

/* readers_writers.c - reader-preference solution */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#include <errno.h>
#include <string.h>

#define NUM_READERS 5
#define NUM_WRITERS 2
#define NUM_READS 3
#define NUM_WRITES 3

int shared_data = 0;          /* the shared resource */
int reader_count = 0;        /* number of active readers */

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
sem_t resource_gate;

static void wait_for_resource(void) {
    while (sem_wait(&resource_gate) == -1) {
        if (errno == EINTR) continue;
        perror("sem_wait");
        exit(EXIT_FAILURE);
    }
}

static void check_thread(int rc) {
    if (rc != 0) {
        fprintf(stderr, "pthread error: %s\n", strerror(rc));
        exit(EXIT_FAILURE);
    }
}

void *reader(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < NUM_READS; i++) {
        /* Entry section for readers */
        pthread_mutex_lock(&mutex);
        reader_count++;
        int active_readers = reader_count;
        if (reader_count == 1) {
            /* First reader locks out writers */
            wait_for_resource();
        }
        pthread_mutex_unlock(&mutex);
    }
}

```

```

        /* Reading section (multiple readers can be here) */
        printf("Reader %d reads shared_data = %d "
               "(readers active: %d)\n",
               id, shared_data, active_readers);
        usleep(rand() % 200000);

        /* Exit section for readers */
        pthread_mutex_lock(&mutex);
        reader_count--;
        if (reader_count == 0) {
            /* Last reader allows writers */
            sem_post(&resource_gate);
        }
        pthread_mutex_unlock(&mutex);

        usleep(rand() % 300000); /* delay between reads */
    }
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < NUM_WRITES; i++) {
        /* Writer needs exclusive access */
        wait_for_resource();

        shared_data++;
        printf("Writer %d wrote shared_data = %d\n",
               id, shared_data);
        usleep(rand() % 300000);

        sem_post(&resource_gate);

        usleep(rand() % 500000); /* delay between writes */
    }
    return NULL;
}

int main() {
    pthread_t r_threads[NUM_READERS], w_threads[NUM_WRITERS];
    int r_ids[NUM_READERS], w_ids[NUM_WRITERS];

    if (sem_init(&resource_gate, 0, 1) == -1) {
        perror("sem_init");
        return EXIT_FAILURE;
    }

    srand(42);

```

```

    for (int i = 0; i < NUM_READERS; i++) {
        r_ids[i] = i;
        check_thread(pthread_create(&r_threads[i], NULL,
                                    reader, &r_ids[i]));
    }
    for (int i = 0; i < NUM_WRITERS; i++) {
        w_ids[i] = i;
        check_thread(pthread_create(&w_threads[i], NULL,
                                    writer, &w_ids[i]));
    }

    for (int i = 0; i < NUM_READERS; i++) {
        check_thread(pthread_join(r_threads[i], NULL));
    }
    for (int i = 0; i < NUM_WRITERS; i++) {
        check_thread(pthread_join(w_threads[i], NULL));
    }

    printf("Final shared_data = %d\n", shared_data);
    sem_destroy(&resource_gate);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Compile and run:

```

gcc -pthread -o rw readers_writers.c
./rw

```

Reader-Preference vs Writer-Preference

The solution above has **reader preference** (also called the "first readers-writers problem"): as long as there is any reader active, new readers can join in, and writers must wait. This can lead to **writer starvation**: if readers arrive continuously, a writer might wait forever.

The opposite approach, **writer preference** (the "second readers-writers problem"), gives priority to writers: once a writer is waiting, no new readers are allowed to start. This can lead to reader starvation.

In practice, **pthread_rwlock_t** provides a built-in reader-writer lock that handles these details for you:

```
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;

/* Reader */
pthread_rwlock_rdlock(&rwlock);
/* ... read shared data ... */
pthread_rwlock_unlock(&rwlock);

/* Writer */
pthread_rwlock_wrlock(&rwlock);
/* ... write shared data ... */
pthread_rwlock_unlock(&rwlock);
```

The implementation of `pthread_rwlock_t` varies by system, but it typically tries to balance reader and writer priorities.

Summary of Solutions

Problem	Key Challenge	Solution Approaches
Dining Philosophers	Circular wait leading to deadlock	Resource ordering (always pick up lower-numbered fork first) or limit the number of concurrent diners with a semaphore
Reader-Writer	Allowing multiple readers while giving writers exclusive access	Protect reader count with a mutex; first reader waits on a resource semaphore, last reader posts it. Or use <code>pthread_rwlock_t</code> .

Why These Problems Matter

These classic problems are not just academic exercises. They appear everywhere in real systems:

- **Dining Philosophers:** Any system where multiple threads compete for a fixed set of resources. Database connections, network sockets, hardware devices, file locks.
- **Reader-Writer:** Caching systems, databases, configuration files, shared data structures where reads are far more frequent than writes.

Understanding these problems and their solutions gives you the conceptual tools to design correct concurrent systems.

Key Takeaways

- The **Dining Philosophers** problem demonstrates how naive lock acquisition can lead to deadlock.
- **Resource ordering** (always acquiring locks in a consistent global order) is the most practical way to prevent deadlock.
- Limiting the number of concurrent resource consumers (using a semaphore) is another effective deadlock prevention strategy.
- The **Reader-Writer** problem shows that different types of access (read vs write) can have different concurrency requirements.
- Reader-preference solutions can starve writers; writer-preference solutions can starve readers. The built-in `pthread_rwlock_t` attempts to balance both.
- These problems represent patterns that appear throughout real-world systems programming.

Module 6 Summary

Wrap Up

In this module, we covered threads, synchronization primitives, and classic concurrency problems. These are essential topics for any systems programmer. Writing correct concurrent code is one of the most challenging tasks in software engineering, and the concepts you learned here form the foundation for tackling that challenge.

The topics included:

- Threads: what they are and how they differ from processes. Threads share the same address space (code, globals, heap) but have their own stack and registers.
- The pthreads library: creating threads with `pthread_create()`, waiting for them with `pthread_join()`, passing arguments and returning values.
- Race conditions: the unpredictable behavior that occurs when threads access shared data without synchronization. The read-modify-write problem.
- Mutexes: the primary synchronization primitive for protecting critical sections and enforcing mutual exclusion.
- Condition variables: a mechanism for threads to wait for specific conditions, enabling efficient producer-consumer patterns.
- Semaphores: counting-based synchronization primitives, useful for resource management and as an alternative to mutexes plus condition variables.
- Classic concurrency problems: the Dining Philosophers and Reader-Writer problems, along with solutions that prevent deadlock and starvation.

Key Concepts to Remember

- **Threads vs Processes:** Threads are lightweight, share memory, and are cheap to create. Processes are isolated, have separate address spaces, and are more expensive. Use threads when tasks need to share data; use processes when you need isolation.
- **Shared state requires synchronization:** If two or more threads access the same data and at least one writes to it, you must use synchronization. No exceptions.
- **counter++ is not atomic:** A single line of C code can be multiple machine instructions. The read-modify-write pattern is the most common source of race conditions.
- **Mutexes enforce mutual exclusion:** Lock before entering a critical section, unlock when leaving. Always unlock on every code path, including error paths.
- **Condition variables use while loops:** Always check the condition in a while loop, not an if statement, to handle spurious and stolen wakeups.
- **Deadlock requires four conditions:** Mutual exclusion, hold and wait, no preemption, and circular wait. Break any one of these to prevent deadlock. The most practical approach is to acquire locks in a consistent global order.
- **Test concurrent code aggressively:** Race conditions are non-deterministic. A program that works 99 times out of 100 is still buggy. Run tests many times and on different machines.

Common Pitfalls Checklist

Before submitting any concurrent program, check for these common mistakes:

Pitfall	How to Avoid It
Forgetting -pthread when compiling	Always compile with gcc -o prog prog.c -pthread
Passing the address of a loop variable to threads	Use an array so each thread has its own copy of the argument
Returning a pointer to a local variable from a thread	Use malloc for return values; free them after pthread_join
Accessing shared data without a lock	Identify all shared variables and protect every access with a mutex
Forgetting to unlock a mutex (especially on error paths)	Review every code path through the critical section
Using different mutexes for the same shared data	All threads must lock the <i>same</i> mutex for mutual exclusion to work
Using if instead of while with pthread_cond_wait	Always use a while loop to recheck the condition
Deadlock from inconsistent lock ordering	Always acquire multiple locks in the same global order
Not calling pthread_join (resource leak)	Always join every thread you create, or detach it

The pthreads Functions We Covered

Here is a quick reference for all the functions introduced in this module:

Category	Function	Purpose
Threads	pthread_create()	Create a new thread
	pthread_join()	Wait for a thread to finish
Mutexes	pthread_mutex_init()	Initialize a mutex
	pthread_mutex_lock()	Acquire the lock (blocks if held)
	pthread_mutex_trylock()	Try to acquire (non-blocking)
	pthread_mutex_unlock()	Release the lock
	pthread_mutex_destroy()	Free mutex resources
Condition Variables	pthread_cond_init()	Initialize a condition variable
	pthread_cond_wait()	Release mutex and sleep until signaled
	pthread_cond_signal()	Wake one waiting thread
	pthread_cond_broadcast()	Wake all waiting threads
	pthread_cond_destroy()	Destroy a condition variable and free its resources
Semaphores	sem_init()	Initialize an unnamed semaphore
	sem_wait()	Decrement counter (blocks if zero)
	sem_post()	Increment counter (wakes a waiter)
	sem_destroy()	Free semaphore resources

Category	Function	Purpose
Reader-Writer Locks	<code>pthread_rwlock_init()</code>	Initialize a reader-writer lock
	<code>pthread_rwlock_rdlock()</code>	Acquire a read lock (shared)
	<code>pthread_rwlock_wrlock()</code>	Acquire a write lock (exclusive)
	<code>pthread_rwlock_unlock()</code>	Unlock a reader-writer lock (for both readers and writers)
	<code>pthread_rwlock_destroy()</code>	Destroy a reader-writer lock and free its resources

Additional Resources

[POSIX Threads Programming](#) [link to Lawrence Livermore National Laboratory tutorial - one of the most comprehensive pthreads tutorials available]

[Operating Systems: Three Easy Pieces - Introduction to Threads](#) [link to free textbook chapter by Remzi and Andrea Arpaci-Dusseau]

[Operating Systems: Three Easy Pieces - Locks](#) [link to free textbook chapter on mutexes and locks]

[Operating Systems: Three Easy Pieces - Condition Variables](#) [link to free textbook chapter on condition variables]

[Operating Systems: Three Easy Pieces - Semaphores](#) [link to free textbook chapter on semaphores]

[Linux man page: pthreads\(7\)](#) [link to the official pthreads man page]

Module 7 Overview

In Module 7: File I/O and Sockets Programming, we will explore how Unix systems handle input and output at the lowest level and then build on that foundation to write networked programs. Unix is built on a powerful idea: **everything is a file**. Whether you are reading from a disk, writing to a terminal, or communicating over a network, you interact with the same set of system calls. This module traces that idea from opening a file on disk all the way to building a working web server.

We begin with low-level file I/O using the system calls `open`, `read`, `write`, and `close`. We then examine the buffered `FILE*` API provided by the C standard library and learn how to work with directories and file metadata. In the second half of the module, we turn to networking. We study the TCP/IP protocol stack, learn the sockets API that lets programs communicate over a network, and progressively build more capable servers: first an echo server, then a multi-client threaded server, and finally a minimal HTTP web server that ties together file I/O, sockets, and threads.

This module is the capstone of the systems-programming arc of the course. It brings together nearly every concept you have learned: C programming, system calls, processes, and threads. By the end, you will see how a few simple abstractions -- file descriptors and byte streams -- underlie all of modern networked computing.

Learning objectives

At the end of this learning module, you will be able to:

1. Explain the Unix "everything is a file" philosophy and describe what a file descriptor represents
2. Use the low-level I/O system calls `open`, `read`, `write`, and `close` to manipulate files
3. Compare and contrast unbuffered I/O (file descriptors) with buffered I/O (the `FILE*` API)
4. Read directory contents and query file metadata using `opendir`, `readdir`, and `stat`

5. Describe the four layers of the TCP/IP model and explain what TCP provides to applications
6. Explain the purpose of IP addresses, ports, and network byte order
7. Implement a TCP server using the socket API: `socket` , `bind` , `listen` , `accept`
8. Implement a TCP client using `socket` and `connect`
9. Build a multi-client server using threads
10. Construct a minimal HTTP web server that serves static files

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below
2. Read Lessons 1 through 6 and work through the code examples on your own machine
3. Take the module quiz
4. Review additional resources on the Module 7 Summary page

Reading

- Bryant & O'Hallaron: Chapters 10 and 11 (System-Level I/O and Network Programming)

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: File Descriptors and Low-Level I/O

Everything Is a File

One of the most important design principles in Unix is that **everything is a file**. Regular files on disk, directories, terminals, pipes, and even network connections are all accessed through the same interface. A program reads from a keyboard the same way it reads from a file on disk: by calling `read()` on a **file descriptor**.

This is not just a metaphor. The kernel maintains a table of open files for each process, and every open file -- regardless of what it actually represents -- is identified by a small non-negative integer called a **file descriptor**. When you open a file, the kernel returns a file descriptor. When you want to read, write, or close that file, you pass the file descriptor back to the kernel.

File Descriptors

A file descriptor is simply an `int`. It serves as an index into a per-process table that the kernel maintains. Each entry in this table points to an entry in a system-wide **open file table**, which in turn points to the file's **inode** (the on-disk data structure that describes the file).

Every process starts with three file descriptors already open:

File Descriptor	Name	C Constant	Purpose
0	Standard Input	STDIN_FILENO	Reading input (usually the keyboard)
1	Standard Output	STDOUT_FILENO	Writing normal output (usually the terminal)
2	Standard Error	STDERR_FILENO	Writing error messages (usually the terminal)

These constants are defined in `<unistd.h>`. When you call `printf("hello")`, the C library ultimately calls `write(1, "hello", 5)` -- it writes 5 bytes to file descriptor 1 (standard output).

When you open a new file, the kernel assigns the lowest available file descriptor number. So the first file you open will typically get descriptor 3, the next gets 4, and so on.

Opening Files: `open()`

The `open()` system call opens a file and returns a file descriptor. It is declared in `<fcntl.h>`:

```
#include <fcntl.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

The `flags` argument specifies how the file should be opened. You must include exactly one of these access modes:

- `O_RDONLY` -- Open for reading only
- `O_WRONLY` -- Open for writing only
- `O_RDWR` -- Open for both reading and writing

You can combine additional flags using the bitwise OR operator:

- `O_CREAT` -- Create the file if it does not exist. When using this flag, you must provide the `mode` argument (e.g., `0644`) to set the file's permissions.

- `O_TRUNC` -- If the file exists and is opened for writing, truncate it to zero length (erase its contents).
- `O_APPEND` -- Writes are appended to the end of the file.

The return value is a file descriptor on success, or `-1` on error (with `errno` set to indicate the error).

Example: Opening a file for reading

```
int fd = open("data.txt", O_RDONLY);
if (fd == -1) {
    perror("open");
    exit(1);
}
```

Example: Creating a file for writing

```
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("open");
    exit(1);
}
```

The permission mode `0644` means the owner can read and write, and everyone else can only read. This is an octal number -- the leading zero tells the compiler it is octal.

Closing Files: close()

When you are finished with a file, you should close it:

```
#include <unistd.h>

int close(int fd);
```

Closing a file descriptor releases the kernel resources associated with the open file. It returns 0 on success and -1 on error. If your program opens many files without closing them, it will eventually run out of file descriptors.

Common misconception — "`close()` always succeeds, so why check it?" Most of the time it does, but `close()` can return -1 in real situations: the kernel may have buffered earlier writes and only attempt to flush them on close (especially over NFS), and a hardware error or quota violation surfaces here. If you've written data that matters, check the return of `close()` — it's often the only signal you'll get that an earlier `write()` didn't actually make it to disk. The defensive idiom is:

```
if (close(fd) == -1) {
    perror("close");
    /* error handling: maybe re-write, retry, alert the
user */
}
```

On error, the file descriptor is still released even though something went wrong — do not call `close()` a second time.

Reading: `read()`

The `read()` system call reads bytes from a file descriptor into a buffer:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

It attempts to read up to `count` bytes from file descriptor `fd` into the buffer pointed to by `buf`. The return value is:

- A positive number: the number of bytes actually read. This can be **less than** `count`. This is called a **short read** or **partial read**. It happens when there are fewer bytes available than requested (for example, you are near the end of the file).
- **0**: End of file (EOF) has been reached. There is nothing left to read.
- **-1**: An error occurred. Check `errno` for the specific error.

Because `read()` can return fewer bytes than you asked for, robust code must handle partial reads. Typically you call `read()` in a loop:

```

char buf[4096];
ssize_t bytes_read;

while ((bytes_read = read(fd, buf, sizeof(buf))) > 0) {
    // process bytes_read bytes in buf
}
if (bytes_read == -1) {
    perror("read");
}

```

Writing: write()

The `write()` system call writes bytes from a buffer to a file descriptor:

```

#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

```

It attempts to write `count` bytes from `buf` to file descriptor `fd`. Like `read()`, it can return fewer bytes than requested (a **short write**). The return value is:

- A positive number: the number of bytes actually written.
- **-1**: An error occurred.

To ensure all bytes are written, you should loop:

```

ssize_t write_all(int fd, const char *buf, size_t count) {
    size_t total = 0;
    while (total < count) {
        ssize_t written = write(fd, buf + total, count - total);
        if (written == -1) {
            perror("write");
            return -1;
        }
        total += written;
    }
    return total;
}

```

The File Offset and lseek()

Every open file descriptor has a **current file offset**: the position in the file where the next `read()` or `write()` will take place. The offset starts at 0 when you open a file (or at the end, with `O_APPEND`) and advances automatically by the number of bytes read or written -- that is why our read loops move through the file without any explicit position bookkeeping. The `lseek()` system call repositions the offset directly:

```
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

The `whence` argument says what `offset` is relative to: `SEEK_SET` (the start of the file), `SEEK_CUR` (the current offset), or `SEEK_END` (the end of the file). It returns the new offset from the start of the file, or -1 on error. Two common idioms:

```
lseek(fd, 0, SEEK_SET);           // rewind to the start to
re-read the file
off_t size = lseek(fd, 0, SEEK_END); // jump to the end: the
offset IS the file size
```

Error Handling: errno and perror()

When a system call fails (returns -1), it sets the global variable `errno` to an integer code indicating what went wrong. Common values include:

- `ENOENT` -- No such file or directory
- `EACCES` -- Permission denied
- `EBADF` -- Bad file descriptor
- `ENOMEM` -- Out of memory

The `perror()` function prints a human-readable error message to standard error. It takes a string prefix, appends a colon and the message corresponding to the current value of `errno`:

```
int fd = open("nonexistent.txt", O_RDONLY);
if (fd == -1) {
    perror("open"); // prints: open: No such file or directory
    exit(1);
}
```

You must include `<errno.h>` to use `errno` directly and `<stdio.h>` for `perror()` .

Complete Example: Copying a File

Here is a complete program that copies one file to another using low-level I/O. It reads from a source file and writes to a destination file, handling partial reads and writes:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUF_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source> <destination>\n",
argv[0]);
        exit(1);
    }

    // Open source file for reading
    int src_fd = open(argv[1], O_RDONLY);
    if (src_fd == -1) {
        perror("open source");
        exit(1);
    }

    // Create destination file for writing
    int dst_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst_fd == -1) {
        perror("open destination");
        close(src_fd);
        exit(1);
    }

    // Copy loop
    char buf[BUF_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(src_fd, buf, BUF_SIZE)) > 0) {
        // Write all bytes that were read
        char *write_ptr = buf;
        ssize_t remaining = bytes_read;

        while (remaining > 0) {
            ssize_t written = write(dst_fd, write_ptr, remaining);
            if (written == -1) {
                perror("write");
                close(src_fd);
                close(dst_fd);
                exit(1);
            }
            write_ptr += written;
            remaining -= written;
        }
    }
}

```



```
    }  
}  
  
if (bytes_read == -1) {  
    perror("read");  
}  
  
close(src_fd);  
close(dst_fd);  
  
return 0;  
}
```

Compile and run:

```
$ gcc -o filecopy filecopy.c  
$ ./filecopy original.txt copy.txt
```

This program demonstrates several important patterns:

- Always check return values of system calls
- Use a loop to handle partial reads and writes
- Clean up resources (close file descriptors) even on error paths
- Use `perror()` for informative error messages

Complete Example: Reading from Standard Input

Since standard input is just file descriptor 0, we can read from the keyboard using `read()`.

This program reads a line from the user and echoes it back:

```

#include <stdio.h>
#include <unistd.h>

int main(void) {
    char buf[256];

    write(STDOUT_FILENO, "Enter text: ", 12);

    ssize_t n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
    if (n == -1) {
        perror("read");
        return 1;
    }

    buf[n] = '\0'; // null-terminate the string

    write(STDOUT_FILENO, "You typed: ", 11);
    write(STDOUT_FILENO, buf, n);

    return 0;
}

```

Notice that we use `sizeof(buf) - 1` to leave room for the null terminator. The `read()` system call does not add a null terminator -- it reads raw bytes. If you want to treat the data as a C string, you must add the `'\0'` yourself.

Key Takeaways

- In Unix, everything is accessed through file descriptors: regular files, terminals, pipes, and sockets.
- `open()` returns a file descriptor; `close()` releases it.
- `read()` and `write()` are the fundamental I/O operations. Both can return fewer bytes than requested.
- Always check return values and handle errors with `errno` and `perror()`.
- File descriptors 0, 1, and 2 are automatically open as `stdin`, `stdout`, and `stderr`.

Lesson 2: The FILE API and Directories

Why Buffered I/O?

In Lesson 1, we used `read()` and `write()` directly. These are system calls -- each one requires a **mode switch** into the kernel (and back), which is expensive. If your program reads a file one byte at a time using `read(fd, &ch, 1)`, it will make one system call per byte. For a 1 MB file, that is over a million system calls.

The C standard library provides a **buffered I/O** layer on top of file descriptors. Instead of a raw `int` file descriptor, you work with a `FILE *` pointer. Internally, the library maintains a buffer (typically 4096 or 8192 bytes). When you call `fgetc()`, the library does not issue a system call for every byte. Instead, it reads a large chunk into its buffer with a single `read()` call and then serves subsequent `fgetc()` requests from the buffer. This dramatically reduces the number of system calls.

The same idea applies to output. When you call `fprintf()`, the output is written into a buffer. The buffer is flushed (actually written to the file descriptor via `write()`) when it is full, when a newline is written (for line-buffered streams like `stdout`), or when you call `fflush()`.

Opening and Closing Files with FILE*

The `fopen()` function opens a file and returns a `FILE *`:

```
#include <stdio.h>

FILE *fopen(const char *pathname, const char *mode);
```

The `mode` string specifies how the file is opened:

Mode	Description
"r"	Open for reading. File must exist.
"w"	Open for writing. Creates the file or truncates it to zero length.
"a"	Open for appending. Creates the file if it does not exist.
"r+"	Open for reading and writing. File must exist.
"w+"	Open for reading and writing. Creates or truncates.

`fopen()` returns `NULL` on error. Always check:

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("fopen");
    exit(1);
}
```

When finished, close the file with `fclose()` :

```
fclose(fp);
```

This flushes any remaining buffered output and releases the underlying file descriptor.

Standard Streams

Just as every process starts with three open file descriptors (0, 1, 2), the C library provides three pre-opened `FILE *` streams:

- `stdin` -- corresponds to file descriptor 0
- `stdout` -- corresponds to file descriptor 1
- `stderr` -- corresponds to file descriptor 2

Functions like `printf()` write to `stdout` . Functions like `scanf()` read from `stdin` . You can use `fprintf(stderr, ...)` to write error messages.

Reading and Writing with FILE*

The C library provides many functions for buffered I/O. Here are the most important ones:

Character-level I/O

```
int fgetc(FILE *stream);           // read one character, returns
EOF on end
int fputc(int c, FILE *stream);    // write one character
```

Line-level I/O

```
char *fgets(char *buf, int size, FILE *stream); // read a line
(up to size-1 chars)
int fputs(const char *str, FILE *stream);       // write a string
```

`fgets()` reads until it encounters a newline, EOF, or has read `size - 1` characters. For a buffer size of at least 2, a successful call null-terminates the characters read, including the newline if one was read. It returns `NULL` on a read error or when EOF occurs before any characters are read; EOF before any characters leaves the buffer unchanged, and a read error leaves its contents indeterminate. Check the return value before using the buffer.

Formatted I/O

```
int fprintf(FILE *stream, const char *format, ...); // formatted
output
int fscanf(FILE *stream, const char *format, ...);  // formatted
input
```

These work exactly like `printf()` and `scanf()`, but operate on any `FILE *` stream rather than just `stdout/stdin`.

Block I/O

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE
*stream);
```

`fread()` and `fwrite()` read and write blocks of data. The `size` parameter is the size of each element, and `nmemb` is the number of elements. They return the number of elements

(not bytes) successfully read or written.

Flushing the buffer

```
int fflush(FILE *stream);
```

Forces any buffered output to be written to the underlying file descriptor. This is important when you need output to appear immediately -- for example, before calling `fork()` or when writing a prompt before reading input.

Complete Example: A Simple cat Clone

The Unix `cat` command reads files and writes their contents to standard output. Here is a simple version using the FILE* API:

```

#include <stdio.h>
#include <stdlib.h>

void cat(FILE *fp) {
    int ch;
    while ((ch = fgetc(fp)) != EOF) {
        fputc(ch, stdout);
    }
}

int main(int argc, char *argv[]) {
    if (argc == 1) {
        // No arguments: read from stdin
        cat(stdin);
    } else {
        // Process each file argument
        for (int i = 1; i < argc; i++) {
            FILE *fp = fopen(argv[i], "r");
            if (fp == NULL) {
                perror(argv[i]);
                continue;
            }
            cat(fp);
            fclose(fp);
        }
    }

    return 0;
}

```

Notice how this program handles `argc` and `argv`. The first element `argv[0]` is the program name. The remaining elements `argv[1]` through `argv[argc-1]` are the command-line arguments. If no files are given, the program reads from standard input -- just like the real `cat`.

Reading Directories

Directories are a special kind of file. You cannot read a directory with `fopen()` and `fgetc()` -- instead, you use the `opendir()` / `readdir()` / `closedir()` functions declared in `<dirent.h>`:

```
#include <dirent.h>

DIR *opendir(const char *name);
struct dirent *readdir(DIR *dirp);
int closedir(DIR *dirp);
```

`opendir()` opens a directory and returns a `DIR *` handle (or `NULL` on error).
`readdir()` returns a pointer to a `struct dirent` for the next entry in the directory, or `NULL` when there are no more entries. The most important field in `struct dirent` is `d_name`, which is the name of the file.

File Metadata: stat()

To get information about a file -- its size, permissions, type, modification time -- you use the `stat()` system call:

```
#include <sys/stat.h>

int stat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
```

`stat()` takes a pathname; `fstat()` takes an already-open file descriptor. Both fill in a `struct stat` with information about the file. Key fields include:

Field	Type	Description
<code>st_size</code>	<code>off_t</code>	Total size in bytes
<code>st_mode</code>	<code>mode_t</code>	File type and permissions
<code>st_mtime</code>	<code>time_t</code>	Time of last modification
<code>st_nlink</code>	<code>nlink_t</code>	Number of hard links

The `st_mode` field encodes both the file type and permissions. You can test the type with macros:

- `S_ISREG(st_mode)` -- Is it a regular file?
- `S_ISDIR(st_mode)` -- Is it a directory?
- `S_ISLNK(st_mode)` -- Is it a symbolic link? (Use `lstat()` for this.)

Complete Example: A Directory Lister

Here is a program that lists the contents of a directory, showing each entry's name, type, and size -- a simplified version of `ls -l` :

```

#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>

int main(int argc, char *argv[]) {
    const char *dirname = ".";    // default to current directory
    if (argc > 1) {
        dirname = argv[1];
    }

    DIR *dir = opendir(dirname);
    if (dir == NULL) {
        perror("opendir");
        exit(1);
    }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        // Build the full path
        char path[1024];
        snprintf(path, sizeof(path), "%s/%s", dirname, entry->d_name);

        // Get file metadata
        struct stat sb;
        if (stat(path, &sb) == -1) {
            perror(path);
            continue;
        }

        // Determine type
        char type;
        if (S_ISREG(sb.st_mode)) {
            type = '-';
        } else if (S_ISDIR(sb.st_mode)) {
            type = 'd';
        } else {
            type = '?';
        }

        printf("%c %8ld %s\n", type, (long)sb.st_size, entry->d_name);
    }

    closedir(dir);
}

```

```
    return 0;
}
```

Compile and run:

```
$ gcc -o myls myls.c
$ ./mysl /home/student
d      4096  .
d      4096  ..
-      1234  notes.txt
d      4096  projects
-      567   hello.c
```

Buffered vs. Unbuffered: When to Use Which

As a general rule:

- Use the **FILE* API** (`fopen` , `fprintf` , `fgets` , etc.) for most file I/O. It is more convenient, more portable, and faster for small reads/writes due to buffering.
- Use **low-level I/O** (`open` , `read` , `write`) when you need precise control over file descriptors -- for example, with pipes, sockets, or `dup2()` for I/O redirection. Network programming almost always uses file descriptors directly.

You can get the underlying file descriptor from a `FILE *` using `fileno(fp)` , and you can wrap a file descriptor in a `FILE *` using `fdopen(fd, mode)` . However, mixing buffered and unbuffered I/O on the same file is dangerous -- the buffer and the file descriptor can get out of sync.

Key Takeaways

- The FILE* API provides buffered I/O on top of file descriptors, reducing the number of system calls.
- `fopen()` / `fclose()` open and close buffered streams. `fprintf()` , `fgets()` , `fread()` , and `fwrite()` perform buffered reads and writes.
- `fflush()` forces buffered output to be written immediately.
- `opendir()` / `readdir()` / `closedir()` iterate over directory contents.
- `stat()` retrieves file metadata including size, type, and permissions.
- Use `argv` and `argc` to accept command-line arguments in your programs.

Lesson 3: TCP/IP and Socket Basics

From Files to Networks

In the first two lessons, we learned how to read and write files. Now we turn to a more ambitious goal: reading and writing data across a network. The good news is that Unix treats network connections as file descriptors too. Once a network connection is established, you can call `read()` and `write()` on it just as you would with a regular file. The challenge is in establishing that connection in the first place.

Before we write any networking code, we need to understand the protocols that make network communication possible.

The TCP/IP Model

The internet is organized into **layers**. Each layer provides services to the layer above it and uses services from the layer below it. The TCP/IP model has four layers:

Layer	Name	What It Does	Example Protocols
4 (top)	Application	Defines the format and meaning of messages exchanged by programs	HTTP, FTP, SSH, DNS, SMTP
3	Transport	Provides end-to-end communication between processes on different machines	TCP, UDP
2	Internet	Routes packets across networks from source to destination host	IP (IPv4, IPv6)
1 (bottom)	Link	Moves data across a single physical network link	Ethernet, Wi-Fi

As a systems programmer, you will work primarily at the boundary between the Application and Transport layers. You write code that constructs application-level messages and hands them to TCP (or UDP) for delivery. The lower layers handle the details of routing and physical transmission.

What TCP Provides

TCP (Transmission Control Protocol) provides a **reliable, ordered, byte-stream** connection between two processes. Let us unpack each of those properties:

- **Reliable:** TCP uses sequence numbers, acknowledgments, checksums, and retransmission to provide reliable transport. A successful `send()` or `write()` means bytes were accepted locally, not that the peer application received or processed them. Applications that need confirmation must define an acknowledgment in their own protocol; TCP checksums also do not provide cryptographic integrity. If a packet is lost in transit, TCP automatically detects the loss and retransmits it.

- **Ordered:** TCP guarantees that bytes arrive in the same order they were sent. Even if packets take different routes through the network and arrive out of order, TCP reassembles them in the correct sequence before delivering them to your application.
- **Byte stream:** TCP treats data as a continuous stream of bytes, not as discrete messages. There are no message boundaries. If you write 100 bytes and then 200 bytes, the receiver might read them as one 300-byte chunk, or as three 100-byte chunks, or any other combination. It is up to the application to determine where one message ends and the next begins.

This last point is critical and a common source of bugs. You cannot assume that one `write()` on the sender side corresponds to one `read()` on the receiver side. For example:

Sender:	Receiver:
<pre>write(fd, "Hello", 5); write(fd, "World", 5);</pre>	Possible read patterns:
<pre>read of 10)</pre>	<pre>read() -> "HelloWorld" (one</pre>
<pre>(split, 3 + 7)</pre>	<pre>read() -> "Hel"</pre>
<pre>(extreme split)</pre>	<pre>read() -> "loWorld" read() -> "H"</pre>
	<pre>read() -> "elloWorl" read() -> "d"</pre>
All three patterns above are LEGAL. TCP delivers exactly the 10 bytes in the correct order, but the boundaries between the two <code>write()</code> calls are erased. If your protocol needs message framing (e.g., HTTP, JSON-over-TCP), you have to encode the boundaries yourself: a length prefix, a delimiter like <code>"\r\n"</code>, or a fixed message size.	

How TCP Sets Up and Tears Down a Connection

Before any data flows, TCP does a **three-way handshake** to establish that both ends are ready to talk and to agree on starting sequence numbers. In short: the client sends a SYN, the server answers with a SYN/ACK, and the client replies with an ACK -- then the connection is open:

Client

Server

```
| ----- SYN -----> |
| "Hi, I'd like to start a connection. |
| My initial sequence number is X." |
| |
| <----- SYN/ACK ----- |
| "OK. My initial sequence number is Y. |
| I acknowledge yours." |
| |
| ----- ACK -----> |
| "I acknowledge yours." |
| |
| ===== connection open ===== |
| (data can now flow both ways) |
```

At the API level: `connect()` on the client and `accept()` on the server return successfully only after this exchange completes.

Connection close is similar but uses FIN packets and can take place independently in each direction: each side sends a FIN meaning "no more data from me," and the other side acknowledges it with an ACK:

Client

Server

```
| ----- FIN -----> | (client says:
"no more data from me")
| <----- ACK ----- |
| |
| <----- FIN ----- | (server says:
"no more data from me")
| ----- ACK -----> |
| |
| ===== connection closed ===== |
```

After `close()`, the local end of the connection enters `TIME_WAIT` for $\sim 2 \times$ the maximum segment lifetime so any stray late packets get discarded safely. (You'll see the practical effect of this in the next lesson, when we talk about `SO_REUSEADDR`.)

TCP vs. UDP

UDP (User Datagram Protocol) is the other major transport protocol. It provides a much simpler service:

Property	TCP	UDP
Reliable?	Yes -- retransmits lost packets	No -- packets can be lost
Ordered?	Yes -- delivers in order	No -- packets can arrive out of order
Connection?	Yes -- requires setup (handshake)	No -- just send packets
Message boundaries?	No -- byte stream	Yes -- each send is a datagram
Overhead?	Higher (headers, acknowledgments, retransmissions)	Lower (minimal headers)

Use TCP when you need reliable, ordered delivery -- which is most of the time (web pages, file transfers, email, remote shells). Use UDP when low latency matters more than reliability (video streaming, online games, DNS lookups).

In this module, we will focus exclusively on TCP.

IP Addresses

An **IP address** identifies a host (a machine) on a network. IPv4 addresses are 32-bit numbers, conventionally written in **dotted decimal notation**: four decimal numbers separated by dots, where each number represents one byte (0-255).

```
192.168.1.100    -- a private network address
10.0.0.1         -- another private address
128.59.105.24   -- a public address (Columbia University)
127.0.0.1        -- the loopback address (localhost)
```

The address `127.0.0.1` is special. It always refers to the local machine. When you are developing and testing network programs, you will typically connect to `127.0.0.1` (or equivalently, `localhost`) so that both the client and server run on the same machine.

IPv6 addresses are 128 bits and written in hexadecimal with colons (e.g., `::1` for localhost). We will use IPv4 in this module for simplicity.

Ports

An IP address identifies a machine, but a machine can run many programs simultaneously. A **port number** identifies a specific process (or service) on that machine. Port numbers are 16-bit unsigned integers (0 to 65535).

A network connection is identified by a 4-tuple: (**source IP, source port, destination IP, destination port**).

Port numbers are divided into ranges:

- **Well-known ports (0-1023):** Reserved for standard services. Examples: port 80 for HTTP, port 443 for HTTPS, port 22 for SSH. On Unix, binding to these ports requires root privileges.
- **Registered ports (1024-49151):** Can be used by any user. When you write a server for testing, pick a port in this range (e.g., 8080, 9000).
- **Ephemeral ports (49152-65535 per IANA):** Automatically assigned by the OS to clients when they make outgoing connections. You generally do not choose these. Note that Linux uses a different default range — `32768-60999` — configurable via `/proc/sys/net/ipv4/ip_local_port_range` .

Network Byte Order

Different machines may store multi-byte integers in different byte orders. Intel x86 processors use **little-endian** byte order (least significant byte first), but the internet protocols use **big-endian** byte order (most significant byte first). This is called **network byte order**.

When you send multi-byte values (like port numbers and IP addresses) over the network, you must convert them to network byte order. The following functions, declared in `<arpa/inet.h>` , perform the conversion:

Function	Purpose
<code>htons(x)</code>	Host to Network Short -- converts a 16-bit value from host to network byte order
<code>htonl(x)</code>	Host to Network Long -- converts a 32-bit value from host to network byte order
<code>ntohs(x)</code>	Network to Host Short -- converts a 16-bit value from network to host byte order
<code>ntohl(x)</code>	Network to Host Long -- converts a 32-bit value from network to host byte order

On a big-endian machine, these are no-ops. On a little-endian machine (like x86), they reverse the byte order. You should always use these functions when filling in network data structures, so your code is portable.

The Socket Address Structure

To specify an IP address and port number together, we use a `struct sockaddr_in` (for IPv4), declared in `<netinet/in.h>` (shown here slightly simplified -- the real declaration also contains padding, `sin_zero`, which is one reason the examples below zero out the whole structure with `memset()`):

```
struct sockaddr_in {
    sa_family_t    sin_family;    // Address family: AF_INET for
IPv4
    in_port_t      sin_port;      // Port number in network byte
order
    struct in_addr sin_addr;      // IP address in network byte
order
};

struct in_addr {
    uint32_t s_addr;              // IP address as a 32-bit integer
};
```

Here is how you fill in this structure for a server listening on port 8080 on all interfaces:

```

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));           // zero out the structure
addr.sin_family = AF_INET;                // IPv4
addr.sin_port = htons(8080);              // port 8080, converted to
network byte order
addr.sin_addr.s_addr = htonl(INADDR_ANY); // listen on all
interfaces

```

`INADDR_ANY` (which is `0.0.0.0`) tells the server to accept connections on any of the machine's network interfaces.

Converting IP Addresses: `inet_pton()` and `inet_ntop()`

To convert between human-readable IP address strings and the binary format stored in `struct in_addr`, use:

```

#include <arpa/inet.h>

// Presentation to Network: convert string to binary
int inet_pton(int af, const char *src, void *dst);

// Network to Presentation: convert binary to string
const char *inet_ntop(int af, const void *src, char *dst, socklen_t
size);

```

The "p" stands for "presentation" (the human-readable string) and "n" stands for "network" (the binary format).

Example: Converting an IP address string to binary

```

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);

// Convert "127.0.0.1" to binary and store in addr.sin_addr
if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) != 1) {
    fprintf(stderr, "Invalid address\n");
    exit(1);
}

```

Example: Converting binary to a string

```
char ip_str[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &addr.sin_addr, ip_str, sizeof(ip_str));
printf("Address: %s\n", ip_str);    // prints: Address: 127.0.0.1
```

The Headers You Will Need

Network programming in C requires several header files. Here is a summary:

```
#include <sys/socket.h>    // socket(), bind(), listen(), accept(),
connect()
#include <netinet/in.h>    // struct sockaddr_in, INADDR_ANY
#include <arpa/inet.h>      // htons(), htonl(), inet_pton(),
inet_ntop()
#include <unistd.h>         // close(), read(), write()
#include <string.h>         // memset()
#include <stdio.h>          // perror(), printf()
#include <stdlib.h>         // exit()
```

Putting It Together: Preparing an Address

Here is a complete example that sets up a socket address structure and prints it. No actual networking happens yet -- that comes in the next lesson.

```

#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <netinet/in.h>

int main(void) {
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);

    if (inet_pton(AF_INET, "192.168.1.100", &server_addr.sin_addr)
    != 1) {
        fprintf(stderr, "Invalid address\n");
        return 1;
    }

    // Print back the address to verify
    char ip_str[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &server_addr.sin_addr, ip_str,
    sizeof(ip_str));

    printf("Server address: %s\n", ip_str);
    printf("Server port:      %d\n", ntohs(server_addr.sin_port));

    return 0;
}

```

Output:

```

Server address: 192.168.1.100
Server port:      8080

```

Key Takeaways

- The TCP/IP model has four layers: Link, Internet, Transport, and Application. We program at the Transport/Application boundary.
- TCP provides a reliable, ordered byte stream. UDP provides an unreliable datagram service. We use TCP for most applications.
- An IPv4 address is a 32-bit number written in dotted decimal notation. Port numbers are 16-bit unsigned integers.

- Network byte order is big-endian. Use `htons()` / `htonl()` to convert from host to network order, and `ntohs()` / `ntohl()` for the reverse.
- `struct sockaddr_in` holds an IPv4 address and port. `inet_pton()` and `inet_ntop()` convert between string and binary representations of IP addresses.

Lesson 4: Building a Server

The Socket Lifecycle

In the last lesson, we learned the concepts behind TCP/IP networking. Now it is time to write actual networking code. A TCP server goes through a specific sequence of system calls to accept incoming connections. This sequence is sometimes called the **socket lifecycle**:

1. `socket()` -- Create a socket (get a file descriptor)
2. `bind()` -- Assign a local address (IP + port) to the socket
3. `listen()` -- Mark the socket as passive (willing to accept connections)
4. `accept()` -- Wait for and accept an incoming connection (returns a **new** file descriptor for that connection)
5. `read()` / `write()` -- Exchange data with the connected client
6. `close()` -- Close the connection

Let us walk through each step in detail.

Step 1: `socket()` -- Creating a Socket

```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

This creates a new socket and returns a file descriptor. The arguments are:

- `domain` : The address family. Use `AF_INET` for IPv4.
- `type` : The socket type. Use `SOCK_STREAM` for TCP (a reliable byte stream).
- `protocol` : The protocol. Use `0` to let the OS choose the appropriate protocol (TCP for `SOCK_STREAM`).


```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd == -1) {
    perror("socket");
    exit(1);
}
```

At this point, `server_fd` is a file descriptor, but it is not yet associated with any address or port.

Setting `SO_REUSEADDR`

Before binding, it is a good practice to set the `SO_REUSEADDR` option. When a TCP server closes a connection, the connection (a 4-tuple of local IP, local port, remote IP, remote port) lingers in the **TIME_WAIT** state for roughly $2 \times \text{MSL}$ (a couple of minutes) so that any stray packets can be discarded safely. While that connection is in `TIME_WAIT`, the kernel will normally refuse to `bind()` a new socket to the same local address and port — annoying during development, because stopping and restarting your server makes `bind()` fail with "Address already in use." Setting `SO_REUSEADDR` tells the kernel to allow that `bind()` anyway:

```
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt)) == -1) {
    perror("setsockopt");
    exit(1);
}
```

You should include this in every server you write.

Step 2: `bind()` -- Assigning an Address

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t
addrlen);
```

`bind()` associates the socket with a specific IP address and port number. We fill in a `struct sockaddr_in` as we learned in Lesson 3:

```

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY); // listen on all
interfaces
addr.sin_port = htons(8080);              // listen on port 8080

if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) == -1)
{
    perror("bind");
    exit(1);
}

```

Note the cast to `(struct sockaddr *)`. The socket API was designed before C had `void *`, so it uses a generic `struct sockaddr` as the base type. You always cast your address-family-specific structure to this type.

Step 3: listen() -- Marking as Passive

```
int listen(int sockfd, int backlog);
```

`listen()` marks the socket as a **passive** socket -- one that will be used to accept incoming connections rather than to initiate outgoing connections. The `backlog` argument specifies the maximum number of pending connections that can be queued before the server calls `accept()`. A value of 5 to 10 is typical for a simple server:

```

if (listen(server_fd, 5) == -1) {
    perror("listen");
    exit(1);
}
printf("Server listening on port 8080...\n");

```

Step 4: accept() -- Accepting a Connection

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

This is where the server waits for a client to connect. `accept()` **blocks** (pauses execution) until a client connects. When a connection arrives, it returns a **new file descriptor** that

represents the connection to that specific client. The original `server_fd` remains open and continues to listen for more connections.

The `addr` and `addr len` parameters are filled in with the client's address. You can pass `NULL` for both if you do not care who the client is:

```
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);

int client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
&client_len);
if (client_fd == -1) {
    perror("accept");
    exit(1);
}

// Print the client's address
char client_ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &client_addr.sin_addr, client_ip,
sizeof(client_ip));
printf("Client connected from %s:%d\n", client_ip,
ntohs(client_addr.sin_port));
```

This is the key insight of the socket API: **`accept()` returns a new file descriptor**. The listening socket (`server_fd`) and the connected socket (`client_fd`) are two different file descriptors. The server uses `client_fd` to communicate with the client, and `server_fd` to wait for more clients.

Step 5: Exchanging Data

Once you have a connected file descriptor, you can use `read()` and `write()` -- the same system calls we used for file I/O. You can also use `send()` and `recv()`, which are socket-specific versions that offer additional flags:

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

With `flags` set to 0, `send()` and `recv()` behave identically to `write()` and `read()`. We will use `read()` and `write()` in our examples since they reinforce the "everything is a file" concept.

Step 6: Closing the Connection

When the conversation is finished, close the client's file descriptor:

```
close(client_fd);
```

When you are done accepting connections entirely, close the listening socket too:

```
close(server_fd);
```

Complete Example: Echo Server

An **echo server** reads data from a client and sends it back unchanged. Here is a complete, working echo server that handles one client at a time:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 1024

int main(void) {
    // 1. Create socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        exit(1);
    }

    // Allow port reuse
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt));

    // 2. Bind to address
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) ==
-1) {
        perror("bind");
        exit(1);
    }

    // 3. Listen for connections
    if (listen(server_fd, 5) == -1) {
        perror("listen");
        exit(1);
    }
    printf("Echo server listening on port %d...\n", PORT);

    // 4. Accept a client
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);

```

```

    int client_fd = accept(server_fd, (struct sockaddr
*)&client_addr, &client_len);
    if (client_fd == -1) {
        perror("accept");
        exit(1);
    }

    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip,
sizeof(client_ip));
    printf("Client connected from %s:%d\n", client_ip,
ntohs(client_addr.sin_port));

    // 5. Echo loop: read from client, write back
    char buf[BUF_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(client_fd, buf, sizeof(buf))) > 0) {
        // Write all bytes back to the client
        char *ptr = buf;
        ssize_t remaining = bytes_read;
        while (remaining > 0) {
            ssize_t written = write(client_fd, ptr, remaining);
            if (written == -1) {
                perror("write");
                goto done;    // client is gone; stop echoing
            }
            ptr += written;
            remaining -= written;
        }
    }

    if (bytes_read == -1) {
        perror("read");
    } else {
        printf("Client disconnected.\n");
    }

done:
    // 6. Close
    close(client_fd);
    close(server_fd);

    return 0;
}

```

Notice the inner loop in step 5: rather than assuming a single `write()` sends everything, we keep writing until all `bytes_read` bytes have been echoed back (this is the `write_all()` pattern from Lesson 1).

Testing the Echo Server

Compile and start the server in one terminal:

```
$ gcc -o echo_server echo_server.c
$ ./echo_server
Echo server listening on port 8080...
```

In a second terminal, connect using `nc` (netcat) or `telnet` :

```
$ nc localhost 8080
Hello, server!
Hello, server!
Testing 123
Testing 123
```

Every line you type is sent to the server, which echoes it back. Press Ctrl+C or Ctrl+D to disconnect.

You can also use `telnet` :

```
$ telnet localhost 8080
Trying 127.0.0.1...
Connected to localhost.
```

Understanding the Data Flow

Let us trace what happens when a client types "Hello" and presses Enter:

1. The client sends the bytes `Hello\n` (6 bytes) over the TCP connection.
2. The server's `read()` returns 6, placing `Hello\n` in the buffer.
3. The server's `write()` sends those same 6 bytes back.
4. The client receives `Hello\n` and displays it.

When the client disconnects (closes its end of the connection), the server's `read()` returns 0 (EOF), and the echo loop ends.

Limitations of This Server

This server has a major limitation: it can only handle **one client at a time**. After the first client connects, `accept()` is not called again, so additional clients must wait until the first client disconnects. In the next lesson, we will fix this with threads.

Key Takeaways

- The server socket lifecycle is: `socket()` , `bind()` , `listen()` , `accept()` , `read/write`, `close()` .
- `accept()` returns a **new** file descriptor for each client connection. The listening socket is separate from the connected socket.
- Always set `SO_REUSEADDR` on your server socket to avoid "Address already in use" errors during development.
- Once a connection is established, `read()` and `write()` work just like file I/O -- this is the "everything is a file" principle in action.
- Use `nc` (netcat) or `telnet` to test your server.

Lesson 5: Clients and Multi-Client Servers

Writing a TCP Client

In Lesson 4, we tested our server using `nc`. Now let us write our own client program in C. The client lifecycle is simpler than the server's:

1. `socket()` -- Create a socket
2. `connect()` -- Connect to a server at a given IP and port
3. `read()` / `write()` -- Exchange data
4. `close()` -- Close the connection

Notice that the client does not call `bind()` or `listen()`. It does not need to bind to a specific port -- the operating system automatically assigns an ephemeral port when `connect()` is called.

The `connect()` System Call

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *addr, socklen_t
addrlen);
```

`connect()` initiates a TCP connection to the server specified by `addr`. It performs the TCP three-way handshake (SYN, SYN-ACK, ACK). It blocks until the connection is established or an error occurs. Returns 0 on success, -1 on error.

Complete Example: Echo Client

This client connects to our echo server, sends lines of text typed by the user, and prints the echoed responses:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    const char *server_ip = "127.0.0.1";
    if (argc > 1) {
        server_ip = argv[1];
    }

    // 1. Create socket
    int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (sock_fd == -1) {
        perror("socket");
        exit(1);
    }

    // 2. Set up the server address
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) != 1)
    {
        fprintf(stderr, "Invalid address: %s\n", server_ip);
        exit(1);
    }

    // 3. Connect to the server
    if (connect(sock_fd, (struct sockaddr *)&server_addr,
    sizeof(server_addr)) == -1) {
        perror("connect");
        exit(1);
    }
    printf("Connected to %s:%d\n", server_ip, PORT);

    // 4. Send and receive loop
    char send_buf[BUF_SIZE];
    char recv_buf[BUF_SIZE];
```

```

    printf("Type a message (Ctrl+D to quit):\n");

    while (fgets(send_buf, sizeof(send_buf), stdin) != NULL) {
        // Send the line to the server
        size_t len = strlen(send_buf);
        if (write(sock_fd, send_buf, len) == -1) {
            perror("write");
            break;
        }

        // Read the echo back
        ssize_t bytes_read = read(sock_fd, recv_buf,
sizeof(recv_buf) - 1);
        if (bytes_read <= 0) {
            if (bytes_read == 0)
                printf("Server closed connection.\n");
            else
                perror("read");
            break;
        }

        recv_buf[bytes_read] = '\0';
        printf("Echo: %s", recv_buf);
    }

    // 5. Close
    close(sock_fd);
    printf("Disconnected.\n");

    return 0;
}

```

One simplification to note: this client sends each line with a single `write()` and assumes a single `read()` returns the entire echo. That works here because a short message over the loopback interface arrives in one piece, but a robust client would loop on `write()` as in Lesson 1 and keep calling `read()` until the full message arrives, for the short-count reasons Lesson 3 explains.

Also note that this client accepts only a numeric IP address: `inet_pton()` parses dotted-decimal strings like `127.0.0.1`, so passing a hostname like `localhost` would fail with "Invalid address". Translating names into addresses requires `getaddrinfo()`, which is beyond our scope here (Beej's Guide covers it well).

Compile and test against the echo server from Lesson 4:

```
$ gcc -o echo_client echo_client.c
$ ./echo_client
Connected to 127.0.0.1:8080
Type a message (Ctrl+D to quit):
Hello from the client!
Echo: Hello from the client!
```

The Problem with One Client at a Time

Our echo server from Lesson 4 can only handle one client. After `accept()` returns, the server enters the echo loop and stays there until that client disconnects. Any other clients trying to connect must wait.

For a real server, this is unacceptable. A web server might need to handle hundreds or thousands of simultaneous connections. There are several approaches to solving this problem:

- **fork()** -- Spawn a child process for each client (you know this from earlier modules)
- **Threads** -- Spawn a thread for each client (you know this too)
- **select()** / **poll()** / **epoll()** -- Multiplex I/O in a single thread (beyond the scope of this module)

We will use **threads**, since you have already learned about them in the previous module. Threads are a natural fit because all threads in a process share the same address space, which makes sharing data (if needed) straightforward.

The Threaded Server Pattern

The idea is simple:

1. The main thread runs an accept loop, calling `accept()` repeatedly.
2. Each time `accept()` returns a new client file descriptor, the main thread spawns a new thread to handle that client.
3. The handler thread reads from and writes to the client, then closes the connection and exits.
4. Meanwhile, the main thread loops back to `accept()` and waits for the next client.

Passing the file descriptor to the thread

There is a subtle but important issue when passing the client file descriptor to a thread. Consider this incorrect approach:

```
// WRONG -- race condition!
while (1) {
    int client_fd = accept(server_fd, NULL, NULL);
    pthread_create(&tid, NULL, handle_client, &client_fd);
    // Problem: client_fd might be overwritten by the next accept()
    // before the thread has a chance to read it!
}
```

The variable `client_fd` lives on the main thread's stack. If the main thread loops back and calls `accept()` again before the new thread reads the value, the new thread will see the wrong file descriptor. This is a **race condition**.

The solution is to allocate memory for each file descriptor on the heap:

```
while (1) {
    int *client_fd = malloc(sizeof(int));
    *client_fd = accept(server_fd, NULL, NULL);
    if (*client_fd == -1) {
        perror("accept");
        free(client_fd);
        continue;
    }
    int rc = pthread_create(&tid, NULL, handle_client, client_fd);
    if (rc != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(rc));
        close(*client_fd);
        free(client_fd);
        continue;
    }
    rc = pthread_detach(tid); // we won't join this thread
    if (rc != 0) {
        fprintf(stderr, "pthread_detach: %s\n", strerror(rc));
    }
}
```

Each thread gets its own copy of the file descriptor on the heap. The thread is responsible for freeing the memory when it is done.

Complete Example: Threaded Echo Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 1024

void *handle_client(void *arg) {
    int client_fd = *(int *)arg;
    free(arg); // free the heap-allocated file descriptor

    char client_ip[INET_ADDRSTRLEN] = "unknown";
    struct sockaddr_in client_addr = { 0 };
    socklen_t addr_len = sizeof(client_addr);
    if (getpeername(client_fd, (struct sockaddr *)&client_addr,
&addr_len) == 0)
        inet_ntop(AF_INET, &client_addr.sin_addr, client_ip,
sizeof(client_ip));
    printf("Thread %lu handling client %s:%d\n",
        (unsigned long)pthread_self(), client_ip,
ntohs(client_addr.sin_port));

    // Echo loop
    char buf[BUF_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(client_fd, buf, sizeof(buf))) > 0) {
        char *ptr = buf;
        ssize_t remaining = bytes_read;
        while (remaining > 0) {
            ssize_t written = write(client_fd, ptr, remaining);
            if (written == -1) {
                perror("write");
                goto done;
            }
            ptr += written;
            remaining -= written;
        }
    }
}
```

```

done:
    printf("Client %s:%d disconnected.\n", client_ip,
ntohs(client_addr.sin_port));
    close(client_fd);
    return NULL;
}

int main(void) {
    // Ignore SIGPIPE so a write() to a disconnected client
    // returns an error instead of killing the server
    signal(SIGPIPE, SIG_IGN);

    // Create socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        exit(1);
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt));

    // Bind
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) ==
-1) {
        perror("bind");
        exit(1);
    }

    // Listen
    if (listen(server_fd, 10) == -1) {
        perror("listen");
        exit(1);
    }
    printf("Threaded echo server listening on port %d...\n", PORT);

    // Accept loop
    while (1) {
        int *client_fd = malloc(sizeof(int));
        if (client_fd == NULL) {
            perror("malloc");

```

```

        continue;
    }

    *client_fd = accept(server_fd, NULL, NULL);
    if (*client_fd == -1) {
        perror("accept");
        free(client_fd);
        continue;
    }

    pthread_t tid;
    int rc = pthread_create(&tid, NULL, handle_client,
client_fd);
    if (rc != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(rc));
        close(*client_fd);
        free(client_fd);
        continue;
    }
    rc = pthread_detach(tid);
    if (rc != 0) {
        fprintf(stderr, "pthread_detach: %s\n", strerror(rc));
    }
}

close(server_fd);
return 0;
}

```

Note the `signal(SIGPIPE, SIG_IGN)` call at the top of `main()`: a `write()` to a client that has already closed the connection raises **SIGPIPE**, whose default action (recall Module 5's signal dispositions) terminates the whole process -- ignoring it makes `write()` return -1 with `errno` set to `EPIPE` instead, so the handler thread can clean up normally. As in Lesson 4, the echo loop inside `handle_client()` keeps calling `write()` until every byte read has been sent back (this is the `write_all()` pattern from Lesson 1).

Compile with the `-pthread` flag:

```

$ gcc -o threaded_echo threaded_echo.c -pthread
$ ./threaded_echo
Threaded echo server listening on port 8080...

```

Now you can open multiple terminals and connect multiple clients simultaneously. Each client is handled by its own thread.

File Descriptor Lifecycle in Multi-Client Servers

It is important to understand which file descriptors exist and who is responsible for closing them:

File Descriptor	Created By	Used By	Closed By
<code>server_fd</code>	Main thread (<code>socket()</code>)	Main thread (<code>accept()</code>)	Main thread (at shutdown)
<code>client_fd</code> (one per client)	Main thread (<code>accept()</code>)	Handler thread (read/write)	Handler thread

Each `accept()` call creates a new file descriptor. The handler thread owns that file descriptor and is responsible for closing it when the client disconnects. The listening socket (`server_fd`) stays open the entire time the server is running.

If you use `fork()` instead of threads, the situation is more complex: both the parent and child process have copies of the file descriptor, and both must close their copy. With threads, there is only one copy because threads share the same file descriptor table.

Using `pthread_detach()`

In the server above, we call `pthread_detach(tid)` after creating each thread. This tells the system that no other thread will call `pthread_join()` on this thread. When the detached thread finishes, its resources are automatically cleaned up. Without detaching (and without joining), the thread's resources would leak.

A Note on Robustness

The threaded server above is a good starting point, but a production server would need additional features:

- **Thread pool:** Creating a new thread for every client is expensive. A real server would pre-create a pool of threads and assign clients to them.
- **Signal handling:** The server should handle SIGINT (Ctrl+C) gracefully, closing the listening socket and waiting for active clients to finish. We already handle SIGPIPE above by ignoring it, so a client that disconnects mid-write cannot kill the server.
- **Connection limits:** Without limits, an attacker could exhaust the server's resources by opening thousands of connections.

For this course, the thread-per-client model is sufficient. Understanding it is the foundation for understanding more sophisticated approaches.

Key Takeaways

- A TCP client calls `socket()`, `connect()`, read/write, and `close()`. No `bind()` or `listen()` is needed.
- To handle multiple clients, put `accept()` in a loop and spawn a thread for each new connection.
- Pass the client file descriptor to the thread via a heap-allocated pointer to avoid race conditions.
- Use `pthread_detach()` so thread resources are cleaned up automatically.
- Each client connection gets its own file descriptor. The handler thread is responsible for closing it.

Lesson 6: From Echo Server to Web Server

Introduction

In the previous two lessons, we built echo servers and clients. An echo server is useful for learning, but it does not do anything practical. In this lesson, we will build something real: a **minimal HTTP web server**. This ties together everything we have learned in this module -- file I/O, sockets, and threads -- into a single program.

When you type a URL into your browser, the browser connects to a web server using TCP, sends an HTTP request, and receives an HTTP response containing the web page. Our server will handle exactly this: accept TCP connections, parse HTTP requests, read files from disk, and send HTTP responses.

HTTP Basics

HTTP (Hypertext Transfer Protocol) is a text-based application-layer protocol. It follows a simple request-response pattern: the client sends a request, the server sends a response.

HTTP Request Format

An HTTP request looks like this:

```
GET /index.html HTTP/1.1\r\n
Host: localhost:8080\r\n
Connection: close\r\n
\r\n
```

The first line is the **request line**. It has three parts:

- **Method:** `GET` means "retrieve this resource." Other methods include POST, PUT, DELETE, etc. We only need to handle GET.
- **Path:** `/index.html` is the file being requested.
- **Version:** `HTTP/1.1` is the protocol version.

After the request line come zero or more **header lines**, each ending with `\r\n` (carriage return followed by newline). The headers end with a blank line (`\r\n` by itself).

For our minimal server, we only need to parse the first line to extract the method and path. We can ignore the headers.

HTTP Response Format

An HTTP response looks like this:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 41\r\n
Connection: close\r\n
\r\n
<html><body><h1>Hello!</h1></body></html>
```

The first line is the **status line**:

- **Version:** `HTTP/1.1`
- **Status code:** `200` means success. `404` means "not found." `500` means "internal server error."
- **Reason phrase:** `OK` is a human-readable description of the status code.

After the status line come response headers, then a blank line, then the **body** (the actual content). The `Content-Length` header tells the client how many bytes are in the body. The `Content-Type` header tells the client what kind of data it is (HTML, plain text, an image, etc.).

Content Types

The `Content-Type` header tells the browser how to interpret the response body. Here are common types:

File Extension	Content-Type
.html	text/html
.css	text/css
.js	application/javascript (current guidance, RFC 9239, is text/javascript ; both work in practice)
.txt	text/plain
.jpg	image/jpeg
.png	image/png

Parsing an HTTP Request

Our server needs to extract the path from the request line. Here is the parsing logic we will embed in `handle_client()` in the complete server below, shown here as a standalone function so we can study it in isolation:

```
// Read the HTTP request and extract the path.
// Returns 0 on success, -1 on error.
int parse_request(int client_fd, char *method, char *path) {
    char buf[2048];
    ssize_t bytes_read = read(client_fd, buf, sizeof(buf) - 1);
    if (bytes_read <= 0) {
        return -1;
    }
    buf[bytes_read] = '\0';

    // Parse the request line: "GET /path HTTP/1.1\r\n..."
    // sscanf will extract the method and path
    if (sscanf(buf, "%15s %255s", method, path) != 2) {
        return -1;
    }

    return 0;
}
```

This is a simplified parser. A production HTTP server would need to handle multi-packet requests, very long headers, and many edge cases. For our purposes, this works because the request line is almost always contained in the first `read()`.

Serving a Static File

Once we know the requested path (e.g., `/index.html`), we need to open that file from a directory on disk, read its contents, and send them back in an HTTP response. We also need to handle the case where the file does not exist (404 Not Found).

Here is a function that determines the content type from the file extension:

```
const char *get_content_type(const char *path) {
    const char *ext = strrchr(path, '.');
    if (ext == NULL)        return "application/octet-stream";
    if (strcmp(ext, ".html") == 0) return "text/html";
    if (strcmp(ext, ".css") == 0) return "text/css";
    if (strcmp(ext, ".js") == 0)  return "application/javascript";
    if (strcmp(ext, ".txt") == 0) return "text/plain";
    if (strcmp(ext, ".jpg") == 0) return "image/jpeg";
    if (strcmp(ext, ".png") == 0) return "image/png";
    return "application/octet-stream";
}
```

Sending the Response

Here is a helper function that sends an HTTP response with headers and a body:

```

void send_response(int client_fd, int status_code, const char
*status_text,
                    const char *content_type, const char *body,
size_t body_len) {
    char header[512];
    int header_len = snprintf(header, sizeof(header),
        "HTTP/1.1 %d %s\r\n"
        "Content-Type: %s\r\n"
        "Content-Length: %zu\r\n"
        "Connection: close\r\n"
        "\r\n",
        status_code, status_text, content_type, body_len);

    write(client_fd, header, header_len);
    if (body_len > 0) {
        write(client_fd, body, body_len);
    }
}

```

Complete Example: Minimal Web Server

Here is the complete web server. It serves files from a directory called `www` in the current working directory. It uses threads to handle multiple clients.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <pthread.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 4096
#define WEB_ROOT "./www"

const char *get_content_type(const char *path) {
    const char *ext = strrchr(path, '.');
    if (ext == NULL)                return "application/octet-
stream";
    if (strcmp(ext, ".html") == 0)  return "text/html";
    if (strcmp(ext, ".css") == 0)   return "text/css";
    if (strcmp(ext, ".js") == 0)    return
"application/javascript";
    if (strcmp(ext, ".txt") == 0)    return "text/plain";
    if (strcmp(ext, ".jpg") == 0)    return "image/jpeg";
    if (strcmp(ext, ".png") == 0)    return "image/png";
    return "application/octet-stream";
}

void send_response(int client_fd, int status_code, const char
*status_text,
                  const char *content_type, const char *body,
size_t body_len) {
    char header[512];
    int header_len = snprintf(header, sizeof(header),
        "HTTP/1.1 %d %s\r\n"
        "Content-Type: %s\r\n"
        "Content-Length: %zu\r\n"
        "Connection: close\r\n"
        "\r\n",
        status_code, status_text, content_type, body_len);

    write(client_fd, header, header_len);
    if (body_len > 0) {
        write(client_fd, body, body_len);
    }
}

```



```

void send_error(int client_fd, int status_code, const char
*status_text) {
    char body[256];
    int body_len = snprintf(body, sizeof(body),
        "<html><body><h1>%d %s</h1></body></html>",
        status_code, status_text);
    send_response(client_fd, status_code, status_text,
        "text/html", body, body_len);
}

void *handle_client(void *arg) {
    int client_fd = *(int *)arg;
    free(arg);

    // Read the request
    char request[2048];
    ssize_t bytes_read = read(client_fd, request, sizeof(request) -
1);
    if (bytes_read <= 0) {
        close(client_fd);
        return NULL;
    }
    request[bytes_read] = '\0';

    // Parse the request line
    char method[16], path[256];
    if (sscanf(request, "%15s %255s", method, path) != 2) {
        send_error(client_fd, 400, "Bad Request");
        close(client_fd);
        return NULL;
    }

    printf("%s %s\n", method, path);

    // Only handle GET requests.
    // HTTP/1.1 (RFC 9110 §15.5.6) requires that 405 responses
include
    // an "Allow" header listing the supported methods, so we send
a
    // hand-rolled response here instead of going through
send_error.
    if (strcmp(method, "GET") != 0) {
        const char *resp =
            "HTTP/1.1 405 Method Not Allowed\r\n"
            "Allow: GET\r\n"
            "Content-Type: text/html\r\n"
            "Content-Length: 57\r\n"
            "Connection: close\r\n"

```

```

        "\r\n"
        "<html><body><h1>405 Method Not Allowed</h1></body>
</html>";
        write(client_fd, resp, strlen(resp));
        close(client_fd);
        return NULL;
    }

    // If path is "/", serve "/index.html"
    if (strcmp(path, "/") == 0) {
        strcpy(path, "/index.html");
    }

    // Build the file path: WEB_ROOT + path
    // Security note: a real server must prevent "." traversal
attacks
    char filepath[512];
    snprintf(filepath, sizeof(filepath), "%s%s", WEB_ROOT, path);

    // Reject any "." path component. Note: searching for the
substring
    // "." would also reject legitimate filenames like
"my..file.html",
    // so we check for it as a path component instead. A production
server
    // would canonicalize with realpath() and verify the result
still lives
    // inside WEB_ROOT.
    if (strcmp(path, "..") == 0
        || strncmp(path, "../", 3) == 0
        || strstr(path, "/..") != NULL) {
        send_error(client_fd, 403, "Forbidden");
        close(client_fd);
        return NULL;
    }

    // Get file info
    struct stat sb;
    if (stat(filepath, &sb) == -1 || !S_ISREG(sb.st_mode)) {
        send_error(client_fd, 404, "Not Found");
        close(client_fd);
        return NULL;
    }

    // Open and read the file
    int file_fd = open(filepath, O_RDONLY);
    if (file_fd == -1) {
        send_error(client_fd, 500, "Internal Server Error");
        close(client_fd);
    }

```

```

        return NULL;
    }

    // Send the response header
    const char *content_type = get_content_type(filepath);
    char header[512];
    int header_len = snprintf(header, sizeof(header),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: %s\r\n"
        "Content-Length: %ld\r\n"
        "Connection: close\r\n"
        "\r\n",
        content_type, (long)sb.st_size);

    write(client_fd, header, header_len);

    // Send the file contents
    char buf[BUF_SIZE];
    ssize_t n;
    while ((n = read(file_fd, buf, sizeof(buf))) > 0) {
        char *ptr = buf;
        ssize_t remaining = n;
        while (remaining > 0) {
            ssize_t written = write(client_fd, ptr, remaining);
            if (written == -1) {
                perror("write");
                goto done; // client is gone; abandon the
transfer
            }
            ptr += written;
            remaining -= written;
        }
    }

done:
    close(file_fd);
    close(client_fd);
    return NULL;
}

int main(void) {
    // Ignore SIGPIPE so a write() to a client that closed the
    // connection returns an error instead of killing the server
    signal(SIGPIPE, SIG_IGN);

    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        exit(1);
    }

```

```

    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt));

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) ==
-1) {
        perror("bind");
        exit(1);
    }

    if (listen(server_fd, 10) == -1) {
        perror("listen");
        exit(1);
    }

    printf("Web server listening on http://localhost:%d/\n", PORT);
    printf("Serving files from %s\n", WEB_ROOT);

    while (1) {
        int *client_fd = malloc(sizeof(int));
        if (client_fd == NULL) {
            perror("malloc");
            continue;
        }

        *client_fd = accept(server_fd, NULL, NULL);
        if (*client_fd == -1) {
            perror("accept");
            free(client_fd);
            continue;
        }

        pthread_t tid;
        int rc = pthread_create(&tid, NULL, handle_client,
client_fd);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            close(*client_fd);
            free(client_fd);
            continue;
        }
    }

```

```

        rc = pthread_detach(tid);
        if (rc != 0) {
            fprintf(stderr, "pthread_detach: %s\n", strerror(rc));
        }
    }

    close(server_fd);
    return 0;
}

```

As in Lesson 5, `main()` begins by ignoring **SIGPIPE**: browsers routinely abort connections mid-transfer (the user hits Stop or closes the tab), and a `write()` to a socket the client has closed would otherwise raise SIGPIPE and kill the whole server (its default action, from Module 5) -- ignored, `write()` instead returns -1 with `errno` set to **EPIPE**, and the send loop abandons the transfer. Notice also the inner loop when sending the file contents: `write()` may send fewer bytes than requested, so we keep writing until all `n` bytes have gone out (this is the `write_all()` pattern from Lesson 1).

Testing the Web Server

First, create the web root directory and an HTML file:

```

$ mkdir www
$ cat > www/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head><title>My Web Server</title></head>
<body>
<h1>It works!</h1>
<p>This page is being served by our C web server.</p>
</body>
</html>
EOF

```

Compile and run:

```

$ gcc -o webserver webserver.c -pthread
$ ./webserver
Web server listening on http://localhost:8080/
Serving files from ./www

```

Open a browser and go to `http://localhost:8080/`. You should see the HTML page rendered. You can also test with `curl`:

```
$ curl -v http://localhost:8080/
* Connected to localhost (127.0.0.1) port 8080
> GET / HTTP/1.1
> Host: localhost:8080
>
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 161
< Connection: close
<
<!DOCTYPE html>
<html>
...
```

How It All Fits Together

Look at how many concepts from this module (and the entire course) come together in this one program:

- **File descriptors** (Lesson 1): The server uses `open()`, `read()`, `write()`, and `close()` to read files from disk.
- **The FILE API and stat()** (Lesson 2): We use `stat()` to get the file size for the Content-Length header, and `S_ISREG()` to verify we are serving a regular file.
- **TCP/IP** (Lesson 3): The server communicates over TCP. Data is sent as a byte stream.
- **The socket API** (Lesson 4): We use `socket/bind/listen/accept` to set up the server.
- **Threads** (Lesson 5, and earlier modules): Each client is handled in its own thread.
- **String parsing**: We parse the HTTP request to extract the method and path.
- **Memory management**: We heap-allocate the client file descriptor to avoid race conditions.

This is the power of the Unix "everything is a file" philosophy. The same `read()` and `write()` calls work for disk files and network sockets. The same file descriptor abstraction unifies all I/O.

Security Considerations

Our web server includes a basic check for `..` in the path to prevent directory traversal attacks (where a client requests `/../../../../etc/passwd` to read files outside the web root). A production server would need much more robust security:

- Canonicalize the path and verify it falls within the web root
- Limit request sizes to prevent memory exhaustion
- Add timeouts so slow clients cannot hold connections open forever
- Run as an unprivileged user
- Support HTTPS (TLS encryption)

Our server is for learning purposes only. Do not expose it to the internet.

Key Takeaways

- HTTP is a text-based request-response protocol. A request has a method, path, and headers. A response has a status code, headers, and a body.
- A minimal web server parses the request path, opens the corresponding file, and sends it back with appropriate HTTP headers.
- Building a web server ties together file I/O, sockets, threads, and string parsing -- nearly every systems programming concept in this course.
- The "everything is a file" abstraction is what makes this possible: the same `read()` / `write()` interface works for both disk files and network sockets.

Module 7 Summary

What We Covered

Module 7 traced a single idea -- the Unix "everything is a file" philosophy -- from opening a file on disk all the way to building a web server. Here is a summary of each lesson and its key concepts.

Lesson 1: File Descriptors and Low-Level I/O

We learned that a **file descriptor** is a small non-negative integer that the kernel uses to identify an open file, device, pipe, or socket. Every process starts with three open descriptors: `stdin` (0), `stdout` (1), and `stderr` (2). The system calls `open()`, `read()`, `write()`, and `close()` form the foundation of all I/O in Unix. We saw that `read()` and `write()` can return fewer bytes than requested (short reads/writes), and that robust code must loop to handle this.

Lesson 2: The FILE API and Directories

We learned about the C standard library's **buffered I/O** layer (`FILE *`), which wraps file descriptors in a buffer to reduce the number of system calls. Functions like `fopen()`, `fclose()`, `fprintf()`, `fgets()`, `fread()`, and `fwrite()` provide a more convenient interface for most file operations. We also learned to read directory contents with `opendir()` / `readdir()` / `closedir()` and to query file metadata with `stat()`.

Lesson 3: TCP/IP and Socket Basics

We studied the **TCP/IP model** and its four layers (Link, Internet, Transport, Application). TCP provides a reliable, ordered byte stream between two processes. We learned about IP addresses (32-bit numbers in dotted decimal notation), ports (16-bit numbers identifying services), and network byte order (big-endian). The functions `htons()` / `htonl()` / `ntohs()` / `ntohl()` convert between host and network byte order. The `struct sockaddr_in` structure holds an IP address and port for socket operations.

Lesson 4: Building a Server

We built our first TCP server by walking through the **socket lifecycle**: `socket()` to create a file descriptor, `bind()` to assign an address, `listen()` to mark it as passive, and `accept()` to wait for connections. The critical insight is that `accept()` returns a **new** file descriptor for each client -- the listening socket and the connected socket are separate. We built a complete echo server and tested it with `nc`.

Lesson 5: Clients and Multi-Client Servers

We wrote a TCP **client** using `socket()` and `connect()`, then addressed the limitation of our single-client server by using **threads**. The threaded server pattern puts `accept()` in a loop, spawning a new thread for each connection. We learned to pass the client file descriptor via heap allocation to avoid race conditions, and to use `pthread_detach()` for automatic resource cleanup.

Lesson 6: From Echo Server to Web Server

We built a **minimal HTTP web server** that ties together every concept in the module. The server parses HTTP requests (extracting the method and path), reads files from disk using low-level I/O, determines content types, constructs HTTP responses with proper headers, and serves them over TCP sockets -- all while handling multiple clients with threads.

The Big Picture: Everything Is a File

The central theme of this module is that Unix provides a **uniform interface** for all I/O. Whether you are reading from a file on disk, writing to a terminal, piping data between processes, or sending bytes over a network, you use the same fundamental operations: `open` (or `socket + accept`), `read`, `write`, `close`.

This is what allowed us to build a web server that reads files from disk and sends them over the network using the same `read()` and `write()` calls. It is also why the skills you learned in this module -- handling file descriptors, managing partial reads/writes, understanding byte streams -- will serve you well in any systems programming context.

The Socket API at a Glance

System Call	Server	Client	Purpose
<code>socket()</code>	Yes	Yes	Create a new socket (file descriptor)
<code>bind()</code>	Yes	No	Assign a local address and port
<code>listen()</code>	Yes	No	Mark socket as passive (ready to accept)
<code>accept()</code>	Yes	No	Wait for and accept a client connection
<code>connect()</code>	No	Yes	Initiate a connection to a server
<code>read()/recv()</code>	Yes	Yes	Receive data
<code>write()/send()</code>	Yes	Yes	Send data
<code>close()</code>	Yes	Yes	Close a file descriptor

Common Pitfalls

- **Forgetting to handle partial reads/writes:** Both `read()` and `write()` can transfer fewer bytes than requested. Always loop.
- **Not checking return values:** Every system call can fail. Always check for -1 returns and use `perror()` to diagnose errors.
- **Forgetting to close file descriptors:** Leaked file descriptors are a resource leak. Close them when done.
- **Forgetting network byte order:** Port numbers and IP addresses in `struct sockaddr_in` must be in big-endian format. Use `htons()` and `htonl()`.

- **Race conditions in threaded servers:** Do not pass a stack variable's address to a thread. Use heap allocation.
- **Mixing buffered and unbuffered I/O:** Do not use `FILE *` functions and raw `read()` / `write()` on the same file descriptor without care. The buffer can get out of sync.
- **Assuming message boundaries in TCP:** TCP is a byte stream. One `write()` does not correspond to one `read()`.

What Comes Next

The concepts in this module form the basis of virtually all networked software. Web servers, databases, chat applications, distributed systems, and cloud infrastructure all rely on the same socket API and file I/O primitives you learned here. One natural next topic is **event-driven I/O** -- using `select()`, `poll()`, or `epoll()` to let a single thread monitor many sockets at once, which is the scalable alternative to our thread-per-client design and the approach used by servers like nginx and Node.js. Understanding these low-level details will make you a more effective programmer, even when you eventually work with higher-level languages and frameworks that abstract them away.

Additional Resources

- Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective*, Chapters 10 and 11
- Stevens, Fenner, & Rudoff, *Unix Network Programming, Volume 1: The Sockets Networking API* (the definitive reference on socket programming)
- Kerrisk, *The Linux Programming Interface*, Chapters 4-5 (file I/O) and Chapters 56-61 (sockets)
- `man 2 open`, `man 2 read`, `man 2 write`, `man 2 socket`, `man 2 bind`, `man 2 listen`, `man 2 accept`, `man 2 connect` -- the Linux man pages are an excellent reference
- Beej's Guide to Network Programming: <https://beej.us/guide/bgnet/> -- a free, well-written introduction to socket programming in C

Module 8 Overview

In Module 8: Memory Hierarchy and Optimization, we explore how modern computer systems organize and access memory, and how understanding the hardware allows us to write dramatically faster code. We begin at the hardware level with random access memory (RAM) and storage technologies, then examine how the principle of locality allows systems to bridge the enormous speed gap between the CPU and main memory through caching. From there, we turn to optimization: how to measure performance, what the compiler can do for us, what it cannot, and the code-level techniques a programmer can apply to make programs run faster. The module concludes with cache-aware programming, which ties the hardware and software sides together.

This module spans eight lessons that build on each other. Understanding RAM and locality explains why caches work; understanding caches explains why certain code patterns are fast and others are slow; and understanding optimization techniques gives you the tools to exploit that knowledge in practice.

Learning Objectives

At the end of this learning module, you will be able to:

1. Describe the basic hardware technologies used in RAM, including the differences between SRAM and DRAM.
2. Explain the principle of locality (temporal and spatial) and identify locality in code examples.
3. Describe the organization of memory caches, including cache lines, sets, mapping schemes (direct-mapped, set-associative, fully associative), and write policies.
4. Explain when optimization matters and make informed decisions about whether performance gains justify the costs in code complexity and readability.
5. Use gcc optimization flags (`-O0` through `-O3`) effectively and understand what the compiler can and cannot optimize on its own.
6. Measure program performance using timing tools and profilers to identify bottlenecks before attempting to optimize.

7. Apply code-level optimization techniques such as reducing procedure calls, eliminating unnecessary memory accesses, and loop unrolling.
8. Write cache-friendly code by choosing data access patterns and data structure layouts that exploit spatial and temporal locality.

Learning Activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below.
2. Read Lessons 1 through 8 and complete the embedded practice problems in the lessons that include them.
3. Compile and experiment with the provided C code examples. Try varying compiler optimization levels and measuring the performance differences.
4. Take Quiz 8.
5. Review additional resources on the Module 8 Summary page.

Reading

- Bryant & O'Hallaron: Chapter 5 (Optimizing Program Performance)
- Bryant & O'Hallaron: Chapter 6 (The Memory Hierarchy)

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: Random Access Memory (RAM)

NOTE: This is a summary of chapter 6, sections 6.1 - 6.4 in the textbook.

Random Access Memory (RAM)

Random access memory (RAM) is part of the **main memory** system. The main memory system consists of many parts including memory caches and virtual memory. Memory caches sit between the CPU and RAM, holding copies of recently accessed RAM data for faster access. Virtual memory extends RAM by using disk storage: when RAM is full, less-used pages are swapped to disk, and in this sense RAM acts as a fast cache for the full virtual address space backed by disk.

Main memory is not permanent data storage. When the power is turned off on the computer; all data is lost. This is called **volatile** memory. We'll take a brief look at permanent storage (**non-volatile**) technology at the end of this lesson.

RAM Hardware

RAM memory is stored on microchips that contain a grid of cells. Each cell holds one bit of information. There are two types of RAM microchips:

- **static RAM (SRAM)** - A conventional cell uses six transistors. SRAM is used where low latency and modest capacity matter, including CPU caches, register files, and on-chip buffers. It is volatile but does not require periodic refresh while power is supplied.
- **dynamic RAM (DRAM)** - uses a capacitor and transistor to keep a state. It is much smaller than SRAM cells so it can fit more memory in the same space (higher density). However, it is not stable and needs constant refreshing of the cells to keep state.

More on SRAM

A conventional SRAM cell uses **6 transistors per bit** arranged in a bistable circuit. As long as power and operating conditions remain within specification, the cell retains its state

without periodic refresh. SRAM can provide very low latency, but actual access time depends on the array size, process technology, and where it sits in the memory hierarchy.

The downside is cost and density. Six transistors per bit means SRAM cells are physically large, so you cannot fit much SRAM on a chip. SRAM is also more expensive to manufacture per bit than DRAM. For these reasons, SRAM is used where speed matters most and capacity requirements are small: **CPU caches** (L1, L2, and L3).

More on DRAM

DRAM stores each bit as a charge on a tiny **capacitor**, paired with a single transistor that acts as an access switch. This is much simpler than SRAM's six-transistor design, which is why DRAM achieves much higher **density** (more bits per chip) and lower cost per bit.

The trade-off is that capacitors leak charge over time. The memory controller therefore issues periodic **refresh** operations so every row is refreshed within the device's specified retention interval (commonly tens of milliseconds, with commands spread across that interval). Refresh consumes power and temporarily occupies part or all of a DRAM rank.

DRAM random-access latency is typically tens of nanoseconds and is much higher than a small on-chip SRAM cache. DRAM is nevertheless used for **main memory** because it provides high capacity at a much lower cost per bit.

SRAM vs. DRAM Summary

SRAM is generally faster but more expensive and much larger per bit than DRAM. It is therefore used in relatively small on-chip memories such as caches and buffers, while DRAM supplies high-capacity main memory.

SRAM	DRAM	
Transistors per bit	6	1 (+ capacitor)
Access time	~1–2 ns	~50–100 ns
Needs refresh?	No periodic refresh	Yes (all rows within a specified retention interval)
Relative cost per bit	High	Low
Density	Low	High
Primary use	CPU caches (L1, L2, L3)	Main memory

DRAM Structure

Each DRAM chip is broken into many supercells. Each supercell is a rectangular array of DRAM cells. To access one cell, the memory address bit pattern is broken into parts that identify a particular row and column. The 2 dimensional arrangement makes it possible to use fewer connections into each cell. The DRAM chips themselves are packaged into memory modules called DIMM's. These are placed into slots on the memory board of the computer.

How DRAM Access Works

To read data from DRAM, the **memory controller** performs a two-step process:

1. **Row Access Strobe (RAS):** The controller sends a row address, which selects an entire row of the DRAM array. The contents of that row are copied into a **row buffer** (also called a sense amplifier array). This step is relatively slow because it involves charging the sense amplifiers.
2. **Column Access Strobe (CAS):** The controller then sends a column address, which selects specific columns from the row buffer and sends that data back to the controller.

An important consequence of this design: if the next access is to a different column in the **same row**, the row is already open in the buffer and only the CAS step is needed. This is called a **page hit** (or row buffer hit) and is significantly faster than accessing a different row. Accessing a different row requires closing the current row, precharging, and then issuing a new RAS — a much slower operation called a **page miss**.

This is one reason why **sequential memory access patterns are faster** than random patterns: sequential accesses tend to hit the same DRAM row repeatedly, while random accesses cause frequent row switches.

The Memory Bus

The CPU communicates with memory through a **memory bus**, which consists of three logical parts:

- **Address bus:** carries the memory address the CPU wants to read or write.
- **Data bus:** carries the actual data being transferred.
- **Control bus:** carries signals that coordinate the transaction (read/write commands, acknowledgments, etc.).

A typical read transaction works as follows: the CPU places an address on the address bus and signals a read request on the control bus. The memory controller receives the request, retrieves the data from DRAM (using the RAS/CAS process described above), and places it on the data bus. The CPU then reads the data from the bus. This entire round trip — across

the bus, through the controller, into the DRAM, and back — is what makes memory access slow relative to CPU speed.

DDR Generations

Modern DRAM modules use **DDR (Double Data Rate)** technology, which transfers data on both the rising and falling edges of the clock signal, effectively doubling the data rate compared to the original single-data-rate SDRAM. Each successive DDR generation roughly doubles bandwidth over the previous one by increasing the I/O clock rate and prefetch width:

- **DDR** (2000): ~1.6–3.2 GB/s bandwidth
- **DDR2** (2003): ~3.2–8.5 GB/s
- **DDR3** (2007): ~6.4–17 GB/s
- **DDR4** (2014): ~12.8–25.6 GB/s
- **DDR5** (2020): ~25.6–51.2 GB/s

You don't need to memorize these numbers, but the pattern is important: bandwidth has been growing with each generation. However, **latency** (measured in nanoseconds) has improved much more slowly. A DDR5 random access still takes roughly 50–100 ns, not dramatically different from DDR1. This is a key point for understanding performance.

The Processor-Memory Gap

Over the past several decades, CPU speeds have improved at a much faster rate than memory access speeds. Modern processors can execute billions of operations per second, but a single main memory access still takes on the order of 100 nanoseconds — enough time for the CPU to have executed hundreds of instructions. This growing disparity is known as the **processor-memory gap** (or "memory wall").

The processor-memory gap is the fundamental reason the **memory hierarchy** exists. Caches, prefetching, and other hardware techniques all exist to bridge this gap by keeping frequently used data close to the CPU. From a programmer's perspective, this gap is why **cache optimization matters so much**: a program that makes good use of the cache can run orders of magnitude faster than one that constantly waits for data from main memory. This motivates the entire rest of the module.

Non-Volatile Storage

Below main memory in the hierarchy sits **non-volatile** storage: devices that keep their contents when the power goes off. The two technologies you'll encounter are **flash-based solid state drives (SSDs)** and **magnetic disks** (hard drives). Flash memory stores each bit as charge trapped in a floating-gate transistor, so it needs no power to hold its value; a magnetic

disk stores bits as magnetized regions on spinning platters, read by a mechanical arm that must physically move into position.

Both are dramatically slower than DRAM. A random SSD read takes tens of microseconds — hundreds of times slower than DRAM's ~50–100 ns — while a magnetic disk access takes several milliseconds (waiting on the arm and the platter rotation), roughly 100,000 times slower than DRAM. Bandwidth follows the same ordering: a modern SSD sustains a few GB/s and a hard disk a few hundred MB/s, versus tens of GB/s for DDR memory.

This is why the memory hierarchy doesn't stop at RAM. Just as caches hold the most-used portion of RAM, RAM itself acts as a cache for data whose permanent home is on disk or SSD — this is exactly the virtual memory arrangement mentioned at the start of this lesson. The same locality principles that make caches work also make RAM effective as a cache for storage.

Lesson 2: Locality

Locality

Accessing main memory is slow compared with CPU execution and cache access; this does not mean DRAM latency gets worse with each hardware generation. But there are some things that can be done in hardware and software to mitigate this problem. In hardware, the use of **memory caches** are one of the ways to keep from accessing main memory. We'll talk about this in the next section.

In software, taking advantage of what is known as **locality** helps speed up the running of your programs. We'll talk about locality first in this section and then discuss how to use it in optimization later in this module.

What is Locality

Below is the **locality principle**. Basically it comes down to two facts about memory access patterns:

- **temporal locality:** A memory location that was recently accessed will most likely be accessed again soon.
- **spatial locality:** A memory location that was recently accessed will most likely have nearby memory locations accessed soon.

Common misconception: these are not the same thing, and they can show up independently. *Temporal* locality is about the same address being touched again ("you'll come back to this exact byte"). *Spatial* locality is about *nearby* addresses being touched ("you'll go to the next byte over"). A loop counter has high temporal locality (the same word is read each iteration) but low spatial locality (it's just one location). A linear scan through a 1 GB array has high spatial locality (next element is right next door) but low temporal locality (you don't revisit). Programs that have both are the easiest for the cache to speed up.

There are several ways computer systems can exploit locality. One way is by using memory caches; we'll discuss this in the following section. Another way is through writing software

that displays both spatial and temporal locality; we'll discuss this later when we talk about optimization.

Temporal Locality Examples

Temporal locality occurs when the same memory location is accessed repeatedly over a short period of time. The most common example is a variable that is read or written inside a loop:

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += a[i]; /* sum is accessed every iteration (good temporal
locality) */
}
```

Here, the variable `sum` is read and written on every iteration of the loop. Because it is accessed so frequently, the hardware will keep it in the cache (or even in a register), making each access very fast. The loop counter `i` also exhibits excellent temporal locality for the same reason.

Spatial Locality Examples

Spatial locality occurs when memory locations near each other are accessed in close succession. The most important example is **sequential array access**:

```
/* Good spatial locality: stride-1 access */
for (int i = 0; i < n; i++)
    sum += a[i];
```

Each access to `a[i]` is immediately followed by an access to `a[i+1]`, which is the next location in memory. When the hardware loads a cache line containing `a[i]`, it also loads several subsequent elements, so the next few iterations will be cache hits.

Contrast this with poor spatial locality:

```
/* Poor spatial locality: stride-M access (column traversal of row-
major array) */
int a[N][M]; /* N rows, M columns */
for (int i = 0; i < N; i++)
    sum += a[i][0]; /* jumps by an entire row (M ints) each time
*/
```

Here, each access jumps over an entire row of the 2D array. The elements accessed are far apart in memory, so loading a cache line for one access does not help with the next access.

Stride Patterns

The **stride** of an access pattern is the distance (in elements) between successive memory accesses. The most common stride patterns are:

- **Stride-1 (sequential):** Each access is to the next element in memory. This gives the best spatial locality. Examples include iterating over a 1D array or traversing a 2D array row by row.
- **Stride-k:** Each access skips $k-1$ elements. For example, accessing every other element of an array is stride-2. Accessing column 0 of a row-major 2D array with N columns is stride- N .

The general rule is: **the smaller the stride, the better the spatial locality**. Stride-1 access is ideal because every element loaded into a cache line gets used. Larger strides mean that most of the data brought into the cache goes unused before being evicted, wasting cache capacity and bandwidth.

Instruction Locality

Locality applies not just to data but also to **instructions**. When the CPU fetches instructions from memory, the same locality principles apply:

- **Temporal locality:** Loops execute the same set of instructions repeatedly. The instructions in the loop body are fetched over and over, so they stay in the instruction cache.
- **Spatial locality:** Instructions are stored sequentially in memory, so when the CPU fetches one instruction, the next instruction is likely in the same cache line.

This is why most CPUs have separate **instruction caches** and **data caches** (the L1 cache is typically split into L1i for instructions and L1d for data). The instruction stream has excellent locality almost by default, and a dedicated cache can exploit this without interference from data accesses.

Evaluating Locality in Code

When analyzing a program for locality, a practical approach is to look at the **innermost loops** and ask two questions:

1. **Does it reuse the same data?** Variables that are read and written on every iteration (like accumulators, loop counters, and frequently accessed pointers) exhibit temporal locality.

2. **Does it access memory sequentially?** Array accesses with stride-1 patterns exhibit spatial locality. Accesses that jump around in memory (large strides, pointer chasing through linked lists, random indexing) exhibit poor spatial locality.

Programs that have **both** good temporal and good spatial locality in their innermost loops tend to run much faster, because the cache can serve most of their memory requests without going to main memory.

Quantitative Example: Row-Major vs. Column-Major

Consider summing all elements of a 2D array. Here are two approaches:

```
/* Version 1: Row-major traversal (good locality) */
int sum = 0;
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        sum += a[i][j];

/* Version 2: Column-major traversal (poor locality) */
int sum = 0;
for (int j = 0; j < N; j++)
    for (int i = 0; i < N; i++)
        sum += a[i][j];
```

Both versions compute the same result, but their locality properties are very different. In Version 1, the inner loop varies `j`, which steps through the columns of a single row — this is stride-1 access with excellent spatial locality. In Version 2, the inner loop varies `i`, which steps through different rows at the same column — this is stride-N access with poor spatial locality.

For large N, Version 2 can be **5–20 times slower** than Version 1, even though they perform exactly the same number of additions. Cache behavior is a major reason; TLB misses, prefetching, memory bandwidth, and compiler transformations can also affect the measured difference. This example foreshadows the cache-aware programming techniques we will study in Lesson 8.

Lesson 3: Memory Caches

Common misconception — "the cache is some kind of software." The CPU caches we describe in this lesson are *hardware*, baked directly into the processor die alongside the cores. You don't allocate them, you don't write to them, you can't even directly observe them — the hardware moves data into and out of them transparently as your program executes regular load and store instructions. (When a Java/Python programmer says "I added a cache" they usually mean an in-memory dictionary they wrote themselves; that's a software cache and is unrelated to what we cover here.)

Memory Hierarchy and Memory Caches

To better exploit locality to speed up delivery of memory data, memory is designed in layers that form a hierarchy. Figure 6.21 in the textbook shows this hierarchy where levels at the top of the memory have faster access times and therefore are more desirable space to store values in. However, because of increased cost and decreased amount of space, these levels are limited in size and can't hold all the data from main memory needed to run a process. Still, they can help by storing a portion of memory that is currently being worked on. This assumes that the software being processed has good spatial and temporal locality.

Memory hierarchy (smaller, faster, more expensive at the top):

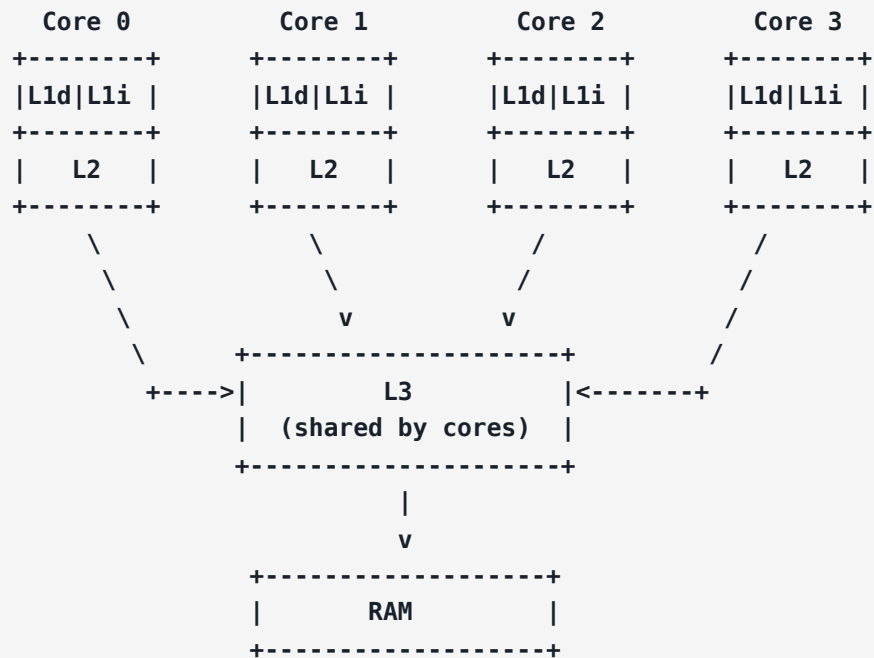
+-----+	~1 cycle, ~1 KB
Registers	
+-----+	
L1 cache	~4 cycles, 32-64 KB per core
+-----+	
L2 cache	~12 cycles, 256 KB - 2 MB per core
+-----+	
L3 cache	~40 cycles, 16-64 MB shared
+-----+	
Main RAM	~200 cycles, GB scale
+-----+	
SSD / disk	microseconds (SSD), ms (HDD)
+-----+	

Each level is substantially larger and slower than the one above it (the exact ratios vary by level). The CPU walks down this stack on every load/store: registers first, then L1, L2, L3, RAM.

From fastest to slowest, the levels are: registers, the L1, L2, and L3 caches, main RAM, and finally SSD or disk.

Because of the restraints between size and speed (the larger the cache, the slower the access times), memory caches have become hierarchical in nature over time. The L1 cache is small (typically 32–64 KB per core); it needs to be small in order to keep the access times short. The L2 cache is larger (typically 256 KB to ~2 MB per core on current Intel/AMD desktop chips) and holds more memory but has slower access times. There may also be an L3 cache that is much larger and shared across cores (commonly 16–64 MB on contemporary desktop CPUs, more on server-class chips).

Multi-core cache layout (typical 4-core x86 desktop):



L1 and L2 are private to each core; L3 is shared.

L1 itself is typically split into separate instruction (L1i) and data (L1d) caches so loading data and fetching instructions do not compete for the same cache port.

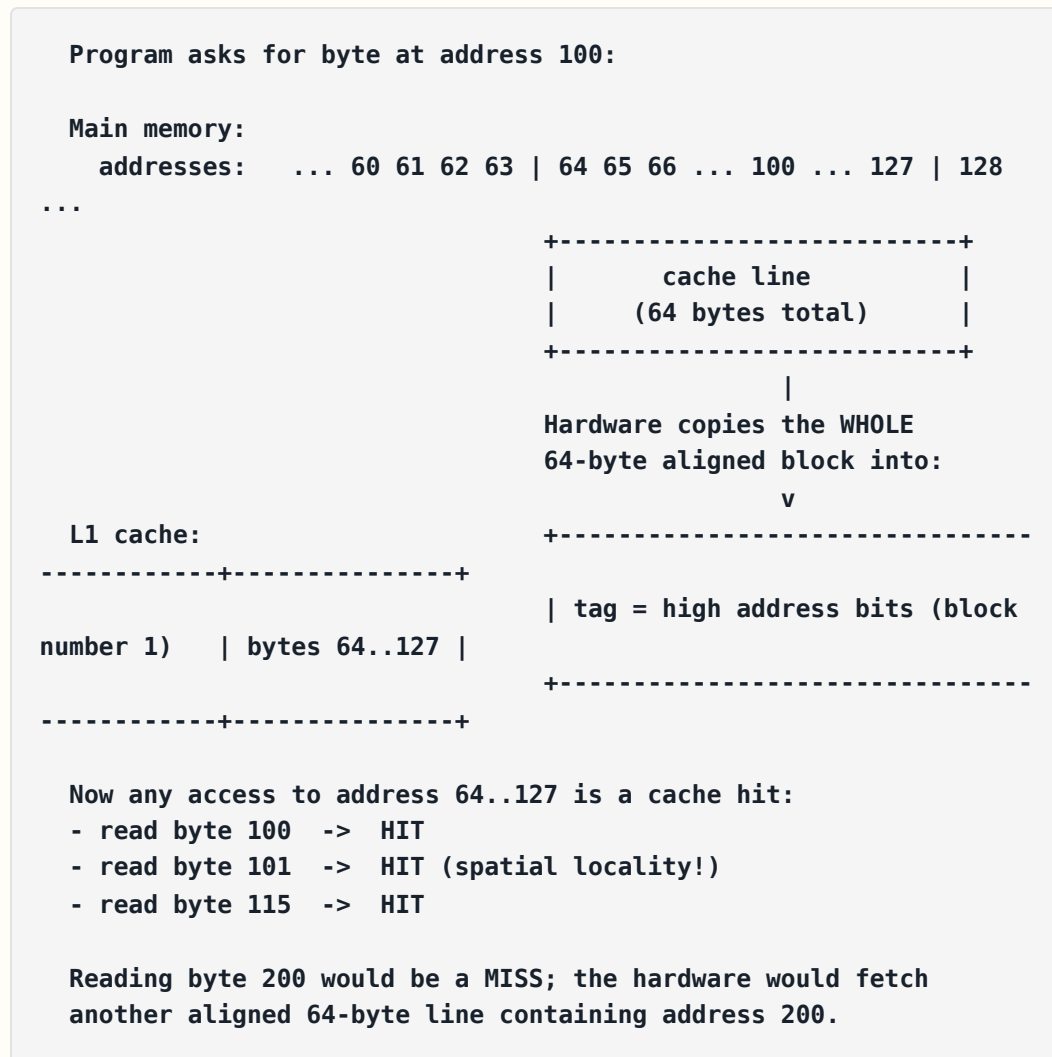
In many designs (called **inclusive** caches), each larger cache holds a copy of everything in the smaller cache above it. Other designs use **non-inclusive** or **exclusive** policies where the levels may hold different data. The general idea is that the most frequently accessed data ends up in L1 (the fastest cache), and less recently used data migrates to L2 or L3. The larger caches serve as a backstop for data that does not fit in the smaller, faster caches above them.

How Do Caches Work

Note that for the rest of this lesson we focus on a single core. In modern CPUs, the cache system is much more complex. Typically, each core will have its own L1 cache(s) and possibly L2 cache but the L3 cache will be shared between cores (see the multi-core layout above). Also, the L1 cache is typically two caches: one for data and one for instructions. This way loading data doesn't interfere with loading instructions.

When the CPU needs to get a value at a requested address, it looks in the highest cache (L1) and goes down to the lowest cache (L3). If the value is found in one of the caches, this is called a **hit**. A **miss** is when the value is not in any cache and needs to be gotten from RAM.

Here is a very simplified example of how caches work. When a process starts running, none of the process's memory is in any of the caches. At some moment, it needs to read a portion of memory, say 1 byte at address 100. The hardware goes to main memory and loads a **cache line** (a fixed-size block, typically 64 bytes on modern CPUs) containing that address into the caches. The cache line would cover, say, addresses 64 to 127. This line is loaded into both L1 and L2 (we are ignoring L3 for the moment). The hardware loads an entire cache line rather than just the one byte to take advantage of **spatial locality**: nearby bytes are likely to be needed soon. The next time the program accesses any address within that same cache line (e.g., address 101 or 115), it will be a cache hit — no need to go back to RAM.



In reality, caches are much larger than this and so it would be inefficient to just fill up the entire cache with one block of memory. Caches are broken into cache lines which are the smallest block of memory that can be read from or written to memory. On modern x86 CPUs, cache lines are 64 bytes. Historically, cache line sizes have ranged from 32 to 128 bytes on various architectures. The process's memory is broken up into line-size portions of blocks and

mapped to one or more cache lines. In the next section, we will talk about how memory is mapped to cache line(s).

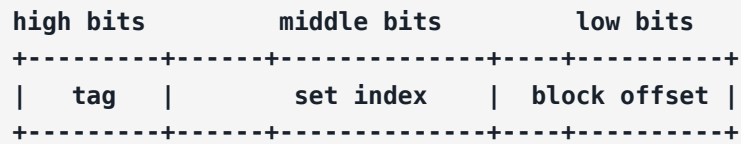
Cache Mapping

There are three types of mapping schemes:

- **Direct-Mapped:** Here more than one memory block is mapped to one and only one cache line. Usually this is done by using the middle bits of the first address of a memory block and finding the corresponding cache line **index**. Since more than one part of memory can be located in this line, there is also a corresponding **tag** value, typically the upper bits of the address, that is written into the cache line for the CPU to determine which block of memory is in the cache.

When the CPU looks for a value, it determines the index from the middle bits of the address, then it finds the cache line. It then compares the tag in the cache line to the tag from the address. If they match then this is a hit and the value is used. If the tags don't match, then it will **evict** the current block by overwriting it with the new memory block, updating the tag to the new block. A tag mismatch is a miss, but not always the same kind of miss: it could be a **compulsory miss** (the block has never been loaded before) or a **capacity miss** (the working set is simply too big for the cache). It is a **conflict miss** when the block was evicted because another block mapped to the same index, even though the rest of the cache still had room.

Address breakdown for a direct-mapped cache:



^^^^^^^^

compared to tag stored
in the cache line

^^^^^^^^^^^^^^^^

which byte within
the 64-byte line

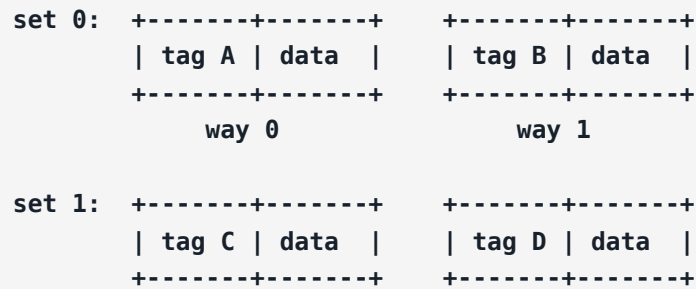
Direct-mapped cache (4 lines, for illustration only):

index	valid	tag	data (64 bytes)
00	1	0x4A	...
01	1	0x12	...
10	0	-	(empty)
11	1	0x4A	...

Two memory blocks with different tags but the **SAME** middle bits both map to the same row. Each new arrival kicks out the previous occupant. That's a conflict miss.

- **Set Associative Mapping:** Direct mapping causes a lot of conflict misses. To help avoid this problem, caches use set associative mapping. Here the cache is grouped into sets of cache lines. Each line can hold one chunk of memory that can be read and written to. Each set has a fixed number of lines. Set associative caches are called N-way associative caches where N is the number of lines in each set.
 - For example, instead of having one index map to one cache line, you can have one index map to 2 cache lines (2-way associative). This way if there is already a block in one cache line, the new block can be stored in the other cache line. This helps to reduce conflict misses. However, if all cache lines fill up, then the hardware has to determine which of the cache line's block of data to **evict** (overwrite with the new data). Hardware caches typically use **least recently used** (LRU) or, more commonly in practice, a cheaper approximation like **pseudo-LRU**, **FIFO**, or **random** replacement. (LFU — least frequently used — is more often seen in software caches such as page replacement and web caches; it is rare in hardware.)

2-way set-associative cache (2 sets, 2 ways per set):



An address hashes to a SET (using the middle bits) and can live in EITHER way within that set. On miss, the LRU occupant of the set is evicted.

An N-way set-associative cache with one set is the same as fully-associative; an N-way set-associative cache with one way is the same as direct-mapped.

- **Fully Associative Mapping:** This mapping scheme does away with mapping altogether. Each memory block can be put into any cache line. This is not used much because of the huge cost of searching the entire cache to find the correct block losing the main purpose of having a cache: quick access. **Typically, caches are N-way set associative.**

Fully associative cache (one set with all the ways):



Any incoming block can occupy any line. To check for a hit the hardware compares the address tag against EVERY tag in parallel. Great hit rates, but expensive in transistors -- this is why fully associative is used only for small structures (e.g., the TLB) or not at all.

Worked Example: Decoding an Address

Let's put the **tag**, **set index**, and **block offset** pieces together with real numbers. Suppose we have a **32 KB, 4-way set-associative** cache with **64-byte** cache lines, and 32-bit addresses. First, work out the geometry of the cache:

Total lines: 32768 bytes / 64 bytes per line = 512 lines
Sets: 512 lines / 4 lines per set = 128 sets

Block offset bits: 64 bytes per line = 2^6 -> 6 bits
Set index bits: 128 sets = 2^7 -> 7 bits
Tag bits: 32 - 7 - 6 -> 19 bits

Now decode the address 0x1234ABCD with this breakdown:

0x1234ABCD = 0001 0010 0011 0100 1010 1011 1100 1101

	19 tag bits		7 index bits		6 offset bits	
	0001001000110100101		0101111		001101	

The low 6 bits (001101 = 13) select the byte within the 64-byte line. The next 7 bits (0101111 = 47) select the set. The remaining 19 high bits form the tag, 0x91A5. So to look up this address, the hardware goes to set 47, compares 0x91A5 against the tags of the 4 lines in that set, and if one matches (and is valid), it's a hit — the requested value is at byte 13 of that line.

Three Kinds of Cache Misses

Cache misses are traditionally categorized into **three Cs**:

- **Compulsory miss** (cold miss): the very first reference to a block. The block has never been in the cache before, so it must be fetched. Even an infinitely large cache cannot avoid these.
- **Capacity miss**: the program's working set is larger than the cache, so blocks that were evicted are later re-referenced. Increasing cache size reduces these.
- **Conflict miss**: the cache has room overall, but the specific set the block maps to is full because too many active blocks share the same index. Increasing associativity reduces these (a fully-associative cache has none).

Cache Writes

The above mapping works for both reading and writing to memory. For reading, this isn't too much of a problem but for writing, we need to decide if and when we should update main memory. Since we are trying to avoid accessing main memory, we don't want to do a write every time a value is updated (this is called **write through**). For example, what if the value is written to over and over again? We would be wasting time writing a value that would be written over again soon. On the other hand, we may need to read the updated value later in the running of the program so we do need to update memory at some point.

One way to avoid excessive writes to memory is to wait until a line is evicted to update main memory. This is called a **write back**. This requires being able to keep track of whether a value has been updated by using a **dirty bit**. If this is set to 1, the value will be written back to main memory.

There's also the question of what happens on a **write miss** (when the address being written to is not in the cache). With **write-allocate** (also called "fetch on write"), the cache first loads the block from the next level into the cache, then performs the write on the cached copy. This is typically paired with write-back. With **no-write-allocate** (also called "write around"), the write goes directly to the next level of memory without loading the block into the cache. This is typically paired with write-through.

More Resources on Memory Cache

Memory caches are a highly complex subject. I'm expecting students to know just the basic concepts and definitions of what's in bold-face above. The textbook has some examples of cache behavior too but if you want to you can look at these videos on YouTube. Except for the last one on writes, they go through a 'toy example' of accessing the cache so you can see basically how it's all put together:

Here's an example of a direct cache:

[YouTube: Cache Access Example \(Part 1\)](#)

Here's an example of a set-associative cache:

[YouTube: Cache Access Example \(Part 2\)](#)

Here is an explanation of writes (it's a slightly better explanation than what is in the textbook):

[YouTube: Writing to Caches](#)

Lesson 4: Optimization - Why and When?

NOTE: This is a summary of chapter 5, sections 5.1 - 5.6, 5.8, 5.13-5.14 and chapter 6, sections 6.6 - 6.7 in the textbook.

Introduction

The main focus of this summary is on how to optimize your code to run faster. There are other optimizations that are also important, such as memory or space optimization, but here we will concentrate on time optimization or performance.

Why Optimize and When?

We want to write programs that are readable and work correctly but we also need to write programs with good performance as well. In this section we are going to concentrate on getting programs to run faster; this is called optimization.

Some optimizing techniques cause a decrease in readability. An example is loop unrolling which causes the code to become longer and more complex. So one should decide if optimization is worth it before deciding to do it. It may not matter to the end user if a program runs 2 seconds longer. On the other hand, if the program is running 2 hours longer then optimization, however ugly it looks, will be worth it. And a programmer should never compromise correctness of the program for performance. A program is useless if it doesn't give the right output no matter how much bragging a programmer might do about how much faster it runs.

Modern compilers do a good job in optimizing code. However, a programmer can accidentally limit the compiler's ability to optimize code. A programmer can help out by writing "compiler friendly" code that helps the compiler to optimize code better.

Other methods of helping make programs run faster exist. Running programs in parallel on multi-core or multiple systems is one method. However, even if we are using several computers or cores to do the optimizing it doesn't hurt to make sure each individual program is running at optimal speed.

Using performance tools such as profilers helps the optimization process. We won't be discussing profilers in this course much but in this module's lab you will get to use a simple profiler available on the PCC Linux system called **gprof**. I will be covering how to use this program during this module's lecture.

We'll look at various techniques that can be used to write optimized code. These techniques may not always work. Often it's a matter of trial and error to find a technique that works.

Amdahl's Law

Amdahl's Law gives us a formula for the maximum speedup we can achieve by optimizing part of a program. Suppose a fraction α of the program's runtime is spent in some component, and we speed up that component by a factor of k . The overall speedup is:

$$\text{Speedup} = 1 / ((1 - \alpha) + \alpha/k)$$

For example, suppose 80% of a program's runtime is spent in a single function, and we make that function 4x faster. Then:

$$\begin{aligned}\text{Speedup} &= 1 / ((1 - 0.8) + 0.8/4) \\ &= 1 / (0.2 + 0.2) \\ &= 1 / 0.4 \\ &= 2.5x \text{ overall speedup}\end{aligned}$$

Even though we made the function 4x faster, the overall program is only 2.5x faster because the remaining 20% of the runtime is unchanged.

The key lesson from Amdahl's Law: **optimizing a function that accounts for 5% of runtime can yield at most about a 1.05x overall speedup (roughly 5.26% faster), even if that function becomes infinitely fast.** This is why profiling (measuring where the time is actually spent) is so important — you need to know which parts of your program dominate the runtime before deciding where to focus your optimization effort.

The Optimization Workflow

Experienced programmers follow a disciplined process when optimizing. Here is the recommended workflow:

1. **Write correct code first.** Get the program working and producing the right output. Do not think about performance yet.
2. **Profile to find the hot spots.** Use a profiler (like `gprof` or `perf`) or the `time` command to measure where the program spends its time.

3. **Optimize the hot spots.** Apply optimization techniques to the functions and loops that consume the most runtime.
4. **Measure again to verify.** After making changes, measure the performance again to confirm that your optimization actually helped. Sometimes a change that seems like it should be faster is not, due to compiler behavior or hardware effects.
5. **Repeat if needed.** If the program is still not fast enough, go back to step 2 and profile again. The hot spots may have shifted after your first round of optimization.

The most important rule is: **never optimize without measuring first.** Guessing where the bottleneck is almost always leads to wasted effort on code that does not matter.

What "Hot Spots" Look Like

In practice, a small fraction of the code accounts for the vast majority of the runtime. This is sometimes called the **90/10 rule**: roughly 90% of the execution time is spent in 10% of the code. The hot spots are usually:

- Inner loops that process large data sets (e.g., iterating over every element of a large array)
- Functions that are called millions of times (e.g., comparison functions in a sorting algorithm)
- Sections of code with poor cache behavior (e.g., traversing large data structures with random access patterns)

These are where optimization effort should be focused. Code that runs once at startup, or a function called only a handful of times, is almost never worth optimizing.

Types of Optimization

There are several levels at which you can optimize a program, roughly ordered from highest impact to lowest:

1. **Algorithmic optimization:** Choosing a better algorithm — for example, replacing an $O(n^2)$ sort with an $O(n \log n)$ sort. This typically gives the biggest wins and should always be considered first. No amount of low-level tuning can compensate for a fundamentally inefficient algorithm.
2. **Compiler optimization:** Letting `gcc -O2` (or `-O3`) do the work. The compiler can perform many transformations automatically: register allocation, instruction scheduling, function inlining, and more. This is essentially "free" performance — you just turn on the flag. We cover this in Lesson 5.
3. **Code-level optimization:** Reducing unnecessary work in loops — eliminating redundant function calls, using local variables instead of repeated memory references, loop

unrolling. We cover these techniques in Lesson 7.

4. **Memory-level optimization:** Writing code with cache-friendly access patterns — stride-1 array traversals, loop tiling, choosing the right data layout. We cover this in Lesson 8.

When NOT to Optimize

Optimization is not always the right choice. Here are some situations where you should think twice:

- **The program already runs fast enough.** If the user is satisfied with the performance, there is no reason to spend time making it faster. Engineering time is valuable.
- **It's a one-time script.** If you will run the program once and never again, spending an hour optimizing code to save 30 seconds of runtime is a poor trade-off.
- **Readability would suffer greatly.** If an optimization makes the code much harder to understand and maintain, the long-term cost may outweigh the short-term speed gain.
- **Correctness is at risk.** Never sacrifice correctness for performance. A fast program that produces wrong results is worse than useless.

The famous quote, "premature optimization is the root of all evil" (attributed to Donald Knuth), is often misunderstood. It does not mean you should never optimize. It means you should not optimize **prematurely** — that is, before you know where the bottleneck is and before you have measured the impact. Optimization guided by measurement and focused on actual hot spots is not premature; it is good engineering.

Lesson 5: gcc Optimization Levels and Limitations

gcc Optimization Levels and Limitations

Probably a good place to start in optimizing your program is to make sure you use the right level of optimization when compiling with gcc. For looking at assembly code, we've been having you reduce optimization to a minimum in gcc by using the switch `-Og` which also makes the code easier to run in a debugger (gdb). This is so that you can see assembly code that follows closely with your C code.

In practice you want to use the highest level of optimization that is appropriate for your project. The standard in the industry for gcc optimization level is `-O2`. Why not `-O3`? While `-O3` does not introduce bugs on its own, it optimizes more aggressively in ways that can expose **latent bugs** in code that has undefined behavior (such as signed integer overflow or strict aliasing violations). Code that appeared to work at `-O0` may break at `-O3` because the optimizer is free to assume undefined behavior never happens. Additionally, `-O3` tends to increase the size of the executable (due to aggressive inlining and loop unrolling), which can hurt instruction cache performance. The default for compiling in gcc is `-O0` which disables optimization (the compiler still performs basic code generation, but does not attempt to optimize). This is usually a good one for doing development but you should use at least `-O1` or `-O2` in a production build.

The Standard `gcc` Optimization Levels at a Glance

Flag	What it does
<code>-O0</code>	No optimization. Default. Best for stepping through code in a debugger; the generated assembly closely follows the C source.
<code>-Og</code>	"Optimize for debugging." Enables optimizations that don't interfere with the debugger; recommended over <code>-O0</code> when you want both decent code and a debugging experience.
<code>-O1</code>	Basic optimizations: dead-code elimination, simple inlining, constant folding. Modest compile-time cost.
<code>-O2</code>	Most production-quality optimizations: stronger inlining, instruction scheduling, common-subexpression elimination, loop optimizations that don't blow up code size. The usual default for release builds.
<code>-O3</code>	Adds aggressive optimizations on top of <code>-O2</code> : more inlining, a more permissive vectorization cost model, and aggressive loop transformations. GCC 12 and later already enable vectorization at <code>-O2</code> , with a more conservative cost model. Can grow code size and can expose latent UB; benchmark before assuming it's faster than <code>-O2</code> .
<code>-Os</code>	Optimize for <i>size</i> : enables most of <code>-O2</code> but disables transformations that grow code. Good for embedded systems, instruction-cache-sensitive code, or anywhere binary size matters.
<code>-Ofast</code>	<code>-O3</code> plus optimizations that <i>break strict standards compliance</i> — most importantly

Flag	What it does
	-ffast-math , which lets the compiler reassociate floating-point operations and ignore IEEE 754 corner cases (NaN/Inf, signed zero, exact rounding). Can be a big win for numerical code, but only if you have explicitly verified that your program tolerates non-IEEE behavior.

gcc Optimization Blockers

When compiling code, gcc takes a conservative approach to optimizing. One example given in the textbook is the code:

```
void twiddle1(long *xp, long *yp)
{
    *xp += *yp;
    *xp += *yp;
}

void twiddle2(long *xp, long *yp)
{
    *xp += 2*(*yp);
}
```

the function `twiddle1()` would be a good candidate for optimization to `twiddle2()` (adding the value `*yp` twice is the same as multiplying by 2). But `gcc` compiles this to two adds. Why? because it is possible for `xp` and `yp` to be pointing to the same memory space. This will make the second add dependent on the outcome of the first add's results.

For example, let's say that `xp` and `yp` have the same address. At this address, the initial value is 3. For `twiddle1()`, `xp` after the first line is 6 and after the second line is 12 (note that the first line replaces `xp` with 6; $6+6 = 12$). For `twiddle2()`, `xp` is 9 ($3 + 2*3 = 9$). This effect is called **memory aliasing** where two pointers can possibly point to the same memory location.

It's possible that `xp` and `yp` will never point to the same memory space but `gcc` doesn't know this at compile time. So it plays it safe and doesn't try to optimize the `twiddle1()` to match `twiddle2()`. Note that if `xp` and `yp` are not pointers, `gcc` will be able to optimize to one shift and then one add. So there's a slight performance downside in using pointers as parameters in this case but this may be needed to achieve pass by reference.

Another such blocker is function calls. This example is from the textbook:

```
long f();

long func1()
{
    return f() + f() + f() + f();
}

long func2()
{
    return 4*f();
}
```

Here `f()` is defined in another file. We would probably guess that `gcc` will most likely compile `func1()` to something like `func2()`. But it doesn't; it does four calls to `f()`. This again is done because it is possible for the return value between calls to `f()` to be different each time.

For example, if `f()` is defined as:

```
long counter = 0;

long f() {
    return counter++;
}
```

Then `func1()` is going to have a different result than `func2()`. `gcc` tends to be very conservative about making any guesses about what a function will return if the function is outside the file that is currently being compiled. Remember that the compilation step is done on each individual file without referencing any other files. This may also prevent a function from being inlined if the function is outside of the file being compiled.

Lesson 6: How to Measure Performance

How to Measure Performance

In order to understand how well our optimization is working, we need to be able to measure how fast our programs are running. The textbook defines something called cycles per element (CPE) which it then uses for its optimization example. This is fine but not necessarily available outside of the book's code.

Most optimization tools in the industry (benchmarks, profilers, etc) use **CPU time**. CPU time is the amount of time in seconds a CPU was processing this particular program (as opposed to other programs that were also using the CPU). This is more accurate than straight clock time as it doesn't measure the times that the program was inactive.

The time Command

In this module's lab, you can use the command `time` at the terminal to get CPU time. For example, if we want to know how much CPU time it takes to run a matrix multiplication program, we type at the command line:

```
$ time ./matmul 1000
real    0m2.341s
user    0m2.312s
sys     0m0.024s
```

Each line of the output means something specific:

- **real** — The total **wall-clock time**: how long the program took from start to finish, including time spent waiting for I/O and time when other processes were using the CPU.
- **user** — The **CPU time** spent executing your program's own code (user mode). This is the time the CPU was actually doing work for your process.
- **sys** — The **CPU time** spent in the kernel on behalf of your program (system mode) — things like file I/O, memory allocation, and other operating system services.

The sum of `user` + `sys` gives the total CPU time. For **CPU-bound programs** (programs that spend most of their time computing), you will see `user` $\approx$ `real`, because the program is actively using the CPU almost the entire time. For **I/O-bound programs** (programs that spend a lot of time waiting for disk or network), you will see `real` $\gg$ `user`, because the program is idle while waiting for data.

Note that `real` can also be **less** than `user` + `sys` if the program uses multiple CPU cores, since wall-clock time measures elapsed time but CPU time is summed across all cores.

The `time` command is very useful for getting an overall feel for how fast your program is running, but its downside is that it doesn't say **where** in the code any slowdowns are. For that, we need a profiler.

Profiling with gprof

A **profiler** is a tool that measures how much time is spent in each function of your program. The profiler we will use is `gprof`, which is available on the PCC Linux system. Here is the workflow:

```
gcc -pg -O2 -o program program.c    # compile with profiling
enabled (-pg flag)
./program                            # run the program (generates
gmon.out)
gprof program gmon.out               # print the profile report
```

The `-pg` flag tells `gcc` to insert profiling instrumentation into the compiled code. When you run the program, it writes profiling data to a file called `gmon.out`. Then `gprof` reads that file and produces a human-readable report.

Reading a Flat Profile

The most useful part of `gprof`'s output is the **flat profile**, which lists each function and how much time was spent in it. Here is an example of what it might look like:

%	cumulative	self		self	total	
time	seconds	seconds	calls	ms/call	ms/call	name
78.23	1.83	1.83	1	1830.00	1830.00	
multiply_matrices						
12.41	2.12	0.29	100000	0.00	0.00	
compute_element						
5.13	2.24	0.12	1	120.00	120.00	
initialize_matrix						
2.14	2.29	0.05	1	50.00	50.00	read_input
2.09	2.34	0.05	1	50.00	50.00	print_result

The columns you should focus on are:

- **% time:** The percentage of total runtime spent in this function. This is the most important column.
- **self seconds:** The actual number of seconds spent in this function (not counting functions it calls).
- **calls:** How many times this function was called.

In this example, `multiply_matrices` takes **78%** of the runtime while `print_result` takes only **2%**. This tells you exactly where to focus your optimization effort. By Amdahl's Law (from Lesson 4), even if you made `print_result` infinitely fast, the overall speedup would be negligible. But making `multiply_matrices` even 2x faster would give a significant overall improvement.

CPE: Cycles Per Element

The textbook uses a metric called **CPE (Cycles Per Element)** to measure the efficiency of code that processes arrays or vectors. CPE is defined as:

$$\text{CPE} = (\text{total CPU cycles}) / (\text{number of elements processed})$$

A lower CPE means better performance. The advantage of CPE as a metric is that it is **independent of the array size**: whether you process 1,000 elements or 1,000,000 elements, a well-optimized function should have roughly the same CPE. This makes it useful for comparing different implementations of the same algorithm without worrying about input size.

For example, if a function takes 4,000 cycles to process an array of 1,000 integers, its CPE is 4.0. If you optimize the function to take 2,000 cycles for the same array, the CPE drops to 2.0 — a 2x improvement. The textbook's optimization chapters use CPE extensively to track progress through successive improvements to a running example.

Benchmarking Tips

Getting reliable performance measurements requires some care. Here are some practical tips:

- **Run the program multiple times and take the minimum.** The minimum runtime is closest to the program's true performance. Higher values include noise from OS scheduling, other processes, and interrupts.
- **Use large enough inputs.** The runtime should be at least a few seconds. Very short runtimes are dominated by startup overhead and measurement noise.
- **Be aware of cold caches.** The first run of a program (or the first run after changing the input) may be slower because the caches are empty. If you are comparing two implementations, run each one a few times to "warm up" the caches before recording the measurement.
- **Disable CPU frequency scaling if possible.** Modern CPUs can change their clock speed dynamically (turbo boost, power saving modes). This can cause inconsistent measurements. On Linux, you can set the CPU governor to "performance" mode if you have the right permissions.
- **Minimize background activity.** Close other programs and avoid running benchmarks while the system is under load from other tasks.

Other Useful Tools

Beyond `time` and `gprof`, there are other performance tools worth knowing about:

- **`perf stat`** — A Linux tool that gives you access to hardware performance counters. It can report the number of cache misses, branch mispredictions, instructions per cycle, and other low-level metrics. Very useful for understanding **why** a program is slow, not just where.
- **`valgrind --tool=callgrind`** — A tool that simulates a cache and records detailed information about function calls and cache behavior. The output can be visualized with **`kcachegrind`**, which shows call graphs and per-line cache miss counts.
- **`valgrind --tool=cachegrind`** — Similar to callgrind but focused specifically on cache simulation. It reports the number of cache hits and misses for each line of source code, which is invaluable when optimizing cache behavior (as discussed in Lesson 8).

You do not need to master all of these tools for this course, but knowing they exist is valuable. In the lab, we will focus on `time` and `gprof`.

Lesson 7: How to Optimize Code

How to Optimize Code

The textbook works through a multi-stage optimization example for the next several sections. I think I'll leave the full example to the textbook. Remember to skip 5.7 and 5.9 through 5.12.

Before we get to the list of techniques, here is the unifying frame — every code-level optimization technique below ultimately does one of two things:

1. **Reduce the work the CPU has to do** (eliminate redundant function calls, fuse operations, pull invariants out of loops, replace expensive operations with cheap ones).
2. **Expose parallelism the CPU could already exploit** (loop unrolling with multiple accumulators lets the CPU's pipelined ALUs work on independent additions in parallel; cache-friendly access patterns let the memory system keep the pipeline fed).

If you ever forget the specifics of a technique, ask: which of those two is it? That's almost always enough to remind you why it works.

Below are the optimization techniques that you should know about for the lab or final exam (optimization will be on the final):

1. **Eliminate Function Calls:** Eliminate any function call inside a loop that will always return the same value each time through the loop. Also don't put these calls in the conditional and update parts of the for loop's header as well. There is no reason to keep calling the function every time through the loop if it is guaranteed to give the same value.

A good example that the textbook talks about is `strlen()`. It's common to get the length of a string as an upper bound for stepping through a string's characters. Many programmers tend to put the call to `strlen()` inside the for loop's conditional:

```
for(int i=0; i < strlen(str); i++)  
    ....
```

but the loop's condition is checked $N+1$ times (where N is the size of the string), so `strlen()` ends up being called $N+1$ times only to return the same value. This is N too many

times. A better idea is to call it above the loop and put the value inside a local variable:

```
int size = strlen(str);
for(int i=0; i < size; i++)
....
```

This way there is only one function call and the value is probably going to be stored in a register for faster access. In fact, the `strlen()` function itself goes through the string to find the null value on the end so it is already doing N iterations. Putting it inside the for loop's conditional, it will now do $N*N$ or N^2 iterations which is very costly.

2. **Reducing Function Calls:** Function calls have non-trivial overhead, so whenever gcc can see the function's body and prove it is safe, it will 'inline' the function (paste the body in at the call site). The main thing that *blocks* inlining is gcc not having access to the function body — for example, the callee lives in a different translation unit and you didn't enable link-time optimization (`-flto`), or the call goes through a function pointer or a recursive function. Inlining also runs into limits set by the compiler's heuristics (large bodies, very high call counts).

So think about whether a function call is necessary inside the loop if the code itself can just be written into the loop. An example from the textbook is the function `get_vec_element()`. This function is just basically stepping through the array one element at a time; the code for this function can just be copied into the loop for faster performance in accessing array elements (no function call mechanisms to set up and break down).

Deep, non-tail-recursive functions can be slow for the same reason — lots of nested calls and stack-frame setup. However, recursion isn't always slow: modern compilers often perform *tail-call optimization* that turns a tail-recursive call into a plain jump (no extra frame, no overhead), and many algorithms (tree traversal, divide-and-conquer like quicksort/mergesort) are clearer and not measurably slower as recursion. The lesson is "prefer iteration when the recursion is deep and non-tail," not "all recursion is bad."

Of course, you can't eliminate all function calls, but do look at function calls inside hot loops as good candidates for optimization.

3. **Eliminate Unneeded Memory References:** Try to avoid directly reading from and writing to arrays and pointers. The compiler usually cannot safely keep them in a register (because of possible aliasing — see the twiddle example in Lesson 5), so each access goes through the memory system — at best a cache hit, at worst main memory. It's best to use a local variable when possible. Local variables end up being stored in registers rather

than memory references. So look at how `combine4()` function (figure 5.10) used the variable `acc` to avoid a read and write to memory. It can't avoid the read to `data[i]` entirely but it did cut down the number of memory references from 3 to 1. `acc` is an example of an `accumulator` which is very useful in optimizing code.

4. **Loop Unrolling:** This is an interesting technique that unfortunately makes code a bit unreadable and may not always work but it's worth trying as it can boost performance under some circumstances. Sometimes `gcc` will use this method if it feels it's safe to do so.

Loop unrolling involves doing two or more iterations at one time before going back to the top of the loop. For example:

```
for(int i=0; i < size; i++)
{
    sum += array[i];
}

can be transformed into

int i;
for(i=0; i < size-4; i += 5)
{
    sum += array[i];
    sum += array[i+1];
    sum += array[i+2];
    sum += array[i+3];
    sum += array[i+4];
}

for(; i < size; i++)
{
    sum += array[i];
}
```

Note that the last loop is to pick up any leftover array values at the end of the array. Also, note that we need to be careful not to increment past the array size. So basically the first loop stops short of the array size and the rest is picked up in the second loop.

This is an example of a "5 x 1" loop. The first number represents the number of iterations being done inside the for loop and the second number is the number of accumulators you are using inside the loop. Accumulators are local variables set up to hold the accumulation of results. Using more than one helps with performance as well.

You can use any number of iterations but there usually is a point where the unrolling stops being effective. How many to use is a matter of trial and error.

Why does this work? It's not really about the jumps — modern CPUs predict a loop's backward branch almost perfectly, so the jump itself is nearly free. The real win is reduced **loop overhead**: the counter increment and the conditional test now happen once per five elements instead of once per element, so a larger fraction of the executed instructions are doing useful work. The bigger payoff, though, comes from combining unrolling with multiple accumulators, which exposes instruction-level parallelism (see the 5 x 2 version next).

You can also help boost things by using two or more accumulators. (this is a 5 x 2 loop):

```
int sum1 = 0;
int sum2 = 0;
int i;
for(i=0; i < size-4; i += 5)
{
    sum1 += array[i];
    sum2 += array[i+1];
    sum1 += array[i+2];
    sum2 += array[i+3];
    sum1 += array[i+4];
}

for(; i < size; i++)
{
    sum1 += array[i];
}
int sum = sum1 + sum2;
```

This speeds up code by making each sum independent from each other creating an opportunity for the CPU to use parallelization (or pipelining) when doing the arithmetic on each line. The trick is to alternate the use of each accumulator.

5. Write Cache Friendly Code: (from Section 6.5 and 6.6)

Earlier in this module (Lesson 3) we talked about how memory caches work and how they use locality to speed up accessing memory. The principle: access memory with a **stride-1** pattern (step through arrays in the same order they are laid out in memory) and reuse values while they are still in the cache (temporal locality). Lesson 8 gives this technique the full treatment, including the row-major vs. column-major matrix examples, so see that lesson for the details.

Lesson 8: Cache-Aware Programming

NOTE: This lesson builds on the memory hierarchy concepts from Chapter 6, Sections 6.6–6.7 in the textbook, and extends the optimization techniques covered in Lessons 4–7.

Introduction: Why Cache Behavior Matters

In earlier lessons we looked at code-level optimizations such as reducing function calls, eliminating redundant memory references, and loop unrolling. These are important techniques, but they operate at the level of individual instructions. In this lesson, we focus on a different and often more impactful factor: **how your program accesses memory**.

Recall from our study of the memory hierarchy that modern systems have multiple levels of cache between the CPU and main memory. A typical L1 cache access takes about 1–4 CPU cycles, while a main memory access can take 100 or more cycles. That is roughly a **25–100x difference** in access time. When a program repeatedly misses the cache, the CPU stalls waiting for data, and performance can degrade by orders of magnitude. The good news is that by understanding how caches work, we can write code that keeps the data we need close to the processor.

The two key principles that make caches effective are:

- **Temporal locality:** If a program accesses a memory location, it is likely to access the same location again soon. Caches exploit this by keeping recently accessed data.
- **Spatial locality:** If a program accesses a memory location, it is likely to access nearby locations soon. Caches exploit this by loading entire *cache lines* (typically 64 bytes) at a time.

Cache-aware programming means writing code that maximizes both forms of locality so that the cache can do its job effectively.

Row-Major vs. Column-Major Array Traversal

C stores two-dimensional arrays in **row-major order**: the elements of each row are stored contiguously in memory. This means that `a[0][0]` is followed immediately by `a[0][1]`, then `a[0][2]`, and so on. The first element of the next row, `a[1][0]`, comes after the last element of the first row.

This storage layout has a direct consequence for cache performance. When we iterate over the array by varying the column index in the inner loop (row-by-row), we access memory sequentially. Each cache line loaded from memory provides multiple useful values, and we use all of them before moving on. This gives us excellent spatial locality.

When we instead vary the row index in the inner loop (column-by-column), we jump across entire rows with each access. If the rows are large, each access may land in a different cache line, and by the time we come back to a given row, the cache line we loaded earlier may have been evicted. This wastes the cache bandwidth and causes many more cache misses.

Example: Summing a 2D Array

Consider two ways of summing all elements of an $N \times N$ array:

Row-major traversal (cache-friendly):

```
/* Good: varies column index (j) in inner loop.
 * Accesses memory sequentially. */
int sum_row_major(int a[N][N]) {
    int sum = 0;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Column-major traversal (cache-unfriendly):

```

/* Bad: varies row index (i) in inner loop.
 * Jumps across rows, causing cache misses. */
int sum_col_major(int a[N][N]) {
    int sum = 0;
    for (int j = 0; j < N; j++)
        for (int i = 0; i < N; i++)
            sum += a[i][j];
    return sum;
}

```

Both functions compute exactly the same result. But for large N (say, $N = 4096$), the row-major version can run **5–20 times faster** than the column-major version, depending on the system. The difference is entirely due to cache behavior. The row-major version has a cache miss roughly once per cache line (every 16 integers for a 64-byte cache line with 4-byte ints), while the column-major version may have a cache miss on nearly every access.

Try this on your own machine. Save the two functions above into a file

`traversal.c` together with a tiny `main` that allocates an $N \times N$ `int` array, fills it, and times each version. (Use `clock_gettime(CLOCK_MONOTONIC, &ts)` for wall time, or just `time ./a.out`.) Wall time is fine here because the loop is CPU-bound and you'll run it on an otherwise idle machine; see Lesson 6 for the wall-time vs. CPU-time distinction. Then compile and run:

```

gcc -O2 -o traversal traversal.c
./traversal      # prints both timings

# repeat with N = 1024, 2048, 4096, 8192

```

Report:

1. What's the row-major / column-major ratio at each N ?
2. At what N does the gap appear? (Hint: roughly when the inner-loop working set stops fitting in L1.)
3. Does the gap shrink, grow, or stay constant as N keeps increasing?

You should see numbers in the 5–20 $\times$ range. Don't take the textbook's word for it — see it on your machine. The qualitative shape (row-major much faster than column-major, with the gap growing as N exceeds cache capacity) is universal across hardware, even if the exact ratio varies.

Loop Tiling (Blocking)

Sometimes the natural algorithm for a computation does not have good locality, even when we traverse arrays in row-major order. The classic example is matrix multiplication. In the standard three-nested-loop algorithm, the inner loop accesses one row of the first matrix and one column of the second matrix. The row access has good spatial locality, but the column access does not. For large matrices, the column of the second matrix will not fit in the cache, causing repeated cache misses.

Loop tiling (also called **blocking**) is the technique of breaking the computation into smaller blocks that fit in the cache. Instead of processing entire rows and columns, we process small sub-matrices (tiles) of size $B \times B$, where B is chosen so that the working set of a single tile operation fits comfortably in the cache.

Example: Matrix Multiplication Without and With Blocking

Standard matrix multiply (no blocking):

```
/* Standard ijk matrix multiply.
 * The inner loop scans a column of b, which has poor
 * spatial locality for large matrices. */
void matmul(int n, double a[n][n], double b[n][n], double c[n][n])
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++)
                sum += a[i][k] * b[k][j];
            c[i][j] = sum;
        }
}
```

Blocked matrix multiply:

```

/* Blocked matrix multiply.
 * Process B x B sub-blocks so that the working data
 * fits in the cache. Typical good values for B are
 * 16, 32, or 64 depending on cache size. */
#define B 32

void matmul_blocked(int n, double a[n][n], double b[n][n], double
c[n][n]) {
    for (int ii = 0; ii < n; ii += B)
        for (int jj = 0; jj < n; jj += B)
            for (int kk = 0; kk < n; kk += B)
                /* Multiply the B x B sub-blocks */
                for (int i = ii; i < ii + B && i < n; i++)
                    for (int j = jj; j < jj + B && j < n; j++) {
                        double sum = c[i][j];
                        for (int k = kk; k < kk + B && k < n; k++)
                            sum += a[i][k] * b[k][j];
                        c[i][j] = sum;
                    }
}

```

The blocked version processes small $B \times B$ tiles of each matrix. Within each tile, the data fits in the cache, so accesses to both `a` and `b` are served from cache rather than main memory. For large matrices (e.g., 1024×1024 or larger), the blocked version typically runs **2–8 times faster** than the unblocked version. The optimal block size B depends on the cache size of your system; a common starting point is to choose B so that three $B \times B$ sub-matrices fit in the L1 data cache. One important caveat: unlike the unblocked version, which overwrites each `c[i][j]`, the blocked version *accumulates into* `c` (note the `double sum = c[i][j];` line), so `c` must be zero-initialized before the call for the two functions to compute the same result.

Structure of Arrays (SoA) vs. Array of Structures (AoS)

The way we organize our data structures also affects cache performance. Consider a program that manages a collection of particles, each with an x-coordinate, y-coordinate, z-coordinate, mass, and velocity. The natural approach in C is an **Array of Structures (AoS)**:

```

/* Array of Structures (AoS) layout.
 * Each particle's fields are stored together. */
struct Particle {
    double x, y, z;
    double mass;
    double vx, vy, vz;
};

struct Particle particles[NUM_PARTICLES];

```

If we frequently need to iterate over all particles and access only the x-coordinates (for example, to compute an average x position), we end up loading entire 56-byte structures into cache lines when we only need the 8-byte `x` field. Six-sevenths of the cache capacity is wasted on fields we are not using.

An alternative is the **Structure of Arrays (SoA)** layout:

```

/* Structure of Arrays (SoA) layout.
 * Each field is stored in its own contiguous array. */
struct Particles {
    double x[NUM_PARTICLES];
    double y[NUM_PARTICLES];
    double z[NUM_PARTICLES];
    double mass[NUM_PARTICLES];
    double vx[NUM_PARTICLES];
    double vy[NUM_PARTICLES];
    double vz[NUM_PARTICLES];
};

struct Particles particles;

```

Now when we iterate over all x-coordinates, we access a contiguous array of doubles. Every byte loaded into the cache is a value we actually need. The SoA layout provides much better spatial locality for computations that access only a subset of fields.

When to Use Each Layout

- **Use AoS** when you typically access most or all fields of each element together (e.g., processing one particle at a time with all its attributes).
- **Use SoA** when you typically iterate over all elements but access only one or two fields at a time (e.g., computing the sum of all x-coordinates, or updating all velocities).

In practice, high-performance scientific computing, game engines, and data processing systems often use SoA layouts for this reason.

Avoiding Unnecessary Memory Accesses

We touched on this in Lesson 7 when discussing how to eliminate unnecessary memory references, but it bears repeating in the context of cache performance. Every write to memory through a pointer forces the value to be stored in the cache (and eventually to main memory), even if the value will be updated again immediately. By accumulating intermediate results in a local variable, we allow the compiler to keep the value in a register, avoiding repeated cache accesses entirely.

Example: Accumulating in a Local Variable

Before (repeated memory writes):

```
/* Bad: writes to *result on every iteration.
 * The compiler may not be able to optimize this
 * away because result could alias elements of a. */
void sum_array(double *a, int n, double *result) {
    *result = 0.0;
    for (int i = 0; i < n; i++)
        *result += a[i];
}
```

After (accumulate in a local variable):

```
/* Good: accumulates in a local variable that the
 * compiler can keep in a register. Only one write
 * to *result at the end. */
void sum_array(double *a, int n, double *result) {
    double sum = 0.0;
    for (int i = 0; i < n; i++)
        sum += a[i];
    *result = sum;
}
```

The improved version is not only friendlier to the cache but also removes the potential aliasing problem: the compiler cannot assume that `result` does not point into the array `a`, so it may not be able to optimize the first version on its own. By using a local variable, we make the optimization explicit and portable.

Data Alignment

A data value is **naturally aligned** when its memory address is a multiple of its size. For example, a 4-byte `int` is naturally aligned when stored at an address that is a multiple of 4, and an 8-byte `double` is naturally aligned at a multiple of 8.

When data is not aligned, a single value can straddle two cache lines. Accessing such a value requires loading two cache lines instead of one, which doubles the cache traffic for that access. Modern x86 processors can handle unaligned accesses in hardware, but there is still a performance penalty, especially when it causes extra cache line loads.

In most cases, the C compiler handles alignment for you. The compiler inserts padding between structure fields to ensure natural alignment. However, there are situations where alignment can be disrupted:

- Using `__attribute__((packed))` or `#pragma pack` to eliminate padding in structures.
- Casting pointers to different types and accessing data through the cast pointer.
- Using custom memory allocators that do not guarantee alignment.

Unless you have a specific reason to override the compiler's alignment (such as matching a hardware register layout or a network protocol format), it is best to let the compiler manage alignment naturally.

Example: Struct Padding and Alignment

```

/* The compiler may insert padding to align fields. */
struct Example {
    char a;      /* 1 byte */
                /* 3 bytes of padding */
    int b;       /* 4 bytes, aligned to 4-byte boundary */
    char c;      /* 1 byte */
                /* 7 bytes of padding */
    double d;    /* 8 bytes, aligned to 8-byte boundary */
};
/* Total size: 24 bytes (not 14) due to padding. */

/* Reordering fields can reduce padding: */
struct ExampleBetter {
    double d;    /* 8 bytes */
    int b;       /* 4 bytes */
    char a;      /* 1 byte */
    char c;      /* 1 byte */
                /* 2 bytes of padding */
};
/* Total size: 16 bytes. Same data, less wasted space. */

```

By ordering structure fields from largest to smallest, we can minimize the amount of padding the compiler inserts, reducing the structure's memory footprint and improving cache utilization.

Practical Advice

Here is a summary of practical guidelines for writing cache-aware code:

1. **Profile first.** Use tools like **gprof**, **perf**, or **valgrind --tool=cachegrind** to identify where your program spends its time and where cache misses are occurring. Do not guess.
2. **Focus on hot loops.** The inner loops that process large data sets are where cache performance matters most. Optimizing a loop that runs ten times is not worth the effort; optimizing a loop that runs ten million times can be transformative.
3. **Prefer sequential access patterns.** Traverse arrays in the order they are laid out in memory. In C, this means row-major order for 2D arrays.
4. **Use blocking for matrix and grid computations.** When an algorithm must access data in patterns that do not naturally have good locality (such as accessing both rows and columns of a matrix), use loop tiling to keep the working set in the cache.
5. **Think about data layout.** Choose between AoS and SoA based on your access patterns. If you iterate over a large collection and only touch one or two fields, SoA will often be faster.

6. **Accumulate in local variables.** Avoid unnecessary writes to memory in tight loops. Let the compiler use registers for intermediate values.
7. **Let the compiler handle alignment.** Do not disable struct padding unless you have a specific reason. Order struct fields from largest to smallest to minimize padding.
8. **Test on realistic data sizes.** Cache effects are often invisible on small inputs. A matrix multiply might show no difference between blocked and unblocked versions for 32 x 32 matrices, but a dramatic difference for 1024 x 1024 matrices.

Conclusion

Cache-aware programming is one of the most practical optimization skills a systems programmer can develop. The techniques are not complicated: traverse data sequentially, keep working sets small, and choose data layouts that match your access patterns. But the performance impact can be enormous, because the gap between cache and main memory speeds is so large. By combining the code-level optimizations from Lesson 7 with the cache-aware techniques from this lesson, you can write programs that make efficient use of the entire memory hierarchy, from registers to main memory.

Module 8 Summary

Wrap Up

In this module, we studied memory from the hardware level and then used that knowledge to write faster code. We began with the underlying storage technologies — SRAM and DRAM — and saw how the enormous speed gap between the CPU and main memory motivates the entire memory hierarchy. We explored how the principle of locality allows caching to be effective, and how cache organization (direct-mapped, set-associative) determines hit rates and performance. From there, we turned to optimization: when it matters, what the compiler can do automatically, how to measure where time is actually spent, and the code-level techniques a programmer can apply. We concluded with cache-aware programming, where understanding the hardware directly translates into dramatic performance improvements.

Key Takeaways

- **The memory hierarchy** is a series of storage levels (registers, L1/L2/L3 caches, main memory, disk) where each level serves as a cache for the level below. Faster levels are smaller and more expensive per byte; slower levels are larger and cheaper. Locality is what makes this hierarchy effective.
- **Locality** comes in two forms: temporal (recently accessed data will likely be accessed again soon) and spatial (data near recently accessed data will likely be accessed soon). Writing code with good locality is one of the most impactful optimizations a programmer can make.
- **Caches** exploit locality by storing recently used data in fast SRAM close to the CPU. Cache organization — including cache lines, sets, mapping schemes, and write policies — determines how effectively the cache can reduce average memory access time.
- **Correctness comes first.** Optimization should only be pursued after a program is working correctly, and only when measurement shows that the performance improvement is meaningful. Not every program needs to be optimized.
- **The compiler is a powerful optimizer.** Using gcc's optimization flags (`-O1` , `-O2` , `-O3`) can yield significant speedups automatically. However, the compiler must produce code that behaves identically in all cases, so it cannot make optimizations that depend on assumptions about pointer aliasing or side effects. Writing compiler-friendly code helps the compiler do its job better.
- **Measure before you optimize.** Profiling tools like **gprof** show exactly which functions and loops consume the most time. Always profile first and focus optimization efforts on the hot spots.
- **Code-level techniques** include reducing procedure calls in loops, eliminating unnecessary memory references (accumulating in registers instead), loop unrolling, and exploiting instruction-level parallelism.
- **Cache-aware programming** is one of the most impactful optimizations. Traversing 2D arrays in row-major order, using loop tiling for matrix computations, and choosing data layouts that match access patterns can yield order-of-magnitude speedups by keeping data in cache.

Additional Resources

[Cache Memories](#) (Video Lecture from Carnegie Mellon University)

[Program Optimization](#) (Video Lecture from Carnegie Mellon University)

Module 9 Overview

In Module 9, we explore how the operating system manages memory and how programs are loaded and run. We begin with virtual memory — the mechanism that gives each process its own private address space while sharing a single pool of physical memory. With that foundation in place, we examine linking: how object files are combined into executables, how shared libraries work, and why dynamic linking depends on virtual memory concepts like position-independent code and memory-mapped files. Finally, we study dynamic memory allocation — how **malloc** and **free** manage the heap, what happens under the hood, and how to avoid the common pitfalls that plague C programs. The module concludes with garbage collection, the automatic memory management approach used by languages like Java, Python, and Go.

These topics build on each other. Understanding virtual memory explains the address space within which programs are loaded and within which malloc operates. Understanding linking explains how code and data actually get placed into that address space. And understanding malloc explains how the programmer manages the heap region of that space at runtime.

Learning Objectives

At the end of this learning module, you will be able to:

1. Explain how virtual memory works, including virtual and physical addresses, page tables, the TLB, and page faults.
2. Describe the purpose of demand paging and swapping, and explain how the operating system manages physical memory.
3. Explain the difference between static and dynamic linking, including the roles of the linker, symbol resolution, and relocation.
4. Describe how shared libraries work, including position-independent code (PIC), the PLT, and the GOT.
5. Use the malloc/free API correctly in C programs to allocate and deallocate heap memory.
6. Explain how malloc manages the heap internally, including free lists, block structure, and allocation strategies.

7. Identify and avoid common dynamic memory errors such as memory leaks, dangling pointers, double frees, and buffer overflows.
8. Use Valgrind to detect memory errors including leaks, invalid reads and writes, use of uninitialized memory, and double frees.
9. Explain the basic concepts of garbage collection, including mark-and-sweep, reference counting, and generational collection, and contrast automatic memory management with C's manual approach.

Learning Activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Complete the reading listed below.
2. Read Lessons 1 through 5 and complete the embedded practice problems.
3. Compile and experiment with the provided C code examples. Pay special attention to the malloc/free examples.
4. Run your C programs under Valgrind with `--leak-check=full` to check for memory errors. Practice reading Valgrind output and fixing each reported issue.
5. Take Quiz 9.
6. Review additional resources on the Module 9 Summary page.

Reading

- Bryant & O'Hallaron: Chapter 7 (Linking)
- Bryant & O'Hallaron: Chapter 9 (Virtual Memory)

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: Virtual Memory

NOTE: This is a summary of chapter 9, sections 9.1 - 9.8 in the textbook.

Why Virtual Memory Exists

In the early days of computing, programs accessed physical memory directly. This created serious problems: a bug in one program could overwrite the memory of another program or even the operating system itself. If two programs happened to use the same memory addresses, they could not run at the same time. And if a program needed more memory than was physically available, it simply could not run.

Virtual memory solves all of these problems by introducing a layer of abstraction between the addresses a program uses and the actual physical memory hardware. With virtual memory:

- **Isolation:** Each process gets its own private address space. Process A cannot see or modify Process B's memory, even by accident.
- **Sharing:** The physical RAM is shared among all running processes, but each process believes it has the entire address space to itself.
- **Overcommitment:** The total virtual memory used by all processes can exceed the amount of physical RAM, because the OS can use disk as overflow storage.

Virtual vs. Physical Addresses

When your C program accesses a variable, the CPU does not send the variable's address directly to the memory hardware. Instead, the CPU generates a **virtual address (VA)**. A hardware component called the **Memory Management Unit (MMU)** translates this virtual address into a **physical address (PA)** before the memory access proceeds.

An ordinary pointer to a live object in a user process denotes a location in that process's virtual address space, and debuggers normally display virtual addresses. Null, one-past, invalid, integer-derived, and implementation-specific pointer representations need not denote an accessible location. Physical addresses are normally hidden from application code.



Pages

Virtual memory divides both the virtual address space and physical memory into fixed-size blocks called **pages**. On most systems, a page is **4 KB** (4096 bytes). The virtual address space is divided into **virtual pages (VPs)** and physical memory is divided into **physical pages (PPs)**, also called **page frames**.

At any given time, each virtual page is in one of three states:

- **Unallocated:** The page has not been created by the OS. It takes up no space anywhere.
- **Cached:** The page is allocated and currently stored in a physical page in RAM.
- **Uncached:** The page is allocated but currently stored only on disk (it has been swapped out or has not yet been loaded).

The following diagram shows how virtual pages map to physical pages. Notice that not every virtual page is in physical memory at the same time, and that two different processes can have virtual pages that map to different physical pages:

Process A Virtual Address Space		Physical Memory (Page Frames)
+-----+		+-----+
VP 0 (code)	----->	PP 0: A's VP 0
+-----+		+-----+
VP 1 (data)	----->	PP 2: A's VP 1
+-----+		+-----+
VP 2 (heap)	----->	PP 3: A's VP 2
+-----+		+-----+
VP 3 (on disk)	(no map)	PP 1: B's VP 0
+-----+		+-----+
		PP 4: B's VP 1
		+-----+
Process B Virtual Address Space		
+-----+		
VP 0 (code)	----->	(PP 1)
+-----+		
VP 1 (data)	----->	(PP 4)
+-----+		

Page Tables

The mapping from virtual pages to physical pages is stored in a data structure called the **page table**. Each process has its own page table, maintained by the operating system. The page table is an array of **page table entries (PTEs)**. Each PTE contains:

- **Present bit:** Does this entry currently provide a usable translation to a physical page frame? A cleared bit causes a page fault, after which the OS decides whether the address is valid and how to supply it.
- **Physical page number (PPN):** If present, which physical page frame holds this virtual page.
- **Permission bits:** Can this page be read? Written? Executed? These bits enforce memory protection.
- **Dirty bit:** Has this page been modified since it was loaded from disk?
- **Accessed/reference bit:** Has this translation been used? Operating systems can use this as one input to page-replacement decisions.

Exact PTE fields are architecture-specific. Information about a non-present page's backing file, swap slot, or virtual-memory area may live partly in software-managed PTE bits and partly in other kernel data structures; a literal hardware PTE does not simply contain the word "disk."

Page Table (one per process)							
Index	Present	PPN	Read	Write	Exec		
0	1	3	1	0	1	(code: read+execute)	
1	1	7	1	1	0	(data: read+write)	
2	1	5	1	1	0	(heap: read+write)	
3	0	--	1	1	0	(OS metadata says file/swap-backed)	
4	0	---	-	-	-	(unallocated)	

To translate a virtual address, the hardware splits it into two parts:

- **Virtual Page Number (VPN):** The upper bits, used as an index into the page table.
- **Virtual Page Offset (VPO):** The lower bits (12 bits for 4 KB pages), which identify the byte within the page. The offset is the same in both the virtual and physical address.

```
Virtual Address (e.g., 32-bit system with 4 KB pages):
+-----+-----+
| Virtual Page Number (VPN) | Page Offset (12) |
|      (20 bits)           |      bits         |
+-----+-----+

Translation:
VPN ---> look up in page table ---> PPN

Physical Address:
+-----+-----+
| Physical Page Number (PPN) | Page Offset (12) |
+-----+-----+
```

The TLB (Translation Lookaside Buffer)

Every memory access requires an address translation, but a cached translation does not require rereading the page tables. Without that cache, a single-level table would add one page-table access before the data access. On x86-64, a 4-level hierarchy (or 5 levels with LA57) can require four or five page-table-entry reads on a TLB miss, plus the data access; huge-page mappings can end the walk earlier. Processor caches may satisfy some of those reads, but uncached walks would be prohibitively expensive.

The solution is the **Translation Lookaside Buffer (TLB)**, a small, fast hardware cache inside the MMU that stores recently used page table entries. When the CPU generates a virtual address:

1. The MMU checks the TLB for the VPN.
2. **TLB hit:** The PPN is immediately available. No extra memory access needed. This is the common case.
3. **TLB miss:** The MMU performs a *page-table walk*, traversing each level of the page hierarchy in main memory (4 levels on standard x86-64, 5 with LA57), and stores the final translation in the TLB (possibly evicting an old entry).

Programs with good locality often achieve high TLB hit rates. TLBs are therefore essential to practical virtual-memory performance, while workloads with large or irregular working sets can still spend significant time on page-table walks.

Vocabulary checkpoint

Before going on to page faults, make sure these terms feel solid — you'll see all of them again in the rest of the module:

- **Virtual address (VA)** — the address your program sees, e.g., the value of a C pointer.
- **Physical address (PA)** — the address used in the processor's physical address space (normally memory, but some ranges can select memory-mapped devices).
- **Page** — the fixed-size unit (typically 4 KiB on x86-64) that VM operates on.
- **Frame** (or **page frame**) — the same-sized unit on the physical-memory side.
- **Virtual page (VP) / physical page (PP)** — one specific page; VP is in the process's virtual address space, PP is a chunk of physical RAM.
- **Page table** — the kernel-managed data structure that maps virtual pages to physical pages, one per process.
- **Page table entry (PTE)** — one entry in the page-table hierarchy, containing a physical frame number or next-level pointer plus present, permission, accessed, and other architecture-defined bits.
- **VPN / VPO / PPN** — virtual page number / virtual page offset / physical page number. The VA splits into VPN+VPO; translation produces PPN, and the offset is reused to form the PA.
- **MMU** — the hardware unit inside the CPU that does the VA-to-PA translation on every memory access.
- **TLB** — a small hardware cache of recently used page-table entries, sitting next to the MMU.
- **Page fault** — the exception raised for a non-present translation or an access-permission violation; the kernel handler decides whether to supply a page, change a mapping, or reject the access.

If any of those words are still fuzzy, scroll back up before continuing — the rest of the lesson assumes them.

Page Faults

A **page fault** occurs when translation cannot complete with the requested permissions. Common causes include first access to a valid demand-zero or file-backed mapping, a

swapped-out page, copy-on-write, stack growth, a protection violation, or a completely unmapped address. The kernel handler first determines which case occurred:

1. The CPU generates a virtual address and the MMU looks up the PTE.
2. The translation is non-present or disallows the requested access, so the MMU triggers a **page fault exception**.
3. The CPU transfers control to the OS **page fault handler**.
4. The handler consults the process's virtual-memory mappings. If the address or access is invalid, it normally arranges a signal such as `SIGSEGV` ; there is no page to load.
5. For a valid fault, the handler may allocate a zero-filled page, copy an existing page for copy-on-write, find a page already in the page cache, or read data from an executable, mapped file, or swap.
6. If a page frame must be reclaimed, a clean file-backed page may simply be discarded; dirty file-backed data may need writeback, and dirty anonymous data may require swap if swap is available.
7. The handler installs or updates the translation and permissions, then returns. The faulting instruction is normally retried.

A **minor** fault that needs no storage I/O is much cheaper than a **major** fault that must wait for a file or swap device, though both cost far more than a TLB hit. Locality helps keep translations and the working set resident.

Demand Paging

Operating systems use **demand paging**: many mappings are populated only as they are touched rather than reading or allocating every page in advance. Some startup pages and page-table structures are already resident, while first touches of other regions may map page-cache data, allocate zero-filled memory, or read from storage. Over time, the process's **working set** — the pages it is actively using — tends to accumulate in physical memory.

This is efficient because many programs have large address spaces but only use a small fraction of their pages during any given phase of execution.

Swapping

Under memory pressure, the OS reclaims page frames. It may drop clean file-backed pages (which can be reread), write dirty file-backed pages to their files, or move anonymous pages to a configured **swap area**. Systems can run without swap. Practical kernels approximate recency and use multi-list or generation-based reclaim policies rather than implementing perfect LRU.

If the system is constantly swapping pages in and out because the working set of all active processes exceeds physical memory, this is called **thrashing**. Thrashing devastates performance because the system spends most of its time doing disk I/O instead of useful computation.

Memory-Mapped Files

Virtual memory also enables **memory-mapped files**. The `mmap()` system call maps a file (or portion of a file) into a process's virtual address space. Once mapped, the process can read and write the file as if it were an array in memory. The OS handles loading pages from the file on demand (using the same page fault mechanism) and writing modified pages back to the file.

Memory-mapped files are used for:

- Loading program code and shared libraries (the **text** segment is memory-mapped from the executable file).
- Sharing memory between processes (two processes can `mmap` the same file).
- Efficient file I/O for large files (avoids explicit read/write system calls).

Process Memory Layout Revisited

Now that we understand virtual memory, we can revisit the memory layout of a process with a deeper understanding. Recall the picture from Module 5 Lesson 5 (Processes), which showed each process living in its own private address space with kernel space mapped at the top.

Back then we said "each process gets its own address space" without explaining how. Now we can: every region in the layout below is a range of *virtual* addresses. The OS sets up the page table so that each region maps to appropriate physical pages (or is marked as unallocated).



This diagram uses 32-bit addresses to keep the numbers readable. On the x86-64 machines you are using, the layout is the same but the numbers are larger: virtual addresses are 48 bits, user space runs from `0x0` up to `0x00007fffffffffff`, and kernel space occupies `0xffff800000000000` and above.

Each of these regions has different page permissions:

- **Text:** Read and execute, but not write. The code is memory-mapped from the executable file.
- **Data and BSS:** Read and write, but not execute.
- **Heap:** Read and write. Grows upward as malloc requests more memory (via brk/sbrk or mmap).
- **Stack:** Read and write. Grows downward as functions are called.
- **Kernel space:** Not accessible to user-mode code. Any access triggers a segmentation fault.

A **segmentation fault** occurs when a process tries to access a virtual address that is either unallocated or that violates the permission bits in the page table entry. The MMU detects the

violation and raises an exception, which the OS converts into a SIGSEGV signal that typically terminates the process.

Putting It All Together

Let us trace what happens when your C program executes `int x = *ptr;` where `ptr` holds the virtual address `0x00405024` :

1. The CPU sends the virtual address `0x00405024` to the MMU.
2. The MMU extracts the VPN (upper 20 bits: `0x00405`) and the offset (lower 12 bits: `0x024`).
3. The MMU checks the TLB for VPN `0x00405` .
4. **If TLB hit:** The PPN is retrieved immediately. Skip to step 6.
5. **If TLB miss:** The MMU walks the applicable page-table levels. A present, permitted translation can be cached in the TLB. Otherwise a page fault occurs, and the OS may supply a page, handle copy-on-write, or reject the access depending on the mapping.
6. The physical address is formed by concatenating the PPN with the offset `0x024` .
7. The load checks the cache hierarchy and accesses main memory on a cache miss; the resulting value is placed in a register and then used for `x` .

Lesson 2: Static and Dynamic Linking

Why is linking in the virtual-memory module? At first glance, "how a program is built" looks like it belongs back in Module 3. The reason it's here is that one of the two ways to link — **dynamic linking** — is built directly on top of virtual memory. A shared library like `libc` isn't copied into your executable; instead, the dynamic loader uses `mmap` to map the library's pages into your process's virtual address space at load time, and the runtime then patches up function-call addresses on the fly. Several other concepts later in this module (PIC, GOT, PLT, lazy binding) only make sense once you know that the kernel can map arbitrary file contents into a process's address space at any time. So: virtual memory first, linking second.

Review: What the Linker Does

In Module 3 Lesson 1, we saw the build process: source files are assembled into object files (`.o`), and then the **linker** combines those object files into a single executable. But what exactly is the linker doing?

The linker has two main jobs:

1. **Symbol resolution:** matching every reference to a symbol (function or variable name) with exactly one definition of that symbol.
2. **Relocation:** assigning final memory addresses to each section and patching the code so that every reference points to the correct address.

In Module 3 Lesson 8, we linked multiple `.o` files together (`main.o`, `readInt.o`, `writeInt.o`). That was **static linking** in its simplest form. This lesson goes deeper into how the linker resolves symbols and introduces the distinction between static and dynamic linking.

Symbol Resolution

Every object file has a **symbol table** — a list of the symbols it defines and the symbols it references. When you write:

```

        .global myfunction      # export the label below
myfunction:
    sub $8, %rsp               # align stack before the call
    call readInt               # defined in another file
    add $8, %rsp
    ret

```

The label defines `myfunction`; `.global` makes it visible to other files. The call refers to `readInt`, which remains an **undefined symbol** in this object file. A `.global` or `.extern` directive alone is not a function definition. The linker's job is to match every undefined symbol in every object file with exactly one definition.

If a symbol is referenced but never defined, you get the familiar error:

```

ld: main.o: in function `_start':
main.s:(.text+0x5): undefined reference to `readInt'

```

If a symbol is defined in two different object files, you may get a **multiple definition** error (more on this below).

Strong, Weak, and COMMON Symbols

For the ELF object files used here, distinguish ordinary global definitions, explicitly weak symbols, and COMMON symbols:

- **Strong definitions:** ordinary externally visible function and object definitions. Multiple strong definitions of the same name normally cause a link error.
- **Weak definitions:** marked explicitly, for example with GAS `.weak` or GCC `__attribute__((weak))`. When both definitions are included in a link, a strong definition overrides a weak one. Multiple weak definitions do not normally cause a duplicate-definition error; do not rely on selection by size.
- **COMMON symbols:** declarations of storage made with `.comm`, or C tentative definitions such as file-scope `int x;` compiled with `-fcommon`. These are not ELF weak symbols. GNU ld merges matching COMMON symbols, reserving the largest requested size; an ordinary definition overrides COMMON storage.

Accidentally using the same tentative name in two files under `-fcommon` can silently share storage. Use `static` for an object private to one file; for a deliberately shared object, put one definition in a source file and an `extern` declaration in a header.

Modern GCC note: Since GCC 10, `-fno-common` is the default. File-scope `int x;` then normally produces a strong BSS definition, so defining it in two files causes a multiple-definition error. `-fcommon` restores COMMON allocation, not weak binding.

Viewing Symbol Tables

You can inspect a file's symbol table with `nm` :

```
$ nm main.o
                 U readInt
                 U writeInt
0000000000000000 T _start
```

`T` means the symbol is defined in the text section (a strong symbol). `U` means it is undefined (referenced but not defined here). Other common codes: `D` = initialized data, `B` = BSS (uninitialized data), `W` = weak, `C` = COMMON.

For more detail, `objdump -t` shows the full symbol table with sizes and section information.

Static Linking

In **static linking**, the linker copies all needed code and data from the input object files (and static libraries) into one self-contained executable. This is what we have been doing with `ld` :

```
$ ld -o program main.o readInt.o writeInt.o
```

A **static library** (`.a` file) is simply an archive of `.o` files. When you link against a static library, the linker pulls in only the object files that contain symbols your program actually references.

Pros and Cons

Advantages	Disadvantages
Self-contained: the executable has everything it needs	Large binary: every program gets its own copy of library code
Fast startup: no runtime symbol resolution needed	No shared updates: to fix a library bug, you must relink every program
Portable: runs on any compatible system without needing libraries installed	Wastes memory: if 10 programs use the same library, 10 copies are in RAM

Dynamic Linking and Shared Libraries

A **shared library** (also called a **dynamic library**) is a `.so` file ("shared object") on Linux. Instead of being copied into the executable at link time, the library is loaded into memory at **runtime** and shared among all programs that use it.

When you compile a C program normally, `gcc` uses dynamic linking by default:

```
$ gcc -o hello hello.c          # dynamically linked (default)
$ gcc -static -o hello hello.c  # statically linked
```

The size difference is dramatic:

```
$ gcc -o hello hello.c
$ ls -l hello
-rwxr-xr-x 1 user user 16696 hello

$ gcc -static -o hello_static hello.c
$ ls -l hello_static
-rwxr-xr-x 1 user user 900424 hello_static
```

The statically linked version is over 50× larger because it includes the entire C library.

Inspecting Dynamic Dependencies

You can see which shared libraries an executable depends on with `ldd` :

```
$ ldd hello
linux-vdso.so.1 (0x00007ffd3b1f2000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
(0x00007f2a1c600000)
/lib64/ld-linux-x86-64.so.2 (0x00007f2a1c82d000)
```

The key entries are:

- `libc.so.6` — the C standard library (`printf`, `malloc`, etc.)
- `ld-linux-x86-64.so.2` — the **dynamic linker** itself, responsible for loading shared libraries at program startup

Pros and Cons

Advantages	Disadvantages
Small executables: library code is not duplicated	Startup cost: the dynamic linker must resolve symbols at load time
Shared memory: one copy of the library serves all processes	Dependency: the program won't run if a required <code>.so</code> is missing
Easy updates: fix a bug in the library and all programs benefit	Version conflicts: different programs may need different library versions

PLT and GOT: How Dynamic Calls Work

When your program calls a function from a shared library (like `printf`), the call doesn't jump directly to `printf`'s code — the address isn't known until runtime. Instead, the call goes through two data structures:

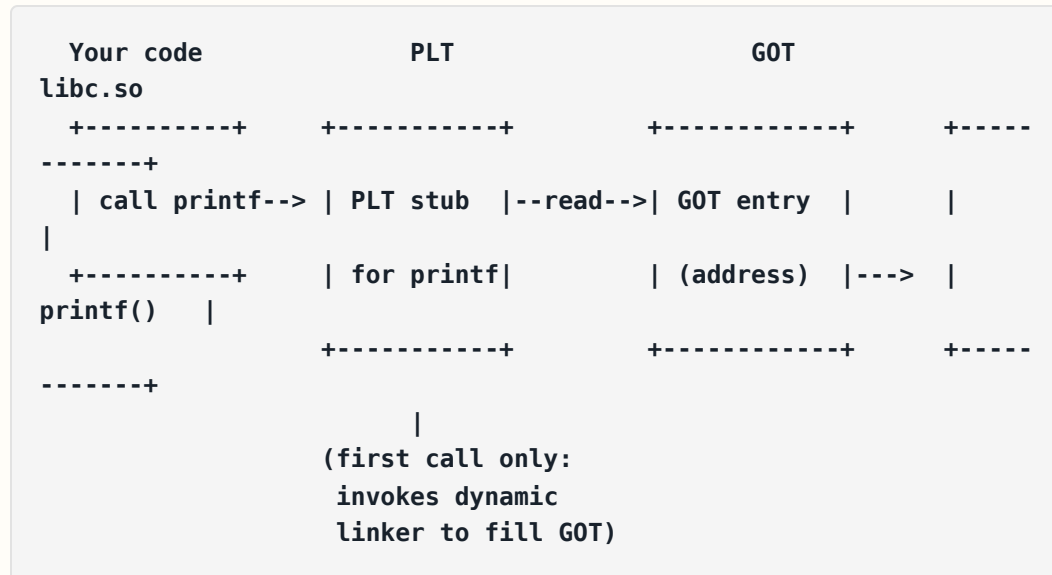
- The **PLT** (Procedure Linkage Table) — a small array of code stubs in your executable, one per external function.
- The **GOT** (Global Offset Table) — a table of addresses that gets filled in by the dynamic linker.

Here is the high-level flow:

1. Your code calls `printf`, which actually jumps to the PLT entry for `printf`.
2. **First call:** The PLT entry jumps to the dynamic linker, which looks up the real address of `printf` in the shared library, writes it into the GOT, and then jumps to `printf`.

3. **Subsequent calls:** The PLT entry reads the (now filled-in) address from the GOT and jumps directly to `printf`. No dynamic linker involved.

With lazy PLT binding enabled, the first call to each such function pays a one-time lookup cost. Eager binding (for example `-Wl, -z, now` or `LD_BIND_NOW=1`) resolves these entries at load time instead. After that, calls go through the GOT with only one extra level of indirection — very fast.



You don't need to memorize the mechanics of lazy binding. The key idea is: **the PLT/GOT provides indirection for dynamically resolved calls; lazy binding adds a first-call lookup, while eager binding does that work at load time.**

Creating a Shared Library

To build a shared library from C source:

```
$ gcc -shared -fPIC -o libmymath.so mymath.c
```

The `-fPIC` flag generates **position-independent code**, which is required for shared libraries (the library may be loaded at any address). The `-shared` flag tells the linker to produce a `.so` instead of an executable.

To link against it:

```
$ gcc -o program program.c -L. -lmymath
```

And to run it (since the library is not in a standard path):


```
$ LD_LIBRARY_PATH=. ./program
```

Summary

Static Linking	Dynamic Linking	
When linking happens	Build time (by <code>ld</code>)	Load time / run time (by <code>ld-linux</code>)
Library format	<code>.a</code> (archive of <code>.o</code> files)	<code>.so</code> (shared object)
Executable size	Large (contains library code)	Small (just references)
Memory usage	Each process has its own copy	Shared across processes
Updates	Must relink to pick up fixes	Just replace the <code>.so</code> file
Tools	<code>ld</code> , <code>ar</code>	<code>ld</code> , <code>ldd</code> , <code>LD_LIBRARY_PATH</code>

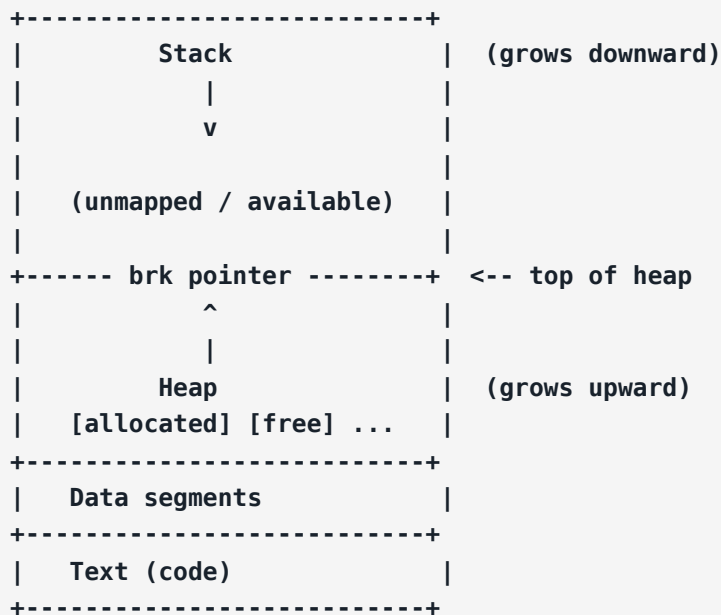
Lesson 3: Dynamic Memory Allocation

NOTE: This is a summary of chapter 9, sections 9.9 - 9.11 in the textbook.

The Heap

In the previous lesson, we saw the memory layout of a process. The **heap** is the region of virtual memory that sits between the initialized data segment and the stack. Unlike the stack, which is managed automatically by function calls and returns, the heap is managed explicitly by the programmer through library functions.

The top of the heap is tracked by a pointer called **brk** (the "break"). When a program needs more heap space, the OS moves **brk** upward using the **brk()** system call (or its library wrapper **sbrk()**). In practice, you almost never call these directly. Instead, you use the **malloc** family of functions, which call **brk/sbrk** (or **mmap** for large allocations) internally.



The malloc/free API

The C standard library provides four functions for dynamic memory allocation, all declared in `<stdlib.h>`:

- **malloc(size_t size)** — Allocates `size` bytes of uninitialized memory. Returns a pointer to the beginning of the allocated block, or `NULL` if the request fails.
- **free(void *ptr)** — Deallocates the block of memory pointed to by `ptr`. The pointer must have been returned by a previous call to `malloc`, `calloc`, or `realloc`.
- **calloc(size_t count, size_t size)** — Allocates memory for an array of `count` elements, each of `size` bytes, and initializes all bytes to zero.
- **realloc(void *ptr, size_t size)** — Resizes a previously allocated block to `size` bytes. May move the block to a new location (the old pointer becomes invalid). Returns a pointer to the new block.

Here is a basic example of proper usage:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Allocate an array of 10 integers
    int *arr = (int *)malloc(10 * sizeof(int));
    if (arr == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }

    // Use the allocated memory
    for (int i = 0; i < 10; i++) {
        arr[i] = i * i;
    }

    // Print the values
    for (int i = 0; i < 10; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // Free the memory when done
    free(arr);
    arr = NULL; // Good practice: set pointer to NULL after
    freeing

    return 0;
}

```

Important rules:

- **Always check** the return value of malloc. It returns NULL if allocation fails.
- **Always free** memory that you allocate, when you are done with it.
- **Never use** memory after freeing it.
- **Never free** the same pointer twice.
- **Only free** pointers returned by malloc/calloc/realloc — never free a stack variable or a string literal.

How malloc Works Internally

Understanding a simple allocator model helps you write better code and debug memory problems. A textbook allocator can maintain a **free list** linking available heap blocks. Production allocators commonly use multiple size classes, bins, per-thread caches, trees, and

mappings rather than one linked list, but the block-splitting and coalescing ideas below remain useful.

Block Structure

Each block on the heap — whether allocated or free — has a **header** that stores metadata.

The header typically contains:

- **Size** of the block (including the header itself).
- **Allocated bit**: indicates whether this block is currently in use (1) or free (0). This bit is often stored in the lowest bit of the size field, since block sizes are always multiples of 8 (or 16) and the lowest bits would otherwise be zero.

A heap with three blocks (one free between two allocated):



Header detail:



When malloc returns a pointer to the user, it points to the **payload** — the usable memory just after the header. The user never sees the header. When free is called, the implementation uses the pointer to find the header (by subtracting the header size) and marks the block as free.

Blocks may also include optional **padding** at the end to satisfy **alignment requirements**. On most 64-bit systems, malloc returns addresses aligned to 16-byte boundaries. This means block sizes are always rounded up to multiples of 16.

Allocation Strategies

In this simplified free-list allocator, an allocation searches available blocks. Common textbook placement strategies include:

- **First Fit:** Search the free list from the beginning and use the first block that is large enough. This is fast but tends to leave many small fragments near the beginning of the list.
- **Next Fit:** Like first fit, but start the search from where the last search left off (instead of from the beginning). This spreads allocation more evenly across the heap but can have worse fragmentation than first fit in practice.
- **Best Fit:** Search the entire free list and use the smallest block that is large enough. This minimizes wasted space within the chosen block but is slower (must examine every free block) and can leave behind many tiny, unusable fragments.

Each strategy involves tradeoffs between allocation speed (throughput) and memory utilization. No single strategy is optimal for all workloads.

Fragmentation

Fragmentation is wasted memory that reduces the effective capacity of the heap. There are two types:

Internal Fragmentation

Internal fragmentation is wasted space *inside* an allocated block. It occurs because:

- The allocator rounds up the requested size to satisfy alignment constraints (e.g., requesting 20 bytes but getting a 32-byte block).
- There is a minimum block size (the block must be large enough to hold the header and free-list pointers).
- The header itself takes up space that the user did not request.

User requests 20 bytes. On a 64-bit system with 16-byte alignment and a 16-byte allocator header, the allocator gives a 48-byte block:

```
+-----+-----+-----+
| Header | Payload (20 bytes) | Padding |
|16 bytes|                   | (12 bytes)|
+-----+-----+-----+
|<----- 48 bytes ----->|
```

Internal fragmentation = 48 - 16 (header) - 20 (requested) = 12 bytes wasted

(Glibc's ptmalloc2 uses an 8–16 byte header on 64-bit systems and rounds payload sizes up to a multiple of 16 bytes. Older 32-bit textbook examples used a 4-byte header.)

External Fragmentation

External fragmentation is when there is enough *total* free memory on the heap to satisfy a request, but no single free block is large enough. The free memory is scattered in small chunks between allocated blocks.

Heap state: [alloc 16] [free 8] [alloc 32] [free 8] [alloc 16] [free 8]

Total free memory: 24 bytes

Request: malloc(20) -- FAILS! No single free block has 20 bytes.

External fragmentation is the more serious problem and is difficult to eliminate entirely. Unlike languages with garbage collection (such as Java), C cannot move allocated blocks to compact free memory, because any pointer to the moved block would become invalid.

Coalescing

When a block is freed, the allocator checks whether the adjacent blocks (immediately before and after in memory) are also free. If so, it **coalesces** (merges) them into a single larger free block. This reduces external fragmentation.

Before freeing the middle block:

```
[free 16] [alloc 32] [free 16]
```

After free() is called on the middle block:

```
[free 16] [free 32] [free 16]
```

After coalescing:

```
[free 64]    <-- one large block instead of three small ones
```

To coalesce with the *previous* block efficiently, many implementations use **boundary tags**: a copy of the header is placed at the end of each block (a **footer**). This allows the allocator to quickly find the size and allocation status of the previous block by looking at the footer just before the current block's header.

Common malloc Errors

Dynamic memory allocation is a major source of bugs in C programs. Here are the most common errors:

Memory Leaks

A **memory leak** occurs when allocated memory is never freed. The memory remains allocated for the lifetime of the process, even though the program no longer needs it. Over time, memory leaks can consume all available memory.

```
/* MEMORY LEAK: allocated memory is never freed */
void memory_leak_example() {
    for (int i = 0; i < 1000; i++) {
        int *p = (int *)malloc(1000 * sizeof(int));
        // use p for something...
        // OOPS: forgot to call free(p)!
    }
    // 1000 blocks are now leaked -- 4 MB of wasted memory
}
```

Dangling Pointers (Use After Free)

A **dangling pointer** is a pointer that refers to memory that has already been freed. Using a dangling pointer is undefined behavior: the memory may have been reallocated for a different purpose, so reading or writing through the pointer can corrupt other data or crash the program.


```

/* DANGLING POINTER: using memory after it has been freed */
void dangling_pointer_example() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    free(p);

    // BAD: p now points to freed memory
    printf("%d\n", *p); // undefined behavior!
    *p = 99;           // undefined behavior -- may corrupt other
data!
}

```

Double Free

Calling **free()** on the same pointer twice is undefined behavior. It can corrupt the free list data structure, leading to crashes or exploitable security vulnerabilities.

```

/* DOUBLE FREE: freeing the same pointer twice */
void double_free_example() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    free(p);
    free(p); // BAD: double free -- undefined behavior!
}

```

Buffer Overflow (Heap Overflow)

Writing past the end of an allocated block overwrites the header of the next block or other heap metadata. This can corrupt the free list and cause malloc or free to crash later — often far from the line of code that caused the problem, making these bugs very difficult to find.

```
/* BUFFER OVERFLOW: writing past the end of an allocated block */
void buffer_overflow_example() {
    int *arr = (int *)malloc(5 * sizeof(int)); // 5 ints = 20
    bytes

    for (int i = 0; i <= 5; i++) { // BUG: should be i < 5, not i
    <= 5
        arr[i] = i; // when i == 5, writes past the allocated
    block!
    }

    free(arr); // may crash here because the heap metadata is
    corrupted
}
```

Correct Usage

Here is an example that demonstrates proper dynamic memory management:

```

/* CORRECT: proper malloc/free usage */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *duplicate_string(const char *src) {
    // Allocate enough space for the string plus the null
    terminator
    size_t len = strlen(src) + 1;
    char *copy = (char *)malloc(len);
    if (copy == NULL) {
        return NULL; // allocation failed
    }
    memcpy(copy, src, len);
    return copy; // caller is responsible for freeing this
}

int main() {
    char *greeting = duplicate_string("Hello, world!");
    if (greeting == NULL) {
        fprintf(stderr, "Out of memory\n");
        return 1;
    }

    printf("%s\n", greeting);

    free(greeting); // free when done
    greeting = NULL; // prevent accidental reuse

    return 0;
}

```

Detecting Memory Errors with Valgrind

The memory errors described above are notoriously hard to find by inspecting the code — a program with a dangling pointer or a slow leak can appear to run correctly for a long time before failing.

The next lesson covers **Valgrind**, the tool for finding exactly these kinds of bugs.

Lesson 4: Debugging Memory Errors with Valgrind

Introduction

In Lesson 3, we saw the common memory errors that plague C programs: leaks, dangling pointers, double frees, and buffer overflows. We also saw that these bugs can be subtle and hard to find — a heap buffer overflow might not crash until much later, when `malloc` or `free` tries to use corrupted metadata. The symptoms often appear far from the actual bug, making manual debugging extremely frustrating.

Valgrind is the tool that finds these bugs for you. It monitors every memory access your program makes and reports errors at the exact instruction that causes them — not when the symptoms appear later. In this lesson, we will learn how to use Valgrind systematically to detect and fix every major category of dynamic memory error.

What Valgrind Is

Valgrind is a **dynamic analysis framework** that runs your program on a synthetic CPU. Rather than executing your program directly on the hardware, Valgrind translates each machine instruction and executes it in a controlled environment where every memory access can be monitored.

Valgrind's default tool, **Memcheck**, monitors every memory access and tracks which bytes are allocated, freed, and initialized. For each byte of memory, Memcheck maintains two pieces of metadata:

- **Addressability:** Is this byte part of a currently allocated block? Or has it been freed, or was it never allocated?
- **Definedness:** Has this byte been written to with a known value, or does it still contain uninitialized data?

This lets Memcheck detect errors at the exact instruction that causes them. When your program reads from freed memory, writes past the end of an array, or branches based on an

uninitialized value, Valgrind reports it immediately — with the exact line number and a full stack trace.

Compiling for Valgrind

Valgrind works on compiled binaries, but it gives much better output when your program is compiled with the right flags:

- `-g` — Include debug symbols so that Valgrind can report source file names and line numbers instead of raw addresses.
- `-O0` — Disable optimization so that the compiled code structure matches what you wrote. With optimization enabled, the compiler may reorder, inline, or eliminate code, making Valgrind's reports harder to map back to your source.

Example:

```
gcc -g -O0 -o myprogram myprogram.c
```

You should always compile with `-g`. The `-O0` flag is strongly recommended during development and debugging. Without it, Valgrind still works, but some error reports may point to unexpected lines due to compiler transformations.

Running Valgrind

The basic syntax is:

```
valgrind ./myprogram
```

This runs your program under Memcheck and reports any memory errors. For more detailed output, use additional flags:

```
valgrind --leak-check=full ./myprogram
```

The `--leak-check=full` flag gives detailed information about each memory leak, including the exact location where the leaked memory was allocated.

```
valgrind --leak-check=full --show-leak-kinds=all ./myprogram
```

The `--show-leak-kinds=all` flag reports all types of leaks, including "still reachable" blocks (memory that was not freed but was still pointed to at program exit). By default, Valgrind only reports "definitely lost" and "possibly lost" blocks.

If your program takes command-line arguments, just add them after the program name:

```
valgrind --leak-check=full ./myprogram arg1 arg2
```

Reading Valgrind Output

Valgrind's output has a distinctive format. Every line is prefixed with `==PID==`, where PID is the process ID of your program. This makes it easy to distinguish Valgrind's messages from your program's own output.

A typical error report looks like this:

```
==12345== Invalid write of size 4
==12345==    at 0x4005A3: main (example.c:6)
==12345== Address 0x5204054 is 0 bytes after a block of size 20
alloc'd
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345== by 0x400593: main (example.c:4)
```

Each error report has several parts:

- **Error description:** The first line describes what went wrong (e.g., "Invalid write of size 4").
- **Stack trace:** The indented lines show where the error occurred, from the most recent function call down to the caller. With `-g`, these show file names and line numbers.
- **Additional context:** Valgrind often provides extra information, such as where the relevant memory block was allocated or freed.

At the end of the run, Valgrind prints a **heap summary** and a **leak summary**:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 400 bytes in 1 blocks
==12345==    total heap usage: 3 allocs, 2 frees, 1,424 bytes
allocated
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 400 bytes in 1 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345==           suppressed: 0 bytes in 0 blocks
```

The leak categories mean:

- **Definitely lost:** Memory that is not pointed to by anything — a definite leak. This is the most important category.
- **Indirectly lost:** Memory that is only reachable through a "definitely lost" block. For example, if a linked list's head node is leaked, all the subsequent nodes are "indirectly lost."
- **Possibly lost:** Memory that is pointed to by an interior pointer (a pointer to the middle of a block rather than the beginning). This sometimes indicates a leak and sometimes does not.
- **Still reachable:** Memory that was not freed but is still pointed to at program exit. This is usually not a serious problem, but it indicates a missing `free` .

Detecting Invalid Reads and Writes

An **invalid write** occurs when your program writes to memory it does not own — for example, writing past the end of a heap-allocated array. An **invalid read** is the same for reading. These are the heap buffer overflows and underflows we discussed in Lesson 3.

Consider this program with an off-by-one error:

```
#include <stdlib.h>

int main() {
    int *arr = (int *)malloc(5 * sizeof(int));
    for (int i = 0; i <= 5; i++) { // off-by-one: writes past end
        arr[i] = i * 10;
    }
    free(arr);
    return 0;
}
```

When `i` is 5, the loop writes to `arr[5]`, which is one element past the end of the allocated block (which only has indices 0 through 4). This might not crash — it might silently corrupt the heap metadata stored after the block — but Valgrind catches it immediately:

```
==12345== Invalid write of size 4
==12345==    at 0x4005B3: main (overflow.c:5)
==12345== Address 0x5204054 is 0 bytes after a block of size 20
alloc'd
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400593: main (overflow.c:4)
```

The message tells us:

- The error is an "Invalid write of size 4" (writing a 4-byte `int`).
- It happened at line 5 of `overflow.c` (the `arr[i] = i * 10;` line).
- The address being written to is "0 bytes after a block of size 20" — meaning it is immediately past the end of a 20-byte block (which is 5 ints of 4 bytes each).
- That block was allocated at line 4 of `overflow.c` (the `malloc` call).

The fix is straightforward: change the loop condition from `i <= 5` to `i < 5`.

Detecting Use of Uninitialized Memory

Memory returned by `malloc` is not initialized — it contains whatever data was previously stored at that location. Reading this data before writing to it is undefined behavior, and bugs of this kind can produce different results on different runs.

Consider this program:


```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));
    if (*p > 0) {          // reading uninitialized memory
        printf("positive\n");
    }
    free(p);
    return 0;
}
```

The variable `*p` was never assigned a value, so the `if` condition reads uninitialized memory. Valgrind reports:

```
==12345== Conditional jump or move depends on uninitialised
value(s)
==12345==    at 0x4005A3: main (uninit.c:6)
```

This error means that a branch decision (the `if`) depends on a value that was never initialized. The fix is to assign a value to `*p` before reading it, or use `calloc` instead of `malloc` to get zero-initialized memory.

By default, Valgrind tells you where the uninitialized value was *used* but not where it came from. To trace it back to its origin, use the `--track-origins=yes` flag:

```
valgrind --track-origins=yes ./uninit
```

This adds an extra message showing that the uninitialized value was created by the `malloc` at a specific line, which is especially useful in larger programs where the allocation and the use are far apart.

Detecting Use After Free

A **use-after-free** error occurs when your program accesses memory through a pointer after that memory has been freed. The pointer becomes a **dangling pointer** — it still holds the old address, but that address is no longer valid.

Consider this program:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    free(p);
    printf("%d\n", *p); // use after free
    return 0;
}
```

After `free(p)`, the memory at `p` has been returned to the heap. Reading `*p` on the next line is undefined behavior. It might print 42, or it might print garbage, or it might crash — depending on whether the freed block has been reused. Valgrind catches it reliably:

```
==12345== Invalid read of size 4
==12345==    at 0x4005C3: main (useafterfree.c:8)
==12345== Address 0x5204040 is 0 bytes inside a block of size 4
free'd
==12345==    at 0x4C2BDEC: free (vg_replace_malloc.c:530)
==12345==    by 0x4005B3: main (useafterfree.c:7)
==12345== Block was alloc'd at
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400593: main (useafterfree.c:5)
```

Valgrind's report gives you three critical pieces of information:

- **What happened:** An "Invalid read of size 4" at line 8 — the `printf` that reads `*p`.
- **Where the block was freed:** At line 7 — the `free(p)` call.
- **Where the block was originally allocated:** At line 5 — the `malloc` call.

The fix is to not access memory after freeing it. A common defensive practice is to set the pointer to `NULL` after freeing: `free(p); p = NULL;`. This way, any subsequent dereference will cause an immediate segfault rather than silently reading stale data.

Detecting Memory Leaks

A **memory leak** occurs when your program allocates memory but never frees it. The memory remains allocated until the program exits, wasting resources. In a long-running program (a server, for example), leaks accumulate and can eventually exhaust available memory.

Consider this program:

```
#include <stdlib.h>

int main() {
    int *arr = (int *)malloc(100 * sizeof(int));
    arr[0] = 42;
    // forgot to free arr
    return 0;
}
```

Running with `--leak-check=full`:

```
valgrind --leak-check=full ./leak
```

Valgrind reports:

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 400 bytes in 1 blocks
==12345==    total heap usage: 1 allocs, 0 frees, 400 bytes
allocated
==12345==
==12345== 400 bytes in 1 blocks are definitely lost in loss record
1 of 1
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400553: main (leak.c:4)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 400 bytes in 1 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345==    suppressed: 0 bytes in 0 blocks
```

The report tells us that 400 bytes (100 ints times 4 bytes each) are "definitely lost," and it shows the exact line where the leaked block was allocated (line 4 of `leak.c`). The "1 allocs, 0 frees" in the heap summary is a quick way to see that something was allocated and never freed.

The fix is to add `free(arr);` before `return 0;`.

Detecting Double Frees

A **double free** occurs when your program frees the same block of memory twice. This corrupts the heap's internal data structures and can lead to crashes, incorrect behavior, or

security vulnerabilities.

Consider this program:

```
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    free(p);
    free(p); // double free
    return 0;
}
```

Valgrind reports:

```
==12345== Invalid free() / delete / delete[] / realloc()
==12345==    at 0x4C2BDEC: free (vg_replace_malloc.c:530)
==12345==    by 0x4005A3: main (doublefree.c:7)
==12345== Address 0x5204040 is 0 bytes inside a block of size 4
free'd
==12345==    at 0x4C2BDEC: free (vg_replace_malloc.c:530)
==12345==    by 0x400593: main (doublefree.c:6)
==12345== Block was alloc'd at
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345==    by 0x400573: main (doublefree.c:4)
```

The report shows:

- An "Invalid free()" at line 7 — the second `free(p)` .
- The block was previously freed at line 6 — the first `free(p)` .
- The block was originally allocated at line 4 — the `malloc` call.

The fix is to remove the second `free(p)` . As with use-after-free, setting the pointer to `NULL` after freeing (`free(p); p = NULL;`) is a good defensive practice. Calling `free(NULL)` is defined by the C standard to do nothing, so a second `free(p)` after `p = NULL` is harmless.

A Complete Debugging Example

Let us look at a more realistic example: a function that builds a dynamically allocated array of strings. This program has several bugs.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char **make_names(int count) {
    char **names = (char **)malloc(count * sizeof(char *));

    for (int i = 0; i <= count; i++) {    // Bug 1: off-by-one
        names[i] = (char *)malloc(20);
        sprintf(names[i], "name_%d", i);
    }

    return names;
}

void print_and_free(char **names, int count) {
    for (int i = 0; i < count; i++) {
        printf("%s\n", names[i]);
        free(names[i]);
    }
    // Bug 2: forgot to free names itself
    // Bug 3: also leaked names[count] from the off-by-one
}

int main() {
    char **names = make_names(3);
    print_and_free(names, 3);
    return 0;
}

```

Running under Valgrind with `--leak-check=full` reveals all three bugs:

```

==12345== Invalid write of size 8
==12345==    at 0x400623: make_names (example.c:9)
==12345== Address 0x5204048 is 0 bytes after a block of size 24
alloc'd
==12345==    at 0x4C2AB80: malloc (vg_replace_malloc.c:299)
==12345==    by 0x4005F3: make_names (example.c:6)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 44 bytes in 2 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks

```

(The exact split between "definitely lost" and "indirectly lost" depends on how the allocator rounds up the 24-byte `names` request. If Valgrind's reachability scan can find the pointer to `names[3]` stored at offset 24, you'll see `definitely lost: 24 / indirectly lost: 20`; otherwise both blocks are classified as definitely lost. Either way, both leaks are reported.)

Bug 1 (off-by-one): The loop condition `i <= count` should be `i < count`. When `count` is 3, the array `names` has space for 3 pointers (indices 0, 1, 2), but the loop writes to `names[3]`, which is past the end. **Fix:** Change the loop condition to `i < count`.

Bug 2 (leaked array): The function `print_and_free` frees each string but not the array of pointers itself. **Fix:** Add `free(names);` at the end of `print_and_free`.

Bug 3 (leaked extra string): Because of the off-by-one, an extra string `names[count]` was allocated but is never freed by `print_and_free` (which only iterates up to `count - 1`). Fixing Bug 1 eliminates this bug as well, since the extra allocation never happens.

Here is the corrected code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char **make_names(int count) {
    char **names = (char **)malloc(count * sizeof(char *));

    for (int i = 0; i < count; i++) {          // Fixed: i < count
        names[i] = (char *)malloc(20);
        sprintf(names[i], "name_%d", i);
    }

    return names;
}

void print_and_free(char **names, int count) {
    for (int i = 0; i < count; i++) {
        printf("%s\n", names[i]);
        free(names[i]);
    }
    free(names);                                // Fixed: free the
array itself
}

int main() {
    char **names = make_names(3);
    print_and_free(names, 3);
    return 0;
}

```

Running the corrected program under Valgrind:

```

==12345== HEAP SUMMARY:
==12345==      in use at exit: 0 bytes in 0 blocks
==12345==    total heap usage: 4 allocs, 4 frees, 84 bytes allocated
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts

```

A clean report: zero errors, zero leaks.

Tips for Using Valgrind

- **Always compile with `-g`** and preferably `-O0`. Without debug symbols, Valgrind's reports show raw addresses instead of file names and line numbers, making them much harder to interpret.
- **Run with `--leak-check=full` as a habit.** This should be your default Valgrind invocation. Without it, Valgrind reports the total bytes leaked but not where the leaks originated.
- **Fix errors in order.** The first memory error can cause cascading effects. An invalid write might corrupt data that leads to additional errors later. Fix the first reported error, re-run Valgrind, and repeat.
- **Valgrind slows your program down roughly 10–50x.** This is normal and expected. Valgrind is instrumenting every memory access, so this overhead is unavoidable. If your program normally takes a few seconds, it will take a minute or two under Valgrind. For programs with very large inputs, consider testing under Valgrind with a smaller input first.
- **A clean Valgrind report should be the standard.** For any C program that uses dynamic memory, you should aim for 0 errors and 0 leaks. If Valgrind reports no issues, you can be confident that your program has no heap memory errors — at least on the execution path you tested.
- **Use `--track-origins=yes`** to trace uninitialized values back to their source. This flag makes Valgrind slower but produces more informative reports for "uninitialised value" errors, showing where the problematic memory was allocated rather than just where it was used.

Practical tip: Make Valgrind a routine part of your development workflow, not something you run only when something goes wrong. A good practice is to run your program under Valgrind every time you test — especially after making changes to code that allocates or frees memory. Many professional C development teams run Valgrind as part of their automated test suite, failing the build if any memory errors are detected. The few extra seconds of Valgrind runtime can save you hours of debugging mysterious crashes.

Lesson 5: Garbage Collection

The Problem: Manual Memory Management is Hard

In Lesson 3, we saw the `malloc / free` API and the many ways it can go wrong: memory leaks, dangling pointers, double frees, and heap buffer overflows. These bugs are difficult to find, difficult to reproduce, and often lead to security vulnerabilities.

The root cause is that **the programmer is responsible for deciding when to free every block of memory**. In a large program with complex data structures — linked lists, trees, graphs, objects pointing to other objects — tracking which memory is still in use becomes extremely error-prone.

Garbage collection (GC) automates some or all object-lifetime management. Tracing collectors reclaim objects not **reachable** from program roots; reference counting reclaims an object when its count reaches zero and needs an additional strategy for cycles. Languages including Java, Python, Go, JavaScript, C#, and Ruby use automatic memory-management techniques.

Reachability

The key idea behind **tracing** garbage collectors is reachability. A block is **reachable** if the collector can reach it by following references starting from a **root**. Roots commonly include:

- Local variables on the stack
- Global/static variables
- CPU registers

If no chain of pointers from any root leads to a block, the program has no way to access it, and the memory can safely be reclaimed. This is the same criterion you use mentally when deciding whether to call `free()` — the garbage collector just automates it.

Reference Counting

Reference counting is the simplest form of garbage collection. Each object maintains a **reference count** — the number of pointers currently pointing to it. When a pointer is assigned to point to an object, the count increases; when a pointer is reassigned or goes out of scope, the count decreases. When the count reaches zero, the object is immediately freed.

```
a = new Object()    # Object refcount = 1
b = a               # Object refcount = 2
a = null            # Object refcount = 1
b = null            # Object refcount = 0 --> freed immediately
```

The Cycle Problem

Reference counting has a fundamental flaw: it cannot reclaim **cycles**. If object A points to object B and object B points back to object A, both have a reference count of at least 1 — even if nothing else in the program can reach either of them.

```
A.next = B          # B refcount = 1
B.prev = A          # A refcount = 1
(remove all other references to A and B)

# A refcount = 1 (from B.prev), B refcount = 1 (from A.next)
# Neither will ever reach 0 --> MEMORY LEAK
```

Languages that rely on reference counting deal with cycles in different ways:

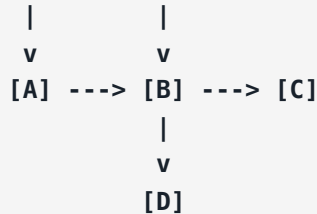
- **CPython**, the standard Python implementation, uses reference counting as its primary mechanism and also runs a cyclic garbage collector. Other Python implementations may use different strategies.
- **Swift** uses reference counting and requires the programmer to break cycles manually using `weak` references.
- **Rust** uses `Rc` (reference counted pointers) and `Weak` for the same purpose, though Rust's ownership model avoids most GC needs entirely.

Mark-and-Sweep

Mark-and-sweep is a tracing garbage collector that handles cycles correctly. It works in two phases:

1. **Mark phase:** Starting from the roots (stack, globals, registers), follow every pointer and **mark** each reachable object. This is essentially a graph traversal (DFS or BFS) over the object graph.
2. **Sweep phase:** Walk through the entire heap. Any object that was **not** marked is unreachable — free it. Clear all marks for the next collection cycle.

Roots: [stack] [globals]



[E] ---> [F] (E and F are not reachable from any root)

Mark phase: A, B, C, D are marked (reachable from roots)

Sweep phase: E and F are not marked --> freed

Mark-and-sweep handles cycles naturally: if A and B point to each other but neither is reachable from a root, neither gets marked, and both are freed in the sweep phase.

A basic mark-and-sweep implementation pauses application threads while it traces and sweeps. Pause duration depends on heap size, live data, and collector design and can be unacceptable for latency-sensitive or real-time software. Production collectors layer substantial optimizations on the basic idea: **Go** uses concurrent tracing; **JavaScript** engines such as V8 combine generational copying with marking and compaction; modern **Java** collectors such as G1, ZGC, and Shenandoah use region-based and mostly concurrent techniques. Treat textbook mark-and-sweep as an algorithmic skeleton, not a description of every production collector.

Generational GC

Empirically, most objects die young — they are allocated, used briefly, and become garbage. This observation is called the **generational hypothesis**, and it motivates an important optimization.

A **generational collector** divides the heap into two or more **generations**:

- The **young generation** (or "nursery") holds recently allocated objects. It is collected frequently, which is fast because most young objects are already dead.

- The **old generation** holds objects that have survived several young collections. It is collected rarely, which amortizes the higher cost of scanning long-lived objects.

Objects that survive enough young-generation collections are **promoted** to the old generation. This optimization dramatically reduces the average cost of garbage collection. It is used by **Java** (the HotSpot JVM) and **.NET**.

Compacting and Copying Collectors

Recall from Lesson 3 that **external fragmentation** is free memory scattered into chunks that cannot satisfy a larger contiguous request. A C allocator can mitigate fragmentation with size classes, coalescing, and separate mappings, but it generally cannot transparently relocate a live allocation because raw pointers to it would become invalid.

A **precise moving collector** can compact managed objects because the runtime knows which references must be updated. Collectors with conservative references or pinned objects may be unable to move everything. Two basic moving approaches are:

- **Compacting collector:** After mark-and-sweep, slide all surviving objects to one end of the heap, eliminating all gaps. The free space becomes one large contiguous block.
- **Copying collector:** Divide memory into two halves. Allocate from one half; when it fills up, copy all reachable objects to the other half (packed tightly), then swap the roles of the two halves.

In their basic models, both pack movable live objects and remove gaps among them. Real collectors may retain fragmentation because of pinned objects, regions, large-object spaces, or alignment. The tradeoff is the cost of copying objects and updating references.

Why C Doesn't Have Garbage Collection

C allows raw pointer arithmetic: you can cast integers to pointers, store pointers in integers, compute pointer offsets, and so on. This means the runtime **cannot reliably determine what is a pointer and what is not**. Consider:

```

int *p = (int *)malloc(sizeof(int));
*p = 42;
uintptr_t secret = (uintptr_t)p;    // pointer stored as an
integer
p = NULL;                          // the only "visible" pointer
is gone
// ... later ...
int *q = (int *)secret;             // recover the pointer from
the integer
printf("%d\n", *q);                // still works -- the block is
still needed!

```

If a garbage collector were managing this memory, it would see that no pointer variable currently refers to the block (only the integer `secret` does, and the GC cannot tell that an integer holds an address). The collector might reclaim the block, and then dereferencing `q` would be a dangling-pointer bug — even though the program legitimately still needs that memory. (Strictly speaking, this is how a *precise* collector — one that knows exactly which values are pointers — would behave; the *conservative* collector described next makes the opposite choice, keeping anything that looks like a pointer, which is why this particular example would survive under it.)

Furthermore, since C pointers are raw addresses, a collector cannot **move** objects — there is no way to find and update every copy of a pointer. This is exactly what Lesson 3 noted: "Unlike languages with garbage collection, C cannot move allocated blocks to compact free memory."

Conservative GC: Boehm GC

There is one exception: the **Boehm-Demers-Weiser conservative garbage collector** is a drop-in replacement for `malloc` / `free` in C and C++ programs. It works by treating **every value in memory that looks like it could be a pointer** (i.e., falls within the heap address range) as if it were one.

This is "conservative" because it may keep objects alive that are not actually reachable — a random integer that happens to look like a heap address will prevent that block from being collected. But it will never incorrectly free memory that is still in use — provided the program keeps at least one *undisguised*, pointer-looking value referring to each live block. A reference that is disguised (XOR-encoded, say, or written out to a file) no longer looks like a pointer, so it defeats even the conservative collector; the `uintptr_t` example above survives only because the raw address is stored intact in memory. The Boehm GC is used in

practice by several language runtimes and as a leak-prevention measure in some C/C++ applications.

Tradeoffs Summary

Manual (C)	Reference Counting	Tracing GC (Mark-and-Sweep)	
Who frees memory?	Programmer (free)	Runtime (when refcount = 0)	Runtime (periodically)
Handles cycles?	N/A	No (needs extra mechanism)	Yes
Pauses?	No collector pauses	Usually incremental, but cascading releases and cycle collection can pause	Basic collector stops the world; production designs may trace or relocate concurrently
Can compact?	No (raw pointers)	Not typically	Yes (if moving/copying)
Overhead	No GC overhead (malloc/free have their own cost)	Count updates on every assignment	Periodic tracing of all reachable objects
Lifetime safety	Manual lifetime errors are possible	Avoids many dangling references; cycles can leak without extra tracing	Prevents manual free/use-after-free within the managed heap, but does not prevent all program or memory-safety bugs
Used by	C, C++ (commonly)	CPython, Swift ARC, Rust Rc / Arc (opt-in)	Java, Go, JavaScript, C#

Module 9 Summary

Wrap Up

In this module, we studied how the operating system manages memory and how programs are loaded, linked, and given access to the heap. We began with virtual memory — the abstraction that gives each process a private address space while sharing physical memory. We then examined linking: how the linker resolves symbols, combines object files, and supports shared libraries through position-independent code. From there, we studied dynamic memory allocation with malloc and free, including the internal data structures that manage the heap. We concluded with garbage collection, the automatic alternative to manual memory management used by most modern languages.

Key Takeaways

- **Virtual memory** is an abstraction provided by the OS and hardware together. Each process gets a private, usually sparse virtual address space. The MMU translates mapped addresses through page tables, and the TLB caches recent translations. Page faults let the kernel populate demand-zero or file-backed pages, perform copy-on-write, restore swapped pages, or reject invalid accesses.
- **Linking** is the process of combining object files into an executable. The **static linker** resolves symbol references, handles strong vs. weak symbols, and produces a self-contained binary. The **dynamic linker** defers library loading until runtime, using the PLT and GOT to resolve function addresses lazily. Dynamic linking saves disk space and memory when multiple programs share the same libraries.
- **Position-independent code (PIC)** allows shared libraries to be loaded at any virtual address. This is essential for ASLR and for sharing physical pages of library code between processes.
- **Dynamic memory allocation** with malloc/free gives the programmer explicit control over heap memory. A simple allocator can maintain free lists, choose blocks with placement policies such as first fit, split oversized blocks, and coalesce adjacent free blocks. Production allocators use more elaborate bins, caches, and mappings.
- **Common memory errors** in C — memory leaks, dangling pointers, double frees, and buffer overflows — are a major source of bugs and security vulnerabilities. Tools like valgrind can detect these errors and should be used routinely on any C program that uses dynamic memory allocation.
- **Valgrind** is a dynamic analysis tool that detects memory errors at the exact instruction that causes them. Running programs under Valgrind with `--leak-check=full` catches leaks, invalid reads and writes, use of uninitialized memory, and double frees. A clean Valgrind report should be the standard for any C program that uses dynamic memory.
- **Garbage collection** automates memory management. Mark-and-sweep traces reachable objects from the roots and frees everything else. Reference counting is simpler but cannot handle cycles. Generational collectors exploit the observation that most objects die young, collecting the nursery frequently and the old generation rarely.

Additional Resources

[Virtual Memory: Concepts](#) (Video Lecture from Carnegie Mellon University)

[Virtual Memory: Systems](#) (Video Lecture from Carnegie Mellon University)

[Linking](#) (Video Lecture from Carnegie Mellon University)

[Dynamic Memory Allocation: Basic Concepts](#) (Video Lecture from Carnegie Mellon University)

[Dynamic Memory Allocation: Advanced Concepts](#) (Video Lecture from Carnegie Mellon University)

[Valgrind Quick Start Guide](#) — Official documentation for getting started with valgrind

Module 10 Overview

In Module 10: Kernel Organization, you will explore the heart of every operating system: the kernel. Throughout this course, you have been using kernel services every time you called `fork()`, sent a signal, created a thread, opened a socket, or allocated memory. Now we step back and examine how the kernel itself is designed and organized.

This module covers the two major approaches to kernel architecture — monolithic kernels and microkernels — along with their tradeoffs and modern variations. You will learn:

- What a kernel does and how it mediates between user programs and hardware
- How monolithic kernels like Linux place major kernel services in one privileged address space
- How microkernels take the opposite approach, running most services in user space
- How modern systems blend these approaches and what tradeoffs drive those decisions

Learning objectives

At the end of this learning module, you will be able to:

1. Explain the role of a kernel, including how it manages processes, memory, devices, and file systems on behalf of user programs
2. Compare and contrast monolithic and microkernel designs, identifying the key architectural differences and their consequences
3. Describe system call dispatch mechanisms, including how the CPU transitions between user mode and kernel mode
4. Evaluate tradeoffs between kernel architectures in terms of performance, reliability, security, and verifiability

Learning activities

You are expected to do the following tasks in order to achieve the above objectives:

1. Read Lessons 1 through 4 and complete the embedded practice problems

2. Complete Discussion 10
3. Take Quiz 10
4. Review additional resources on the Module 10 Summary page

Prerequisites

This module builds on concepts from throughout the course. You should be comfortable with:

- C programming and system calls (`fork()` , `exec()` , `wait()` , `open()` , `read()` , `write()`)
- Processes, signals, and threads (Modules 5-6)
- File I/O, sockets, and inter-process communication (Module 7)
- Memory hierarchy and optimization (Module 8)
- Virtual memory and dynamic memory (Module 9)

If you have questions, you can post them in the course lobby or send your instructor an email.

Lesson 1: What a Kernel Does

Introduction

When you have called `fork()` to create a process, opened a file with `open()`, or connected to a remote server with a socket, you were asking the operating system's **kernel** to do work on your behalf. Even `malloc()`, which normally serves small requests from a user-space allocator, periodically asks the kernel for larger regions of virtual memory. The kernel is the core of the operating system and remains resident from boot until shutdown. Everything else — your shell, your text editor, your web browser, every program you have written in this course — depends on services it provides.

In this lesson, we will look at what the kernel actually does, how it separates itself from user programs, and how your programs communicate with it through system calls.

What Is a Kernel?

On a bare-metal operating system, the kernel is the lowest-level general-purpose software layer and runs with the CPU privileges needed to manage memory, processors, and devices. Firmware, management processors, and a hypervisor (when present) may operate beneath or outside that kernel, so even a kernel does not necessarily control every physical resource.

Think of the kernel as a referee and a resource manager. It decides which process gets to run on the CPU and for how long. It keeps each process's memory separate so one buggy program cannot corrupt another. It translates your program's request to "read a file" into the specific sequence of disk operations needed to retrieve the data. Without a kernel, every program would need to include its own code for talking to hardware, and there would be nothing to prevent one program from interfering with another.

Kernel Responsibilities

The kernel handles several major categories of work:

- **Process management:** Creating new processes (`fork()`), loading programs into them (`exec()`), scheduling which process runs on which CPU core, and cleaning up when processes terminate. The kernel maintains a process table that tracks every running

process, its state (running, ready, sleeping, zombie), its register values, and its memory mappings.

- **Memory management:** Implementing virtual memory so each process thinks it has its own private address space. The kernel maintains page tables that map virtual addresses to physical addresses, handles page faults when a process accesses memory that is not currently in RAM, and decides which pages to evict to disk when physical memory is full. You studied this in the previous module on virtual memory.
- **Device management:** Providing device drivers that know how to communicate with specific hardware: disk controllers, network cards, USB devices, GPUs, keyboards, and so on. The kernel abstracts these into a uniform interface so your program can use `read()` and `write()` without knowing the details of the underlying hardware.
- **File systems:** Organizing data on disk into files and directories. The kernel implements file system formats (ext4, XFS, NTFS, etc.) and provides the familiar interface of `open()`, `read()`, `write()`, and `close()`. It also handles permissions, so one user's files are protected from another user.
- **Networking:** Implementing the entire TCP/IP networking stack. When you create a socket and call `connect()`, the kernel handles the TCP three-way handshake, manages packet buffering, implements congestion control, and routes data between your program and the network hardware.
- **Inter-process communication (IPC):** Providing mechanisms for processes to communicate: pipes, signals, shared memory, message queues, and sockets (including Unix domain sockets for local communication).

User Mode vs. Kernel Mode

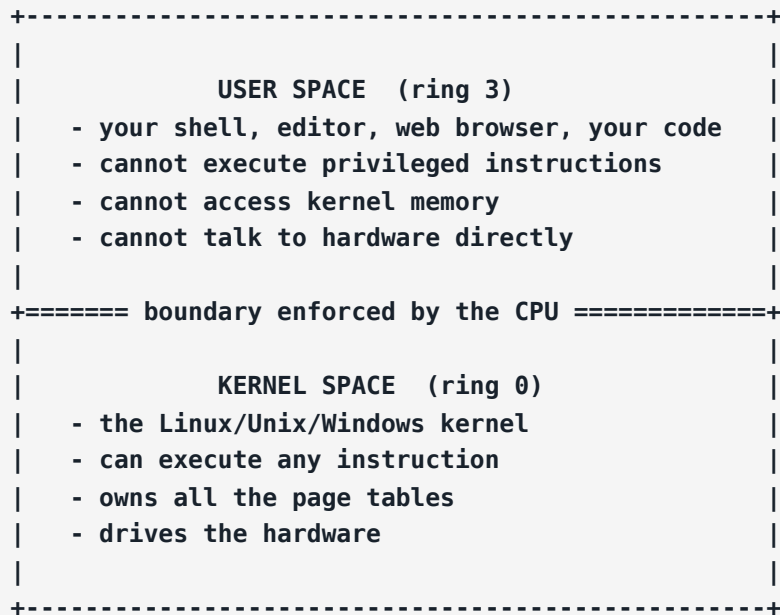
A critical question arises: if the kernel has complete control over the hardware, how do we prevent a buggy or malicious user program from taking over the machine? The answer is built into the CPU hardware itself.

Modern CPUs have at least two **privilege levels** (also called **protection rings**). On x86 processors, there are four rings numbered 0 through 3, but in practice only two are used:

- **Ring 0 (kernel mode / supervisor mode):** The kernel runs here. Code running in ring 0 can execute any CPU instruction, access any memory address, and interact directly with hardware devices. This is the most privileged level.
- **Ring 3 (user mode):** All user programs run here. Code running in ring 3 is restricted. It cannot execute privileged instructions (like `hlt` to halt the CPU or instructions that modify page tables), it cannot access memory belonging to the kernel or other processes, and it cannot directly communicate with hardware.

The CPU enforces these restrictions in hardware. If a user-mode program tries to execute a privileged instruction, the CPU generates a **general protection fault** (GPF, interrupt 13), which the kernel catches and handles — typically by killing the offending process. This is related to, but distinct from, the **page fault** (interrupt 14) that occurs when a program accesses invalid memory. In both cases, the kernel typically delivers a `SIGSEGV` signal ("segmentation fault") to the process, but the underlying hardware exceptions are different.

Privilege rings on x86 (only rings 0 and 3 are used by Linux):



**The boundary is crossed only at controlled entry points:
the `syscall` instruction, hardware interrupts, and faults.**

This hardware-enforced separation is fundamental to system security and stability. Ordinary user-mode loads, stores, and privileged instructions cannot directly access kernel or another process's protected memory. User programs instead request kernel services through controlled entry points such as **system calls**. Kernel vulnerabilities and hardware flaws can still break this isolation, which is why those entry points require careful validation.

System Calls: The User-Kernel Interface

A **system call** (or **syscall**) is the mechanism by which a user program requests a service from the kernel. You have been using system calls throughout this course — `fork()`, `exec()`, `wait()`, `open()`, `read()`, `write()`, `close()`, `socket()`, `mmap()`, `kill()`, and many others are all system calls (or thin wrappers around system calls provided by the C library).

How a System Call Works

Let us trace what happens when your C program calls `write(fd, buf, n)` :

1. **The C library wrapper:** The `write()` function in your program is actually a wrapper function provided by the C library (glibc on Linux). This wrapper sets up the arguments according to the system call convention and then executes a special CPU instruction.
2. **The syscall instruction:** On modern x86-64 Linux, the wrapper executes the `syscall` instruction. (On older 32-bit x86 systems, it was `int 0x80` , a software interrupt.) Before executing this instruction, the wrapper places the **system call number** in the `%rax` register. For `write()` , the syscall number is 1. The arguments go in registers `%rdi` , `%rsi` , and `%rdx` — the file descriptor, buffer pointer, and byte count, respectively.
3. **CPU mode switch:** The `syscall` instruction causes the CPU to switch from ring 3 (user mode) to ring 0 (kernel mode). The hardware saves the user program's instruction pointer into `%rcx` and the flags into `%r11` , then jumps to a predefined **system call entry point** in the kernel. (The user stack pointer is *not* saved by `syscall` itself; `swapgs` gives the entry stub access to kernel per-CPU data. Separate instructions save the user stack pointer and load the kernel stack pointer; `swapgs` does not change `%rsp` .)
4. **Syscall dispatch:** The kernel validates the syscall number and selects its handler. A **system call table** is a useful model of this mapping, but the dispatch implementation varies by kernel version: Linux 6.12 x86-64 uses generated switch-based dispatch. Number 1 selects the write handler (shown as `sys_write()` below).
5. **Kernel handles the request:** The kernel's `sys_write()` function validates the arguments (is the file descriptor valid? does the buffer pointer point to memory the process is allowed to read? etc.), then performs the actual I/O operation — copying data from the user's buffer into a kernel buffer and ultimately to the device.
6. **Return to user mode:** When the kernel function completes, it places the return value in `%rax` and returns through an architecture-appropriate path (commonly `sysretq` , with `iretq` used when required). On success, `%rax` holds the number of bytes written; on failure, the raw kernel interface returns a *negative errno value* (for example, `-EPERM` = -1, `-EBADF` = -9, or `-EINVAL` = -22). A C library wrapper translates values in the kernel's error range into the conventional return value `-1` and sets `errno` . The CPU is then back in ring 3 at the instruction after `syscall` .

This entire sequence — switching to kernel mode, dispatching to the right handler, doing the work, and returning — happens every time you make a system call. It is extremely fast (a few hundred nanoseconds on modern hardware), but it is not free. The mode switch, the saving

and restoring of registers, and the various security checks all take time. This overhead becomes important when we discuss kernel architecture in the following lessons.

Path of a single write(fd, buf, n) call:



To recap the diagram: the glibc wrapper loads the syscall number and arguments into registers and executes `syscall`; on the kernel side, the entry stub runs `swaps` and switches to a

kernel stack before selecting the write handler by syscall number; and on the way back, glibc translates a negative `%rax` (a `-errno` value) into the familiar `-1` with `errno` set before your program continues.

The System Call Table

The system call table is simply an array where each index corresponds to a syscall number, and each entry is a function pointer to the kernel function that handles that call. On Linux x86-64, a few examples:

Syscall Number (%rax)	Name	Kernel Function
0	read	sys_read
1	write	sys_write
2	open	sys_open
3	close	sys_close
39	getpid	sys_getpid
57	fork	sys_fork
59	execve	sys_execve
60	exit	sys_exit
62	kill	sys_kill

x86-64 Linux defines hundreds of system calls; the exact set depends on the kernel version and architecture. You can inspect the authoritative table in the kernel source file `arch/x86/entry/syscalls/syscall_64.tbl`, or the generated system headers installed on your Linux system.

Context Switches

A **context switch** occurs when the kernel stops running one process and starts running another. This is how the kernel implements multitasking — giving the illusion that many processes are running simultaneously on a machine with a limited number of CPU cores.

During a context switch, the kernel must:

1. **Preserve the current task's execution state:** The kernel saves the architectural state needed to resume the task, including its stack and register context. Linux tracks the task in a `task_struct` and keeps additional memory-management and processor state in structures referenced from it.
2. **Switch the memory mapping:** Load the page table base register (CR3 on x86) with the new process's page table pointer, so virtual addresses now map to the new process's physical memory.
3. **Restore the state of the new process:** Load the new process's saved register values, instruction pointer, and stack pointer from its task struct.
4. **Resume execution:** The CPU continues executing from wherever the new process left off.

Context switches are triggered by several events: a timer interrupt may allow the kernel to preempt a task, a system call may block (such as `read()` waiting for disk I/O), or a task may explicitly yield. The kernel's **scheduler** decides which runnable task to run next. Linux used the **Completely Fair Scheduler (CFS)** selection algorithm for many years and began transitioning its fair scheduling class to **EEVDF (Earliest Eligible Virtual Deadline First)** in Linux 6.6. Both use virtual runtime accounting to pursue fairness, while EEVDF adds eligibility and virtual-deadline selection.

Context switches are expensive relative to normal instruction execution. A typical context switch on modern x86 hardware takes on the order of 1-10 microseconds, depending on how much state needs to be saved and whether the TLB (translation lookaside buffer) must be flushed. This cost is one reason why creating too many processes or threads can degrade performance — the system spends too much time switching between them and too little time doing actual work.

Connecting It All Together

Let us revisit some of the things you have done in this course and see them through the lens of kernel services:

- When you called `fork()`, you asked the kernel to create a new entry in the process table, copy the parent's page tables (using copy-on-write optimization), and set up a new task struct.
- When you called `exec()`, you asked the kernel to replace the current process's memory image with a new program loaded from disk, setting up new code, data, and stack segments.
- When you called `wait()`, you asked the kernel to put your process to sleep until one of its children changed state, and to give you the child's exit status.

- When you registered a signal handler with `sigaction()`, you asked the kernel to record a function pointer and, when the specified signal arrives, to interrupt the process and redirect execution to your handler.
- When you opened a file with `open()`, the kernel allocated a file descriptor, looked up the file in the file system, and created internal data structures to track your position in the file.
- When you created a socket and called `connect()`, the kernel managed the entire TCP handshake, set up buffer space for sending and receiving data, and handled all the low-level packet formatting and network routing.
- When you called `malloc()`, the C library either carved memory from a pre-allocated pool or called the `brk()` / `mmap()` system call to ask the kernel for more virtual memory pages.

Every one of these operations required crossing the user-kernel boundary. The kernel was doing the real work behind the scenes all along. In the next three lessons, we will examine how that kernel itself can be organized, and the significant consequences that different organizational choices have for performance, reliability, and security.

Lesson 2: Monolithic Kernels

Introduction

Now that we understand what a kernel does, the natural question is: how do you organize all of that functionality? There is a lot of code involved — process management, memory management, file systems, device drivers, networking, security — and the way you structure it has profound consequences for performance, reliability, and maintainability.

The most straightforward and historically dominant approach is the **monolithic kernel**. In this design, the kernel's major services — the scheduler, memory manager, file systems, device drivers, and networking stack — run in one privileged address space and communicate primarily through direct function calls. User applications and system services still run outside the kernel.

What "Monolithic" Means

The word "monolithic" comes from "monolith" — a single, large block. In a monolithic kernel, core kernel services share one privileged address space. Some code may be linked into the boot-time kernel image and other code may be loaded later as a module, but there is no memory-protection boundary between, say, file-system code and a loaded device driver.

This is in contrast to the microkernel approach (which we will study in Lesson 3), where most of these services run as separate user-space processes and communicate through message passing.

To visualize the difference, consider what happens when a user program reads a file:

In a monolithic kernel:

1. User program calls `read()` → syscall into kernel
2. Kernel's VFS layer determines which file system to use
3. File system code (e.g., ext4) calls the block I/O layer
4. Block I/O layer calls the disk driver
5. Driver talks to hardware, data flows back up through the layers

6. Data is copied to the user's buffer, syscall returns

All of steps 2-5 happen within the kernel, in ring 0, with direct function calls between components. In this simplified path, there are **two** user/kernel crossings: syscall entry and return. A real read may block, be interrupted, or involve a user-space file system, adding scheduling or communication work.

Linux: The Canonical Monolithic Kernel

Linux is the most widely used monolithic kernel in the world. It runs on everything from smartphones (Android) and embedded devices to the vast majority of the world's servers and supercomputers. It is an excellent example to study because it demonstrates both the strengths and challenges of the monolithic approach.

Linux Kernel Architecture

The Linux kernel is organized into several major subsystems, all running in kernel space:

- **System call interface:** The entry point for all user requests. As we discussed in Lesson 1, the syscall instruction transfers control to kernel code, which dispatches to the appropriate handler based on the syscall number in `%rax`.
- **Process scheduler:** The scheduler manages which tasks run on which CPU cores and for how long. As covered in Lesson 1, Linux used CFS selection for many years and began transitioning its fair scheduling class to EEVDF in Linux 6.6; both aim for fair resource distribution and good interactive responsiveness.
- **Virtual File System (VFS):** An abstraction layer that provides a uniform interface for all file systems. When your program calls `open()` or `read()`, VFS determines which underlying file system (ext4, XFS, btrfs, NFS, etc.) should handle the request and calls its specific implementation. This is why you can use the same `read()` call regardless of whether the file is on a local disk, a network share, or even a virtual file in `/proc`.
- **Memory manager:** Handles virtual-to-physical address translation, page allocation and deallocation, the page cache (keeping frequently accessed disk data in RAM), swapping pages to disk when memory is low, and memory-mapped I/O. It manages the page tables that the MMU hardware uses for address translation.
- **Device drivers:** Code that knows how to communicate with specific hardware devices. The `drivers/` tree is the largest source category in typical Linux releases, although exact line counts and percentages change from release to release. Each driver translates generic kernel I/O requests into the register operations, DMA transfers, and interrupt handling that particular hardware requires.

- **Network stack:** Implements the complete TCP/IP protocol suite: Ethernet frame handling, IP routing, TCP connection management, UDP, ICMP, and much more. When you use sockets, every layer of the network protocol is implemented in kernel code.

Because all of these subsystems run in the same address space, they can interact efficiently. The VFS layer can call directly into ext4 code, which can call directly into the block I/O layer, which can call directly into a disk driver. Each of these is just a C function call — fast, simple, and with negligible overhead.

The Advantages of Monolithic Design

The monolithic approach has several significant advantages:

- **Performance:** This is the big one. Since all kernel components share the same address space and privilege level, communication between them is just a function call. There is no need to switch between address spaces, copy messages between processes, or cross privilege boundaries within the kernel. A file system can call a device driver with the same speed that any C function calls another C function. This makes operations like file I/O and networking very fast.
- **Simplicity of communication:** Kernel subsystems can share data structures directly. The VFS layer can pass a pointer to a buffer directly to a device driver. There is no need to serialize data into messages, send them through IPC channels, and deserialize them on the other end.
- **Mature ecosystem:** The monolithic approach has been refined over 50+ years of Unix development. The APIs, design patterns, and debugging tools are well-established. Thousands of drivers exist for Linux because the driver model, while complex, is well-documented and well-understood.

Loadable Kernel Modules

One common criticism of a statically linked monolithic kernel is deployment inflexibility: must you rebuild and reboot to add hardware support? Traditional Unix systems were often configured that way, but modern monolithic kernels support **loadable kernel modules (LKMs)**.

A loadable kernel module is a piece of kernel code that can be loaded into the running kernel at runtime, without rebooting the system. When loaded, the module's code becomes part of the kernel — it runs in ring 0 and has full access to all kernel data structures and functions. When unloaded, its code is removed from memory.

On Linux, you manage modules with these commands:


```
# List all currently loaded modules
lsmod

# Load a module (e.g., a USB storage driver),
# automatically resolving any dependencies
sudo modprobe usb-storage

# (the lower-level insmod command loads a single module,
# but requires the full path to the .ko file and does not
# resolve dependencies)

# Remove a module
sudo modprobe -r usb-storage

# Show information about a module
modinfo usb-storage
```

Most device drivers in Linux are implemented as loadable modules. When you plug in a USB device, the kernel's hotplug system detects the new hardware, identifies which driver module is needed, and loads it automatically. When you unplug the device, the module can be unloaded. This gives the monolithic kernel much of the flexibility that was once thought to be exclusive to microkernels.

However, it is important to understand that loadable modules do not provide any isolation. A loaded module has the same privilege and access as any other kernel code. If a module has a bug, it can corrupt kernel memory and crash the entire system just as easily as if it were compiled directly into the kernel. Modules are a deployment convenience, not a security or reliability boundary.

The Disadvantages of Monolithic Design

The monolithic approach comes with serious drawbacks:

- **Reliability risk:** Since kernel components share an address space and high privilege, a sufficiently serious bug in any one of them can bring down or compromise the entire system. A buggy device driver can overwrite critical kernel data structures and cause a **kernel panic**. Drivers are a large and historically important source of kernel defects, and the monolithic design provides no process-style memory isolation between a loaded driver and the rest of the kernel.
- **Large trusted computing base (TCB):** In security analysis, the "trusted computing base" is the set of code that must be correct for the system to be secure. In a monolithic kernel, the TCB includes the entire kernel — every driver, every file system, every

subsystem. A vulnerability in any of these components can be exploited to gain full control of the system.

- **Difficulty of whole-kernel formal verification:** Linux's size, configuration space, concurrency, hardware interfaces, and rate of change make a comprehensive machine-checked proof far beyond current verification projects. Formal methods are still useful for individual kernel components and properties; the difficulty is proving the entire general-purpose kernel stack.
- **Complexity:** With everything in one address space, the potential for unintended interactions between components is high. A change in the memory manager might subtly affect the behavior of a file system. The kernel's internal APIs are not as well-defined as external APIs, because there has historically been less need for strict boundaries between internal components.

The Size of a Monolithic Kernel

The Linux source tree gives a sense of the scale involved. Exact counts depend heavily on the release, configuration, generated files, and counting method, so the table below is a qualitative guide rather than a current release census:

Category	Relative scale / role
Device drivers	Largest source-tree category
Architecture-specific code	Large; supports many processor families
File systems	Many local, network, and virtual file systems
Networking	Protocol stacks, filtering, and device-independent networking
Core kernel	Scheduler, memory management, locking, and core infrastructure

The vast majority of this code is device drivers. Every graphics card, every network adapter, every sound card, every USB gadget needs its own driver code. This is both a strength (Linux supports an enormous range of hardware) and a weakness (all of that code runs with full kernel privileges, and driver code is statistically the buggiest part of the kernel).

Other Monolithic Kernels

Linux is not the only monolithic kernel. The monolithic design has been the dominant approach throughout operating system history:

- **Traditional Unix (and its descendants):** The original Unix kernel developed at Bell Labs in the 1970s was monolithic. Its descendants — including the various BSD variants — followed the same approach.
- **FreeBSD:** A modern, high-quality monolithic Unix kernel. FreeBSD is widely used in servers (Netflix's streaming infrastructure runs on FreeBSD) and forms the basis of Sony's PlayStation operating system.
- **Windows NT kernel:** Microsoft's documentation, and most OS textbooks, describe NT as a *hybrid* kernel (we'll cover what that means in Lesson 4) because environment subsystems run as user-mode server processes while a Mach-influenced kernel layer underneath does the privileged work. That said, the bulk of OS services — the Executive (which includes the file systems), along with the window manager and graphics drivers that sit alongside it — all run in kernel mode in a single address space, which is why NT's practical performance characteristics resemble a monolithic kernel. Microsoft moved the graphics subsystem (win32k.sys) into the kernel starting with Windows NT 4.0 for performance reasons, pushing it further in that direction.

A Day in the Life of a Monolithic Kernel

To make this concrete, let us trace what happens inside a monolithic kernel like Linux when you run a simple command like `cat file.txt`:

1. Your shell calls `fork()`: the kernel's process management code creates a new process, copies the parent's page tables (with copy-on-write), allocates a new PID, and adds an entry to the process table.
2. The child process calls `execve("/bin/cat", ...)`: the kernel opens the executable, reads its ELF headers, replaces the process image with new virtual-memory mappings, and transfers control to the ELF entry point. For a dynamically linked program, that entry point is normally the dynamic linker, which maps the required shared libraries before starting `cat`.
3. `cat` calls `open("file.txt", O_RDONLY)`: the kernel's VFS looks up the file in the directory cache, checks permissions, allocates a file descriptor, and returns it.
4. `cat` calls `read(fd, buf, 4096)` in a loop: the kernel checks the page cache. If the data is already cached in RAM (from a previous read), it copies it directly to the user buffer — no disk access needed. If not, the kernel issues a request to the block I/O layer,

which asks the disk driver to fetch the data. The process sleeps until the disk interrupt signals completion, then the data is copied to the user buffer.

5. `cat` calls `write(1, buf, n)` to output to stdout: the kernel's terminal driver handles the output, sending characters to your terminal emulator.
6. `cat` calls `exit(0)`: the kernel releases most resources, records the exit status, and notifies the parent. A small zombie entry remains until the parent calls `wait()` or a related function; only then can that final process record be removed.

Every one of these steps involves multiple kernel subsystems cooperating. In a monolithic kernel, this cooperation happens through direct function calls, making the whole operation efficient and fast. In the next lesson, we will see how a microkernel handles the same operations differently — and what the consequences are.

Lesson 3: Microkernel Architectures

Introduction

In the previous lesson, we saw that monolithic kernels put everything into kernel space: drivers, file systems, networking, and more. This gives excellent performance because components communicate through direct function calls, but it means that a bug in any component can crash the entire system. The microkernel takes the opposite approach: make the kernel as **small as possible**, and run everything else as separate user-space processes.

The idea is simple and elegant. If device drivers and file systems run as user-mode processes, an ordinary memory fault in one server can be contained to that process, and a supervisor may be able to restart it. This is much stronger isolation than an in-kernel driver receives, although recovery is not automatic or guaranteed: a failed server may leave requests or hardware in an inconsistent state, and unrestricted device DMA can bypass process isolation unless an IOMMU is configured.

The Microkernel Idea

In a microkernel design, the kernel itself provides only the most fundamental services — the absolute minimum needed to build everything else:

- **Minimal process/thread management:** Creating and destroying processes and threads, scheduling them on CPUs.
- **Minimal memory management:** Setting up address spaces, mapping memory pages.
- **Inter-process communication (IPC):** A mechanism for processes to send messages to each other.

That is essentially it. Everything else — device drivers, file systems, the networking stack, even some aspects of security policy — runs as a **user-space server**. These servers are regular processes running in ring 3, just like your own programs. They communicate with each other and with applications through the microkernel's IPC mechanism.

File Read: The Microkernel Way

To understand the difference, let us trace a file read operation in a microkernel system.

Compare this to the monolithic sequence from Lesson 2:

1. User program calls `read()` → a library translates this into an IPC message to the **file system server**.
2. The microkernel delivers the message to the file system server (which is a user-space process).
3. The file system server determines which disk blocks to read and sends an IPC message to the **disk driver server** (another user-space process).
4. The microkernel delivers this message to the disk driver server.
5. The disk driver server interacts with the hardware (the microkernel may provide limited, controlled access to device registers, or the driver may use memory-mapped I/O granted by the microkernel).
6. The disk driver server sends the data back to the file system server via IPC.
7. The file system server processes the data and sends it back to the user program via IPC.

Notice how many IPC messages and protection-boundary crossings are involved. In the monolithic example, steps 2-5 used direct calls within kernel space. In this microkernel example, communication between separate servers enters the kernel and resumes a receiver, and it may also require copying or mapping message data. Optimized IPC fast paths reduce these costs, but the additional boundaries are a central performance tradeoff.

IPC: The Critical Mechanism

Because user-space services in a microkernel compose through message passing, IPC performance is critical to many common OS operations. Slow IPC especially hurts paths that cross several services.

IPC in a microkernel typically works like this:

1. The sending process prepares a message (which might be as small as a few registers' worth of data or as large as a page of memory).
2. The sender makes a system call to the microkernel: "send this message to process X."
3. The microkernel validates the request, copies the message (or maps the memory page), and switches to the receiving process.
4. The receiving process reads the message and processes it.

Early microkernels suffered because their IPC paths were expensive. The first generation (including early Mach) treated IPC as a general-purpose mechanism with features such as message queuing, port-based addressing, and complex access control. Depending on the

hardware and operation, IPC could be orders of magnitude slower than a local function call. When a file read requires several IPC exchanges, that overhead compounds.

This led many in the systems community to dismiss microkernels as impractical. But as we will see, later designs proved that IPC can be made much faster.

The L4 Family

The L4 microkernel, designed by Jochen Liedtke in the mid-1990s, was a response to the "microkernels are too slow" criticism. Liedtke's insight was that previous microkernels were slow not because the microkernel concept was inherently flawed, but because they were poorly implemented. He set out to prove that a microkernel could have IPC performance comparable to a monolithic kernel's function calls.

Liedtke's original L4 implementation achieved IPC performance that was **20 times faster** than Mach. He accomplished this by:

- Making the kernel extremely small (the original version was roughly 6,400 lines of x86 assembly, compiling to about 12 KB of code)
- Designing IPC to be as simple and fast as possible: synchronous (blocking), with small messages passed directly in registers rather than being copied through memory
- Carefully optimizing the mode-switch path, keeping it to as few instructions as possible

The L4 family spawned many descendants, but the most important for our purposes is **seL4**.

seL4: The Formally Verified Microkernel

seL4, developed by researchers at NICTA (now Data61) in Australia, is a landmark formally verified microkernel. For specific supported configurations, machine-checked proofs connect the implementation to a formal specification and establish security properties under stated assumptions. This provides evidence fundamentally different from testing, but it is not an unconditional claim that every seL4 build, specification, hardware platform, or surrounding user-space system is bug-free.

What the seL4 verification proves:

- For verified configurations, the C implementation refines the abstract specification
- For configurations with binary verification, the machine code refines the verified C implementation, removing the compiler and linker from the trusted assumptions
- Configured systems can enforce integrity, confidentiality, and availability properties described by the security proofs

- Functional correctness rules out classes of implementation defects such as buffer overflows and undefined behavior in the verified code, subject to proof assumptions; boot code, some low-level hardware interfaces, unverified features, and timing channels are outside some or all proof scopes

The verified seL4 kernel is roughly 10,000 lines of C, and its original functional-correctness proof required about 20 person-years of effort. Applying comparable whole-kernel proofs to Linux would be vastly harder not just because Linux is much larger, but because it has an enormous configuration space, frequent changes, concurrency, and thousands of hardware-facing components. Formal methods can still verify selected Linux subsystems and properties.

seL4 has been deployed and evaluated in high-assurance defense, aerospace, and autonomous-system projects. Its value is precise assurance about the verified kernel boundary; the rest of a deployed system must still be designed, configured, and verified appropriately.

Mach

The Mach microkernel, developed at Carnegie Mellon University in the 1980s, is historically one of the most influential microkernels, though its legacy is complicated.

Mach was designed to be a flexible, portable microkernel that could run existing Unix programs. It introduced several important concepts:

- **Ports:** Mach's IPC is based on "ports" — kernel-managed communication channels. Processes send messages to ports, and other processes receive from ports. Port rights (send right, receive right) control which processes can communicate.
- **Memory objects:** Mach unified the concepts of virtual memory and IPC. Files, device memory, and shared memory regions are all represented as memory objects that can be mapped into a process's address space.
- **Tasks and threads:** Mach separated the concepts of a "task" (an address space plus resources) and a "thread" (a unit of execution within a task). A single task can contain multiple threads. This was influential on later threading designs.

However, Mach had a significant problem: performance. Its IPC was complex and slow, involving multiple memory copies and kernel calls. Running Unix services on top of Mach added substantial overhead — early Mach-based Unix systems were on the order of 25-65% slower than native monolithic Unix kernels on real workloads, depending on the benchmark.

Despite these performance issues, Mach's influence lives on. Apple's macOS and iOS use the **XNU kernel**, which incorporates a heavily modified version of the Mach microkernel

combined with components from BSD (a monolithic Unix kernel). We will discuss this hybrid approach in Lesson 4.

QNX

QNX is a commercial microkernel-based real-time operating system developed by QNX Software Systems (now owned by BlackBerry). It is widely deployed in embedded systems where reliability, deterministic timing, and safety certification matter.

QNX runs in:

- **Automobiles:** The infotainment systems and digital instrument clusters in many cars run QNX. Some advanced driver-assistance systems (ADAS) use it as well.
- **Medical devices:** QNX products are used in surgical equipment, patient monitoring, and medical imaging; safety-oriented editions and development evidence can help manufacturers pursue the applicable regulatory approvals.
- **Nuclear power plants:** Safety-critical control systems in nuclear facilities run on QNX.
- **Industrial automation:** Factory robots and process control systems rely on QNX.

QNX Neutrino keeps scheduling, IPC, interrupt handling, and timers in the microkernel while drivers, protocol stacks, and file systems normally run in memory-protected user space.

The key advantage in these environments is **fault containment and the possibility of recovery**. A supervised user-space driver can often be terminated and restarted without corrupting kernel memory. Whether service can continue safely depends on the driver, hardware, application architecture, and recovery policy; a microkernel boundary is an enabling mechanism, not a complete safety guarantee.

MINIX

MINIX holds a special place in operating systems history. Created by Andrew Tanenbaum in 1987 as a teaching operating system for his textbook *Operating Systems: Design and Implementation*, MINIX was the direct inspiration for Linux. Linus Torvalds was a student who used MINIX and decided to write his own kernel — which became Linux.

MINIX 3 (released in 2005) was redesigned with a focus on reliability through its microkernel architecture. In MINIX 3:

- The microkernel is approximately 6,000 lines of code
- Device drivers run as separate user-space processes

- If a driver crashes, a **reincarnation server** detects the failure and restarts the driver automatically, often without any visible disruption to the user
- Each driver runs with the minimum privileges it needs (principle of least privilege)

MINIX 3 demonstrations show its reincarnation server restarting selected user-space drivers after injected failures. Recovery works when the failure has been contained and the driver and device can be returned to a consistent state. In-kernel driver recovery is harder because a fault may already have corrupted shared kernel state, though it is not logically impossible in every case.

Public reports based on Intel presentations have described a MINIX-derived environment in version 11 of Intel's Management Engine firmware on Skylake-generation systems. Intel has not published enough implementation detail to support sweeping claims about every modern Intel processor or a precise device count, so this is best treated as an interesting reported deployment rather than a defining MINIX 3 fact.

The Tanenbaum-Torvalds Debate

In 1992, a famous argument erupted on the comp.os.minix Usenet newsgroup between Andrew Tanenbaum and Linus Torvalds about the future of kernel design. It remains one of the most entertaining and educational exchanges in computing history.

Tanenbaum's position: "Linux is obsolete." He argued that monolithic kernels were a step backward, that microkernels were the future, and that any OS designed as a monolith in 1991 was fundamentally flawed in its architecture. He pointed out that all the academic research pointed toward microkernels, and predicted that within a few years, monolithic kernels would be replaced by more modern designs.

Torvalds's position: Linux works. He argued that the monolithic design was pragmatic and performant, that microkernel overhead was real and significant, and that the theoretical elegance of microkernels did not outweigh the practical advantages of the monolithic approach. He was building something that people could use *now*, not a research project for the future.

More than 30 years later, the debate remains useful rather than decisively settled. Linux shows that monolithic kernels can scale across an extraordinary range of hardware, while QNX, seL4, MINIX 3, and other systems show the value of small privileged cores and fault-isolated services. Real designs choose among performance, hardware support, assurance, compatibility, and recovery requirements; architecture alone does not guarantee any one outcome.

Advantages and Disadvantages Summary

Aspect	Monolithic Kernel	Microkernel
Performance	Excellent (direct function calls)	Overhead from IPC message passing
Reliability	A privileged component bug can affect the whole system	User-space service faults can often be contained
Security	Large attack surface	Small trusted computing base
Formal verification	Whole-kernel proof is extremely difficult at general-purpose scale	Small kernels have been comprehensively verified for specific configurations (seL4)
Driver recovery	Harder because drivers share privileged state	Often possible when the failure and device state are contained
Complexity	One large codebase, but straightforward communication	Many small processes, but complex IPC patterns
Hardware support	Extensive (Linux: thousands of drivers)	Limited (smaller ecosystems)
Development model	All developers work on one codebase	Components can be developed independently

Lesson 4: Tradeoffs and Modern Approaches

Introduction

In the previous two lessons, we studied the two "pure" approaches to kernel design: monolithic kernels that put everything in kernel space, and microkernels that keep the kernel minimal and run everything else in user space. In practice, modern operating systems rarely fall neatly into one category. Real systems make pragmatic engineering tradeoffs, and new approaches have emerged that blur the traditional boundaries. In this lesson, we examine hybrid designs, alternative architectures, and the emerging technologies that are reshaping how we think about kernel organization.

The Performance Question

The central debate between monolithic and microkernel designs has always been about performance. Let us put some numbers on it.

In the early days of microkernels (late 1980s, early 1990s), Mach's feature-rich IPC path was far more expensive than a direct in-kernel function call. Workloads built from multiple user-space servers could therefore suffer substantial slowdowns; published results varied by Mach version, hardware, and workload rather than yielding one universal ratio.

Jochen Liedtke's L4 work demonstrated that a much smaller, carefully designed IPC path could be dramatically faster than Mach's. Absolute latency still depends on the processor, kernel, message size, and whether a benchmark measures one-way IPC or a round trip, so architecture should be compared with measurements on the target workload.

However, even small per-operation overheads add up. Consider a workload that performs 100,000 file system operations per second (not unusual for a database server). If each operation requires 4 extra mode switches at 500 nanoseconds each, that is 200 milliseconds of overhead per second — 20% of a CPU core spent just on mode switching. For many workloads this is acceptable, but for high-performance servers it is significant.

The performance gap has narrowed dramatically since the 1990s, but extra protection-boundary crossings can still matter. General-purpose desktop and server systems commonly use monolithic or hybrid kernels, while microkernels are prominent in a number of embedded and high-assurance systems. Neither architecture by itself guarantees performance or reliability.

Hybrid Kernels

A **hybrid kernel** attempts to combine the performance advantages of a monolithic kernel with some of the architectural benefits of a microkernel. The idea is to run most services in kernel space (for performance) while maintaining some degree of modularity and message-passing structure.

macOS / iOS: XNU

Apple's operating systems (macOS, iOS, iPadOS, watchOS, tvOS) all run on the **XNU kernel**. The name "XNU" is sometimes informally back-formed as "XNU is Not Unix" (echoing GNU) — but Apple does not formally expand the acronym, and the gloss is somewhat misleading anyway, because XNU provides a full POSIX-compatible Unix interface.

XNU is a genuine hybrid. It combines:

- **Mach microkernel:** Provides the lowest-level services: task and thread management, virtual memory, IPC (Mach ports). The Mach layer handles the fundamental abstractions.
- **BSD layer:** A substantial chunk of FreeBSD code, running in kernel space, provides the Unix personality: the POSIX API, the file system stack, the networking stack, the process model, and security features.
- **I/O Kit:** Apple's object-oriented driver framework, written in a restricted subset of C++, running in kernel space.

The key insight is that while XNU has a microkernel *inside* it, most of the OS services still run in kernel mode, just as they would in a monolithic kernel. The BSD and I/O Kit code runs in the same address space as the Mach microkernel. The Mach IPC mechanism is available and used for some inter-component communication, but the performance-critical paths use direct function calls within kernel space.

In practice, XNU keeps many performance-critical services together in one privileged address space while retaining Mach abstractions. This is why it is commonly described as a hybrid rather than a pure microkernel.

Windows NT

Windows NT (the kernel underlying all modern versions of Windows, from Windows XP through Windows 11) was designed by Dave Cutler with a microkernel-inspired architecture. The original design envisioned several "environment subsystems" running as user-space servers, with a small kernel providing core services.

In practice, Windows NT evolved in a monolithic direction over time:

- The graphics subsystem (win32k.sys) was moved from user space into kernel space starting with Windows NT 4.0, for performance reasons.
- Most device drivers run in kernel mode.
- The Windows Driver Framework (WDF) introduced "user-mode drivers" (UMDF) for some device classes, bringing back a touch of microkernel philosophy for devices where performance is less critical (like USB accessories).

Microsoft's documentation sometimes calls NT a "hybrid kernel," and this is fair — it retains some microkernel-inspired structure (the HAL, the object manager, the separation of kernel and executive) while running most code in kernel mode for performance.

DragonFly BSD

DragonFly BSD, forked from FreeBSD in 2003 by Matthew Dillon, takes yet another hybrid approach. It uses a messaging subsystem for communication between kernel components, similar to microkernel IPC, but all the components still run in kernel space. The goal is to get the software-engineering benefits of message passing (clean interfaces, testability, potential for future user-space migration) without the performance cost of actually crossing privilege boundaries.

Unikernels

Unikernels take a radically different approach. Instead of running a general-purpose OS with a clear separation between user and kernel, a unikernel compiles the application and the OS libraries it needs into a **single binary** that runs directly on the hardware (or, more commonly, on a hypervisor like Xen or KVM).

In a unikernel:

- There is **no user/kernel separation**. The application runs in ring 0 alongside the OS code. There is only one address space and one privilege level.
- There is **no process model**. The unikernel runs a single application. There is no shell, no `fork()`, no multiprocessing.

- Only the OS components the application actually needs are included. If the application does not use the network, the network stack is not compiled in.

The results are striking:

- **Tiny binaries:** A unikernel web server might be 5-10 MB, compared to a full Linux installation of hundreds of MB or GB.
- **Boot time:** Unikernels can boot in milliseconds, compared to seconds or minutes for a traditional OS.
- **Potentially smaller attack surface:** The image can omit a shell, unused services, and unnecessary interfaces. This reduces available code and post-exploitation conveniences, but a vulnerability in the included application or library OS can still compromise the whole instance.
- **Performance:** Many unikernels avoid an internal user/kernel syscall boundary and general-purpose process scheduling, although hypervisor transitions, interrupts, and thread scheduling can still occur.

Examples of unikernel projects include:

- **MirageOS:** Written in OCaml, designed for cloud deployments. MirageOS compiles applications into standalone virtual machine images.
- **IncludeOS:** Written in C++, designed for cloud services. An IncludeOS web server can be as small as a few megabytes.
- **Unikraft:** A project that provides modular, configurable unikernel components, allowing you to build custom unikernels with only the OS primitives you need.

The major limitation is that unikernels are **single-purpose**. You cannot run multiple applications, you cannot log in and poke around, and debugging is challenging because there are no standard OS tools available. They are best suited for cloud microservices and serverless functions, where each instance runs exactly one application.

Containers and Virtualization

Containers represent yet another approach to the problem of isolation — one that works within a monolithic kernel rather than replacing it.

Docker containers, the most widely known container technology, run on Linux's monolithic kernel but use kernel features to provide process-level isolation:

- **Namespaces:** Isolate processes so they have their own view of the system — their own process IDs, network interfaces, file system mount points, user IDs, and hostnames. With separate sibling PID namespaces, processes cannot see one another's PIDs. Containers

can also share a PID namespace or use the host namespace; visibility depends on configuration, and ancestor PID namespaces can see descendant processes.

- **Cgroups (control groups):** Account for and control resources such as CPU time, memory, process counts, and block I/O. Correct limits reduce resource interference, but an unconstrained or privileged container can still exhaust host resources.
- **Seccomp-BPF:** Filter system calls so a container process can be restricted to an allowlist of permitted calls, reducing exposed kernel interfaces.

Containers use ordinary process address spaces, so CPU page-table protection does isolate their user memory just as it does for other processes. The important limitation is that all containers trust the same host kernel: a kernel vulnerability can cross container boundaries, and namespaces do not create a second privilege boundary inside that kernel.

Virtual machines (VMs) give each workload a guest kernel and use hardware virtualization to isolate guests. Xen and VMware ESXi are conventionally called type-1 hypervisors. KVM is implemented inside the Linux kernel and places guest execution under hardware virtualization; whether a full KVM-based stack is labeled type 1 or type 2 depends on the taxonomy, but the useful point is that guest kernels are separated by virtualization hardware and a host/hypervisor boundary.

eBPF: Microkernel Flexibility in a Monolithic Kernel

eBPF (extended Berkeley Packet Filter) allows suitably privileged software to load programs that run in constrained contexts in the Linux kernel.

Here is how it works:

1. A user writes a small program in a restricted subset of C.
2. This program is compiled to eBPF bytecode.
3. The eBPF verifier statically checks control flow, bounded execution, types, pointer use, and access rules for the selected program type. Programs using bounded loops are allowed; programs the verifier cannot establish as safe are rejected.
4. If the verifier approves it, the bytecode is JIT-compiled to native machine code and attached to a kernel event (a system call, a network packet arrival, a scheduler decision, etc.).
5. When the event occurs, the eBPF program runs as native or interpreted code under the constraints established by the verifier and the program-type API.

This brings some of the flexibility and safety benefits of microkernels to the monolithic Linux kernel:

- **Safety:** Accepted programs are constrained by verifier rules and helper APIs, greatly reducing risk compared with arbitrary kernel modules. Verifier, JIT, helper, and kernel bugs can still cause vulnerabilities, so this is not an absolute guarantee.
- **Flexibility:** Users can extend kernel behavior without writing kernel modules or rebuilding the kernel.
- **Performance:** eBPF programs run inside the kernel at native speed, avoiding the IPC overhead of microkernel user-space servers.

eBPF is used for networking (high-performance packet filtering and load balancing), security (system call filtering), observability (tracing kernel and application behavior), and more. Companies like Meta, Google, and Cloudflare use eBPF extensively in production.

Formal Verification: The Reliability Frontier

As discussed in Lesson 3, seL4 was the first general-purpose kernel with a machine-checked functional-correctness proof and has accumulated additional binary and security proofs. Those results apply to specified configurations and assumptions; they do not automatically cover every supported feature, platform, or user-space component.

The verified kernel is roughly 10,000 lines of C. For applicable configurations, the proof stack establishes properties such as functional correctness, integrity, and confidentiality under its formal model and assumptions. Comparing source-line counts alone does not measure verification difficulty, but it does illustrate why a deliberately small trusted computing base helps:

Kernel	Scale	Verification status
seL4	Small verified kernel (roughly 10,000 lines of C)	Strong machine-checked proofs for documented configurations; scope varies by platform and feature
MINIX 3	Small microkernel plus user-space services	Designed for fault isolation; not accompanied by a seL4-style whole-kernel proof stack
QNX Neutrino	Commercial microkernel and user-space services	Safety-certified product variants exist; certification is different from a functional-correctness proof
Linux	Very large, fast-changing general-purpose kernel ecosystem	Formal methods are used on selected components and properties, not as a comprehensive proof of the full kernel

Formal verification can establish universal properties within a model that testing cannot exhaustively demonstrate. Assurance still depends on the specification, proof assumptions, verified configuration, hardware, boot path, and system setup, so verification complements rather than eliminates testing, certification, and system engineering.

A small microkernel can make the trusted computing base and proof target manageable. It does not make isolation absolute: user-space policy mistakes, unverified code, DMA, hardware faults, timing channels, or unmet assumptions can still violate a deployed system's goals.

Where Each Approach Shines

Different kernel architectures are suited to different use cases. The following table summarizes the practical landscape:

Use Case	Preferred Architecture	Examples
General-purpose desktops and servers	Monolithic	Linux, FreeBSD
Consumer desktop/mobile OS	Hybrid	macOS/iOS (XNU), Windows (NT)
Embedded real-time systems	Microkernel	QNX (cars, medical devices)
Safety-critical/high-assurance systems	Microkernel (verified)	seL4 (military, aerospace)
Cloud microservices	Unikernel or containers	MirageOS, Docker/Kubernetes on Linux
High-performance networking/observability	Monolithic with eBPF	Linux with eBPF programs

No single architecture is "best." The right choice depends on what you value most: raw performance, fault isolation, provable correctness, hardware support, or development velocity.

The Future

Several developments are reshaping the landscape of kernel design:

Rust in the Linux Kernel

Rust support entered the mainline Linux kernel in version 6.1 (released in 2022). Rust's safe subset and ownership model prevent important classes of memory-safety and data-race bugs when abstractions uphold their contracts. Kernel Rust still uses `unsafe` code and interfaces with C and hardware, so it reduces risk rather than making defects impossible.

Rust addresses a different layer of the problem from microkernel isolation. Safer implementation can prevent many defects before code runs; protection boundaries can contain some failures that remain. A robust system can benefit from both techniques.

CHERI Hardware Capabilities

CHERI (Capability Hardware Enhanced RISC Instructions) is a research hardware architecture from the University of Cambridge and SRI International. CHERI adds hardware-enforced capabilities that carry bounds and permissions and constrain memory access. Pure-capability code uses capabilities for its pointers; hybrid code can mix capability pointers with conventional integer-address pointers. The protection obtained depends on the compilation model and how capabilities are assigned.

This can provide fine-grained memory protection *within* an address space. Research systems explore using capabilities to compartmentalize privileged code without a process boundary. Such designs still have transition, compatibility, and capability-management costs; whether they match a microkernel's isolation or outperform it is an empirical design question, not an automatic consequence of CHERI.

Convergence

The trend in modern operating systems is convergence: monolithic kernels are adopting ideas from microkernels (loadable modules, user-mode drivers, eBPF sandboxing, Rust safety), while microkernel-based systems are finding ways to reduce IPC overhead and improve performance. The sharp line between the two architectures is blurring.

As a systems programmer, understanding both approaches — and the engineering tradeoffs between them — will serve you well regardless of which specific systems you work with. The fundamental tension between performance and isolation is a theme that appears throughout computing, from kernel design to database architecture to distributed systems.

Module 10 Summary

Wrap Up

In this module, you explored the heart of every operating system: the kernel. You learned what the kernel does, how it separates user programs from privileged hardware access, and the two fundamental approaches to organizing a kernel — monolithic and microkernel designs. You also examined modern hybrid approaches and emerging technologies that are reshaping the landscape.

Throughout this course, you have been using kernel services with every system call: creating processes with `fork()`, managing memory, communicating over sockets, handling signals. Now you understand how the software behind those services can be organized, and why that organization matters for performance, reliability, and security.

Key Takeaways

- **The kernel is the core of the operating system.** It runs in privileged mode (ring 0 on x86), manages all hardware resources, and provides services to user programs through system calls. Every `fork()`, `read()`, `write()`, and `socket()` call you have made in this course crossed the user-kernel boundary via the `syscall` instruction.
- **System calls are the interface between user programs and the kernel.** A system call works by placing the `syscall` number in `%rax`, executing the `syscall` instruction, which causes a hardware mode switch to ring 0. The kernel selects the appropriate handler using the `syscall` number, performs the requested operation, and returns to user mode.
- **Monolithic kernels place major kernel services in one privileged address space.** Linux is the canonical example. Scheduling, file systems, networking, and most drivers communicate primarily through direct calls. This can be efficient, but a sufficiently serious privileged-component bug can affect the whole system. Loadable modules provide deployment flexibility, not process-style isolation.
- **Microkernels keep a smaller privileged core and run many services in user space.** Process/thread primitives, address-space management, and IPC remain in the kernel while services such as drivers and file systems can run as protected server processes. This makes some faults easier to contain and can make comprehensive verification tractable, at the cost of additional IPC and system complexity.
- **A central tradeoff is communication cost vs. protection boundaries.** Direct calls within one privileged address space are cheap; crossing isolated address spaces costs more but can contain faults. Implementation quality, hardware, workload, and system configuration also determine real performance and reliability.
- **Hybrid kernels combine elements of both approaches.** macOS (XNU) uses a Mach microkernel internally but runs most services in kernel space. Windows NT was designed with microkernel ideas but evolved toward monolithic for performance. These hybrids reflect the pragmatic reality that neither extreme is ideal for all situations.
- **Modern alternatives continue to evolve.** Unikernels eliminate the user/kernel boundary entirely for single-purpose workloads. Containers provide process-level isolation within a monolithic kernel. eBPF allows safe, verified code to run inside the Linux kernel. Rust in the kernel provides memory safety at compile time.

CHERI hardware capabilities may eventually provide in-address-space isolation at native speed.

- **The right architecture depends on the use case.** Monolithic and hybrid kernels are common in general-purpose computing; microkernels are important in many embedded and high-assurance systems; virtual machines, containers, and some unikernels serve cloud workloads. There is no single "best" design — only tradeoffs matched to requirements.

Connecting to the Course

This module completes a journey through the layers of a computer system that began in Module 1:

Module	Layer	Connection to Kernel Organization
Module 1	Data representation	The kernel and all its data structures are built from the same integer and memory representations you learned first
Module 2	Floating point	Hardware representation of data, including the tradeoffs designers must make at every level of a system
Modules 3-4	Assembly and machine code	System calls use specific instructions (<code>syscall</code>) and register conventions that are defined at the assembly level; hardware privilege levels (ring 0 vs. ring 3) are enforced by the CPU itself
Module 5	Processes and signals	Process creation, scheduling, and signal delivery are core kernel responsibilities
Module 6	Threads	The kernel schedules threads; synchronization combines user-space operations with kernel assistance when threads need to block or wake
Module 7	File I/O and networking	The file system and networking stack are kernel subsystems (in monolithic kernels) or

Module	Layer	Connection to Kernel Organization
		user-space servers (in microkernels)
Module 8	Memory hierarchy and optimization	Caches exploit locality to bridge the CPU-memory speed gap; understanding the memory hierarchy is essential for writing performant code and for understanding how the kernel manages memory
Module 9	Virtual memory, linking, and dynamic memory	Virtual memory, page tables, and the TLB are managed by the kernel's memory subsystem; linking determines how programs are loaded into the virtual address space; malloc/free operate within the heap region of that address space
Module 10	Kernel organization	How all of these subsystems are structured and how they interact

Understanding kernel organization gives you the architectural context for everything else you have learned. When you know how the kernel is structured, you can better understand why system calls have the performance characteristics they do, why context switches are expensive, why a segmentation fault kills your process, and why operating system design involves fundamental engineering tradeoffs that have been debated for over 30 years.

Additional Resources

[The Linux Kernel Archives](#) — the official source of the Linux kernel, where you can browse the actual source code of a monolithic kernel

[The seL4 Microkernel](#) — the homepage of the world's first formally verified general-purpose OS kernel

[L4 Microkernel Family](#) — overview of the L4 family of microkernels, from Liedtke's original to modern descendants like seL4, Fiasco.OC, and NOVA

[MINIX 3](#) — Andrew Tanenbaum's reliable microkernel OS, with documentation on its fault-tolerant design

[QNX 8.0 Developer Documentation](#) — technical documentation for the QNX real-time microkernel OS

[The Tanenbaum-Torvalds Debate \(1992\)](#) — the original Usenet thread where Andrew Tanenbaum and Linus Torvalds debated monolithic vs. microkernel design

[eBPF.io](#) — an introduction to eBPF, the technology bringing sandboxed extensibility to the Linux kernel

[Unikraft](#) — a modular unikernel development framework

Integer and Floating Point Reference

Quick reference for Modules 1–2 topics: C integer types, two's complement, conversions, bit-level operators, and IEEE 754 floating point.

C Integer Types and Sizes (LP64)

Type	Size (bytes)	Width (bits)
<code>char</code>	1	8
<code>short</code>	2	16
<code>int</code>	4	32
<code>long</code>	8	64
<code>long long</code>	8	64
pointer (any <code>T *</code>)	8	64

LP64 is the model used by Linux on x86-64: `long` and pointers are 64 bits, `int` stays 32.

Signed and Unsigned Ranges

For a signed type of width `w` bits (two's complement):

$$TMin = -2^{(w-1)} \quad TMax = 2^{(w-1)} - 1$$

For an unsigned type of width `w` bits:

$$UMin = 0 \quad UMax = 2^w - 1$$

w	TMin	TMax	UMax
8	-128	127	255
16	-32,768	32,767	65,535
32	-2,147,483,648	2,147,483,647	4,294,967,295
64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	18,446,744,073,709,551,615

- Handy anchors: $|TMin| = TMax + 1$ and $UMax = 2 * TMax + 1$.

Two's Complement Essentials

- Negation rule:** $-x == \sim x + 1$ (flip every bit, then add 1).
- Sign bit weight:** the top bit contributes $-2^{(w-1)}$, not $+2^{(w-1)}$; all other bits keep their usual positive weights.
- There is exactly one zero (unlike sign-magnitude), so the range is **asymmetric**: one more negative value than positive.
- The asymmetry bites:** $-TMin$ cannot be represented. Evaluating it is undefined behavior in C; do not rely on the wrapped hardware bit pattern looking like TMin again.
- Same adder hardware works for signed and unsigned addition; only overflow detection and comparisons differ.

Conversions Between Integer Types

Conversion	Bit-level effect
Signed ↔ unsigned, same size	On our GCC/Linux target, the bits are unchanged. Conversion to unsigned is defined modulo 2^w , so <code>(unsigned)-1 == UMax</code> . An unrepresentable conversion to signed is implementation-defined in C.
Widening a signed value	Sign-extend: replicate the sign bit into the new upper bits.
Widening an unsigned value	Zero-extend: fill the new upper bits with 0.
Narrowing to unsigned	Defined modulo 2^w ; ordinary hardware keeps the low bits.
Narrowing to signed	Value is preserved if representable; otherwise the result is implementation-defined (or an implementation-defined signal may be raised). Common GCC/Clang targets keep the low bits, but portable code must not assume that.

- **Integer promotions:** in expressions, `char` and `short` are first promoted to `int` before arithmetic.
- **Usual arithmetic conversions:** when a signed and an unsigned operand of the *same rank* meet, the signed one is converted to unsigned.
- **The classic trap:** `-1 < 1U` is **false** — the `-1` converts to `UMax` (4,294,967,295), which compares greater than 1.

```
int x = -1;
unsigned u = 1;
if (x < u) ... /* NOT taken: x converts to 4294967295 */
```

Bitwise vs Logical Operators

Bitwise	Logical	Difference
<code>&</code>	<code>&&</code>	Bitwise works bit by bit; logical treats any nonzero value as true.
<code> </code>	<code> </code>	Logical returns only 0 or 1; bitwise returns a full-width result.
<code>~</code>	<code>!</code>	<code>~0x41 == 0xFFFFFBE</code> but <code>!0x41 == 0</code> ; <code>!!x</code> normalizes to 0/1.
<code>^</code>	(none)	XOR has no logical counterpart; <code>x ^ x == 0</code> .

- Logical `&&` and `||` short-circuit: the right operand may never be evaluated.
- Typo hazard: `if (p & p->next)` is invalid C because bitwise `&` requires integer operands. Use `if (p && p->next)`; short-circuiting avoids dereferencing a null `p`.

Shift Semantics

- **Left shift:** `x << k` multiplies by 2^k (bits shifted out are discarded, zeros shifted in). For signed operands, a negative left operand or an unrepresentable result causes undefined behavior, even for `-1 << 0`.
- **Right shift, unsigned:** logical shift — zeros fill from the left; `x >> k` is `x / 2^k` (rounding toward zero).
- **Right shift, signed:** arithmetic shift — copies of the sign bit fill from the left, so negatives stay negative. The C standard calls this implementation-defined, but every mainstream compiler (gcc, clang, MSVC) does arithmetic shift.
- Arithmetic right shift rounds toward *negative infinity*, so `-3 >> 1 == -2`, not `-1`.
- Shifting by `k >= w` or by a negative amount is undefined behavior (x86 masks the count to the low bits, but do not rely on it).

Hex and Binary Nibble Table

Each hex digit is exactly one nibble (4 bits); memorizing this table converts any value.

Hex	Binary	Decimal	Hex	Binary	Decimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

IEEE 754 Layout

Format	Sign	Exponent (e)	Fraction (f)	Bias
float (single)	1 bit	8 bits	23 bits	127
double	1 bit	11 bits	52 bits	1023

Value = $(-1)^s \times M \times 2^E$, where M and E depend on the exponent field:

Case	Exponent field	E	M	Represents
Normalized	not all 0s, not all 1s	$e - \text{bias}$	$1.f$ (implied leading 1)	Ordinary values
Denormalized	all 0s	$1 - \text{bias}$	$0.f$ (no implied 1)	Values near 0, including ± 0 (when $f = 0$)
Special	all 1s	—	—	$\pm\infty$ if $f = 0$; NaN if $f \neq 0$

Extreme	float	double
Smallest positive normalized	$2^{-126} \approx 1.18 \times 10^{-38}$	$2^{-1022} \approx 2.23 \times 10^{-308}$
Largest normalized	$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$	$(2 - 2^{-52}) \times 2^{1023} \approx 1.8 \times 10^{308}$
Smallest positive denormalized	$2^{-149} \approx 1.4 \times 10^{-45}$	$2^{-1074} \approx 4.9 \times 10^{-324}$

- Denormalized numbers fill the gap between 0 and the smallest normalized value (gradual underflow), spaced evenly.
- The denormalized E is $1 - \text{bias}$ (not $0 - \text{bias}$) so normalized and denormalized ranges join smoothly.
- $\text{NaN} \neq \text{NaN}$ — any comparison involving NaN (other than $!=$) is false.

Rounding

Mode	Behavior
Round-to-nearest-even (default)	Nearest representable value; exact ties go to the value with an even (0) last bit.
Round toward zero	Truncate toward 0.
Round down (toward $-\infty$)	Always round toward negative infinity.
Round up (toward $+\infty$)	Always round toward positive infinity.

- **Guard/round/sticky in one line:** hardware keeps the two bits just below the last kept bit (guard, round) plus a sticky bit that ORs everything lower, which is enough to round correctly as if the result were computed exactly.
- Round-to-nearest-even avoids the statistical upward bias of always rounding halves up.

Floating Point Pitfalls

- **0.1 is not exact:** like $1/3$ in decimal, 0.1 repeats forever in binary; stored `double` 0.1 is actually 0.1000000000000000055511...
- **Use `==` deliberately:** exact comparison is correct for values that should be identical (including comparisons with 0 in many algorithms), but it is usually wrong for independently rounded approximations. For approximate comparisons, choose an absolute and/or relative tolerance appropriate to the problem; a single fixed epsilon is not scale-independent.
- **Addition is not associative:** in `float`, `(3.14f + 1e10f) - 1e10f == 0.0f` because 3.14f is absorbed, but `3.14f + (1e10f - 1e10f) == 3.14f`.
- **Large ints lose precision:** `float` has a 24-bit significand (1 implied + 23 stored), so integers above $2^{24} = 16,777,216$ are not all representable — `(float)16777217 == 16777216.0f`. For `double` the limit is 2^{53} .
- With IEEE 754's default round-to-nearest mode, overflow goes to $\pm\infty$ rather than wrapping, and gradual underflow passes through subnormals on the way to 0. Directed rounding modes and nonstandard flush-to-zero modes change those details.

Common Mistakes Checklist

- Mixing signed and unsigned in comparisons (`-1 < 1U` is false).

- Assuming `-x` is always positive when `x` is negative (fails for TMin).
- Expecting sign extension when widening an *unsigned* value, or zero extension when widening a *signed* one.
- Confusing `&` with `&&` or `~` with `!`.
- Shifting by the type's full width or more (undefined behavior).
- Assuming right shift of a negative number divides exactly like `/` (it rounds toward negative infinity, not zero).
- Forgetting the bias when reading an exponent field: E is `e - bias` for normalized values, `1 - bias` for denormalized.
- Forgetting the implied leading 1 in a normalized significand.
- Using `==` when independently rounded results are intended to be merely close, or using one fixed absolute epsilon at every scale.
- Assuming every `int` fits exactly in a `float` (only up to `2^24`).

_start Assembly Syntax Reference

This reference matches the style used in this course: GNU assembler (`as`), GNU linker (`ld`), hand-written x86-64 assembly, and direct entry at `_start` (no C runtime).

Minimal Program Skeleton

```
.text
.global _start
_start:
    mov $60, %rax    # syscall: exit
    xor %rdi, %rdi    # status 0
    syscall
```

Build and Run

```
$ as -o prog.o prog.s
$ ld -o prog prog.o
$ ./prog
$ echo $?
```

Multi-file link example:

```
$ as -o main.o main.s
$ as -o readInt.o readInt.s
$ as -o writeInt.o writeInt.s
$ ld -o app main.o readInt.o writeInt.o
```

Core Syntax Rules (AT&T)

- Order is `source, destination` (not destination, source).
- Immediates use `$` (example: `$42`).
- Registers use `%` (example: `%rax`).

- Instruction size suffixes: `b` (1 byte), `w` (2), `l` (4), `q` (8).
- 32-bit register writes (example: `movl ... , %eax`) zero-extend into the upper 32 bits of the 64-bit register.
- `#` starts a comment.

Sections and Directives

Directive	Purpose
<code>.text</code>	Executable code section.
<code>.data</code>	Writable initialized data.
<code>.section .rodata</code>	Read-only constants (strings, tables). Note: GAS does <i>not</i> recognize a bare <code>.rodata</code> shorthand — you must write <code>.section .rodata</code> .
<code>.bss</code>	Zero-initialized/uninitialized storage.
<code>.global name</code>	Export label for linker visibility.
<code>.extern name</code>	Declare symbol defined in another file.
<code>.byte/.word/.long/.quad</code>	Define 1/2/4/8-byte data values.
<code>.asciz "..."</code>	NUL-terminated string.
<code>.equ NAME, value</code>	Symbolic constant.

Addressing Forms

General memory operand form:

`displacement(base, index, scale)`

- Computed address = `displacement + base + index*scale`.
- `scale` must be 1, 2, 4, or 8.
- RIP-relative form for static data: `label(%rip)`.

```
movq 16(%rsp), %rax
movq (%rbx,%rcx,8), %rdx
lea arr(%rip), %r8
```

Data Movement and Arithmetic

- `mov` does not allow memory-to-memory operands in one instruction.
- `lea` computes addresses/arithmetic; it does not read memory contents.
- `idiv` uses the pair `%rdx:%rax`; run `cqto` before signed 64-bit division.

```
movq %rax, %rbx
movq $123, %rax
movq (%rsi), %rax
lea (%rdx,%rdx,4), %rdx      # rdx = rdx*5
addq %rbx, %rax
subq $8, %rsp
imulq $3, %rax, %rax
cqto
idivq %rcx
```

Compare, Test, and Jumps

```
cmpq %rbx, %rax      # flags from (rax - rbx)
testq %rax, %rax     # flags from (rax & rax)
```

Type	Common Jumps
Equality	<code>je</code> , <code>jne</code>
Signed	<code>jl</code> , <code>jle</code> , <code>jg</code> , <code>jge</code>
Unsigned	<code>jb</code> , <code>jbe</code> , <code>ja</code> , <code>jae</code>
Zero/nonzero	<code>jz</code> , <code>jnz</code>

Stack and Calls

```
pushq %rax
popq %rbx
call readInt
ret
```

- `pushq` is equivalent to `subq $8,%rsp` then store at `(%rsp)` .
- `popq` loads from `(%rsp)` then adds 8 to `%rsp` .
- Keep stack alignment consistent before `call` sites when interoperating with ABI-compliant routines.

Linux x86-64 Syscall Convention

Field	Register
Syscall number	<code>%rax</code>
Arg 1..6	<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%r10</code> , <code>%r8</code> , <code>%r9</code>
Return value	<code>%rax</code>
Clobbered by <code>syscall</code>	<code>%rcx</code> , <code>%r11</code>

Common syscall numbers used in this course:

- `read` = 0
- `write` = 1
- `exit` = 60

Function Call Convention (Contrast with Syscalls)

Calls made with `call` follow the System V AMD64 ABI. It is close to the syscall convention above, but not identical — compare the two tables carefully.

Field	Function call (<code>call</code>)	Syscall (<code>syscall</code>)
Arg 1..6	<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> , <code>%r9</code>	<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%r10</code> , <code>%r8</code> , <code>%r9</code>
Return value	<code>%rax</code>	<code>%rax</code>
Caller-saved (may be clobbered)	<code>%rax</code> , <code>%rcx</code> , <code>%rdx</code> , <code>%rsi</code> , <code>%rdi</code> , <code>%r8</code> – <code>%r11</code>	Only <code>%rax</code> (return value), <code>%rcx</code> , <code>%r11</code>
Callee-saved (preserved)	<code>%rbx</code> , <code>%rsp</code> , <code>%rbp</code> , <code>%r12</code> – <code>%r15</code>	Everything except <code>%rax</code> , <code>%rcx</code> , <code>%r11</code>
Stack alignment	<code>%rsp</code> must be 16-byte aligned at the <code>call</code> instruction	No alignment requirement

- **Argument 4 differs:** functions use `%rcx` , syscalls use `%r10` (the `syscall` instruction itself overwrites `%rcx` , so the kernel cannot receive an argument there).
- The `syscall` instruction clobbers `%rax` (return value), `%rcx` , and `%r11` ; other registers survive.
- A `call` is only guaranteed to preserve the callee-saved registers `%rbx` , `%rsp` , `%rbp` , `%r12` – `%r15` . Assume every other register is trashed by the callee.
- If your function uses a callee-saved register, push it on entry and pop it before `ret` .
- Because `call` pushes an 8-byte return address, `%rsp` is 16-byte aligned *before* the call but off by 8 at function entry; account for this before making further ABI-compliant calls.

Common Patterns

Write a string to stdout

```

        .section .rodata
msg:    .asciz "Hello\n"
len = . - msg - 1

        .text
        .global _start
_start:
    mov $1, %rax
    mov $1, %rdi
    lea msg(%rip), %rsi
    mov $len, %rdx
    syscall

    mov $60, %rax
    xor %rdi, %rdi
    syscall

```

Read bytes from stdin into a buffer

```

        .section .bss
buf:    .skip 64

        .text
        .global _start
_start:
    mov $0, %rax
    mov $0, %rdi
    lea buf(%rip), %rsi
    mov $64, %rdx
    syscall                # %rax = bytes read

    mov $60, %rax
    xor %rdi, %rdi
    syscall

```

On return, `%rax` holds the number of bytes actually read. A return value of 0 means EOF; a negative value indicates an error.

Exit with computed status

```

    mov $60, %rax
    mov %r12, %rdi
    syscall

```


`echo $?` reports the low 8 bits (0-255).

Common Mistakes Checklist

- Using `main` instead of `_start` when linking directly with `ld`.
- Forgetting `.global _start`.
- Reversing operand order (AT&T is source then destination).
- Forgetting `$` on immediates.
- Using invalid memory-to-memory arithmetic/move forms.
- Using signed jump for unsigned compare (or vice versa).
- Forgetting that `syscall` overwrites `%rax` with return value.
- Failing to preserve needed registers/values across calls and syscalls.

Useful Commands

```
$ objdump -d prog.o
$ gdb ./prog
(gdb) break _start
(gdb) stepi
(gdb) info registers
(gdb) x/8xg $rsp
```

Scope note: This page is a syntax and convention reference for the course's `_start` workflow. It is not a full x86-64 instruction manual.

Processes, Signals, and Exceptions Reference

Quick reference for Module 5 topics: exceptional control flow, process control, and signal handling on Linux/x86-64.

Exception Classes

Class	Source	Typical Return Behavior	Example
Interrupt	Asynchronous external event	Resume next instruction	Timer interrupt, device I/O
Trap	Synchronous intentional exception	Resume next instruction	<code>syscall</code>
Fault	Synchronous potentially recoverable exception	Retry faulting instruction if fixed	Page fault
Abort	Synchronous unrecoverable exception	Terminate process	Serious hardware/internal failure

Process Lifecycle Syscalls

```
pid_t pid = fork();
if (pid == 0) {
    execl("/bin/ls", "ls", "-l", (char *)0);
    _exit(127);    // only runs if exec fails
}
int status;
waitpid(pid, &status, 0);
```

- `fork()` : returns 0 in child, child PID in parent, -1 on error.
- `execve()` family: replaces current process image, does not return on success.
- `waitpid()` : reaps child and prevents zombies.
- Use `_exit()` in child error paths after `fork()` if needed.

Zombies and Orphans

- Zombie: child has exited but parent has not reaped it yet.
- Orphan: parent exits first; child is re-parented (typically to PID 1/systemd).
- Install SIGCHLD handling or call `waitpid()` reliably to avoid zombie buildup.

Signal Basics

Signal	Default Action	Typical Source
<code>SIGINT</code>	Terminate	Ctrl+C
<code>SIGTSTP</code>	Stop	Ctrl+Z
<code>SIGKILL</code>	Terminate	Administrative kill, cannot catch/block/ignore
<code>SIGSTOP</code>	Stop	Cannot catch/block/ignore
<code>SIGCHLD</code>	Ignore (commonly)	Child state change
<code>SIGALRM</code>	Terminate	Timer expiration

Installing Handlers (Recommended: `sigaction`)

```
volatile sig_atomic_t got_sigint = 0;

void on_sigint(int signo) {
    got_sigint = 1;
}

struct sigaction sa;
sa.sa_handler = on_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
```

- `signal()` is simple but less precise across platforms; prefer `sigaction()` .
- Keep handler work minimal; set a flag and return.
- Use `volatile sig_atomic_t` for shared handler flags.

Async-Signal Safety

- Safe in handlers: low-level async-signal-safe functions such as `write()` .
- Unsafe in handlers: `printf()` , `malloc()` , most stdio and non-reentrant library code.
- If output is needed inside a handler, prefer `write(STDOUT_FILENO, ...)` .

Blocking and Unblocking Signals

```
sigset_t mask, prev;
sigemptyset(&mask);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, &prev);

// critical section

sigprocmask(SIG_SETMASK, &prev, NULL);
```

- `SIG_BLOCK` : add to blocked set.
- `SIG_UNBLOCK` : remove from blocked set.
- `SIG_SETMASK` : replace blocked set exactly.

Process Groups and kill

- `kill(pid, sig)` : signal one process if `pid > 0` .
- `kill(0, sig)` : signal all processes in caller's process group.
- `kill(-pgid, sig)` : signal all processes in process group `pgid` .

```
$ kill -9 12345      # process 12345
$ kill -9 -15111     # process group 15111
```

Common Wait Macros

```
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    int sig = WTERMSIG(status);
}
```

Reliable SIGCHLD Reaping Loop

```
void on_sigchld(int signo) {
    int saved = errno;
    for (;;) {
        pid_t pid = waitpid(-1, NULL, WNOHANG);
        if (pid > 0) continue;
        if (pid == -1 && errno == EINTR) continue;
        break;
    }
    errno = saved;
}
```

This pattern reaps all available terminated children and avoids leaving zombies.

Common Mistakes Checklist

- Forgetting to reap children, causing zombies.
- Calling non-async-signal-safe functions inside signal handlers.

- Assuming blocked standard signals queue multiple times (they do not).
- Ignoring return values and errno from `fork` / `exec` / `waitpid` / `sigaction` .
- Using `exit()` in a post-fork child error path when `_exit()` is required.

Concurrency and Threads Reference

Quick reference for Module 6 topics: pthreads, race conditions, mutexes, condition variables, semaphores, and deadlock patterns.

Thread Model: Shared vs Private

Per-Thread (Private)	Per-Process (Shared by Threads)
Registers, program counter, stack	Code segment, heap, globals, open file descriptors

Compile/Link Flags

```
$ gcc -pthread -O2 -g program.c -o program
```

Use `-pthread` for correct compile + link behavior with pthreads.

Create and Join Threads

```
void *worker(void *arg) {  
    return NULL;  
}  
  
pthread_t tid;  
pthread_create(&tid, NULL, worker, arg_ptr);  
pthread_join(tid, NULL);
```

- Start routine signature: `void *fn(void *arg)` .
- Passing pointer to loop variable is a common race bug; pass stable per-thread data.

Race Condition Pattern

```
// not atomic:
counter++;
// often compiles to load, increment, store
```

Two threads interleaving these steps can lose updates.

Mutex Basics

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);
// critical section
pthread_mutex_unlock(&m);
```

- Always unlock on all return/error paths.
- `pthread_mutex_trylock()` tests lock availability without blocking.

Condition Variables

```
pthread_mutex_lock(&m);
while (!condition_is_true) {
    pthread_cond_wait(&cv, &m);
}
// condition holds
pthread_mutex_unlock(&m);
```

- Use `while`, not `if`, because of spurious wakeups.
- `pthread_cond_wait` atomically releases the mutex and sleeps, then re-locks on wake.
- Use `pthread_cond_signal` (wake one) or `pthread_cond_broadcast` (wake all).

Semaphores

```
sem_t s;  
sem_init(&s, 0, initial_value);  
sem_wait(&s);    // P operation (decrement/block)  
sem_post(&s);    // V operation (increment/wake)  
sem_destroy(&s);
```

- Binary semaphore (`initial_value=1`) can enforce mutual exclusion.
- Counting semaphore tracks available units/resources.

Producer/Consumer Semaphore Pattern

```
sem_t empty, full, mutex;  
// empty = N, full = 0, mutex = 1  
  
// producer  
sem_wait(&empty);  
sem_wait(&mutex);  
// insert item  
sem_post(&mutex);  
sem_post(&full);  
  
// consumer  
sem_wait(&full);  
sem_wait(&mutex);  
// remove item  
sem_post(&mutex);  
sem_post(&empty);
```

Deadlock: Four Conditions

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Most practical prevention in systems code: enforce a global lock acquisition order.

Reader-Writer Lock

```
pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;

pthread_rwlock_rdlock(&rw); // many readers can hold concurrently
pthread_rwlock_unlock(&rw);

pthread_rwlock_wrlock(&rw); // writer is exclusive
pthread_rwlock_unlock(&rw);
```

Common Pthreads APIs

Task	API
Create thread	<code>pthread_create</code>
Join thread	<code>pthread_join</code>
Detach thread	<code>pthread_detach</code>
Initialize mutex	<code>pthread_mutex_init</code>
Lock/unlock mutex	<code>pthread_mutex_lock</code> / <code>pthread_mutex_unlock</code>
Condition wait/signal	<code>pthread_cond_wait</code> / <code>pthread_cond_signal</code>
RW lock	<code>pthread_rwlock_rdlock</code> / <code>pthread_rwlock_wrlock</code>

Common Mistakes Checklist

- Sharing stack addresses that go out of scope with worker threads.
- Returning with a mutex still locked.
- Using `if` around `pthread_cond_wait` instead of `while`.
- Assuming operation order without synchronization.
- Inconsistent lock ordering across code paths (deadlock risk).
- Using `volatile` as a replacement for mutex/atomic synchronization.

Debugging Tips

```
$ gdb ./app
(gdb) info threads
(gdb) thread apply all bt
```

```
$ valgrind --tool=helgrind ./app      # thread error detector
$ valgrind --tool=drd ./app          # alternative thread checker
```

File I/O and Sockets Reference

Quick reference for Module 7 topics: Unix file descriptors, buffered I/O, directories, and TCP socket programming.

Standard File Descriptors

FD	Name
0	stdin
1	stdout
2	stderr

Low-Level File I/O

```
int fd = open(path, O_RDONLY);
ssize_t n = read(fd, buf, sizeof(buf));
ssize_t m = write(fd2, buf, n);
close(fd);
```

- `open` returns -1 on error.
- `read` returns 0 at EOF.
- `write` may perform partial writes; loop until all bytes are written.

Open Flags

```
open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Flag	Meaning
<code>O_RDONLY</code>	Open for reading only
<code>O_WRONLY</code>	Open for writing only
<code>O_RDWR</code>	Open for read/write
<code>O_CREAT</code>	Create if missing (requires mode)
<code>O_TRUNC</code>	Truncate existing file
<code>O_APPEND</code>	Append writes at end

Buffered FILE* I/O

```
FILE *fp = fopen("in.txt", "r");
int c = fgetc(fp);
fclose(fp);
```

- FILE* is buffered in user space; usually faster than byte-at-a-time `read` .
- Use `fflush(fp)` to force buffered output to be written.

Directory Iteration

```
DIR *d = opendir(".");
struct dirent *ent;
while ((ent = readdir(d)) != NULL) {
    // ent->d_name
}
closedir(d);
```

TCP Socket Lifecycle (Server)

```
int listenfd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
bind(listenfd, (struct sockaddr *)&addr, sizeof(addr));
listen(listenfd, backlog);
int connfd = accept(listenfd, (struct sockaddr *)&client, &clen);
```

Order is important: `socket` -> `bind` -> `listen` -> `accept` .

TCP Client Lifecycle

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *)&server, sizeof(server));
write(fd, req, req_len);
read(fd, buf, sizeof(buf));
close(fd);
```

Address Conversion

```
struct sockaddr_in a;
memset(&a, 0, sizeof(a));
a.sin_family = AF_INET;
a.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &a.sin_addr);
```

- `htons` / `htonl` : host -> network byte order.
- `ntohs` / `ntohl` : network -> host byte order.
- `inet_pton` : parse text IP to binary form.

Partial Read/Write Rules

- TCP is a byte stream; message boundaries are not preserved.
- One `write()` can be observed as multiple `read()` calls.
- Loop until protocol-level message is complete.

Threaded Server FD Handoff Pattern

```
int client_fd = accept(...);
int *arg = malloc(sizeof(*arg));
*arg = client_fd;
pthread_create(&tid, NULL, handle_client, arg);
pthread_detach(tid);
```

Passing `&client_fd` directly from loop stack variable can race and hand wrong FD to threads.

HTTP Mini-Server Essentials

- Request line: method, path, version.
- Response starts with status line, then headers, blank line, then body.
- Set at least `Content-Length` and `Content-Type` .
- Path sanitization: reject `..` to prevent directory traversal.

Useful Headers

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

Common Mistakes Checklist

- Ignoring partial writes/reads on sockets.
- Forgetting to close both connection FDs and listening FDs where appropriate.
- Calling `bind` without `SO_REUSEADDR` and hitting restart delays.
- Using host byte order directly in `sin_port` .
- Not checking return values (`-1`) and `errno`.

Memory Hierarchy and Optimization Reference

Quick reference for Module 8 topics: RAM/caches, locality, compiler optimizations, and measurement-driven tuning.

Hierarchy at a Glance

Level	Typical Property
Registers	Smallest, fastest
L1/L2/L3 cache (SRAM)	Fast, limited capacity
Main memory (DRAM)	Larger, slower
Disk/SSD	Persistent, much slower

Locality Rules

- **Temporal locality:** recently used data tends to be reused soon.
- **Spatial locality:** nearby addresses tend to be accessed together.
- Smaller stride generally improves spatial locality.
- In C row-major arrays, varying the last index in inner loops often gives stride-1 access.

Cache Concepts

- Hit: data found in cache; miss: fetch from lower level.
- Common cache line size is 64 bytes.
- Miss types: compulsory (cold), conflict, capacity.
- Write policies: write-through vs write-back (dirty bit tracking).

Struct Layout and Padding

```
struct A {  
    char a;  
    int b;  
    char c;  
    double d;  
};
```

- Compiler inserts padding to satisfy alignment constraints.
- Ordering fields from larger to smaller often reduces padding.

Amdahl's Law

$$\text{overall_speedup} = 1 / ((1 - p) + p/s)$$

- `p` : fraction of runtime affected.
- `s` : speedup of optimized part.
- If `p` is small, max whole-program speedup is limited.

GCC Optimization Levels

Flag	Use
<code>-O0</code>	No optimization; easiest debug mapping
<code>-Og</code>	Debug-friendly optimizations
<code>-O1</code>	Basic optimization
<code>-O2</code>	Common production baseline
<code>-O3</code>	More aggressive, may increase code size

Useful companions: `-g` for debug symbols, `-pg` for gprof instrumentation.

Optimization Workflow

1. Get correct output first.

2. Measure baseline.
3. Profile hotspots.
4. Optimize the hotspot.
5. Re-measure and verify correctness.

Measuring Performance

```
$ /usr/bin/time -p ./app
real 1.23
user 1.18
sys 0.04
```

- `real` : wall-clock elapsed time.
- `user` : CPU time in user mode.
- `sys` : CPU time in kernel mode.

gprof Quick Start

```
$ gcc -pg -O2 app.c -o app
$ ./app
$ gprof ./app gmon.out | less
```

Classic Loop Pitfalls and Fixes

```
// pitfall
for (int i = 0; i < strlen(s); i++) { ... }

// fix
size_t n = strlen(s);
for (size_t i = 0; i < n; i++) { ... }
```

Cache-Friendly Matrix Tips

- Prefer loop orders with stride-1 access in innermost loops.
- Use blocking/tiling so working submatrices fit cache.
- Avoid unnecessary temporary arrays and scattered access patterns.

Pointer Aliasing Reminder

Compiler must assume pointers may alias unless it can prove otherwise. This can block optimizations that would otherwise be legal.

Common Mistakes Checklist

- Optimizing before profiling.
- Comparing only single-run timings.
- Ignoring cache behavior and memory access patterns.
- Changing algorithms for tiny gains that hurt readability without measured value.
- Assuming source-level simplification always maps to faster machine code.

Virtual Memory, Linking, and Heap Reference

Quick reference for Module 9 topics: address translation, linking, heap allocation, Valgrind, and garbage collection concepts.

Virtual Memory Basics

- User processes use virtual addresses; MMU translates to physical addresses.
- TLB caches recent translations.
- With 4 KB pages, offset uses 12 low-order bits.
- A leaf PTE commonly includes a present bit, physical page-frame number, permissions, and accessed/dirty state; exact fields are architecture-specific.

Page Fault Flow

1. Translation is non-present or disallows the requested access.
2. CPU raises page fault exception.
3. Kernel handler validates the access and may allocate a zero page, handle copy-on-write, map cached data, read storage, or reject the access.
4. Instruction is retried or process is signaled (for invalid access).

Static vs Dynamic Linking

Mode	Trait
Static	Library code copied into executable (larger binary)
Dynamic	Libraries resolved at load/runtime (smaller binary, shared libs)

```
$ gcc main.c -o dyn_app
$ gcc -static main.c -o static_app
$ ldd dyn_app
```

PLT and GOT (Dynamic Linking)

- PLT: call stubs for external functions.
- GOT: runtime-resolved addresses used by those stubs.
- First call may trigger resolver; later calls usually jump directly via GOT.

Symbol Strength (Common Rules)

- Ordinary global function and object definitions are strong unless explicitly marked weak.
- ELF weak binding (`.weak`) is distinct from COMMON storage (`.comm`), or tentative C definitions with `-fcommon`). GCC defaults to `-fno-common` since version 10.
- An included strong definition overrides weak or COMMON definitions of that name. Multiple COMMON declarations merge using the largest size; weak selection does not follow that size rule.

Heap Allocation APIs

```
void *p = malloc(n);
void *q = calloc(count, size);
void *r = realloc(p, new_n);
free(p);
```

- `malloc` returns `NULL` on failure.
- `calloc` allocates and zero-initializes.
- `realloc` may move block; always use returned pointer carefully.
- Double-free and use-after-free are undefined behavior.

Fragmentation Terms

- **Internal fragmentation:** wasted bytes inside allocated blocks.
- **External fragmentation:** free memory exists but split into unusable pieces.
- **Coalescing:** merging adjacent free blocks.

Valgrind Quick Commands

```
$ gcc -g -O0 app.c -o app
$ valgrind --leak-check=full ./app
$ valgrind --track-origins=yes ./app
```

Report	Meaning
Invalid read/write	Out-of-bounds or bad pointer access
Definitely lost	Leaked block with no remaining pointers
Indirectly lost	Reachable only through definitely lost blocks

Garbage Collection Concepts

- Reference counting fails on cycles unless cycle detection is added.
- Mark-and-sweep: mark reachable from roots, then sweep unreachable.
- Generational GC collects young objects frequently and old objects less often.
- C's unrestricted pointer arithmetic makes moving GC difficult.

Common Mistakes Checklist

- Forgetting to free on all paths.
- Using pointers after `free` .
- Writing one byte past array bounds.
- Assuming linking errors are compiler errors; often missing objects/libraries.
- Ignoring page-fault/performance effects of access patterns.

Kernel Organization Reference

Quick reference for Module 10 topics: kernel responsibilities, privilege transitions, architecture styles, and practical design tradeoffs.

What the Kernel Does

- Manages CPU scheduling and context switches.
- Manages virtual memory, page tables, and protection.
- Provides file systems, networking, and device access.
- Exposes services via system calls.
- Enforces isolation between processes and between user/kernel mode.

Privilege Levels

Mode	Typical Capabilities
User mode (ring 3)	No direct privileged instructions or device control
Kernel mode (ring 0)	Full privileged access to CPU/MMU/devices

System Call Path (x86-64 Linux)

1. User code places syscall number in `%rax` and args in syscall registers.
2. CPU executes `syscall` and switches to kernel mode.
3. Kernel dispatches through syscall table to handler.
4. Kernel returns result in `%rax` and resumes user mode.

Common user-space wrapper path: `libc wrapper -> syscall instruction -> kernel handler -> return`.

Context Switch Essentials

- Save current thread/process register state.
- Select next runnable task.
- Restore next state and memory mapping context.
- Switching tasks incurs overhead (cache/TLB disruption + scheduler work).

Kernel Architecture Styles

Style	Core Idea	Strength	Tradeoff
Monolithic	Most services run in kernel space	High performance via direct calls	Large trusted code base; faults can be catastrophic
Microkernel	Minimal kernel, services in user space	Fault isolation and smaller trusted core	More IPC/mode-switch overhead
Hybrid	Mix of both approaches	Pragmatic performance/reliability balance	Architectural complexity

Monolithic Kernel Notes

- Linux is monolithic with loadable kernel modules.
- Drivers, file systems, scheduler, networking, and VM all run in privileged space.
- Module loading improves deployability, not protection isolation.

Microkernel Notes

- Kernel keeps only essentials: low-level scheduling, memory primitives, IPC.
- Drivers and services run as user-space servers.
- If a server crashes, system can often recover without full kernel panic.

IPC and Message Passing

- Microkernels rely heavily on IPC for service composition.

- Fast IPC design is central to microkernel performance.
- Copying and context switches can dominate if IPC path is inefficient.

Scheduling Terms

Term	Meaning
Preemption	OS interrupts running task to run another
Time slice (quantum)	CPU time budget before potential switch
Latency	Time from ready-to-run until execution
Throughput	Amount of work completed per unit time

Reliability and Security Design Points

- Smaller trusted computing base can simplify auditing and formal verification.
- Compartmentalization limits blast radius of component bugs.
- Monolithic kernels reduce boundary crossings but increase privileged attack surface.

Modern Practical Directions

- Containers: process isolation using kernel namespaces/cgroups.
- eBPF: verifier-checked bytecode executed in-kernel for observability, networking, and other constrained hooks.
- Rust in kernel components: memory-safety improvements for new code.
- Specialized kernels (e.g., RTOS/microkernels) in safety-critical systems.

Common Mistakes Checklist

- Assuming loadable modules are isolated like user-space processes.
- Treating all context switch costs as constant; cache/TLB effects matter.
- Assuming microkernels are always too slow for real systems.
- Assuming monolithic kernels are inherently insecure; engineering quality still dominates outcomes.

- Ignoring workload: architecture choice depends on goals (latency, throughput, isolation, certifiability).